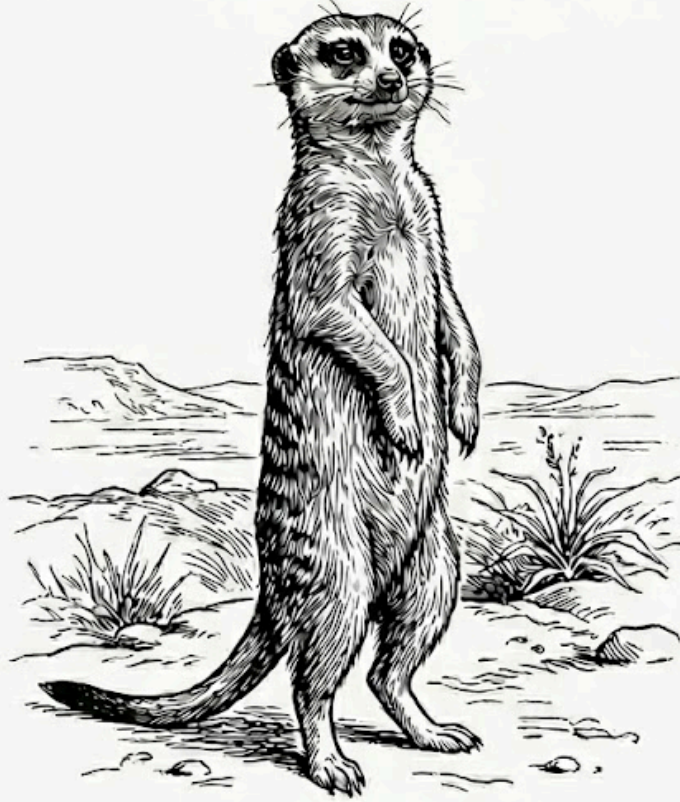


# OxiDB

## DOKÜMAN VERİTABANI

Ölçeklenebilir, NoSQL Doküman Tabanlı Çözümler Rehberi



Barış AKIN ve Anthropic

Yayın Yılı: 2026 • © 2026 Barış AKIN ve Anthropic. Tüm hakları saklıdır.

# **OxiDB Doküman Veritabanı**

Ölçeklenebilir, NoSQL Doküman Tabanlı Çözümler Rehberi

Barış AKIN ve Anthropic

2026

## Önsöz

Bu kitap ne anlatıyor  
Bu kitap kime göre  
Bu kitabın yöntemi  
Kitabın düzeni

### 1 Veri, Veritabanı ve Veritabanı Yönetim Sistemleri

- 1.1 Veri nedir, enformasyon nedir
- 1.2 Kalıcılık: neden bellek yetmez
- 1.3 Neden sadece dosya kullanmıyoruz
- 1.4 Veritabanı ile veritabanı yönetim sistemi
- 1.5 Gömülü kütüphane mi, sunucu mu
- 1.6 Bu kitabın kuracağı zihinsel harita

### 2 Veri Modelleri: Hiyerarşikten Belgeye

- 2.1 Veri modeli neyi belirler
- 2.2 Hiyerarşik model: dünya bir ağaçtır
- 2.3 Ağ modeli: bağlantılar her yöne
- 2.4 İlişkisel model: büyük ayrışma
- 2.5 İlişkisel modelin gölgesi: parçalanma ve uyumsuzluk
- 2.6 Anahtar-değer modeli: en yalın biçim
- 2.7 Belge modeli: yapılandırılmış bütünlük
- 2.8 Diğer modeller ve manzaranın bütünü

### 3 İlişkisel Modelden Belge Modeline: Neden ve Ne Zaman

- 3.1 Modelin kralı değil, hizmetkârı: erişim örüntüleri
- 3.2 Belge modeline geçişin dört itici gücü
- 3.3 Belge modelinin parladığı durumlar
- 3.4 Belge modelinin zorlandığı durumlar
- 3.5 Çoğaltma ödünleşimine yeniden bakış
- 3.6 “Ya hep ya hiç” yanlıgısı ve melez gerçeklik
- 3.7 Karar vermeden önce sorulacak sorular

### 4 Belge Modeli Derinlemesine: JSON, Şema Esnekliği, Gömme ve Referans

- 4.1 Bir belge neyden yapılır
- 4.2 Kendini tanımlayan veri
- 4.3 JSON nedir ve nereden geldi
- 4.4 JSON neden kazandı
- 4.5 Belgeler ve koleksiyonlar
- 4.6 “Şemasızlık” yanlıgısı: şema-okumada ve şema-yazmada
- 4.7 Belge modelinin kalbindeki karar: gömmek mi, atıfta bulunmak mı
- 4.8 Atomikliğin sınırı belgedir

### 5 Depolama Motorları: Sayfa Tabanlı, Append-Only, LSM ve B-Ağaçları

- 5.1 Depolama motoru ne yapar
- 5.2 Her şeyi belirleyen kısıt: disk belleğe benzemez
- 5.3 Temel gerilim: okumaya mı, yazmaya mı eniyilemek
- 5.4 Sayfa tabanlı B-ağaçları: yerinde güncelleme
- 5.5 Append-only ve log-yapılı yaklaşımlar: asla üzerine yazma
- 5.6 LSM ağaçları: log-yapılının olgun hâli
- 5.7 Üç büyütme: kaçınılmaz üçgen

- 5.8 Belleğin rolü ve veriyi belleğe yansıtmak
- 5.9 Bu bölümün bıraktığı yer ve bir sonrakine köprü
- 6 Dayanıklılık: Write-Ahead Log, fsync ve Çökmeden Kurtarma
  - 6.1 “Yazdım” demek neden yetmez
  - 6.2 fsync: veriyi gerçekten diske zorlamak
  - 6.3 İkinci tehlike: yarım kalmış yazma
  - 6.4 Çözüm: önce niyetini yaz
  - 6.5 Kurtarma: enkazın içinden geri dönmek
  - 6.6 Yarım yazmayı yakalamak: sağlama toplamları
  - 6.7 Günlük sonsuza dek büyüyemez: denetim noktası
  - 6.8 Dayanıklılığın tayfı: katı, gevşek ve grup commit
  - 6.9 WAL nereye oturur
- 7 İndeksleme: B-Ağacı İndeksleri, Bileşik İndeksler ve Ters İndeks
  - 7.1 Tarama sorunu
  - 7.2 İndeks nedir: kitabın arkasındaki dizin
  - 7.3 İndeksin bedeli: bedava hız yoktur
  - 7.4 Sıralı indeksler: hem eşitlik hem aralık
  - 7.5 Seçicilik: her indeks aynı ölçüde işe yaramaz
  - 7.6 Bileşik indeksler: birden çok alan birlikte
  - 7.7 Kapsayan indeksler: belgeye hiç dokunmamak
  - 7.8 Ters indeks: metni aranabilir kılmak
  - 7.9 İndeksler de dayanıklı olmalı
  - 7.10 Bu bölümün bıraktığı yer
- 8 Sorgu İşleme: Ayrıştırma, Planlama ve İndeks Seçimi
  - 8.1 Soru ile yol arasındaki fark
  - 8.2 Birinci aşama: ayrıştırma
  - 8.3 İkinci aşama: planlama
  - 8.4 Birden çok koşulu birleştirmek ve en seçiciyi seçmek
  - 8.5 İndeks yoksa: tarama ve süzgeç
  - 8.6 Sıralama, atlama ve sınırlama
  - 8.7 Yalnızca gerekeni döndürmek
  - 8.8 Bildirimsel olmanın asıl kazancı
  - 8.9 Bu bölümün bıraktığı yer
- 9 Toplama: Pipeline Modeli, Gruplama ve Pencere Fonksiyonları
  - 9.1 Süzmenin ötesi: veriyi özetlemek
  - 9.2 Pipeline modeli: montaj hattı
  - 9.3 Pipeline’ın aşamaları
  - 9.4 Sıranın önemi
  - 9.5 Gruplamanın derinliği
  - 9.6 Pencere fonksiyonları: satırı yok etmeden hesaplamak
  - 9.7 Toplamanın performans doğası
  - 9.8 Bu bölümün bıraktığı yer
- 10 İşlemler: ACID, Yalıtım, Kilitleme, MVCC ve OCC
  - 10.1 İşlem nedir: bölünmez bir bütün
  - 10.2 Dört güvence: ACID
  - 10.3 Yalıtımın çözdüğü sorun: eşzamanlılık anormallikleri

- 10.4 Yalıtım düzeyleri: bir tayf
- 10.5 Yalıtımı sağlamanın üç felsefesi
- 10.6 Belge dünyasında işlemler: tek belge kolay, çoğu zor
- 10.7 Bedava güvence yoktur
- 10.8 Bu bölümün bıraktığı yer

#### 11 Eşzamanlılık ve Tutarlılık

- 11.1 Neden birden çok kopya
- 11.2 Yeni sorun: kopyalar anlaşmazlığa düşebilir
- 11.3 Tutarlılık tayfı: katıdan gevşeye
- 11.4 Bölünme anı ve kaçınılmaz seçim
- 11.5 Güçlü tutarlılık nasıl sağlanır: otorite ve çoğunluk
- 11.6 Ayarlanabilir tutarlılık
- 11.7 Tutarlılık ihtiyacı doğruluktan doğar
- 11.8 Bu bölümün bıraktığı yer

#### 12 Ölçeklendirme: Replikasyon, Konsensüs ve Sharding

- 12.1 Neden tek makine yetmez
- 12.2 Replikasyon: aynı verinin kopyaları
- 12.3 Lider çökünce: failover ve iki büyük tehlike
- 12.4 Konsensüs: çoğunlukla anlaşmak
- 12.5 Sharding: veriyi bölmek
- 12.6 Yönlendirme ve parçalar-arası sorgu
- 12.7 İyi bir parça anahtarı ve yeniden dengelemenin yükü
- 12.8 İkisini birleştirmek
- 12.9 Yine aynı ders: dağıtım bedava değildir
- 12.10 Bu bölümün bıraktığı yer

#### 13 Bellek, Önbellek ve Disk Ödünleşimi

- 13.1 Temel asimetri ve oyunun özü
- 13.2 Çalışma kümesi ve uçurum etkisi
- 13.3 Önbellek: belleği akılcıca kullanmak
- 13.4 Tahliye: yer açmak için ne atılır
- 13.5 Önbelleğin kendi maliyeti ve sınırlama zorunluluğu
- 13.6 İki felsefe: belleğe öncelikli ve diske öncelikli
- 13.7 İşletim sistemine güvenmek: belleğe yansıtma
- 13.8 Sıkıştırma: yer, işlemci ve sıfır-kopya üçgeni
- 13.9 Bu bölümün bıraktığı yer

#### 14 Güvenlik: Kimlik Doğrulama, Yetkilendirme, Şifreleme ve Denetim

- 14.1 Tehdit ve katmanlı savunma ilkesi
- 14.2 Kimlik doğrulama: sen kimsin
- 14.3 Yetkilendirme: ne yapabilirsin
- 14.4 Şifreleme: çalınsa bile okunamaz
- 14.5 Denetim: kim, ne zaman, ne yaptı
- 14.6 Güvenlik bir bütündür ve ödünleşimlerle doludur
- 14.7 Kısım II'nin sonu ve Kısım III'e geçiş

#### 15 OxiDB'ye Genel Bakış ve Mimari

- 15.1 OxiDB nedir
- 15.2 Katmanların kuş bakışı görünümü

- 15.3 Çekirdeğin ötesi: ek yüzeyler
- 15.4 Çalışma alanının yapısı ve istemciler
- 15.5 Bir isteğin yaşam döngüsü
- 15.6 Üçüncü kısmın yol haritası
- 16 OxiDB'nin Depolama Katmanı: In-RAM ve Disk-First, mmap, .bdat / .btree
  - 16.1 Çekirdek: kimlikten baytlara
  - 16.2 İki kip, iki felsefe
  - 16.3 Dosyaların dili: .btree, .bdat ve .bopts
  - 16.4 Belleğe yansıtmanın somut etkisi
  - 16.5 Append-only olmanın sonuçları
  - 16.6 Sıkıştırma: yer, işlemci ve sıfır-kopya
  - 16.7 Açılış ve kurtarmaya kısa bir bakış
  - 16.8 Bu bölümün bıraktığı yer
- 17 OxiDB'de WAL, Dayanıklılık ve Kurtarma; Katı ve Gevşek Senkronizasyon
  - 17.1 OxiDB'nin yazma-öncesi günlüğü
  - 17.2 Katı dayanıklılık: varsayılan tercih
  - 17.3 Grup commit: bedeli toplu yazmada eritmek
  - 17.4 Gevşek senkronizasyon: hızı seçmek
  - 17.5 Kurtarma: çökmeden geri dönmek
  - 17.6 Denetim noktası ve günlüğün dizginlenmesi
  - 17.7 WAL her iki kbin de önünde durur
  - 17.8 Bu bölümün bıraktığı yer
- 18 OxiDB'de İndeksler: Alan, Bileşik ve mmap Tabanlı Disk İndeksleri
  - 18.1 Alan indeksleri: sıralı bir eşleme
  - 18.2 Türler arası sıralama: IndexValue inceliği
  - 18.3 Bileşik indeksler ve örnek kuralı
  - 18.4 İndeksten yanıt: belgeye dokunmadan saymak
  - 18.5 İndeks destekli sıralama ve erken sonlanma
  - 18.6 Disk-öncelikli indeksler: indeksi de bellekten çıkarmak
  - 18.7 Diğer indeks türleri
  - 18.8 İndekslerin bedeli ve dayanıklılığı
  - 18.9 Bu bölümün bıraktığı yer
- 19 OxiDB'nin Sorgu Motoru: Operatörler, İndeks Destekli Yollar ve Bayt Düzeyinde Süzme
  - 19.1 Sorgunun ayrıştırılması ve operatör dağılımı
  - 19.2 İndeksli koşullar ile süzgeç koşulları
  - 19.3 Yürütme: indeks, süzgeç ve tarama
  - 19.4 Bayt düzeyinde süzme: çözmeden elemek
  - 19.5 Tekil işlemlerde erken sonlanma
  - 19.6 Sunucu yoluyla ilişki
  - 19.7 Bildirimsel özgürlüğün yansıması
  - 19.8 Bu bölümün bıraktığı yer
- 20 OxiDB'nin Toplama Pipeline'ı: Gruplama, \$facet ve Pencere Fonksiyonları
  - 20.1 Pipeline ve aşamaları
  - 20.2 Baştaki süzmeyi öne çekmek
  - 20.3 Akış halinde gruplama
  - 20.4 Çok-yönlü analiz: \$facet

- 20.5 Pencere fonksiyonları
- 20.6 Toplamanın performans gerçeği: disk-öncelikli durum
- 20.7 Bu bölümün bıraktığı yer
- 21 OxiDB'de İşlemler: İyimsen Eşzamanlılık ve Üç Fazlı Commit
  - 21.1 OxiDB neden iyimsen yolu seçti
  - 21.2 İyimsen akışın üç fazı
  - 21.3 Dayanıklılıkla bağ
  - 21.4 Kilitlenmeye karşı tasarımıyla bağlılık
  - 21.5 Tek belge ile çok belge
  - 21.6 Küme durumunda işlemler
  - 21.7 İyimsenliğin bedeli
  - 21.8 Bu bölümün bıraktığı yer
- 22 OxiDB'de Sıkıştırma: Ölü Alan ve Otomatik Tetikleme
  - 22.1 Ölü alan nereden gelir
  - 22.2 Sıkıştırma ne yapar
  - 22.3 Zor kısım: yaşayan sistemde güvenle yapmak
  - 22.4 İndeksler neden sağ kalır
  - 22.5 Açık çağrıdan otomatik tetiklemeye
  - 22.6 Sıkıştırmanın bedeli ve dengesi
  - 22.7 Bu bölümün bıraktığı yer
- 23 OxiDB'nin Ek Yüzeyleri: Tam Metin Arama, Blob Depolama, Şifreleme ve PITR
  - 23.1 Tam metin arama: ters indeksin somutu
  - 23.2 Blob depolama: büyük ikili nesnelere yer açmak
  - 23.3 Dururken şifreleme: depolama sınırında koruma
  - 23.4 Zaman-noktasına kurtarma: çökmenin ötesinde
  - 23.5 Ortak iplik: tek motor, isteğe bağlı yetenekler
  - 23.6 Bu bölümün bıraktığı yer
- 24 OxiDB'nin Sunucu Katmanı: OxiWire Protokolü, Kimlik Doğrulama, RBAC ve Denetim
  - 24.1 Çerçeveleme sorunu ve OxiWire protokolü
  - 24.2 Bağlantı modeli ve el sıkışma
  - 24.3 Kimlik doğrulama: SCRAM ile parolayı hattan geçirmeden
  - 24.4 Yetkilendirme: rol tabanlı geçit
  - 24.5 Aktarım şifrelemesi ve denetim
  - 24.6 İsteğin sunucudaki yolu
  - 24.7 Bu bölümün bıraktığı yer
- 25 OxiDB'de Ölçeklendirme: Raft Kümesi ve OxiPool ile Sharding
  - 25.1 Raft kümesi: replikasyon ve konsensüs
  - 25.2 Kümenin doğrulanması
  - 25.3 OxiPool: sharding ile veriyi bölmek
  - 25.4 Dürüst sınırlar
  - 25.5 İkisini birleştirmek
  - 25.6 Bu bölümün bıraktığı yer
- 26 Uyumluluk Katmanları ve İstemciler
  - 26.1 Çekirdeğe iki kapı
  - 26.2 Diğer dillere köprü: FFI
  - 26.3 İstemci kütüphaneleri

- 26.4 Uyumluluk katmanları: başka protokolleri konuşmak
- 26.5 Tek çekirdek, çok yüz
- 26.6 Bu bölümün bıraktığı yer

## 27 Bellek Optimizasyonu ve Karşılaştırmalı Değerlendirme

- 27.1 Belleği yerleşik yığından çıkarma yolculuğu
- 27.2 Bellek ölçümünün dürüst yüzü
- 27.3 Karşılaştırmının felsefesi
- 27.4 OxiDB nerede kazanır
- 27.5 OxiDB nerede berabere kalır
- 27.6 OxiDB nerede kaybeder ve neden
- 27.7 Karşılaştırmının asıl dersi: dayanıklılık merceği
- 27.8 İlkelerin bir bütün oluşturması
- 27.9 Bu bölümün ve ana metnin sonu

## Ek A — Sözlük

## Ek B — Kaynaklar

- 27.10 Genel: veri-yoğun sistemlerin temelleri
- 27.11 Veri modelleri ve ilişkisel temel
- 27.12 JSON ve belge biçimleri
- 27.13 Depolama motorları ve indeksleme
- 27.14 İşlemler, eşzamanlılık ve tutarlılık
- 27.15 Ölçeklendirme ve konsensüs
- 27.16 OxiDB'nin kendi kaynakları
- 27.17 Son bir söz

# Önsöz

Bir veritabanı, ilk bakışta sıradan bir araçtır: veriyi koyarsınız, sonra geri alırsınız. Oysa bu basit vaadin altında, bilgisayar biliminin en zarif ve en zorlu problemlerinden bazıları yatar. Elektrik kesildiğinde verinin kaybolmaması nasıl garanti edilir? Binlerce kullanıcı aynı anda aynı kaydı değiştirmeye kalktığında düzen nasıl korunur? Milyonlarca belge arasından aradığınız tek kaydı, diski baştan sona taramadan saniyenin binde birinde nasıl bulursunuz? Veri tek bir makineye sığmaz hale geldiğinde, onu nasıl bölersiniz ve böldüğünüzde tutarlılığı nasıl sürdürürsünüz? Bu kitap, işte bu soruların peşinden gider.

## Bu kitap ne anlatıyor

Kitap iki yolculuğu birleştirir. Birincisi **kavramsal** yolculuktur: belge veritabanlarının ne olduğunu, neden var olduklarını ve içeride nasıl çalıştıklarını temelden kurar. Veriyle başlar; veriyi kalıcı kılmanın neden zor olduğunu, veri modellerinin tarih boyunca nasıl evrildiğini, belge modelinin bu evrimde nereye oturduğunu anlatır. Ardından bir belge veritabanının iç mimarisini katman katman açar: verinin diske nasıl yazıldığı, çökmeden nasıl kurtarıldığı, indekslerin aramayı nasıl hızlandırdığı, sorguların nasıl işlendiği, işlemlerin tutarlılığı nasıl koruduğu ve sistemin tek makinenin ötesine nasıl ölçeklendiği.

İkinci yolculuk **somut** olandır: bu kavramların gerçek bir sistemde nasıl hayata geçtiğini, OxiDB adlı bir belge veritabanı motoru üzerinden adım adım gösterir. Soyut bir ilkeyi öğrendikten sonra, onun gerçek bir mühendislik kararına nasıl dönüştüğünü, hangi ödünleşimlerin yapıldığını ve neden öyle yapıldığını görürsünüz. Böylece kitap yalnızca “ne” değil, “neden böyle” sorusunu da yanıtlar.

## Bu kitap kime göre

Bu kitap, veritabanlarını kullanan ama içlerinde ne olup bittiğini merak eden yazılım geliştiricileri; sistemlerin nasıl kurulduğunu derinlemesine anlamak isteyen mühendisler; ve bir veri sisteminin altındaki ilkeleri kavramak isteyen öğrenciler için yazıldı. Belirli bir programlama diline ya da belirli bir ürünün kullanımına hâkim olmanız gerekmiyor. Bilgisayarların nasıl çalıştığına dair temel bir aşinalık — bir dosyanın diskte durduğunu, belleğin geçici olduğunu, bir programın komutları sırayla yürüttüğünü bilmek — yeterlidir.

## Bu kitabın yöntemi

Bilinçli bir tercihle, kitapta **örnek kod yoktur**. Amaç, sizi belirli bir sözdizimine bağlamadan, kavramların kendisini anlatmaktır. Bir fikrin özünü düz metinle, benzetmelerle ve adım adım akıl yürütmeye anlatmak, çoğu zaman bir kod parçasından daha kalıcı bir kavrayış bırakır. Kod ezberlenir ve unutulur; bir mekanizmanın *neden* öyle çalıştığını bir kez gerçekten anladığınızda ise o bilgi sizinle kalır.

Kitap kademeli ilerler. Her kavram, kendinden öncekilerin üzerine kurulur: dayanıklılığı anlamadan işlemleri, indekslemeyi anlamadan sorgu işlemeyi tam kavrayamazsınız. Bu yüzden bölümleri sırayla okumanız önerilir. Yine de her bölüm, kendi başına da anlamlı olacak şekilde, gerektiğinde önceki kavramları kısaca hatırlatarak yazılmıştır.

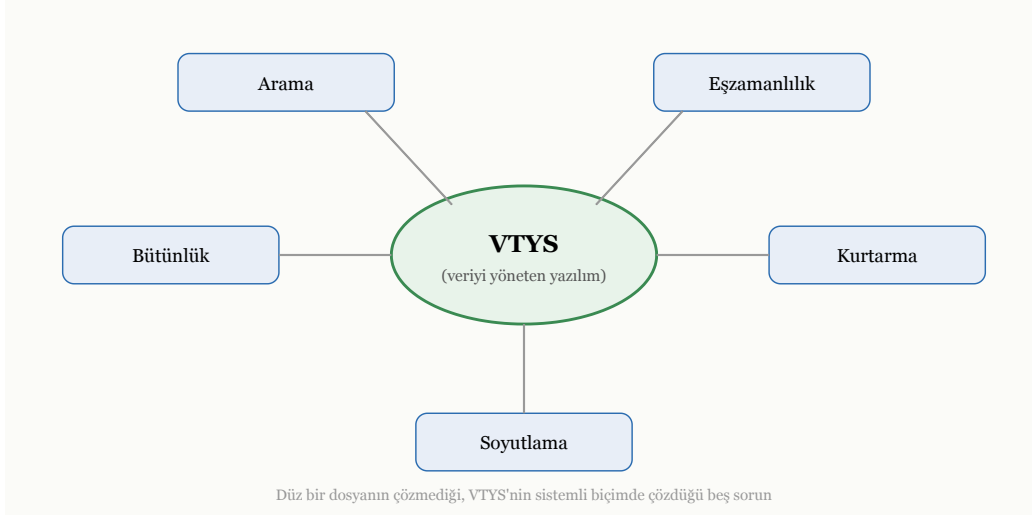
## Kitabın düzeni

Kitap üç kısma ayrılır. **Birinci Kısım** temelleri kurar: veri ve veritabanı kavramı, veri modellerinin tarihi ve belge modelinin doğası. **İkinci Kısım**, herhangi bir belge veritabanının içeride nasıl çalıştığını genel ilkeler düzeyinde anlatır — depolama, dayanıklılık, indeksleme, sorgu, işlemler ve ölçeklendirme. **Üçüncü Kısım**, tüm bu ilkelerin OxiDB’de nasıl somutlaştığını adım adım gösterir.

Şimdi başlayalım. İlk durağımız, sandığınızdan daha derin bir soru: aslında bir veritabanı tam olarak nedir ve onu basit bir dosyadan ayıran nedir?

# 1 Veri, Veritabanı ve Veritabanı Yönetim Sistemleri

Her veritabanı, son derece mütevazı bir ihtiyaçtan doğar: bir şeyi hatırlamak. Bir bilgiyi bugün kaydedip yarın, hatta yıllar sonra geri almak isteriz. Bu ihtiyaç o kadar temeldir ki çoğu zaman üzerinde düşünmeyiz; oysa onu güvenilir biçimde karşılamak, bilgisayar mühendisliğinin en derin problemlerinden birini açar. Bu bölümde, bir veritabanını bir avuç dosyadan ayıran şeyin ne olduğunu temelden kuracağız. Henüz “belge” veritabanından söz etmeyeceğiz; önce her veritabanının çözmek zorunda olduğu evrensel sorunları anlamamız gerekiyor.



Şekil 1 — Bir VTYS'nin düz dosyadan farkı: sistemli biçimde çözdüğü beş temel sorun.

## 1.1 Veri nedir, enformasyon nedir

İşe en baştan, “veri” sözcüğünden başlayalım. Veri, ham gerçeklerin kaydedilebilir biçimidir: bir sayı, bir ad, bir tarih, bir ölçüm. Tek başına veri sessizdir; anlamı, ona bir bağlam verdiğimizde belirir. “37” bir veridir; onun bir hastanın vücut sıcaklığı olduğunu söylediğimizde enformasyona, yani anlam taşıyan veriye dönüşür. Veritabanı, ham veriyi bağlamıyla birlikte, yani hangi sayının neyi ifade ettiğini de koruyacak şekilde saklamakla görevlidir.

Bu ayrım önemlidir, çünkü bir veritabanının değeri yalnızca sakladığı baytlardan değil, o baytları **yapılandırma** biçiminden gelir. Aynı “37” sayısı, bir tablodaki “sıcaklık” sütununda durduğunda ya da bir belgedeki **sıcaklık** alanında durduğunda anlamını kazanır. Verinin nasıl yapılandırıldığı — buna veri modeli diyeceğiz ve bir sonraki bölümün konusu olacak — bir veritabanının kimliğini belirleyen en önemli karardır.

## 1.2 Kalıcılık: neden bellek yetmez

Bir program çalışırken verisini bellekte (RAM’de) tutar. Bellek hızlıdır ve kullanışlıdır, ama bir kusuru vardır: **uçucudur**. Elektrik kesildiğinde, program kapandığında ya da makine yeniden başladığında bellekteki her şey silinir. Oysa hatırlamak istediğimiz veri, programın ömründen uzun yaşamak zorundadır. Bir bankanın hesap bakiyesi, uygulama kapandı diye sıfırlanamaz.

Çözüm açıktır: veriyi **kalıcı depolamaya**, yani diske yazmak. Disk, elektrik gittiğinde içeriğini korur. Ama bu çözüm, beraberinde yeni bir dünya dolusu problem getirir; çünkü disk, belleğe hiç benzemez. Bellek nanosaniyeler mertebesinde erişilirken, geleneksel bir diskte bir konuma erişmek milisaniyeler alabilir — milyonlarca kat yavaş. Modern katı hal sürücüler (SSD’ler) bu uçurumu daraltsa da, disk hâlâ belleğe kıyasla kat kat yavaştır ve çok farklı kurallarla çalışır. Disk, veriyi büyük bloklar halinde okuyup yazar; küçük, dağınık erişimleri sevmez; ve en önemlisi,

bir yazma işleminin gerçekten fiziksel olarak tamamlanıp tamamlanmadığı, sandığınızdan çok daha incelikli bir sorudur. İkinci kısımda bu inceliğin üzerine uzun uzun eğileceğiz; şimdilik şunu aklınızda tutun: veriyi kalıcı kılmak, onu yalnızca diske göndermek değil, oraya **gerçekten ulaştığınızdan emin olmaktır**.

### 1.3 Neden sadece dosya kullanmıyoruz

Mademki veriyi diske yazacağız, neden bir veritabanına ihtiyaç duyalım? Sonuçta işletim sistemi bize dosyalar sunar; veriyi bir dosyaya yazıp gerektiğinde okuyabiliriz. Birçok basit uygulama tam olarak bunu yapar ve bu yeterlidir. Sorunlar, veri büyüdükçe, birden çok kullanıcıyla paylaşıldıkça ve üzerinde karmaşık sorular sormaya başladıkça ortaya çıkar. Bir dosyayla baş başa kaldığınızda, aşağıdaki sorunların her biriyle tek tek, elle uğraşmak zorunda kalırsınız.

**Arama sorunu.** Diyelim ki bir dosyada bir milyon müşteri kaydı var ve belirli bir e-posta adresine sahip olanı arıyorsunuz. Yapısı olmayan bir dosyada tek seçeneğiniz, dosyayı baştan sona okumaktır. Bir milyon kaydı taramak yavaştır; üstelik aramayı her tekrarladığınızda baştan taramanız gerekir. Bir veritabanı, indeks adı verilen yardımcı yapılarla bu sorunu çözer: aradığınız kaydın yerini, dosyayı taramadan, doğrudan bulmanızı sağlar. Yedinci bölüm tümüyle bu mekanizmaya ayrılmıştır.

**Eşzamanlılık sorunu.** Tek bir kullanıcı varken her şey basittir. Ama iki kişi aynı anda aynı kaydı değiştirmeye kalktığında ne olur? Biri kaydı okur, üzerinde çalışırken diğeri aynı kaydı değiştirir ve kaydeder; sonra ilk kişi kendi değişikliğini kaydeder ve diğeri değişikliğini hiç haberi olmadan ezer. Buna “kayıp güncelleme” denir ve düz dosyalarla çalışırken sessizce, fark edilmeden gerçekleşir. Bir veritabanı, eşzamanlı erişimi düzene sokan mekanizmalarla bu tür bozulmaları engeller. Onuncu ve on birinci bölümler bu konunun derinine iner.

**Bütünlük sorunu.** Verinin yalnızca saklanması değil, **tutarlı** kalması da gerekir. Bir bankada para transferi, bir hesaptan düşme ve diğerine ekleme olmak üzere iki adımdan oluşur. Bu iki adımın arasında sistem çökerse, para bir hesaptan çıkmış ama diğerine girmemiş olabilir — yani buharlaşmıştır. Ya iki adım da olmalı ya da hiçbiri olmamalıdır; arada bir durum kabul edilemez. Düz dosyalarla bu “ya hep ya hiç” garantisini kurmak son derece zordur. Veritabanı, işlem (transaction) kavramıyla bunu sistematik biçimde sağlar.

**Çökmeden kurtulma sorunu.** Sistem tam bir yazma işleminin ortasında çökerse ne olur? Dosyanın yarısı yeni, yarısı eski veriyle, belki de bir kısmı bozuk kalabilir. Yeniden açıldığında veritabanı, kendisini tutarlı bir duruma getirmeyi bilmek zorundadır. Bunu, yazma-öncesi günlük (write-ahead log) gibi tekniklerle yapar; altıncı bölümün konusu budur.

**Soyutlama sorunu.** Son olarak, düz bir dosyayla çalışırken verinin diskte tam olarak nasıl yerleştiğini — hangi baytın nerede durduğunu, kayıtların nasıl ayrıldığını — bizzat düşünmek zorundasınızdır. Bir veritabanı bu ayrıntıları gizler. Siz “şu koşula uyan kayıtları getir” dersiniz; verinin fiziksel yerleşimiyle uğraşmazsınız. Bu soyutlama, hem işinizi kolaylaştırır hem de veritabanının altta yerleşimi değiştirip eniyileme yapmasına olanak tanır.

İşte bir veritabanını basit bir dosyadan ayıran şey budur: bu beş sorunu — arama, eşzamanlılık, bütünlük, kurtarma ve soyutlama — sizin yerinize, sistemli ve güvenilir biçimde çözmesidir.

### 1.4 Veritabanı ile veritabanı yönetim sistemi

Günlük konuşmada “veritabanı” sözcüğünü iki ayrı şey için kullanırız ve bu iki anlamı ayırmak yararlıdır. Dar anlamıyla **veritabanı**, düzenlenmiş verinin kendisidir — diskte duran o kayıt yığını. **Veritabanı yönetim sistemi** (kısaca VTYs) ise o veriyi yöneten yazılımdır: yazma ve okuma isteklerini karşılayan, indeksleri tutan,

eşzamanlılığı düzenleyen, çökmeden kurtaran program. Bu kitapta asıl ilgilendiğimiz şey, ikincisidir: veriyi yöneten makinenin içindeki çarklardır. “OxiDB” dediğimizde de aslında bu makineden, yani VTYS’den söz ederiz.

Bir VTYS’yi, bir kütüphanenin işleyişine benzetebiliriz. Kitaplar (veri) raflarda (disk) durur. Ama bir kütüphaneyi yalnızca raflar değil, işleyiş kurar: kitapların hangi düzene göre dizildiği (veri modeli), aradığınız kitabı fişlerden bulmanızı sağlayan katalog (indeks), aynı kitabı iki kişiye birden vermemeyi sağlayan ödünç defteri (eşzamanlılık denetimi), yangın çıktığında en değerli ciltleri koruyan yedekleme düzeni (dayanıklılık ve kurtarma). VTYS, tüm bu işleyişi yürüten görünmez kütüphanecidir.

## 1.5 Gömülü kütüphane mi, sunucu mu

Bir VTYS’nin uygulamanızla nasıl konuştuğu, baştan verilen önemli bir tasarım kararıdır ve iki uçtan biriyle başlar. Birinci uçta **gömülü** veritabanı vardır: VTYS, uygulamanızın içine bir kütüphane olarak yerleşir, onunla aynı süreçte çalışır. Veriye erişim, bir ağ üzerinden değil, doğrudan fonksiyon çağrılarıyla olur; bu çok hızlıdır ve hiçbir sunucu kurulumu gerektirmez, ama veritabanı yalnızca o tek uygulamaya hizmet eder. İkinci uçta **sunucu** veritabanı vardır: VTYS ayrı bir süreç, hatta ayrı bir makine olarak çalışır; uygulamalar ona ağ üzerinden bağlanır. Bu, aynı veriye birçok uygulamanın aynı anda erişmesini sağlar, ama her isteğin ağdan geçmesinin bir bedeli ve sunucuyu yönetmenin bir yükü vardır.

Bu kitapta ele alacağımız OxiDB, ilginç biçimde her iki uçta da çalışabilen bir tasarıma sahiptir; üçüncü kısımda bunun nasıl mümkün olduğunu göreceğiz. Şimdilik yalnızca şunu bilmek yeterli: aynı temel mekanizmalar — depolama, indeks, işlem, kurtarma — bir VTYS gömülü de çalışsa, ağ üzerinden de hizmet verse, özünde aynıdır. Bu kitabın ikinci kısmında anlatacağımız ilkeler, bu yüzden ikisine de aynı ölçüde uygulanır.

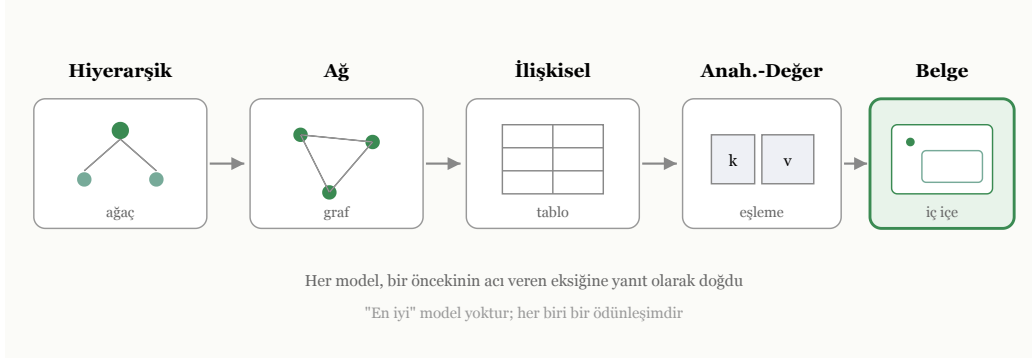
## 1.6 Bu kitabın kuracağı zihinsel harita

Buraya kadar, bir veritabanının çözmek zorunda olduğu sorunları sıraladık. Kitabın geri kalanı, bu sorunların her birinin nasıl çözüldüğünü tek tek açar. Yol haritamızı şöyle özetleyebiliriz. Önce verinin nasıl **modellendiğini** göreceğiz: ham baytları anlamlı yapılara dönüştürmenin tarih boyunca denenmiş yolları ve belge modelinin bu yolların neresinde durduğu. Ardından bir belge veritabanının iç **mimarisine** ineceğiz: verinin diske nasıl **yazıldığı** (depolama motoru), bu yazmanın çökmeye karşı nasıl **güvence altına alındığı** (dayanıklılık), aradığımızı taramadan nasıl **bulduğumuz** (indeksleme), sorularımızın nasıl **yanıtlandığı** (sorgu ve toplama), verinin nasıl **tutarlı** kaldığı (işlemler ve eşzamanlılık) ve sistemin tek makineyi aşınca nasıl **büyüdüğü** (replikasyon ve sharding). Son olarak, tüm bu parçaların gerçek bir motorda, OxiDB’de nasıl bir araya geldiğini somut olarak izleyeceğiz.

Bir sonraki bölümde ilk büyük soruyla, veri modeli sorusuyla başlıyoruz: aynı veriyi düzenlemenin birçok yolu vardır ve bu yolların her birinin kendine özgü güçleri ve zayıflıkları olmuştur. Belge modelinin neden ortaya çıktığını anlamak için, önce ondan önce gelenleri — ve onların hangi sıkıntılara yanıt olarak doğduğunu — görmemiz gerekiyor.

## 2 Veri Modelleri: Hiyerarşikten Belgeye

Bir önceki bölümde, bir veritabanının değerinin yalnızca sakladığı baytlardan değil, o baytları yapılandırma biçiminden geldiğini söylemiştik. Verinin nasıl yapılandırıldığına **veri modeli** denir ve bu, bir veritabanı sisteminin kimliğini belirleyen en temel karardır. Bu bölümde, veri modellerinin tarih boyunca nasıl evrildiğini izleyeceğiz. Bu bir tarih dersi değil; her modelin, kendinden öncekinin acı veren bir eksiğine yanıt olarak doğduğunu görmek, belge modelinin neden ortaya çıktığını ve hangi sorunları çözmeyi vaat ettiğini anlamanın en sağlam yoludur.



Şekil 2 — Veri modellerinin evrimi; her biri bir öncekinin eksiğine yanıt olarak doğdu.

### 2.1 Veri modeli neyi belirler

Bir veri modeli üç soruyu birden yanıtlar. Birincisi: veriyi hangi **biçimde** tutarız — düz bir liste mi, bir tablo mu, iç içe geçmiş bir yapı mı? İkincisi: veri parçaları arasındaki **ilişkileri** nasıl ifade ederiz — bir siparişin bir müşteriye ait olduğunu, bir yazının birçok etikete sahip olduğunu nasıl söyleriz? Üçüncüsü: bu veriye hangi **işlemlerle** erişiriz — onu nasıl arar, filtreler, birleştirir, değiştiririz?

Bu üç sorunun yanıtları birbirine sıkıca bağlıdır. Veriyi tutma biçiminiz, ilişkileri ifade etme imkânlarınızı belirler; ilişkileri ifade etme biçiminiz de hangi soruları kolayca, hangilerini zorlukla sorabileceğinizi belirler. Tarih boyunca her veri modeli, bu üç soruya farklı bir yanıt vermiş; her yanıt bazı işleri kolaylaştırırken bazılarını zorlaştırmıştır. Hiçbir model her açıdan üstün değildir; her biri belirli bir ödünleşimi temsil eder. Yolculuğumuzun özünde bu ödünleşimi izlemek yatar.

### 2.2 Hiyerarşik model: dünya bir ağaçtır

Bilgisayarların iş dünyasına girdiği ilk dönemde, en doğal yapı ağaç biçiminde görünüyordu. Birçok gerçek dünya ilişkisi gerçekten de hiyerarşıktır: bir şirketin departmanları, departmanların çalışanları vardır; bir siparişin kalemleri vardır. **Hiyerarşik model**, veriyi tam da böyle, bir ağaç olarak düzenler. Tepede bir kök kayıt durur; onun altında çocuk kayıtlar, onların altında torunlar uzanır. Her kaydın tek bir ebeveyni vardır ve veriye erişmek, bu ağaçta kökten yapraklara doğru ilerlemek demektir.

Bu model, hiyerarşik olan veriyi temsil etmekte zarif ve hızlıdır. Ama gerçek dünya her zaman temiz bir ağaç değildir, işte tam da burada zorlanır. İki tür sıkıntı ortaya çıkar. Birincisi **çoktan-çoğa ilişkilerdir**. Bir öğrencinin birçok dersi, bir dersin birçok öğrencisi vardır. Bunu bir ağaçla temsil etmek için ya öğrencileri derslerin altına ya da dersleri öğrencilerin altına yerleştirmek zorunda kalırsınız; hangisini seçerseniz seçin, diğer taraftaki veriyi **çoğaltarak** tekrar yazmanız gerekir. Bu çoğaltma hem yer israfıdır hem de tehlikelidir: aynı bilginin iki kopyası zamanla birbirinden ayrı düşer, biri güncellenir diğeri unutulur ve veri tutarsız hale gelir.

İkinci sıkıntı **gezinmenin katılığıdır**. Hiyerarşik bir veritabanında bir kayda ulaşmak için izleyeceğiniz yol baştan bellidir: ağacın yapısının dikte ettiği güzergâhtan gitmek zorundasınız. Eğer veriye, tasarımcının öngörmediği bir açıdan — diyelim ki “şu dersi alan tüm öğrenciler” yerine “şu öğrencinin aldığı tüm dersler” — bakmak isterseniz, ya çok dolambaçlı bir yol izlersiniz ya da hiç yapamazsınız. Sorgu, verinin fiziksel yapısına tutsaktır.

Hiyerarşik modelin bize öğrettiği ders şudur: iç içe geçmiş, ağaç biçimli veriyi bir arada tutmak güçlü ve doğaldır — bu fikri belge modelinde yeniden göreceğiz — ama tek bir katı hiyerarşiye mahkûm olmak ve gezinme yolunu yapıya bağlamak, esnekliği öldürür.

## 2.3 Ağ modeli: bağlantılar her yöne

Hiyerarşik modelin çoktan-çoğa ilişkilerdeki sıkıntısına bir yanıt olarak **ağ modeli** doğdu. Buradaki fikir, kayıtları katı bir ağaca hapsedmek yerine, aralarına istenen her yöne **işaretçiler** koymaktır. Bir öğrenci kaydı, aldığı her derse bir bağlantı taşır; bir ders kaydı, kendisini alan her öğrenciye. Böylece çoktan-çoğa ilişkiler veriyi çoğaltmadan ifade edilebilir.

Bu, ifade gücü açısından bir ilerlemeydi, ama ağır bir bedelle geldi: **karmaşıklık**. Veriye erişmek, artık zihinde tuttuğunuz bir bağlantı labirentinde tek tek işaretçileri izlemek demektir. Programcı, aradığı veriye ulaşmak için “şu kayıttan şu bağlantıyı izle, oradan şuna geç” diye, adım adım bir gezinme yolu yazmak zorundaydı. Veri yapısı değiştiğinde, bu gezinme kodunun büyük bölümü kırılırdı. Veriye nasıl ulaşılacağı bilgisi, uygulamanın içine derinlemesine gömülmüştü; mantıksal yapı ile fiziksel yapı birbirine kenetlenmişti.

Hem hiyerarşik hem de ağ modelinin ortak kusuru buydu: ikisinde de **veriye erişim yolu, verinin yapısına sıkıca bağlıydı**. Ne sorabileceğiniz ve nasıl sorabileceğiniz, tasarımcının baştan kurduğu yapının elverdiğiyle sınırlıydı. Bu kenetlenmeyi kırmak, bir sonraki büyük fikrin işi olacaktı.

## 2.4 İlişkisel model: büyük ayrışma

1970’te, bu kenetlenmeden rahatsız olan bir araştırmacı, radikal bir öneri getirdi. Verinin fiziksel olarak nasıl saklandığı ile mantıksal olarak nasıl düşünüldüğünü birbirinden tamamen **ayırmaı** önerdi. Bu fikir, **ilişkisel model** olarak bilinir ve onlarca yıl boyunca veritabanı dünyasına egemen oldu.

İlişkisel modelin temel yapısı şaşırtıcı derecede sadedir: her şey bir **tablodur**. Bir tablo, sütunları (alanlar) ve satırları (kayıtlar) olan bir ızgaradır; tıpkı bir hesap çizelgesi gibi. Müşteriler bir tabloda, siparişler başka bir tabloda durur. Peki bir siparişin hangi müşteriye ait olduğunu nasıl söyleriz? İşaretçiyle değil — bir **değerle**. Sipariş satırında, ait olduğu müşterinin kimliğini taşıyan bir alan bulunur. İlişki, fiziksel bir bağlantı değil, paylaşılan bir değerdir. Bu inceliğin sonuçları derindir: ilişkiler veriye gömülü işaretçilerden kurtulduğu için, veriyi istediğiniz gibi yeniden düzenleyeb, yeni ilişkiler keşfedebilirsiniz.

Bu ayrışmanın iki büyük armağanı oldu. Birincisi, sorguların **bildirimsel** hale gelmesidir. Artık veriye nasıl ulaşılacağını adım adım tarif etmek yerine, yalnızca **ne** istediğinizi söylersiniz: “şu koşullara uyan müşterilerle şu koşullara uyan siparişleri eşleştir.” Verinin fiziksel olarak nerede durduğunu, hangi indekslerin kullanılacağını sistem kendisi karar verir. Programcı “ne”, sistem “nasıl” sorusuyla ilgilenir. Bu ayrım, sorgu eniyileyci denen ve verinin en hızlı nasıl getirileceğini hesaplayan bileşeni doğurdu; sekizinci bölümde bu fikre döneceğiz.

İkinci armağan **normalleştirmedir**. İlişkisel model, her bilginin tek bir yerde durmasını teşvik eder. Bir müşterinin adresi yalnızca müşteri tablosunda yazar; o müşterinin yüzlerce siparişi olsa da, adres her siparişin içinde tekrarlanmaz, yalnızca müşteri kimliğiyle ona atıfta bulunulur. Böylece adres değiştiğinde tek bir satırı güncellemek yeterli olur; çoğaltmadan kaynaklanan tutarsızlık tehlikesi ortadan kalkar. Hiyerarşik modelin en büyük derdine — veri çoğaltma — temiz bir yanıtı bu.

İlişkisel model, sağlam matematiksel temeli, bildirimsel sorgu gücü ve veri bütünlüğüne verdiği önemle haklı bir egemenlik kurdu ve hâlâ dünyanın verisinin büyük kısmını taşıyor. Ama her güçlü fikir gibi, onun da bir gölgesi vardı.

## 2.5 İlişkisel modelin gölgesi: parçalanma ve uyumsuzluk

İlişkisel modelin verdiği şey — her bilgiyi ayrı, düzgün tablolara bölmek — aynı zamanda onun zorlandığı yeri yaratır. Çünkü uygulamalarda düşündüğümüz “şeyler” çoğu zaman tek bir tabloya sığmaz; birçok tabloya **parçalanmıştır**.

Bir örnek üzerinden düşünelim. Bir e-ticaret siparişi, kavramsal olarak tek bir bütündür: bir müşterisi, bir teslimat adresi, birçok kalemi, her kalemin bir ürünü ve miktarı vardır. İlişkisel model bunu düzgünce ayrı tablolara böler: siparişler, sipariş kalemleri, ürünler, adresler. Bu, depolama açısından temizdir. Ama o siparişi ekranda **bütün olarak** görmek istediğinizde, bu parçaları yeniden bir araya getirmeniz gerekir. Dağılmış satırları paylaşılan değerler üzerinden eşleştirip birleştiren bu işleme **birleştirme** (join) denir. Birleştirme güçlü bir araçtır, ama bedeli vardır: ne kadar çok tabloyu birleştirirseniz, sorgu o kadar karmaşıklaşır ve çoğu zaman o kadar yavaşlar. Bir siparişi göstermek için yarım düzine tabloyu birleştirmek, sık karşılaşılan bir yüküdür.

İkinci ve daha sinsî bir sürtünme, programlama dünyasıyla veritabanı dünyası arasındaki uyumsuzluktan doğar. Modern programlarda veriyi, iç içe geçmiş nesneler olarak düşünürüz: bir “sipariş” nesnesinin içinde bir “müşteri” nesnesi, bir “kalemler” listesi durur. Oysa ilişkisel veritabanında bu, düz tablolara serilmiştir. Programdaki zengin, iç içe nesne ile veritabanındaki düz satırlar arasında sürekli bir çeviri yapmak gerekir: nesneyi parçalayıp satırlara dağıtmak, sonra satırları toplayıp nesneyi yeniden kurmak. Bu sürekli çeviri yüküne **nesne-ilişkisel uyumsuzluk** denir. Yıllarca, bu çeviriyi otomatikleştirmek için koca yazılım katmanları geliştirildi; ama uyumsuzluğun kökü, iki dünyanın veriyi farklı biçimlerde düşünmesinde yatıyordu.

İşte belge modelini doğuran soru tam burada belirir: *Madem uygulamalar veriyi iç içe geçmiş bütünler olarak düşünüyor, veritabanı da onu neden öyle saklamasın?*

## 2.6 Anahtar-değer modeli: en yalın biçim

Belge modeline geçmeden önce, onun en yakın akrabasına, en yalın veri modeline bakalım. **Anahtar-değer modeli**, bir veritabanını dev bir sözlük gibi düşünür: her verinin bir **anahtarı** vardır ve o anahtarla, ona bağlı **değeri** geri alırsınız. Tıpkı bir vestiyer fişi gibi — fişi verirsin, paltonu alırsın.

Bu modelin gücü sadeliğindedir. Anahtarla erişim son derece hızlıdır ve böyle bir sistemi çok büyük ölçeklere yaymak görece kolaydır. Ama bir kısıtı vardır: sistem, değer **içine bakmaz**. Değer, sistem açısından anlamsız bir bayt yığıdır. Bu yüzden “değeri şu koşula uyan kayıtları getir” diyemezsiniz; yalnızca anahtarını bildiğiniz kaydı alabilirsiniz. Değerin içeriğine göre arama, filtreleme ya da grupta yapmak mümkün değildir, çünkü sistem o içeriğin yapısından habersizdir.

Anahtar-değer modeli, “anahtarıyla tek kayıt al” ihtiyacının baskın olduğu durumlarda kusursuzdur. Ama çoğu uygulama, verinin içeriğine göre de soru sormak ister. Belge modeli, tam da bu noktada devreye girer: anahtar-değer modelinin sadeliğini korur, ama değeri sisteme **anlaşılır** kılar.

## 2.7 Belge modeli: yapılandırılmış bütünler

**Belge modeli**, anahtar-değer modelinin değerini opak bir bayt yığını olmaktan çıkarıp, sistemin anlayabildiği, **yapılandırılmış** bir nesne haline getirir. Bu nesneye **belge** denir. Bir belge, alanlardan oluşur; her alanın bir adı

ve bir değeri vardır. Değer basit olabilir — bir sayı, bir metin — ya da **kendisi de iç içe geçmiş** bir belge ya da bir liste olabilir. Yani belgeler ağaç gibi derinleşebilir: bir siparişin içinde müşteri bilgisi, onun içinde adres, sipariş kalemlerinin oluşturduğu bir liste, her kalemin içinde ürün ayrıntıları durabilir — hepsi tek bir belgede, tek bir bütün olarak.

Bu, ilişkisel modelin parçalama derdine doğrudan bir yanıttır. Uygulamanın “bir sipariş” diye düşündüğü şey, veritabanında da tek bir belge olarak durur. Onu okumak için yarım düzine tabloyu birleştirmeniz gerekmez; belgeyi olduğu gibi alırsınız ve uygulamanızdaki nesneye neredeyse birebir karşılık gelir. Nesne-ilişkisel uyumsuzluk büyük ölçüde erir, çünkü iki dünya da veriyi artık aynı biçimde — iç içe geçmiş bütünler olarak — düşünmektedir.

Belge modelinin ikinci ayırt edici özelliği, hiyerarşik modelin tersine, **katı bir şema dayatmamasıdır**. İlişkisel bir tabloda her satır aynı sütunlara sahip olmak zorundadır; sütunlar baştan tanımlanır ve değiştirmek zahmetlidir. Belge modelinde ise her belge kendi alanlarını taşıyabilir. Aynı koleksiyondaki iki belgenin alanları farklı olabilir; bir belgeye yeni bir alan eklemek için önceden bir tanım değiştirmeniz gerekmez. Bu esneklik, verinin biçiminin zamanla geliştiği, her kaydın birbirinin aynı olmadığı durumlarda büyük rahatlık sağlar. Üçüncü bölümde bu esnekliğin hem armağanlarını hem de tuzaklarını ayrıntısıyla ele alacağız; çünkü “şema yok” demek, “şema düşünmek gerekmiyor” demek değildir.

Belge modelinin ödünleşimi de tam buradan doğar. İlişkisel modelin her bilgiyi tek bir yerde tutma disiplinini gevşetir; bir bilgiyi, ait olduğu belgenin içine gömerek tekrarlamayı kabul eder. Bu, okumayı hızlandırır — her şey tek yerde — ama hiyerarşik modelin eski derdini, çoğaltmayı geri çağırır. Belge modelinde verinizi tasarlarken sürekli bir soruyla yüzleşirsiniz: bu bilgiyi belgenin içine **gömmeli** miyim, yoksa ayrı tutup ona **atıfta** mı bulunmalıyım? Bu soru, belge modeliyle çalışmanın kalbinde durur ve dördüncü bölümü tümüyle ona ayıracağız.

## 2.8 Diğer modeller ve manzaranın bütünü

Tamlık için, manzaranın iki parçasından daha kısaca söz edelim. **Graf modeli**, ilişkilerin kendisini birinci sınıf bir varlık haline getirir; verinin değil, veriler arasındaki **bağlantıların** ön planda olduğu durumlar için — sosyal ağlar, öneri sistemleri, yol haritaları gibi — biçilmiş kaftandır. Bir bakıma ağ modelinin, bildirimsel sorgu gücüyle yeniden doğmuş, olgun halidir. **Geniş-sütun** (wide-column) modeli ise, çok büyük ölçekli ve yazma-yoğun yükler için, tabloları satır yerine sütun grupları halinde düzenleyerek farklı bir ödünleşim sunar.

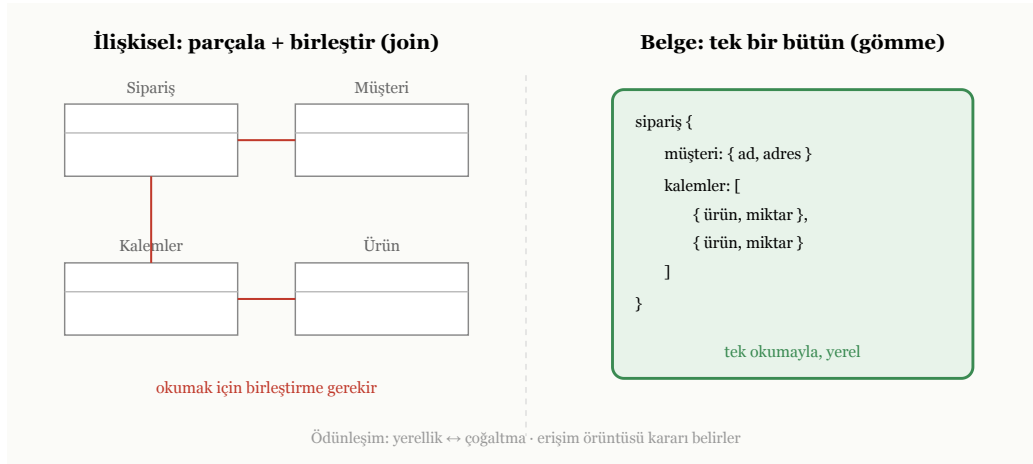
Bütün bu modeller arasında “en iyi” yoktur; yalnızca **farklı ödünleşimler** vardır. Hiyerarşik ve ağ modelleri, veriye erişimi yapıya kenetleyerek esneklik ödedi. İlişkisel model, mantığı fizikten ayırarak büyük bir esneklik ve sorgu gücü kazandı, ama veriyi parçalayarak birleştirme yükünü ve nesne uyumsuzluğunu getirdi. Anahtar-değer modeli, sadelik ve ölçek uğruna içeriğe-göre-sorgudan vazgeçti. Belge modeli, ilişkisel modelin sorgu gücünü büyük ölçüde korurken, veriyi uygulamaların düşündüğü gibi iç içe bütünler halinde tutarak parçalama ve uyumsuzluk dertlerini hafifletti — bunun karşılığında ise çoğaltma ve şema disiplini sorumluluğunu geliştiricinin omuzlarına yükledi.

Bu kitabın geri kalanı belge modeline odaklanıyor; ama bu odağın bir bedel değil, bilinçli bir ödünleşim olduğunu görmek önemlidir. Belge modeli bir gümüş kurşun değildir; belirli bir denge noktasıdır ve o denge noktasının nerede işe yarayıp nerede yetersiz kaldığını bilmek, onu iyi kullanmanın ön koşuludur.

Bir sonraki bölümde, manzaranın en çok karşılaşılan geçişine yakından bakacağız: ilişkisel düşünceden belge düşüncesine geçiş. Bu geçişin neden, ne zaman ve hangi bedellerle yapıldığını anlamak — ve ne zaman *yapılmaması* gerektiğini görmek — belge veritabanlarını gerçekten kavramanın anahtarıdır.

### 3 İlişkisel Modelden Belge Modeline: Neden ve Ne Zaman

Önceki bölümde veri modellerinin manzarasını çizdik ve hiçbirinin mutlak anlamda “en iyi” olmadığını, her birinin bir ödüneşimi temsil ettiğini söyledik. Pratikte en sık karşılaşılan karar, ilişkisel düşünceden belge düşüncesine geçiş kararıdır. Bu bölüm, o kararın anatomisini açar: ekipler neden belge modeline yönelir, bu geçiş hangi durumlarda kazandırır, hangi durumlarda kaybettirir ve karar verirken hangi soruları sormak gerekir. Amacımız bir modeli yüceltmek değil; ne zaman hangisinin doğru araç olduğunu görebilecek bir muhakeme kazandırmaktır. Çünkü yanlış araçla iyi yapılmış bir iş, yine de yanlış iştir.



Şekil 3 — Aynı sipariş verisi: ilişkisel parçalama (join) ile belge gömme yan yana.

#### 3.1 Modelin kralı değil, hizmetkârı: erişim örüntüleri

Bu bölümün üzerine kurulu olduğu tek bir fikir vardır ve onu en baştan söylemek gerekir: **veri modeli, erişim örüntülerini izler**. Yani veriyi nasıl yapılandıracağınıza, soyut bir “doğruluk” ilkesine göre değil, o veriyi nasıl okuyup yazacağınıza göre karar verirsiniz. Aynı gerçek dünya bilgisi — diyelim bir müşteri ve siparişleri — bir uygulamada ilişkisel olarak, bambaşka bir uygulamada belge olarak modellenenebilir ve ikisi de doğru olabilir; çünkü iki uygulama o veriye farklı sorular soruyordur.

Bu yüzden “ilişkisel mi, belge mi” sorusu, soyut olarak yanıtlanamaz. Yanıt, şu somut soruların içinde gizlidir: Veriyi hangi birimler halinde okuyorum? En sık hangi sorguları soruyorum? Bir şeyi değiştirdiğimde nereleri güncellemem gerekiyor? Bir işlem, kaç farklı varlığa birden dokunuyor? Bu bölümün geri kalanı, bu soruların yanıtlarının modeli nasıl belirlediğini gösteriyor.

#### 3.2 Belge modeline geçişin dört itici gücü

Ekipleri ilişkisel modelden belge modeline yönelten gerekçeler, genellikle dört başlık altında toplanır. Bunların her biri, önceki bölümde değindiğimiz bir ilişkisel sürtünmeye verilen bir yanıttır.

**Birincisi, erişim yerelliğidir.** Bir uygulamanın en sık yaptığı iş, çoğu zaman tek bir varlığı bütün olarak okumaktır: bir kullanıcının profilini, bir ürünün tüm ayrıntılarını, bir siparişin tamamını. İlişkisel modelde bu varlık birçok tabloya dağılmıştır ve onu bütün olarak getirmek birleştirme gerektirir. Belge modelinde ise o varlık tek bir belgede, yan yana durur; bir okuma işlemiyle, dağıntık parçaları toplamadan elinize gelir. Veriyi bir arada tutmaya **yerellik** denir ve okuma-yoğun uygulamalarda en büyük kazançlardan biridir; çünkü tek bir yerden okumak, birçok yerden toplayıp birleştirmekten neredeyse her zaman hızlıdır.

**İkincisi, geliştirme hızıdır.** İlişkisel bir tabloda yeni bir alan eklemek ya da yapıyı değiştirmek, önceden tanımlı bir şemayı değiştirmeyi gerektirir; bu, büyük tablolarda zahmetli ve riskli bir işlemdir. Belge modelinde her belge kendi alanlarını taşıyabildiği için, yeni bir alan eklemek çoğu zaman yalnızca onu yazmaya başlamak kadar kolaydır. Gereklerin hızla değiştiği, ürünün biçiminin sürekli evrildiği erken aşama projelerinde bu esneklik, hızı belirgin biçimde artırır. (Dördüncü bölümde bu esnekliğin bedelini de göreceğiz; şimdilik yalnızca itici gücün ne olduğunu saptıyoruz.)

**Üçüncüsü, nesne uyumsuzluğunun azalmasıdır.** Önceki bölümde, programlardaki iç içe nesnelerle ilişkisel tablolar arasındaki sürekli çeviri yükünden söz etmiştik. Belge modeli, veriyi tam da uygulamanın düşündüğü biçimde — iç içe geçmiş bütünler olarak — sakladığı için bu çeviri büyük ölçüde ortadan kalkar. Programdaki nesne ile veritabanındaki belge neredeyse aynı şekle sahip olur; aradaki sürtünme erir.

**Dördüncüsü, dağıtımaya yatkınlıktır.** Veri tek bir makineye sığmaz hale geldiğinde, onu birden çok makineye bölmek gerekir; on ikinci bölümde bunu ayrıntısıyla ele alacağız. Kendi içinde bütün olan, başka kayıtlara işaretçilerle bağlı olmayan belgeler, bu bölmeye doğal olarak yatkındır: her belge bir bütün olduğu için, hangi makineye gideceğine kolayca karar verilebilir ve onu okumak için başka makinelerdeki parçaları toplamak gerekmez. İlişkisel modelin birçok tabloyu birleştiren sorguları ise, veri makinelerine dağıldığında çok daha pahalı hale gelir; çünkü birleştirilecek parçalar farklı makinelerde olabilir.

### 3.3 Belge modelinin parladığı durumlar

Bu dört itici güç bir araya geldiğinde, belge modelinin açık ara doğru seçim olduğu bir profil belirir. Eğer veriniz büyük ölçüde **kendi içinde bütün** varlıklardan oluşuyorsa — yani bir varlığı okuduğunuzda neredeyse her zaman tamamını istiyorsanız ve onu değiştirdiğinizde değişiklik o varlığın içinde kalıyorsa — belge modeli bu veriyi olduğu gibi, doğal biçimde tutar. Bir blog yazısı ve onun yorumları, bir ürün ve onun teknik özellikleri, bir kullanıcı ve onun ayarları; bunların hepsi tek bir belgeye sığan, birlikte okunup birlikte yazılan bütünlerdir.

Belge modeli ayrıca verinin biçiminin **kayıttan kayda değiştiği** durumlarda parlar. Bir ürün kataloğu düşünün: bir kitabın sayfa sayısı, bir gömleğin bedeni ve rengi, bir yazılımın lisans türü vardır. İlişkisel bir tabloda bunların hepsini barındırmak ya devasa ve çoğu boş sütunlu bir tablo gerektirir ya da zahmetli dolaylı yapılar. Belge modelinde her ürün, yalnızca kendisine ait alanları taşır; biçim değişkenliği bir sorun değil, doğal bir durumdur.

Son olarak, **okuma-ağırlıklı ve yerel** iş yükleri belge modeline çok uygundur. Veri bir kez yazılıp çok kez okunuyorsa ve okumalar varlığın tamamını birden istiyorsa, veriyi bir arada tutmanın okuma kazancı, çoğaltmanın yazma maliyetine fazlasıyla baskın gelir.

### 3.4 Belge modelinin zorlandığı durumlar

Şimdi madalyonun öteki yüzüne, yani belge modelinin **kötü** bir seçim olduğu durumlara gelelim. Bunları görmezden gelmek, belge veritabanlarıyla ilgili en yaygın hataların kaynağıdır; çünkü model, kötü oturduğu yerlerde sessizce değil, acı verici biçimde zorlar.

**Yoğun biçimde birbirine bağlı veri.** Eğer veriniz, her yönden sorgulanan çoktan-çoğa ilişkilerle örülüyseniz — sosyal bir ağdaki arkadaşlıklar, bir bilgi grafindeki kavram bağlantıları gibi — belge modeli zorlanır. Çünkü belge modeli bir varlığı bir bütün olarak tutmakta iyidir, ama varlıklar arasındaki zengin, çok yönlü ilişkileri ifade etmekte ilişkisel modelin (ya da graf modelinin) gerisinde kalır. Önceki bölümde gördüğümüz çoktan-çoğa derdi, belge modelinde de yeniden belirir.

**Gelişigüzel, varlıklar arası sorgular.** Belge modeli, veriyi belirli bir okuma örüntüsüne göre düzenlemenizi teşvik eder. Ama uygulamanız zamanla, baştan öngörülmemiş açılardan sorular sormaya başlarsa — “şu koşula uyan

tüm müşterilerin, şu koşula uyan tüm siparişlerini, şu ürünle eşleştir” gibi — ilişkisel modelin bildirimsel birleştirme gücü çok değerli hale gelir. Belge modelinde bu tür varlıklar arası sorgular ya zahmetlidir ya da veriyi baştan farklı düzenlemeyi gerektirir.

**Sık güncellenen, paylaşılan bilgi.** Yerellik için belgeye gömdüğünüz bir bilgi, eğer birçok belgede tekrarlanıyorsa ve sık değişiyorsa, sizi belaya sokar. Bir satıcının adını her siparişin içine gömdüğünüzü düşünün; satıcı adını değiştirdiğinde, o satıcının yüzlerce, binlerce siparişindeki kopyayı tek tek güncellemeniz gerekir. İlişkisel modelin normalleştirilmesi — bilgiyi tek bir yerde tutmak — tam da bu sorunu çözmek için vardı. Bilgi ne kadar paylaşılır ve ne kadar sık değişirse, onu gömmenin bedeli o kadar artar.

**Çok varlığa dokunan işlemler.** Bir işlemin tutarlı kalması gereken birim, tek bir belgeden büyükse, dikkatli olmak gerekir. Belge veritabanları tek bir belge üzerindeki işlemleri kolayca atomik kılar; ama bir işlem birçok ayrı belgeye, ya da dağınık bir kurulumda birçok makinedeki belgeye birden dokunuyorsa, tutarlılık güvencesi karmaşılaşır. (Bu konuyu onuncu ve on ikinci bölümlerde derinlemesine ele alacağız.)

Bu dört durumun ortak noktası şudur: hepsinde de veri, **tek bir bütün olarak okunup yazılan bağımsız varlıklar** profilinden uzaklaşır ve **birbirine bağlı, paylaşılan, çok yönlü sorgulanan** bir ağa dönüşür. Belge modeli ilkinde güçlüyken, ikincisinde ilişkisel model öne geçer.

### 3.5 Çoğaltma ödünleşimine yeniden bakış

Belge modeline geçişin kalbinde, önceki bölümde değindiğimiz bir ödünleşim yatar ve burada onu netleştirmekte yarar var: **çoğaltma**. İlişkisel model, her bilgiyi tek bir yerde tutarak güncellemeyi ucuzlatır ve tutarsızlığı önler, ama okumayı pahalılaştırır — çünkü dağınık parçaları her okumada toplamak gerekir. Belge modeli, bilgiyi ait olduğu belgeye gömerek okumayı ucuzlatır, ama çoğaltmayı kabul ettiği ölçüde güncellemeyi pahalılaştırır ve tutarsızlık riskini geri çağırır.

Bu yüzden geçiş kararı, aslında bir okuma-yazma dengesi kararıdır. Veriniz çok okunup az yazılıyorsa ve okumalar bütünleri istiyorsa, belge modelinin gömme yaklaşımı kazandırır. Veriniz sık güncelleniyor ve paylaşılan bilgiler taşıyorsa, ilişkisel modelin normalleştirilmesi kazandırır. Çoğu gerçek uygulama bu iki ucun arasında bir yeredir; bu yüzden belge modeliyle çalışırken bile, neyi gömüp neye atıfta bulunacağınıza dair sürekli bir muhakeme yürütürsünüz. Dördüncü bölüm tümüyle bu muhakemeye ayrılmıştır.

### 3.6 “Ya hep ya hiç” yanılgısı ve melez gerçeklik

Bu tartışmayı bir savaş gibi sunmak — “ilişkisel mi, belge mi” — gerçeği yansıtmaz. Pratikte üç önemli incelik vardır.

Birincisi, çoğu ciddi sistem **tek bir veritabanı modeline mahkûm değildir**. Bir uygulamanın kullanıcı profillerini bir belge veritabanında, finansal işlemlerini ilişkisel bir veritabanında, oturum bilgilerini bir anahtar-değer deposunda tutması olağandır. Her veriyi, erişim örüntüsüne en uygun araçta saklama yaklaşımına “çok-dilli kalıcılık” denir. Doğru soru çoğu zaman “hangi model” değil, “bu veri parçası için hangi model” sorusudur.

İkincisi, modeller arasındaki sınırlar **bulanıklaşmıştır**. Modern belge veritabanları, başlangıçta yokken, birleştirme benzeri işlemler, çok-belgeli işlemler ve güçlü toplama yetenekleri kazanmıştır; bu kitapta OxiDB üzerinde tam da bunları göreceğiz. Aynı şekilde ilişkisel veritabanları da belge benzeri, iç içe alanları doğrudan saklayıp sorgulayabilen yetenekler edinmiştir. İki dünya, birbirinin güçlü yanlarını ödünç alarak yakınlaşmıştır. Dolayısıyla “belge modeli birleştirme yapamaz” gibi mutlak ifadeler, artık olduğundan daha keskindir.

Üçüncüsü, doğru karar **zamanla değişebilir**. Bir uygulamanın erken aşamasında, gereksinimler belirsizken belge modelinin esnekliği paha biçilmez olabilir; sistem olgunlaştıkça ve erişim örüntüleri sabitlendikçe, bazı parçalar için

daha katı bir yapı tercih edilebilir. Model seçimi tek seferlik, geri dönülmez bir yemin değildir; sistemin yaşamı boyunca yeniden gözden geçirilen bir mühendislik kararıdır.

### 3.7 Karar vermeden önce sorulacak sorular

Bu bölümü, geçiş kararını verirken kendinize soracağınız somut sorularla toplayalım. Bu sorular, soyut tercihi pratik bir muhakemeye çevirir.

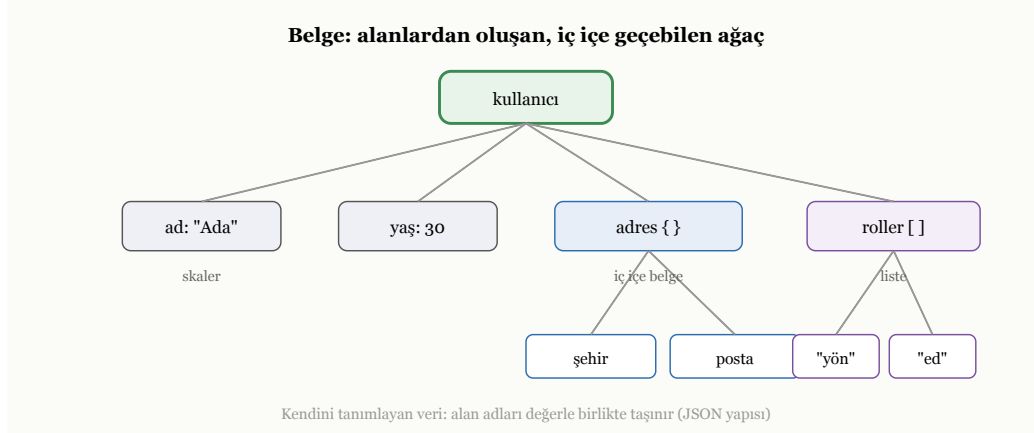
Verimi hangi **birimler** halinde okuyup yazıyorum; bir varlığı çoğunlukla bütün olarak mı, yoksa parçalarına ayrı ayrı mı erişiyorum? Tutarlı kalması gereken **birim** ne kadar büyük; bir işlem tek bir varlığa mı, yoksa birçok varlığa birden mi dokunuyor? Verim ne kadar **birbirine bağlı**; ilişkiler tek yönlü ve sahiplik temelli mi, yoksa her yönden sorgulanan çok-çok ilişkiler mi? Hangi bilgiler **paylaşıyor** ve ne sıklıkta **değişiyor**; bunları gömersem kaç yere kopyalamış olurum? Okuma mı yoksa yazma mı baskın; bu denge gelecekte nasıl değişebilir? Sorgularım baştan **belli** mi, yoksa zamanla öngörülemeyen yeni açılardan sorular bekliyor muyum?

Bu soruların yanıtları, modeli sizin yerinize seçer. Yanıtlar “bağımsız bütünler, yerel erişim, az paylaşım, okuma ağırlıklı, belli sorgular” yönüne işaret ediyorsa belge modeli evinizdir. “Birbirine bağlı, paylaşılan, sık güncellenen, gelişigüzel sorgulanan veri” yönüne işaret ediyorsa ilişkisel model daha rahat edecektir. Çoğu zaman yanıt karışıktır ve bu, çok-dilli kalıcılığa ya da modellerin melezleşmiş yeteneklerine başvurma işaretidir.

Buraya kadar belge modelini, diğer modellerle karşılaştırarak, dışarıdan tanıdık. Artık onun **içine** girme zamanı. Bir sonraki bölümde belge modelini yakın plana alıyoruz: bir belgenin tam olarak neyden yapıldığını, JSON gibi gösterimlerin neden bu kadar yaygınlaştığını, şema esnekliğinin hem armağanını hem tuzağını ve belge modeliyle çalışmanın kalbindeki o tekrar eden kararı — gömmek mi, atıfta bulunmak mı — derinlemesine inceleyeceğiz.

## 4 Belge Modeli Derinlemesine: JSON, Şema Esnekliği, Gömme ve Referans

Önceki üç bölümde belge modelini dışarıdan, diğer modellerle karşılaştırarak tanıdık. Artık onun içine giriyoruz. Bu bölümde bir belgenin tam olarak neyden yapıldığını, JSON gibi gösterimlerin neden bu kadar yaygınlaştığını, “şemasızlık” denen şeyin gerçekte ne anlama geldiğini ve belge modeliyle çalışmanın kalbindeki o tekrar eden karar — gömmek mi, atıfta bulunmak mı — inceleyeceğiz. Bu bölüm, Kısım I’nin son durağı; onu bitirdiğimizde belgenin *ne* olduğunu tam olarak bilecek ve Kısım II’de bir veritabanının bu belgeleri içeride *nasıl* sakladığına geçmeye hazır olacağız.



Şekil 4 — Belge, alanlardan oluşan ve iç içe geçebilen bir ağaçtır.

### 4.1 Bir belge neyden yapılır

Bir belge, özünde, **adlandırılmış alanların** bir koleksiyonudur. Her alanın bir adı (örneğin *ad*, *yaş*, *adres*) ve bir değeri vardır. Değer birkaç temel türden biri olabilir. En basitleri **skaler** değerlerdir: bir metin, bir sayı, bir doğru/yanlış (mantıksal) değeri ya da “değer yok” anlamına gelen boş (null) değer. Ama belge modelini güçlü kılan, değerlerin yalnızca skaler olmak zorunda olmamasıdır.

Bir değer, bir **liste** (dizi) olabilir: sıralı bir öğeler topluluğu, örneğin bir yazının etiketleri ya da bir siparişin kalemleri. Daha da önemlisi, bir değer **başka bir belge** olabilir: alanları olan, iç içe geçmiş bir nesne. Bir kullanıcının *adres* alanı, içinde *sokak*, *şehir*, *posta\_kodu* alanları olan bir alt-belge olabilir. Ve bu iç içe geçme istediğiniz kadar derinleşebilir: listelerin içinde belgeler, belgelerin içinde listeler. İşte belge modelinin ağaç biçimli, hiyerarşik doğası buradan gelir; ikinci bölümde hiyerarşik modelin iç içe veriyi tutmaktaki gücünden söz etmiştik — belge modeli o gücü, ama tek bir katı hiyerarşiye mahkûm olmadan, devralır.

Bu yapı sayesinde, uygulamanızın zihnindeki bir “şey” — bir sipariş, bir profil, bir ürün — tek bir belgede, kendi doğal biçimiyle durabilir. Belgenin şekli, nesnenin şekline neredeyse birebir uyar. Üçüncü bölümde sözünü ettiğimiz nesne uyumsuzluğunun erimesi, tam da bu yapısal benzerlikten kaynaklanır.

### 4.2 Kendini tanımlayan veri

Belge modeliyle ilişkisel model arasındaki en derin yapısal fark, **şemanın nerede durduğudur**. İlişkisel bir tabloda alan adları ve türleri yalnızca bir kez, tablo tanımında yazar; satırların kendisi yalnızca değerleri taşır, hangi değerlerin hangi sütuna ait olduğu tablonun yapısından bilinir. Belge modelinde ise her belge, alan adlarını **kendi**

**içinde** taşır. Yani bir belgeye baktığınızda, ayrı bir tanıma bakmaya gerek kalmadan, hangi değerin neyi ifade ettiğini görürsünüz. Buna **kendini tanımlayan** veri denir.

Bu özelliğin iki yüzü vardır. Armağan tarafı: veri taşınabilir ve esnektir. Bir belgeyi tek başına anlamlandırabilirsiniz; her belge kendi alanlarını taşıdığı için farklı belgeler farklı alanlara sahip olabilir ve yeni bir alan eklemek için merkezi bir tanımı değiştirmek gerekmez. Bedel tarafı: alan adlarının her belgede tekrarlanması yer kaplar — bir milyon belgenin her birinde `kullanici_adi` metnini taşımak, o adı bir kez tanımlamaktan daha savurgandır — ve adların tutarlılığını güvence altına alan merkezi bir otorite yoktur. Bir belgede `telefon`, başkasında `tel`, bir üçüncüsünde `telefon_no` yazarsa, sistem bunu bir hata olarak görmez; bu tutarsızlıkla baş etmek size kalır. Bu, şema esnekliği tartışmasının kalbinde yatan gerilimdir ve birazdan ona döneceğiz.

## 4.3 JSON nedir ve nereden geldi

Belgeyi bir bilgisayarda tutmak başka şeydir, onu insanlar ve programlar arasında **yazıya dökerek aktarmak** başka şeydir. Bir belgeyi bir dosyada saklamak, bir ağ üzerinden göndermek ya da bir kayıt üzerine yazmak için, onun metinsel bir gösterimine ihtiyaç duyarız. İşte JSON, tam da bu işi yapan bir **gösterim biçimidir**. Adı, İngilizce “JavaScript Object Notation” ifadesinin kısaltmasıdır; Türkçeye “JavaScript Nesne Gösterimi” olarak çevrilebilir. Adında JavaScript geçmesine karşın, JSON belirli bir programlama diline bağlı değildir; herhangi bir dilde okunup yazılabilen, dilden bağımsız bir veri değiş-tokuş biçimidir.

JSON’un yapısı şaşırtıcı derecede sadedir ve tam olarak bu bölümün başında anlattığımız belge yapısına karşılık gelir. Yalnızca birkaç yapı taşı vardır. Bir **nesne**, süslü parantezler içinde, ad-değer çiftlerinden oluşan bir topluluktur — yani bir belgenin ta kendisi. Bir **dizi**, köşeli parantezler içinde, sıralı bir değerler listesidir. Ve değerlerin kendisi ya bir metin (çift tırnak içinde), bir sayı, bir mantıksal değer (doğru/yanlış), boş değer (null), ya da yine bir nesne veya dizi olabilir. Değerlerin nesne ve dizi olabilmesi sayesinde JSON, tıpkı belgeler gibi, istenildiği kadar iç içe geçebilir. Yani JSON’un dilbilgisi, belge modelinin yapısının yazıya dökülmüş hâlinde başka bir şey değildir.

JSON’un kökeni öğreticidir. JavaScript dili 1990’ların ortasında, web tarayıcılarında çalışmak üzere doğdu ve bu dilin, nesneleri ve listeleri kısaca yazmaya yarayan kendine özgü bir sözdizimi vardı — buna nesne ve dizi “değişmez gösterimi” (literal) denir. 2000’lerin başında, web sayfalarının sunucularla veri alışverişi yapma ihtiyacı hızla arttı. O dönemde bu iş için yaygın olarak XML adlı, etiketlere dayalı ve oldukça ağır bir biçim kullanılıyordu. Douglas Crockford adlı bir mühendis, JavaScript’in zaten var olan o sade nesne gösteriminin, veri aktarımı için XML’den çok daha hafif ve kullanışlı bir biçim olarak kullanılabileceğini fark etti; bu fikri derleyip toparlayarak, ona bir ad (JSON) ve açık bir tanım kazandırdı ve `json.org` adresinde yayımladı. Yani JSON icat edilmedi denebilir; daha çok, JavaScript’in içinde zaten saklı duran sade bir alt küme keşfedilip bağımsız bir biçim olarak öne çıkarıldı. JSON’un nesnelere bu kadar doğal uymasının nedeni de budur: o, bir programlama dilinin nesne yazma biçiminin doğrudan kendisinden türemiştir.

JSON’un yükselişi, web’in olgunlaşmasıyla el ele gitti. Tarayıcıların sayfayı baştan yüklemeyi sunucusuyla arka planda veri alışverişi yapabildiği tekniklerin yaygınlaşmasıyla, bu alışverişte taşınacak hafif bir biçime ihtiyaç doğdu ve JSON, ağır XML’in yerini hızla aldı. Zamanla, biçimin herkesçe aynı yorumlanması için resmî standartlara da bağlandı: hem yazılım endüstrisi standartları hem de internet standartları, JSON’un dilbilgisini kesin biçimde tanımlayan belgeler yayımladı. Böylece JSON, bir kişinin derlediği pratik bir fıkirden, tüm dünyanın üzerinde anlaştığı resmî bir veri değiş-tokuş biçimine dönüştü ve bugün web servislerinden yapılandırma dosyalarına, kayıt sistemlerinden — bu kitabın konusu olan — belge veritabanlarına kadar her yerde karşımıza çıkıyor.

## 4.4 JSON neden kazandı

JSON'un ne olduğunu ve nereden geldiğini gördükten sonra, onu belge dünyasının ortak diline dönüştüren özellikleri toparlayabiliriz; bu özellikler öğreticidir. JSON, insan tarafından okunabilir düz metindir; bir belgeye bakıp ne olduğunu hemen anlarsınız. Yapısı, tam da az önce anlattığımız belge yapısına uyar: adlandırılmış alanlar, skaler değerler, listeler ve iç içe nesneler. Ve belki en önemlisi, modern programlama dillerindeki nesnelere neredeyse doğrudan karşılık gelir; bir programdaki nesneyi JSON'a çevirmek ve geri almak kolaydır. Bu üç özellik — okunabilirlik, belge yapısına uygunluk ve nesnelere yakınlık — JSON'u belge dünyasının ortak dili yaptı.

Ama JSON'un, bir veritabanının iç gösterimi olarak doğrudan kullanmak için iki önemli eksiği vardır. Birincisi **tür yoksulluğudur**. JSON yalnızca birkaç temel tür tanır; örneğin tarih, zaman ya da ikili (binary) veri için ayrı bir türü yoktur. Bir tarihi JSON'da saklamak için onu bir metne ya da sayıya çevirmek gerekir ve bu çevrimin anlamı, yorumlayan tarafın bilgisine kalır. İkincisi **verimsizliktir**: JSON düz metindir, bu yüzden hem yer açısından savurgandır hem de bir alanı bulmak için metni baştan ayrıştırmak gerekir; sayıları bile metin olarak yazıp yeniden okumak zorundadır.

Bu yüzden ciddi belge veritabanları, JSON'u dışarıya — kullanıcıyla iletişimde — kullanırken, **içeride** daha zengin ve daha verimli bir **ikili gösterime** çevirir. Bu ikili biçimler, JSON'un tanımadığı türleri (tarih, ikili veri, kesin sayılar) ekler ve veriyi, ayrıştırmadan doğrudan üzerinde işlem yapılabilen şekilde kodlar; örneğin bir alanın değerine, tüm belgeyi metin olarak okumadan, doğrudan atlayabilirsiniz. Üçüncü kısımda OxiDB'nin tam olarak böyle bir iç ikili gösterim kullandığını ve bunun sorgu hızına nasıl katkı sağladığını göreceğiz. Şimdilik akılda tutulacak nokta şu: JSON belge modelinin yüzüdür, ama bir veritabanının içinde yaşayan biçim genellikle JSON'un daha zengin, daha sıkı bir akrabasıdır.

## 4.5 Belgeler ve koleksiyonlar

Belgeler tek başlarına durmaz; bir arada gruplanırlar. Bu gruplara genellikle **koleksiyon** denir — ilişkisel dünyadaki tablonun belge dünyasındaki karşılığı. Ama önemli bir farkla: bir tablo, içindeki tüm satırların aynı sütunlara sahip olmasını dayatır; bir koleksiyon ise içindeki belgelerin birbirine benzemesini **zorunlu kılmaz**. Aynı koleksiyonda, farklı alanlara sahip belgeler bir arada durabilir. Pratikte bir koleksiyondaki belgeler genelde benzer biçimdedir — çünkü hepsi aynı tür “şeyi” temsil eder — ama bu benzerlik, sistemin dayattığı bir kural değil, uygulamanın gözettiği bir gelenektir.

Her belgenin, kendisini koleksiyon içinde benzersiz biçimde tanımlayan bir **kimliği** vardır. Bu kimlik, belgeye doğrudan, taramadan erişmenin anahtarıdır; ikinci bölümde anahtar-değer modelinden söz ederken değindiğimiz “anahtarla erişim” fikri burada da yaşar. Belge veritabanları, bir belgeyi kimliğiyle bulmayı her zaman çok hızlı kılacak şekilde tasarlanır. Diğer alanlara göre arama ise — yedinci bölümde göreceğimiz — indekslerin işidir.

## 4.6 “Şemasızlık” yanılgısı: şema-okumada ve şema-yazmada

Belge veritabanları sık sık “şemasız” diye anılır, ama bu niteleme yanıltıcıdır ve düzeltilmesi gerekir. Veri her zaman bir şemaya sahiptir — yani bir biçime, bir beklentiye. Soru, o şemanın **nerede yaşadığı** ve **ne zaman dayatıldığıdır**.

İlişkisel modelde şema **yazma anında** dayatılır: bir satırı yazmadan önce tablonun tanımına uymak zorundadır, aksi halde yazma reddedilir. Buna “şema-yazmada” denir. Şema merkezidir, açıktır ve veritabanı tarafından zorlanır. Avantajı, verinin her zaman beklenen biçimde olmasıdır; veriyi okuyan hiç kimse “acaba bu alan var mı, türü doğru mu” diye endişelenmek zorunda değildir. Dezavantajı, biçimi değiştirmenin pahalı olmasıdır: yeni bir alan eklemek, var olan tüm satırları ilgilendiren bir değişiklik gerektirir.

Belge modelinde ise şema tipik olarak **okuma anında** anlam kazanır: veritabanı, yazdığınız belgenin biçimine karışmaz; biçimi yorumlama işi, veriyi okuyan uygulamaya düşer. Buna “şema-okumada” denir. Şema kaybolmamıştır; yalnızca veritabanından uygulamaya, açıktan örtüğe taşınmıştır. Avantajı esnekliktir: biçimi değiştirmek için merkezi bir tanım güncellemek gerekmez; farklı belgeler farklı biçimlerde olabilir; veri zamanla evrilebilir. Dezavantajı, şemayı zorlama sorumluluğunun uygulamaya geçmesidir.

Bu devrin pratik sonuçları vardır. Esneklik, bir armağandır: gereksinimler değiştiğinde göç (migration) denen o zahmetli, tüm veriyi dönüştüren işlemleri yapmadan, yeni alanları yazmaya başlayabilirsiniz. Ama aynı esneklik bir tuzaktır: veritabanı sizi tutarlı tutmadığı için, zamanla **biçim kayması** oluşabilir — aynı koleksiyonda kimi belgelerde bir alan vardır, kimilerinde yoktur; kiminde sayıdır, kiminde metin; kiminde tel, kiminde telefon. Veriyi okuyan kod, bu çeşitliliğe karşı **savunmacı** olmak, eksik ya da beklenmedik alanlara hazırlıklı olmak zorunda kalır. “Şemasızlık”, “şema düşünmek gerekmiyor” demek değildir; tersine, şema disiplini veritabanı yerine sizin taşımanız gerektiği anlamına gelir. İyi kullanılan belge veritabanlarında bu disiplin yok olmaz; uygulama katmanında, bilinçli biçimde sürdürülür.

#### 4.7 Belge modelinin kalbindeki karar: gömmek mi, atıfta bulunmak mı

Belge modeliyle veri tasarlarırken karşılaşacağınız en sık ve en belirleyici karar şudur: birbiriyle ilişkili iki veri parçasını, birini diğerinin **içine gömerek** mi tek bir belgede tutmalı, yoksa ayrı belgelerde tutup birinden diğerine **atıfta bulunarak** (referansla) mı bağlamalı? Bu kararın iyi verilmesi, belge modelini ustaca kullanmanın özüdür. Üçüncü bölümdeki çoğaltma ödünleşimi, burada somut bir tasarım kararına dönüşür.

**Gömmek**, ilişkili veriyi ana belgenin içine yerleştirmektir. Bir blog yazısının yorumlarını yazının kendi belgesi içinde bir liste olarak tutmak buna örnektir. Gömmenin armağanları, üçüncü bölümde saydığımız yerellik kazanlarıdır: veriyi tek bir okumayla, parçaları toplamadan alırsınız; ve bir belgeyi değiştirmek atomik olduğu için, gömülü veriyi ana veriyle birlikte tutarlı biçimde güncelleyebilirsiniz. Bedeli ise iki katmanlıdır. Birincisi, eğer gömülü bilgi başka belgelerde de tekrarlanıyorsa, çoğaltmanın güncelleme maliyetini ve tutarsızlık riskini üstlenirsiniz. İkincisi ve daha sinsi olanı, **sınırsız büyüme** tehlikesidir: yorumları gömdüğünüz bir yazı yıllar içinde on binlerce yorum biriktirirse, o belge devasa büyür; her okumada — yalnızca başlığı görmek isterseniz bile — tüm o yorum yığını taşımak zorunda kalırsınız. Sınırı belli olmayan, sürekli büyüyen listeleri gömmek, belge modelinin en yaygın tuzaklarından biridir.

**Atıfta bulunmak**, ilişkili veriyi ayrı bir belgede tutup, ona yalnızca kimliğiyle işaret etmektir — ilişkisel modelin paylaşılan-değerle bağ kurma fikrinin belge dünyasındaki karşılığı. Bunun armağanları normalleştirmenin armağanlarıdır: paylaşılan bilgi tek bir yerde durur, çoğaltma olmaz, değiştirmek ucuzdur ve belgeler şişmez. Bedeli ise yerelliğin kaybıdır: ilişkili veriyi almak için birden fazla okuma ya da uygulama tarafında bir birleştirme gerekir.

Bu iki seçenek arasında karar verirken birkaç yol gösterici ilke yardımcı olur. İlişkili veri, ana varlığın **ayrılmaz bir parçasıysa** ve onunla **birlikte okunup yazılıyorsa** — bir adresin bir kullanıcıya, bir sipariş kaleminin bir siparişe ait olması gibi — ve büyümesi **sınırlıysa**, gömmek doğaldır. İlişkili veri **birçok varlık tarafından paylaşılıyorsa, bağımsız olarak sorgulanıyorsa, sık değişiyorsa** ya da **sınırsız büyüyebiliyorsa**, atıfta bulunmak daha sağlamdır. Ve sık karşılaşılan çoktan-çoğa ilişkilerde — ikinci bölümden beri peşimizi bırakmayan o zorlu örüntü — neredeyse her zaman atıf tercih edilir, çünkü gömmek kaçınılmaz biçimde çoğaltmaya yol açar.

Bu kararın tek ve kesin bir doğru yanıtı yoktur; doğru yanıt, üçüncü bölümde gördüğümüz gibi, erişim örüntülerinize bağlıdır. Aynı “kullanıcı ve adres” ilişkisi, bir uygulamada gömülecek, başka birinde referansla bağlanacaktır. Belge modelini iyi kullanmak, bu kararı her ilişki için bilinçli biçimde, erişim örüntülerine bakarak vermektir.

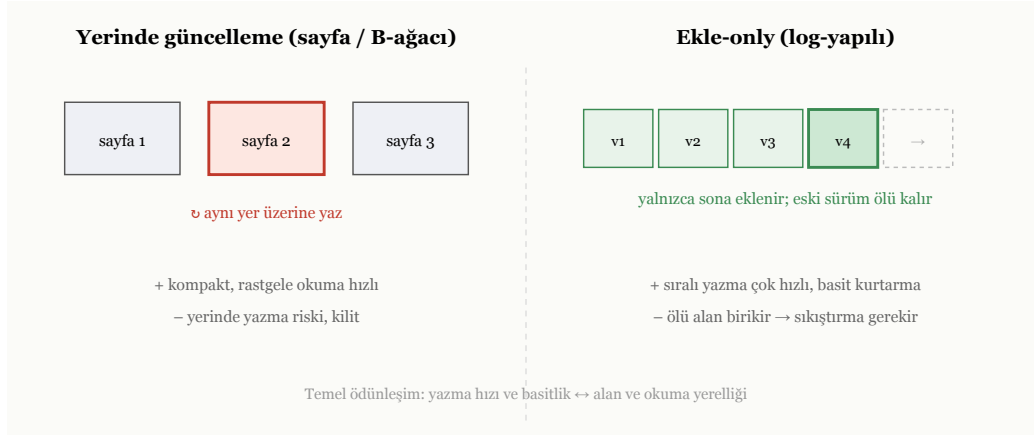
## 4.8 Atomikliğin sınırı belgedir

Son olarak, ileride önemli olacak bir noktayı şimdiden tohumlayalım. Belge veritabanlarında, üzerinde “ya hep ya hiç” güvencesi en kolay verilen birim, tek bir belgedir. Bir belgeyi değiştirdiğinizde, o değişiklik ya tümüyle gerçekleşir ya da hiç gerçekleşmez; arada bir durum olmaz. İşte gömmenin daha önce değinmediğimiz bir avantajı buradan doğar: ilişkili veriyi tek bir belgede topladığınızda, onları birlikte, tek bir atomik işlemde güncelleyebilirsiniz. Veriyi ayrı belgelere dağıttığınızda ise, birden çok belgeyi tutarlı biçimde değiştirmek daha güçlü işlem güvenceleri gerektirir. Bu, üçüncü bölümdeki “tutarlı kalması gereken birim ne kadar büyük” sorusunun belge modelindeki yankısıdır ve onuncu bölümde işlemleri ele alırken bütün ağırlığıyla geri gelecek.

Böylece Kısım I’i tamamlamış olduk. Artık belgenin ne olduğunu — alanlardan, iç içe yapılardan, kendini tanımlayan veriden oluştuğunu; JSON’la gösterilip içeride daha zengin bir ikili biçimde yaşadığını; koleksiyonlarda gruplandığını; şemasının yazma yerine okuma anında anlam kazandığını; ve onunla çalışmanın kalbinde gömme-referans kararının durduğunu — biliyoruz. Şimdiye dek hep *mantıksal* düzeyde, yani veriyi nasıl düşündüğümüz düzeyinde kaldık. Kısım II ile birlikte perdeyi aralıyor ve *fiziksel* düzeye iniyoruz: bir belge veritabanı, tüm bu belgeleri diske tam olarak nasıl yazar, çökmeden nasıl korur, aradığımızı taramadan nasıl bulur ve sorularımızı nasıl yanıtlar? Bir sonraki bölümde işin en altından, depolama motorundan başlıyoruz.

## 5 Depolama Motorları: Sayfa Tabanlı, Append-Only, LSM ve B-Ağaçları

Kısım I boyunca hep mantıksal düzeyde kaldık: veriyi nasıl düşündüğümüzle, belgenin ne olduğuyla ilgilendik. Şimdi perdeyi aralıyor ve fiziksel düzeye iniyoruz. Bir belge veritabanı, tüm o belgeleri diske tam olarak nasıl yazar ve geri okur? Bu sorunun yanıtını veren bileşene **depolama motoru** denir ve o, bir veritabanının kalbidir. Bu bölümde, depolama motorlarının çözmek zorunda olduğu temel gerilimi ve bu gerilime tarih boyunca verilen başlıca yanıtları — sayfa tabanlı B-ağaçlarını ve append-only/log-yapılı yaklaşımları — inceleyeceğiz. Buradan sonraki birkaç bölüm daha teknik olacak, ama yöntemimiz aynı: her tasarımı, çözmeye çalıştığı somut sorundan başlayarak anlamak.



Şekil 5 — Yerde güncelleme ile ekle-only depolamanın temel ödünleşimi.

### 5.1 Depolama motoru ne yapar

Bir depolama motorunu, veritabanının geri kalanından soyutlayarak düşünmek yararlıdır. Üstündeki katmanlar — sorgu işleyici, indeksler, işlem yöneticisi — ona iki temel istekle gelir: “şu kimliğe sahip belgeyi sakla” ve “şu kimliğe sahip belgeyi geri ver.” Depolama motorunun işi, bu basit isteklerin altındaki zor sorunu çözmektir: baytları diskte nereye, hangi düzende yerleştireceğini ve onları nasıl geri bulacağını kararlaştırmak. Bu kararlar, veritabanının ne kadar hızlı yazıp okuyacağını, ne kadar yer kaplayacağını ve çökmeye karşı ne kadar dayanıklı olacağını doğrudan belirler. Bu yüzden depolama motoru tasarımı, veritabanı mühendisliğinin en belirleyici tercihidir.

Önemli bir noktayı baştan ayıralım: depolama motoru, belgenin **içeriğiyle** ilgilenmez. Onun için bir belge, bir kimliğe bağlı bir bayt yığınıdır. Belgenin alanlarını yorumlamak, üzerinde sorgu çalıştırmak üst katmanların işidir; depolama motoru yalnızca o bayt yığınına güvenilir biçimde saklayıp geri vermekle yükümlüdür. Bu ayırım, depolama motorlarının neden farklı veri modellerinin altında — ilişkisel de olsa belge de olsa — aynı ilkelerle çalışabildiğini açıklar.

### 5.2 Her şeyi belirleyen kısıt: disk belleğe benzemez

Birinci bölümde, diskin belleğe hiç benzemediğine değinmiştik. Depolama motorlarını anlamak için, bu farkın somut sonuçlarını derinleştirmemiz gerekir; çünkü bu bölümdeki her tasarım kararı, doğrudan diskin doğasından doğar.

Diskün üç temel huyu vardır. Birincisi, **yavaştır**: belleğe erişim nanosaniyeler alırken, diske erişim kat be kat daha uzun sürer. Bu yüzden depolama motorunun en büyük amacı, gereksiz disk erişimlerinden kaçınmaktır. İkincisi,

disk **ardışık erişimi**, dağınık erişimden çok daha iyi yapar. Diskten art arda gelen bir bölgeyi okumak ya da dosyanın sonuna eklemeye devam etmek hızlıdır; oysa diskin orasından burasından, rastgele konumlara erişmek çok daha yavaştır. Bu fark, geleneksel dönen disklerde uçurum kadar büyüktü; katı hal sürücülerinde daralsa da hâlâ önemlidir. Üçüncüsü, disk veriyi **bloklar halinde** okuyup yazar: tek bir baytı bile değiştirmek istes., sistem o baytı içeren koca bir bloğu okuyup, değiştirip, geri yazar. Yani küçük, dağınık değişiklikler diskte orantısız bir maliyet taşır.

Bu üç huy, depolama motoru tasarımına tek bir buyruk dayatır: **diske ardışık, büyük ve seyrek yaz; rastgele, küçük ve sık erişimden kaçın**. Bu bölümdeki tüm yaklaşımlar, bu buyruğa farklı biçimlerde uymaya çalışan stratejilerdir.

### 5.3 Temel gerilim: okumaya mı, yazmaya mı eniyilemek

Depolama motoru tasarımının kalbinde tek bir gerilim yatar ve onu en baştan görmek, geri kalan her şeyi aydınlatır: bir veri yerleşimi, hem okumayı hem yazmayı aynı anda en iyi yapamaz. Bu ikisi arasında bir ödünleşim vardır.

Neden? Çünkü okumayı hızlandırmak için veriyi **düzenli** tutmak istersiniz — örneğin sıralı, dengeli bir yapıda — ki aradığınızı çabucak bulasınız. Ama veriyi düzenli tutmak, her yeni yazmada o düzeni korumak için diskin ortasına, “doğru yere” müdahale etmeyi gerektirir; bu da rastgele yazma demektir, yani diskin en sevmediği şey. Tersine, yazmayı hızlandırmak için veriyi olduğu gibi, geldiği sırada, dosyanın sonuna ardışık olarak eklersiniz; bu yazma için idealdir, ama o zaman veri düzensiz birikir ve aradığınızı bulmak zorlaşır. Düzen, okumanın dostu ama yazmanın düşmanıdır; düzensizlik ise tersi. İşte iki büyük depolama felsefesi — sayfa tabanlı B-ağaçları ve log-yapılı yaklaşımlar — bu gerilimin iki ucundan doğar.

### 5.4 Sayfa tabanlı B-ağaçları: yerinde güncelleme

Birinci felsefe, veriyi **düzenli tutmayı** önceler ve onlarca yıl boyunca veritabanı dünyasına egemen oldu. Bu yaklaşımda disk, sabit boyutlu **sayfalara** bölünür — her sayfa, belirli sayıda kaydı tutan, diskten tek seferde okunup yazılan bir blok. Veri, bu sayfalar üzerinde **B-ağacı** denen bir yapıyla düzenlenir.

B-ağacını, devasa, çok katmanlı bir dosyalama dolabı gibi düşünebilirsiniz. En üstte, “A–M arası şu çekmecede, N–Z arası bu çekmecede” diyen bir yönlendirme katmanı vardır. Onun altında daha ince ayrımlar, en altta ise verinin kendisini tutan yapraklar bulunur. Bir kaydı aramak için tepeden başlar, her adımda doğru dala saparak yapraklara inersiniz; birkaç adımda, milyonlarca kayıt arasında bile, aradığınıza ulaşırsınız. Üstelik veri sıralı tutulduğu için, “şu değerle bu değer arasındaki tüm kayıtlar” gibi **aralık sorgularını** da verimli yanıtlarsınız — yapraklar arasında yan yana ilerlemek yeterlidir. B-ağacı, hem tekil hem aralık okumalarında olağanüstü iyidir; bu yüzden okuma-yoğun, sorgu-zengin sistemlerin gözdesi olmuştur.

Bedeli ise yazmada ortaya çıkar. Bir kaydı değiştirdiğinizde, B-ağacı onu diskte ait olduğu sayfada, **yerinde** günceller. Bu, az önce diskin en sevmediği şey dediğimiz rastgele yazma demektir: değiştirilecek sayfa diskin neresindeyse, oraya gidilip o sayfa okunur, değiştirilir, geri yazılır. Dahası, ağaç dengesini korumak için sayfaların ara sıra bölünmesi ya da birleşmesi gerekir; bu da ek yazmalar doğurur. Tek bir küçük değişikliğin diske birden çok blok yazılmasına yol açmasına **yazma büyütmesi** (write amplification) denir ve B-ağaçlarının doğal bir maliyetidir. Bir de eşzamanlılık zorluğu vardır: birçok işlem aynı sayfalara aynı anda dokunabileceği için, sayfaların dikkatlice kilitlenmesi gerekir; bunun inceliklerine on birinci bölümde döneceğiz.

Özetle B-ağacı yaklaşımı, okumayı en üst düzeye çıkarmak için veriyi düzenli ve yerinde tutar; bunun karşılığında rastgele yazma ve yazma büyütmesi maliyetini öder.

## 5.5 Append-only ve log-yapılı yaklaşımlar: asla üzerine yazma

İkinci felsefe, gerilimin öteki ucundan başlar ve **yazmayı** önceler. Temel fikri tek bir cümleyle özetlenebilir: **var olan veriyi asla yerinde değiştirme; her yazmayı dosyanın sonuna ekle**. Bunu, sürekli yeni satırlar eklediğiniz, ama hiçbir eski satırı silmediğiniz bir muhasebe defterine benzetebilirsiniz. Bir kaydı güncellemek istediğinizde, eski kaydı bulup değiştirmesiniz; onun yeni hâlini defterin sonuna yazarsınız. En son yazılan, geçerli olanıdır.

Bu yaklaşımın yazmadaki üstünlüğü açıktır: her yazma, dosyanın sonuna **ardışık** bir ekleme olduğu için, diskin en sevdiği erişim biçimidir. Rastgele yazma yoktur; yazma büyütmesi en aza iner. Bu yüzden log-yapılı (append-only) motorlar, yazma- yoğun iş yüklerinde B-ağaçlarını rahatça geçer.

Ama bedava öğle yemeği yoktur; bu yaklaşım iki yeni sorun doğurur. Birincisi **okuma sorunudur**. Bir kaydın en güncel hâli defterin neresinde diye, dosyayı baştan sona taramak kabul edilemez. Bu yüzden append-only motorlar, ayrı bir **dizin** tutar: her kaydın kimliğinden, o kaydın en güncel hâlinin dosyada durduğu konuma bir eşleme. Yazarken bu dizini güncellersiniz; okurken önce dizine bakıp konumu öğrenir, sonra doğrudan o konuma gidersiniz. Yani append-only motor, veriyi düzensiz yazmanın getirdiği okuma zorluğunu, ayrı bir dizinle telafi eder. (Bu dizinin kendisi de bellekte ya da diskte tutulabilir; üçüncü kısımda OxiDB'nin tam olarak böyle bir kimlik→konum dizini kullandığını göreceğiz.)

İkincisi **ölü alan sorunudur**. Madem eski kayıtların üzerine yazmıyoruz, onlar dosyada öylece durmaya devam eder. Bir kaydı yüz kez güncellerseniz, dosyada o kaydın yüz eski kopyası birikir; yalnızca sonuncusu geçerlidir, gerisi **ölü alandır**. Zamanla dosya, çoğu artık geçersiz kayıtlarla şişer. Bu şişkinliği gidermek için, arada bir, dosyayı baştan yazıp yalnızca yaşayan (geçerli) kayıtları yeni bir dosyaya kopyalamak gerekir; ölü kayıtlar geride bırakılır. Bu temizlik işlemine **sıkıştırma** (compaction) denir ve append-only motorların ayrılmaz bir parçasıdır. Sıkıştırmayı, defter dolup taşıtığında oturup yalnızca hâlâ geçerli satırları temiz bir deftere geçirmeye benzetebilirsiniz. (Üçüncü kısımda OxiDB'nin sıkıştırmayı ne zaman ve nasıl yaptığını ayrı bir bölüm ayıracağız.)

## 5.6 LSM ağaçları: log-yapılının olgun hâli

Append-only fikrini bir adım ileri taşıyan ve birçok modern belge ve geniş-sütun veritabanının kalbinde yatan tasarıma **LSM ağacı** denir — açılımı “log-yapılı birleştirmeli ağaç”. Saf append-only yaklaşımının okuma zorluğunu, akıllıca bir düzenlemeyle hafifletir.

LSM'nin fikri şudur: gelen yazmaları önce **bellekte**, sıralı bir yapıda biriktir. Bellek hızlı olduğu için bu yazmalar anında kabul edilir. Bellekteki bu tampon dolduğunda, içeriği tek seferde, **sıralı** biçimde diske bir “parça” olarak yazılır — yine ardışık yazma, yine diskin sevdiği biçim. Zamanla diskte böyle birçok sıralı parça birikir. Arka planda çalışan bir süreç, bu parçaları periyodik olarak **birleştirir** (merge): birkaç sıralı parçayı tek bir büyük sıralı parçaya kaynaştırır, bu sırada ölü kayıtları da ayıklar. Bu birleştirme, append-only modelin sıkıştırmasının daha düzenli, kademeli bir biçimidir.

LSM, yazmada olağanüstü hızlıdır, çünkü her yazma önce belleğe gider ve diske hep ardışık yazılır. Okumada ise bir bedel vardır: bir kaydı ararken, en güncel hâli hangi parçada diye birden çok parçaya bakmak gerekebilir. Bu maliyeti azaltmak için LSM motorları, “bu kayıt bu parçada **kesinlikle yok**” sorusunu diske hiç bakmadan, çok ucuza yanıtlayan olasılıksal yardımcı yapılar kullanır; böylece çoğu boşa aramadan kurtulur. Yine de LSM, doğası gereği, yazmayı okumanın ve arka plan birleştirme yükünün önüne koyan bir tasarımdır.

## 5.7 Üç büyütme: kaçınılmaz üçgen

Bu noktada, depolama motorlarını karşılaştırmak için kullanışlı bir çerçeve ortaya çıkar. Her tasarım, üç tür “büyütme” arasında bir denge kurar ve hiçbirini üçünü birden en aza indiremez; biri azalırken genellikle bir diğeri artar.

**Yazma büyütmesi**, mantıksal olarak yazdığınız bir baytın diske kaç bayt olarak yazıldığıdır. B-ağaçlarında yerinde güncelleme ve sayfa bölünmeleri bunu artırır; LSM’de ise asıl yük, arka plandaki birleştirmenin aynı veriyi defalarca yeniden yazmasından gelir. **Okuma büyütmesi**, mantıksal bir okuma için diskten kaç parçaya bakıldığıdır; B-ağacı burada genellikle iyidir (birkaç sayfa), LSM ise birden çok parçaya bakmak zorunda kalabildiği için daha kötüdür. **Yer büyütmesi**, verinin diskte mantıksal boyutunun kaç katı yer kapladığıdır; append-only ve LSM’de biriken ölü kayıtlar bunu artırır, sıkıştırma azaltır.

Bu üçgen, neden tek bir “en iyi” depolama motoru olmadığını net biçimde gösterir. B-ağaçları okuma büyütmesini en aza indirir, yazma ve belirli durumlarda yer büyütmesini öder; LSM ve append-only yaklaşımlar yazmayı en aza indirir, okuma ve yer büyütmesini öder. Hangisinin doğru olduğu, üçüncü bölümdeki o değişmez ilkeye geri döner: iş yükünüze, yani okuma mı yoksa yazma mı baskın olduğuna bağlıdır.

## 5.8 Belleğin rolü ve veriyi belleğe yansıtmak

Şimdiye dek hep diskten söz ettik, ama hiçbir gerçek depolama motoru yalnızca diske güvenmez; bellekle disk arasında sürekli bir dans vardır. Sık erişilen veriyi bellekte tutmak, onu her seferinde diskten okumaktan kat kat hızlıdır. Bu yüzden depolama motorları, diskin en çok kullanılan parçalarını bellekte tutan bir **önbellek** işletir; sıcak veri bellekte kalır, soğuk veri diskte. Bu önbelleğin ne kadar büyük olacağı, hangi verinin tutulup hangisinin atılacağı, başlı başına bir tasarım konusudur ve on üçüncü bölümü tümüyle ona ayıracağız.

Belleği işin içine katmanın zarif bir yolu daha vardır ve onu burada tohumlamak yararlı olur: bir dosyayı, işletim sistemine söyleyerek, doğrudan belleğe **yansıtmak**. Bu teknikte dosya, sanki belleğin bir parçasıymış gibi davranır; ona erişmek, diskten açıkça okuma çağrısı yapmak yerine, yalnızca bellekteki bir adrese bakmak kadar basit hale gelir. Verinin gerçekten diskten belleğe ne zaman getirileceğine işletim sistemi karar verir ve bellek darda kaldığında o veriyi sessizce geri atabilir. Bu yaklaşım, depolama motorunun kendi önbelleğini elle yönetme yükünü büyük ölçüde işletim sistemine devretmesini sağlar. Üçüncü kısımda OxiDB’nin “disk-öncelikli” kipinde tam olarak bu tekniği kullandığını ve bunun bellek ayak izini nasıl küçülttüğünü ayrıntısıyla göreceğiz.

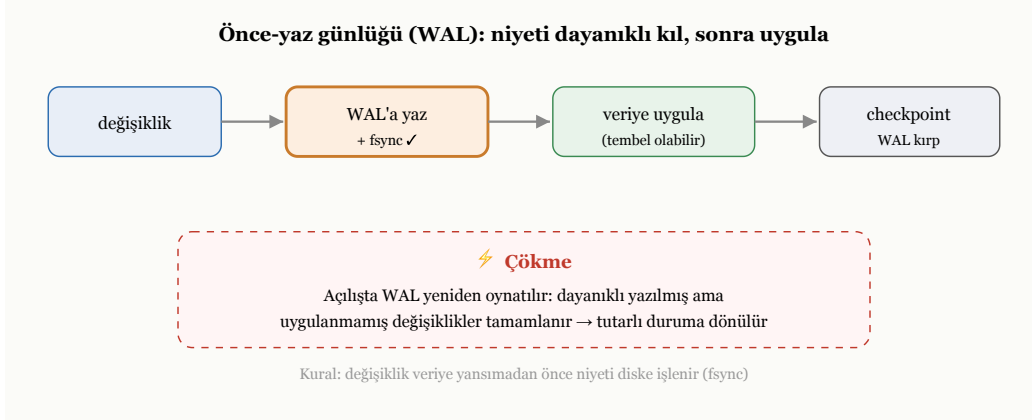
## 5.9 Bu bölümün bıraktığı yer ve bir sonrakine köprü

Bu bölümde, bir belge veritabanının baytları diske nasıl yerleştirdiğini — depolama motorunun işini — ve bu işin kalbindeki okuma-yazma gerilimini gördük. İki büyük felsefeyi tanıdık: veriyi düzenli ve yerinde tutarak okumayı önceleyen sayfa tabanlı B-ağaçları; ve veriyi asla üzerine yazmadan ardışık ekleyerek yazmayı önceleyen append-only ve LSM yaklaşımları. Bu yaklaşımların hiçbirinin mutlak üstün olmadığını, her birinin üç büyütme arasında farklı bir denge kurduğunu gördük.

Ama hangi felsefeyi seçerseniz seçin, henüz konuşmadığımız, sinsi bir sorun geride duruyor. Bir veriyi diske “yazdığınızda”, o veri gerçekten kalıcı olmuş mudur? Sezgi “evet” der, ama gerçek çok daha incelikli. Yazma işlemleri yolda, çeşitli tamponlarda bekliyor olabilir; ve sistem tam o sırada çökerse, “yazdım” sandığınız veri buharlaşabilir — ya da daha kötüsü, yarı yazılmış, bozuk bir hâlde kalabilir. Bir sonraki bölümde, depolama motorlarının en çetin sorumluluğuna, yani bir yazmanın gerçekten dayanıklı olduğundan emin olmaya ve çökmenin ortasından tutarlı biçimde geri dönmeye — yazma-öncesi günlüğe ve fsync’in inceliklerine — eğileceğiz.

## 6 Dayanıklılık: Write-Ahead Log, fsync ve Çökmeden Kurtarma

Önceki bölümün sonunda sinsi bir soru bırakmıştık: bir veriyi diske “yazdığınızda”, o veri gerçekten kalıcı olmuş mudur? Sezgi rahatça “evet” der. Ama bu sezgi yanıltır ve bu yanlıştın bedeli, kaybolan ya da bozulan veridir. Bu bölüm, bir veritabanının en çetin ve en az takdir edilen sorumluluğunu ele alır: bir yazmanın gerçekten dayanıklı olduğundan emin olmak ve sistem en beklenmedik anda çökerse, enkazın içinden tutarlı biçimde geri dönmek. Bu, veritabanını basit bir dosyadan ayıran en derin farklardan biridir.



Şekil 6 — Önce-yaz günlüğü (WAL) akışı ve çökmeden kurtarma.

### 6.1 “Yazdım” demek neden yetmez

Bir programın diske yazma isteği, sandığınız gibi tek adımda diskin yüzeyine ulaşmaz. Aradan birçok katman geçer ve her katman, hız uğruna veriyi geçici olarak bekletir. Programınız veriyi yazdığında, o veri önce işletim sisteminin tuttuğu bir tampona düşer; işletim sistemi, performans için, bu veriyi hemen diske göndermeyebilir — biriktirip sonra topluca yazmayı tercih edebilir. Veri diske gönderildiğinde bile, diskin kendi denetleyicisinin de bir önbelleği vardır; veri orada bekleyip henüz fiziksel ortama işlenmemiş olabilir. Yani “yazdım” dediğiniz an ile verinin gerçekten kalıcı ortama oturduğu an arasında, görünmez bir gecikme ve birçok bekleme noktası vardır.

Sıradan zamanlarda bu katmanlar fark edilmez; veri eninde sonunda diske oturur ve her şey yolundadır. Sorun, tam da bu bekleme noktalarının birinde sistem çökerse — elektrik kesilirse, makine donarsa, süreç aniden ölürse — ortaya çıkar. O an, henüz fiziksel ortama ulaşmamış olan her şey buharlaşır. Programınız “kaydettim” demiş, kullanıcıya “işlem tamam” demiş olabilir; ama o veri katmanların birinde beklerken yok olmuştur. Kullanıcı, yaptığını sandığı işlemin hiç gerçekleşmediğini sonradan öğrenir. Bir bankada, bir siparişte, bir tıbbi kayıta bunun ne anlama geldiğini düşünün.

### 6.2 fsync: veriyi gerçekten diske zorlamak

Bu gecikmenin bir panzehiri vardır. İşletim sistemine, “bu veriyi şimdi, bu an, gerçekten kalıcı ortama yaz ve bunu bana onaylamadan geri dönme” diyebilirsiniz. Bu açık emre, geleneksel adıyla **fsync** denir. fsync döndüğünde, daha önce yazdığınız verinin gerçekten dayanıklı hale geldiğinden emin olabilirsiniz; artık çökme onu silemez.

Ama fsync pahalıdır. Çünkü tam da diskin en sevmediği şeyi yapmaya zorlar: beklemeyi bırakıp veriyi fiilen fiziksel ortama işlemek. Bu, milisaniyeler mertebesinde sürebilir — bilgisayar ölçeğinde upuzun bir zaman. Bir veritabanı her küçük yazma için fsync çağırıyorsa, korkunç yavaş olurdu. İşte buradan, bu bölümün geri kalanını biçimlendiren

gerilim doğar: **dayanıklılık güvenlidir ama yavaştır; onsuz hızlıdır ama tehlikelidir**. İyi bir veritabanı, bu ikisi arasında akıllıca bir denge kurmak zorundadır. (Bir incelik: bazı sistemlerde sıradan fsync, veriyi disk denetleyicisinin önbelleğine bırakıp fiziksel ortama işlendiğini garanti etmeyebilir; gerçek bir dayanıklılık için daha güçlü, daha da pahalı bir “tam boşaltma” emri gerekir. Üçüncü kısımda OxiDB’nin tam da bu güçlü boşaltmayı kullandığını ve bunun tek-kayıt güncellemelerinin hızına nasıl yansıdığını göreceğiz.)

### 6.3 İkinci tehlike: yarım kalmış yazma

Çökmenin yalnızca veriyi yok etmek gibi bir tehlikesi yoktur; ondan daha sinsi bir tehlikesi vardır. Bir yazma işleminin **ortasında** çökme olursa ne olur? Disk veriyi bloklar halinde yazdığı için, büyük bir kaydın bir kısmı yeni değeriyle, bir kısmı eski değeriyle, hatta arada bir yerde tümüyle bozuk kalabilir. Buna **yarım kalmış yazma** (torn write) denir. Sonuç, yalnızca eksik değil, **tutarsız** bir veridir: yarısı bir dünyaya, yarısı başka bir dünyaya ait bir kayıt. Böyle bir kaydı sonradan okumaya çalışmak, anlamsız ya da yanıltıcı sonuçlar doğurabilir.

Birinci bölümde para transferi örneğini hatırlayın: bir hesaptan düş, diğerine ekle. Bu iki adımın arasında çökme olursa, para buharlaşmıştı. Şimdi sorunun daha da temel bir katmanını görüyoruz: yalnızca iki ayrı adım arasında değil, **tek bir kaydın yazılması sırasında** bile, çökme bizi tutarsız bir duruma düşürebilir. Bir veritabanı, hem “ya hep ya hiç” güvencesini vermek hem de yarım yazmalara karşı kendini korumak zorundadır. Bu iki ihtiyaca birden, şaşırtıcı derecede zarif tek bir fikir yanıt verir.

### 6.4 Çözüm: önce niyetini yaz

Bu fikrin adı **yazma-öncesi günlük** — İngilizcesiyle write-ahead log, kısaca WAL. Özü tek bir cümlede toplanır: **asıl veriyi değiştirmeden önce, ne yapacağımı ayrı bir günlüğe yaz ve o günlüğü dayanıklı kıl**. Yani önce niyetini kaydedersin, sonra eylemi gerçekleştirirsin.

Bunu günlük hayattan bir benzetmeyle düşünelim. Önemli bir işe girişmeden önce, ne yapacağınızı bir deftere not aldığınızı varsayın: “şu hesaptan şu kadar düş, şu hesaba şu kadar ekle.” Bu notu aldıktan, yani niyetinizi kalıcı biçimde kaydettikten sonra, işi yapmaya başlarsınız. İşin tam ortasında bayılıp düşerseniz ne olur? Ayıldığınızda defterinizi açar, ne yapmaya niyet ettiğinizi okur ve işi baştan, eksiksiz tamamlarsınız. Defter, niyetinizin kanıtıdır; eylem yarım kalsa bile, defterdeki kayıt sayesinde onu tamamlayabilirsiniz.

WAL tam olarak böyle çalışır. Veritabanı bir değişiklik yapmadan önce, o değişikliğin bir kaydını günlüğe **ekler** ve o kayıt fsync ile dayanıklı kılar. Günlüğe yazmanın güzelliği, önceki bölümde gördüğümüz append-only fikrinin tam da kendisidir: günlük yalnızca sona eklenir, yani diskin sevdiği o hızlı, ardışık yazma biçiminde büyür. Bir değişiklik günlüğe dayanıklı biçimde yazıldığı an, o değişiklik artık **kalıcı kabul edilir** — asıl veri dosyaları henüz güncellenmemiş olsa bile. Çünkü bir çökme olsa dahi, günlükteki kayıt sayesinde o değişikliği yeniden uygulayabiliriz.

İşte WAL’ın asıl zekâsı buradadır. Asıl veri yapısını — ister B-ağacı ister append-only depo olsun — değiştirmek, rastgele yazma içerebilen, yavaş ve riskli bir iştir. WAL, dayanıklılık güvencesini bu yavaş işten **ayırır**: dayanıklılığı hızlı, ardışık günlük yazmasıyla hemen sağlar; asıl veri yapısını güncellemeyi ise sonraya, acele etmeden yapılacak bir işe bırakır. Hız ve güvenlik, böylece aynı anda elde edilir: günlük hızlı ve güvenlidir, asıl güncelleme ise tembel ve huzurludur.

### 6.5 Kurtarma: enkazın içinden geri dönmek

WAL’ın asıl sınavı, çökmeden sonra verdiği vaattir. Sistem yeniden açıldığında, veritabanı kendini tutarlı bir duruma getirmek zorundadır ve bunu günlüğü **okuyarak** yapar. Bu sürece **kurtarma** (recovery) denir.

Kurtarma şöyle ilerler. Veritabanı, günlüğü baştan tarar ve her kaydı inceler. Günlükte dayanıklı biçimde yer alan ama asıl veri dosyalarına henüz yansımamış değişiklikleri bulup **yeniden uygular** — tıpkı bayıldıktan sonra defterini açıp yarım kalan işi tamamlayan kişi gibi. Böylece, çökmeden hemen önce “tamamlandı” denmiş her değişiklik, asıl veriye eksiksiz işlenmiş olur. Buna karşılık, günlüğe tam olarak yazılmamış, yani yarım kalmış kayıtlar atılır; çünkü onların temsil ettiği değişiklikler hiçbir zaman “tamamlandı” sayılmamıştı.

Bu kurtarma sürecinin güvenilir olması için bir özellik şarttır: yeniden uygulama **zararsız tekrarlanabilir** olmalıdır. Yani aynı değişikliği bir kez ya da iki kez uygulamak, sonucu değiştirmemelidir. Çünkü kurtarma sırasında, bir değişikliğin asıl veriye zaten yansıyor yansımadığından her zaman emin olamayız; bu yüzden onu yeniden uygulamak, eğer zaten uygulanmışsa bile, bir zarar vermemelidir. Bu özelliğe sahip işlemlere “etkisiz tekrarlanabilir” denir ve sağlam bir kurtarma tasarımının temelinde yatar.

## 6.6 Yarım yazmayı yakalamak: sağlama toplamaları

Peki kurtarma, günlüğün kendisinin yarım kalmış bir kaydını nasıl fark eder? Sonuçta çökme, en son günlük kaydının tam yazılmasının ortasında da olabilir. İşte burada **sağlama toplamı** (checksum) devreye girer. Veritabanı, her günlük kaydının yanına, o kaydın içeriğinden hesaplanmış küçük bir doğrulama değeri ekler. Kurtarma sırasında her kaydı okurken, bu değeri yeniden hesaplar ve kayda iliştilenmiş olanla karşılaştırır. İkisi uyuşuyorsa kayıt sağlamdır; uyuşmuyorsa, kayıt çökme sırasında yarım kalmış ya da bozulmuştur. Tipik olarak yalnızca günlüğün **en sonundaki** kayıt böyle bozulabilir — çünkü çökme anına dek öncekiler zaten tamamlanmıştı — ve veritabanı o bozuk son kaydı güvenle atıp, ondan önceki son sağlam noktadan devam eder. Böylece yarım kalmış yazma, sessizce bozuk veriye dönüşmek yerine, açıkça fark edilip temizlenir.

## 6.7 Günlük sonsuza dek büyüyemez: denetim noktası

WAL’ın bir sorunu vardır: günlük yalnızca eklenerek büyüdüğü için, zamanla devasa olur. Eğer hiç temizlenmezse, hem yer kaplar hem de kurtarma sırasında baştan sona okunması gereken bir dev haline gelir. Çözüm, günlüğü periyodik olarak güvenle kısaltmaktır.

Bu işleme **denetim noktası** (checkpoint) denir. Mantığı şudur: günlükteki değişiklikler eninde sonunda asıl veri dosyalarına yansıtılır. Veritabanı, belirli aralıklarla durup şundan emin olur: “şu ana kadarki tüm günlük kayıtlarının temsil ettiği değişiklikler, artık asıl veriye güvenle işlenmiş ve dayanıklı hale gelmiştir.” Bu güvence sağlandıktan sonra, o noktaya kadarki günlük kayıtlarına artık ihtiyaç kalmaz — çünkü kurtarma gerekse bile, onların temsil ettiği veri zaten asıl depoda durmaktadır. İşte o eski kayıtlar artık atılabilir ya da günlük yeniden kullanılabilir. Denetim noktası, günlük ile asıl depo arasındaki **senkronizasyon anıdır**: o ana kadar her şeyin yerli yerinde olduğunu mühürler ve günlüğün baştan büyümesine izin verir.

## 6.8 Dayanıklılığın tayfı: katı, gevşek ve grup commit

Bölümün başında, dayanıklılığın güvenli ama yavaş olduğunu söylemiştik. Gerçek sistemler, bu ödünleşimde tek bir noktaya saplanmaz; bir tayf üzerinde tercih yaparlar.

Bir uçta **katı dayanıklılık** vardır: her değişiklik, “tamamlandı” denmeden önce günlüğe yazılır ve fsync ile dayanıklı kılınır. Bu, en güçlü güvencedir — “tamamlandı” dedikten sonra hiçbir çökme veriyi geri alamaz — ama her değişiklik bir fsync beklediği için en yavaş olandır. Öteki uçta **gevşek dayanıklılık** vardır: değişiklikler günlüğe yazılır ama fsync, her değişiklikte değil, arada bir, toplu olarak yapılır. Bu çok daha hızlıdır; ama bir çökme olursa, henüz fsync edilmemiş son birkaç değişiklik kaybolabilir. Yani gevşek kip, hızı küçük bir veri kaybı riski karşılığında satın alır.

İki ucun arasında zarif bir orta yol vardır: **grup commit**. Fikri şudur: aynı anda birçok değişiklik “tamamlandı” olmayı bekliyorsa, onları tek tek fsync etmek yerine, hepsini bir araya getirip **tek bir fsync** ile birden dayanıklı kılmak. Bir fsync, ister bir değişikliği ister yüz değişikliği boşaltsın, kabaca aynı süreyi alır; bu yüzden yüz değişikliği tek fsync’e toplamak, gerçek dayanıklılık güvencesinden hiç ödün vermeden verimi kat kat artırır. Bunu, postaneye her mektup için ayrı ayrı koşmak yerine, eldeki tüm mektupları toplayıp tek seferde götürmeye benzetebilirsiniz. Üçüncü kısımda OxiDB’nin varsayılan olarak katı dayanıklılığı seçtiğini — her commit’i fsync ettiğini — ama toplu eklemelerde bu maliyeti tek bir fsync’e amortize ettiğini ve istendiğinde gevşek kipe geçilebildiğini göreceğiz.

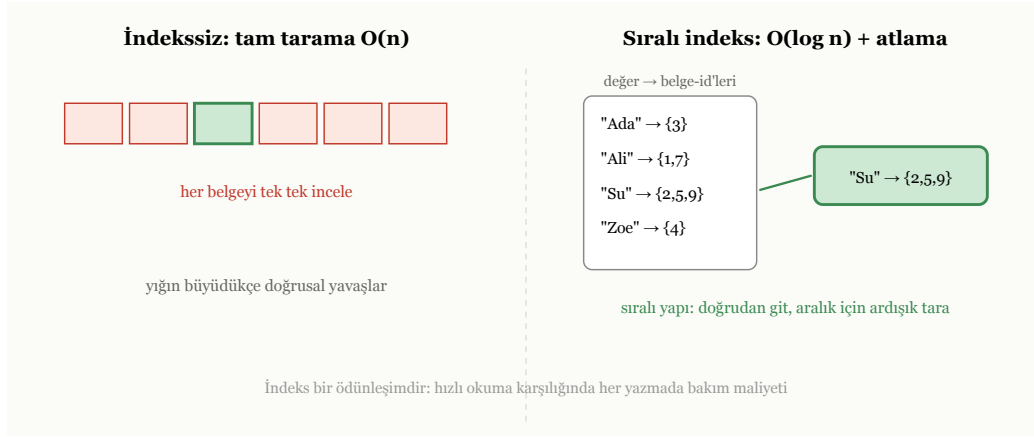
## 6.9 WAL nereye oturur

Bu bölümü, önceki bölümle bağımlı netleştirerek toparlayalım. WAL, beşinci bölümdeki depolama felsefelerinin bir alternatifi değildir; onların **önünde** duran, dayanıklılıktan ve atomiklikten sorumlu ayrı bir katmandır. İster veriyi sayfa tabanlı bir B-ağacında yerinde güncelleyen bir motor olsun, ister append-only bir depo olsun, ikisi de dayanıklılık ve çökme güvenliği için bir WAL’a yaslanabilir. Asıl veri yapısı “nasıl saklayacağım” sorusunu yanıtlar; WAL ise “değiştirirken çökersem ne olacak” sorusunu yanıtlar. İkisi birbirini tamamlar.

Böylece, beşinci ve altıncı bölümler birlikte, bir belge veritabanının en alt katmanını — veriyi diske güvenli biçimde yazma ve çökmeden geri dönme yeteneğini — tamamlamış oldu. Artık verimiz hem kalıcı hem de tutarlı biçimde diskte duruyor. Ama elimizde, çözülmemiş büyük bir sorun daha var. Birinci bölümde saymıştık: bir milyon belge arasından aradığımız tek kaydı, diski baştan sona taramadan nasıl buluruz? Veriyi güvenle saklamayı öğrendik; şimdi onu hızla **bulmayı** öğrenmemiz gerekiyor. Bir sonraki bölümde, veritabanlarını taramaya mahkûm olmaktan kurtaran o yardımcı yapılara — indekslere — eğiliyoruz.

## 7 İndeksleme: B-Ağacı İndeksleri, Bileşik İndeksler ve Ters İndeks

Önceki iki bölümde veriyi diske güvenle yazmayı ve çökmeden geri dönmeyi öğrendik. Artık verimiz kalıcı ve tutarlı biçimde duruyor. Ama birinci bölümde saydığımız sorunlardan en görünür olanı hâlâ çözülmedi: bir milyon belge arasından aradığımız tek kaydı, diski baştan sona taramadan nasıl buluruz? Bu bölüm, veritabanlarını taramaya mahkûm olmaktan kurtaran o yardımcı yapılara — indekslere — eğiliyor. İndeksler, bir veritabanının hızının büyük bölümünün geldiği yerdir; onlarsız her soru, tüm veriyi okumayı gerektirirdi.



Şekil 7 — Tam tarama ile sıralı indeks erişiminin karşılaştırması.

### 7.1 Tarama sorunu

Bir koleksiyonda bir milyon kullanıcı belgesi olduğunu ve belirli bir e-posta adresine sahip olanı aradığınızı düşünelim. Hiçbir yardımcı yapı yoksa, tek seçeneğimiz belgeleri tek tek okuyup e-posta alanına bakmaktır. Aradığımız belge şanslıysa başlarda, şanssızsak en sonda; ortalama olarak yarım milyon belgeyi okumamız gerekir. Buna **tam tarama** (full scan) denir ve maliyeti veriyle birlikte büyür: veri iki katına çıkınca arama da iki kat yavaşlar. Üstelik aynı aramayı her tekrarladığınızda baştan tararsınız; sistem hiçbir şey öğrenmez.

Bu, kabul edilemez bir durumdur. Bir milyon, hatta bir milyar kayıt arasından aradığınızı, neredeyse anında bulabilmeliyiz. Bunun için, verinin yanı sıra, aramayı hızlandıran ayrı bir yapı tutmamız gerekir.

### 7.2 İndeks nedir: kitabın arkasındaki dizin

İndeksin ne olduğunu anlamak için, elinizdeki bu kitabın arkasındaki dizini düşünün. “fsync” kavramının nerede geçtiğini bulmak istediğinizde, kitabı baştan sona okumazsınız; arkadaki dizine bakar, “fsync” sözcüğünün yanında yazan sayfa numarasını görür ve doğrudan o sayfaya gidersiniz. Dizin, kitabın içeriğinin bir kopyası değildir; ondan **türetilmiş**, yalnızca “hangi kavram hangi sayfada” bilgisini sıralı biçimde tutan, küçük ve aranabilir bir yapıdır.

Bir veritabanı indeksi tam olarak budur: belirli bir alanın değerlerinden, o değere sahip belgelerin **konumuna** giden bir eşleme. E-posta alanı için bir indeksimiz varsa, aradığımız e-posta adresini bu indekste buluruz ve indeks bize o adrese sahip belgenin nerede olduğunu doğrudan söyler. Bir milyon belgeyi taramak yerine, küçük ve düzenli indeks yapısında birkaç adımda sonuca ulaşırız. İndeks, asıl veriden türetilmiştir — yani onda yeni bir bilgi yoktur, veri zaten belgelerin içindedir — ama o bilgiyi, aramaya uygun bir düzende yeniden düzenleyerek, bulmayı hızlandırır.

## 7.3 İndeksin bedeli: bedava hız yoktur

İndeksler sihirli görünür, ama bedelsiz değildir ve bu bedeli görmek, onları doğru kullanmanın ön koşuludur. İki maliyet vardır.

Birincisi **yazma maliyetidir**. İndeks, asıl veriden türetildiği için, veri her değiştiğinde indeksin de güncellenmesi gerekir. Yeni bir belge eklediğinizde, yalnızca belgeyi yazmakla kalmaz, o belgenin indekslenen her alanı için ilgili indekse de bir giriş eklersiniz. Bir belgeyi sildiğinizde ya da indekslenen bir alanını değiştirdiğinizde, indeksleri de buna göre düzeltmeniz gerekir. Dolayısıyla her indeks, okumayı hızlandırırken yazmayı bir miktar yavaşlatır. Bir koleksiyonda ne kadar çok indeks varsa, her yazma o kadar çok ek iş doğurur.

İkincisi **yer maliyetidir**: her indeks, diskte ek yer kaplar.

Bu maliyetler, basit ama önemli bir ilkeye götürür: **her alanı indekslemezsiniz; yalnızca üzerinde sık arama yaptığınız alanları indekslersiniz**. Hangi alanları indeksleyeceğiniz, üçüncü bölümdeki o değişmez ilkeye geri döner — erişim örüntünüze bağlıdır. Hangi sorguları sık soruyorsanız, o sorguların süzdüğü alanları indekslersiniz. “Her ihtimale karşı her şeyi indeksleyelim” yaklaşımı, yazmaları boğan ve yer israf eden yaygın bir hatadır.

## 7.4 Sıralı indeksler: hem eşitlik hem aralık

İndeksin *ne* olduğunu anladık; peki *nasıl* yapılır? En yaygın ve en güçlü indeks türü, değerleri **sıralı** tutan yapılardır — beşinci bölümde tanıdığımız B-ağacının indeks olarak kullanılan biçimi. Burada B-ağacı, asıl veriyi değil, **indekslenen alanın değerlerini** sıralı biçimde tutar; her değerın yanında, o değere sahip belgelerin konumu durur.

Değerleri sıralı tutmanın üç ayrı kazancı vardır ve bu, sıralı indeksleri bu kadar değerli kılan şeydir. Birincisi **eşitlik aramasıdır**: belirli bir değere sahip kayıtları, sıralı yapıda hızlıca buluruz. İkincisi **aralık aramasıdır**: “şu değerle bu değer arasındaki tüm kayıtlar” sorusunu, sıralı yapıda o aralığın başına gidip yan yana ilerleyerek yanıtlarız — değerler zaten sıralı olduğu için bu doğaldır. Üçüncüsü ve çoğu zaman gözden kaçanı, **sıralı getirmedir**: bir sorgu sonuçları belirli bir alana göre sıralı isterse ve o alanın sıralı bir indeksi varsa, sıralama işini ayrıca yapmaya gerek kalmaz — indeks zaten o sırayı tutmaktadır. “Şu alana göre sıralı ilk on kayıt” gibi bir istek, tüm veriyi sıralamak yerine, indeksin başından on adım yürüyerek yanıtlanabilir. Üçüncü kısımda OxiDB’nin sıralamayı tam da böyle, indeks üzerinden, taramadan yaptığını göreceğiz.

Sıralı indeksin bir alternatifi, değerleri sıralı tutmayan ama eşitlik aramasını çok hızlı yapan **karma (hash) indekstir**. Karma indeks, bir değeri doğrudan konumuna eşler; eşitlik aramasında çok hızlıdır ama değerleri sıralı tutmadığı için aralık sorgularını ya da sıralı getirmeyi yapamaz. Yalnızca tam eşleşme aradığınız ve aralık ya da sıralama gerekmediği durumlarda yeterlidir. Sıralı indeksler ise daha genel amaçlıdır; bu yüzden veritabanlarının çoğunlukla yaslandığı yapı onlardır.

## 7.5 Seçicilik: her indeks aynı ölçüde işe yaramaz

Bir indeksin ne kadar işe yaradığı, indekslenen alanın **seçiciliğine** bağlıdır. Seçicilik, bir değerin kaç belgeyle eşleştiğiyle ilgilidir. Bir alanın her değeri yalnızca birkaç belgeyle eşleşiyorsa — örneğin e-posta adresi, ki neredeyse her biri benzersizdir — indeks muhteşem çalışır: aradığınız değeri bulur ve sizi doğrudan o birkaç belgeye götürür.

Ama bir alanın yalnızca birkaç farklı değeri varsa, indeksin yararı azalır. Bir “aktif mi” alanını düşünün: değeri ya doğru ya yanlıştır. Bu alanı indekslerseniz, “aktif olanları getir” sorgusu, belki belgelerin yarısını seçer; yarım milyon belgeyi getirmek için indeksten geçmek, neredeyse tüm veriyi taramaktan pek de hızlı değildir. Düşük seçicilikli

alanları indekslemek, çoğu zaman bedelini hak etmez. İyi bir indeks, çok sayıda belge arasından **azını** ayıklayan alanlar üzerine kurulur.

## 7.6 Bileşik indeksler: birden çok alan birlikte

Sorgular çoğu zaman tek bir alanı değil, birkaç alanı birden süzer: “şu bölgedeki, şu yaş aralığındaki, şu durumdaki kullanıcılar.” Bunun için **bileşik indeks** (composite index) kullanılır: tek bir alan yerine, birkaç alanın birleşimi üzerine kurulu bir indeks.

Bileşik indeksin inceliği, alanların **sırasının** önemli olmasıdır. Bir telefon rehberini düşünün: kayıtlar önce soyada, aynı soyad içinde ada göre sıralanmıştır. Bu rehber, “soyadı Yılmaz olanlar” ya da “soyadı Yılmaz, adı Ali olanlar” sorularını kolayca yanıtlar; çünkü sıralama önce soyada göredir. Ama “adı Ali olanlar” sorusunu — soyadından bağımsız olarak — kolayca yanıtlayamaz, çünkü Ali’ler tüm rehberde dağılmıştır. Bileşik indeks de tıpkı böyledir: (bölge, yaş) üzerine kurulu bir indeks, yalnızca bölgeye göre ya da bölge artı yaşa göre aramayı hızlandırır; ama yalnızca yaşa göre aramaya pek yaramaz. Buna **önek kuralı** denir: bileşik indeks, alan sırasının baştan başlayan bir önekini kullanan sorgulara yarar. Bu yüzden bileşik indekste alanları hangi sırayla dizeceğiniz, hangi sorguları hızlandıracağınızı belirleyen önemli bir tasarım kararıdır.

## 7.7 Kapsayan indeksler: belgeye hiç dokunmamak

İndekslerin zarif bir gücü daha vardır. Normalde bir indeks size belgenin **konumunu** verir; sonra o konuma gidip belgeyi okursunuz. Ama bazen, sorgunun ihtiyaç duyduğu tüm bilgi zaten indeksin içinde durur. O zaman belgeye hiç gitmeye gerek kalmaz; yanıt, yalnızca indeksten üretilir.

En arı örneği saymadır. “Şu bölgede kaç kullanıcı var?” sorusunu düşünün. Eğer bölge alanının bir indeksi varsa, o indekste her bölgenin altında hangi belgelerin olduğu zaten yazılıdır; saymak için yalnızca o girişleri saymak yeterlidir, belgelerin hiçbirini açmaya gerek yoktur. Sorgunun ihtiyaç duyduğu her şeyi indeksin tek başına sağlamasına **kapsama** (covering) denir ve bir sorgunun verilebilecek en hızlı yanıtlarından birini doğurur: hiç belge okumadan, yalnızca indeksten. Üçüncü kısımda OxiDB’nin saymayı tam da böyle, belgelere hiç dokunmadan yaptığını göreceğiz.

İndekslerin bir yan faydasını da burada anmak gerekir: **benzersizlik güvencesi**. Bir alanı “benzersiz indeks” olarak işaretlerseniz, indeks o alanda aynı değerin iki kez yazılmasını engeller. Böylece indeks yalnızca aramayı hızlandırmakla kalmaz, bir veri bütünlüğü kuralını da dayatmış olur.

## 7.8 Ters indeks: metni aranabilir kılmak

Şimdiye dek anlattığımız indeksler, bir alanın **tam değeriyle** çalışır: e-posta adresi şuna eşit, yaş şu aralıkta. Ama bambaşka bir arama türü vardır: metnin **içinde** sözcük aramak. “İçinde ‘veritabanı’ sözcüğü geçen tüm belgeleri getir” demek istediğinizde, alanın tam değerine bakan indeksler işe yaramaz; çünkü siz tam eşleşme değil, metnin içinde geçen bir sözcüğü arıyorsunuz. Bu ihtiyaç için tasarlanmış, tümüyle farklı bir yapı vardır: **ters indeks** (inverted index).

Ters indeksin fikri, sıradan indeksin tersini yapmaktır. Sıradan indeks “belge → içindeki değer” ilişkisini tutarken, ters indeks “sözcük → o sözcüğü içeren belgeler” ilişkisini tutar. Bir kitabın arkasındaki dizine yine dönelim; orada her kavramın yanında, o kavramın geçtiği sayfaların listesi vardır. Ters indeks tam olarak budur, ama metindeki her anlamlı sözcük için. Bir belge eklendiğinde, metni anlamlı sözcüklere ayrılır (buna parçalama denir) ve her sözcüğün altına o belge eklenir. Bir sözcüğü aradığınızda, ters indeks size o sözcüğü içeren tüm belgelerin listesini doğrudan verir — metinleri taramadan.

Metin aramanın sıradan aramadan bir farkı daha vardır: **alaka düzeyi**. Bir sözcüğü içeren yüzlerce belge olabilir, ama hepsi aynı ölçüde ilgili değildir. O sözcüğün sık geçtiği, kısa bir belge, o sözcüğün bir kez geçtiği uzun bir belgeden büyük olasılıkla daha ilgilidir. Bu yüzden metin arama sistemleri, sonuçları yalnızca bulmakla kalmaz, **sıralar** da: hangi belgenin sorguya daha alakalı olduğunu, sözcüğün belgede ne sıklıkta geçtiği ve genel olarak ne kadar yaygın bir sözcük olduğu gibi ölçütlere bakarak puanlar. Üçüncü kısımda OxiDB'nin tam metin aramayı, böyle bir ters indeks ve alaka puanlaması üzerine kurduğunu göreceğiz.

## 7.9 İndeksler de dayanıklı olmalı

Son bir bağlantı kuralım. İndeksler asıl veriden türetilmiş olsa da, her sorgudan önce baştan kurulamayacak kadar pahalıdır; bu yüzden onların da kalıcı olması gerekir. İndeksler de, asıl veri gibi, diske yazılır ve bir önceki bölümde gördüğümüz dayanıklılık kaygılarının kapsamına girer. Bir çökme sonrası, indekslerin de asıl veriyle tutarlı bir duruma getirilmesi gerekir; kimi sistemler indeksleri günlükten kurtarır, kimileri ise gerekirse asıl veriyi tarayarak yeniden kurar. İndeks ile veri arasındaki bu tutarlılığın korunması, sağlam bir veritabanının sessiz ama kritik bir görevidir.

## 7.10 Bu bölümün bıraktığı yer

Bu bölümde, veritabanlarını taramaya mahkûm olmaktan kurtaran indeksleri tanıdık. Bir indeksin, asıl veriden türetilmiş, aramayı hızlandıran ayrı bir yapı olduğunu; bunun karşılığında yazmayı yavaşlatıp yer kapladığını; bu yüzden seçici biçimde, erişim örüntülerine göre kurulduğunu gördük. Sıralı indekslerin eşitlik, aralık ve sıralı getirmeyi birden desteklediğini; karma indekslerin yalnızca eşitlikte hızlı olduğunu; seçiciliğin bir indeksin yararını belirlediğini; bileşik indekslerin alan sırasıyla birden çok koşulu hızlandırdığını; kapsayan indekslerin belgeye hiç dokunmadan yanıt üretebildiğini; ve ters indekslerin metni sözcük düzeyinde aranabilir kıldığını öğrendik.

Ama bir indeks, tek başına, yalnızca bir araçtır. Bir kullanıcının sorusunu — “şu bölgedeki, şu yaş aralığındaki, adında şu geçen kullanıcıları, şuna göre sıralı getir” — alıp, bu soruyu hangi indekslerin işe yarayacağına karar veren, veriyi en az iş yaparak süzecek bir plana dönüştüren ayrı bir akıl gerekir. O akıl, sorgu işleyicidir. Bir sonraki bölümde, bir sorunun bir yanıtı nasıl dönüştüğünü — ayırtırmadan planlamaya, indeks seçiminden yürütmeye — adım adım izleyeceğiz.

## 8 Sorgu İşleme: Ayırıştırma, Planlama ve İndeks Seçimi

Önceki bölümde indeksleri tanıdık ve bir indeksin, tek başına, yalnızca bir araç olduğunu söyledik. Bir kullanıcının sorusunu alıp, hangi indeksin işe yarayacağına karar veren, veriyi en az iş yaparak süzecek bir plana dönüştüren ayrı bir akıl gerekir. O akıl, **sorgu işleyicidir** ve bu bölümün konusu, bir sorunun bir yanıtı nasıl dönüştüğüdür. İkinci bölümde, ilişkisel modelin büyük armağanının sorguları “bildirimsel” kılmak olduğunu — yani yalnızca *ne* istediğinizi söyleyip *nasıl* sorusunu sisteme bırakmak olduğunu — söylemiştik. İşte o “nasıl” sorusunu yanıtlayan bileşen, sorgu işleyicidir; ve onun bu işi ne kadar iyi yaptığı, bir veritabanının hızını belirleyen en kritik etkenlerden biridir.



Şekil 8 — Bir sorunun yaşam döngüsü: ayırıştır, planla, çalıştır.

### 8.1 Soru ile yol arasındaki fark

Önce bir ayrımı net kuralım. Bir **sorgu**, bir sorudur: “şu bölgedeki, şu yaş aralığındaki, adında şu geçen kullanıcıları, şuna göre sıralı getir.” Bu soru, *neyi* istediğinizi söyler, ama o sonuca *nasıl* ulaşılacağını söylemez. Sonuca ulaşmanın ise birçok yolu olabilir. Önce bölgeye göre indeksi kullanıp adayları daraltıp sonra yaşa bakabilirsiniz; ya da önce yaşa göre indeksten başlayabilir; ya da hiç indeks kullanmadan tüm belgeleri tarayıp koşulları tek tek deneyebilirsiniz. Bu yolların her biri aynı yanıtı verir, ama maliyetleri taban tabana farklı olabilir — biri saniyenin binde biri, diğeri dakikalar sürebilir.

Sorgu işleyicinin görevi, soruyu bir **yola**, yani bir yürütme planına dönüştürmektir. Bunu bir yol tarifi uygulamasına benzetebilirsiniz: siz yalnızca varış noktanızı söylersiniz; uygulama, olası güzergâhları değerlendirip en hızlısını seçer ve sizi adım adım yönlendirir. Sorgu işleyici de tıpkı böyle, olası yürütme yollarını tartar ve en ucuzunu seçer. Bu işi üç aşamada yapar: ayırıştırma, planlama ve yürütme.

### 8.2 Birinci aşama: ayırıştırma

İlk adım **ayırıştırmadır** (parsing). Kullanıcının sorgusu, sisteme bir biçimde — bir metin ya da yapılandırılmış bir istek olarak — gelir. Sorgu işleyici, bu ham isteği önce kendi anlayabileceği iç bir temsile çevirir: koşulların ve mantıksal bağların bir ağacına. Örneğin “bölge eşittir Ege VE yaş 30 ile 40 arasında” sorgusu, bir “VE” düğümünün altında iki koşulun durduğu küçük bir ağaca dönüşür.

Belge veritabanlarında bu koşullar zengin bir çeşitlilik taşır. En temeli **eşitliktir**: bir alanın belirli bir değere eşit olması. Ardından **karşılaştırmalar** gelir: bir alanın bir değerden büyük, küçük ya da bir aralıkta olması. **Üyelik** koşulları, bir alanın belirli değerler kümesinden birine eşit olup olmadığını sorar. **Mantıksal bağlar** — ve, veya, değil — koşulları birleştirir. Belgeler iç içe ve listeli olabildiği için, belgeye özgü koşullar da vardır: bir alanın **var**

**olup olmadığı**, bir listenin belirli bir öğeyi **içerip içermediği**, bir metnin bir kalıba **uyup uymadığı**. Ayırıştırma aşaması, tüm bu koşulları tanır, geçerliliğini denetler ve onları işlenebilir bir yapıya kavuşturur. Bu aşamanın sonunda, sorgu artık bir metin değil, sistemin üzerinde akıl yürütebileceği yapısal bir nesnedir.

### 8.3 İkinci aşama: planlama

Asıl zekâ ikinci aşamada, **planlamada** belirir. Artık sorgunun yapısını bilen sistem, onu nasıl yürüteceğine karar vermelidir. Buradaki temel karar, yedinci bölümde tanıdığımız araçla doğrudan ilgilidir: **hangi indeks kullanmalı, ya da hiç indeks kullanmamalı mı?**

Mantık şöyle işler. Sorgudaki her koşula bakılır ve o koşulun süzdüğü alanın bir indeksi olup olmadığı kontrol edilir. Eğer bir koşul, indeksli ve seçici bir alan üzerindeyse — örneğin e-posta adresi gibi — bu koşul bir **fırsattır**: indeks kullanarak, tüm belgeleri taramadan, yalnızca o koşula uyan az sayıda **aday** belgeyi doğrudan bulabiliriz. Plan, bu indeksten yararlanmak üzere kurulur.

Burada zarif bir iş bölümü ortaya çıkar. İndeks, adayları **daraltır**; ama sorguda indeksin kapsamadığı başka koşullar da varsa, o koşullar adaylar üzerinde **süzgeç** olarak uygulanır. Diyelim ki bölge alanının indeksi var ama yaş alanının yok. Sistem, önce bölge indeksinden o bölgedeki adayları çeker — belki bir milyondan birkaç bine iner — sonra bu birkaç bin aday üzerinde yaş koşulunu tek tek dener. Tüm belgeleri taramak yerine, indeksin daralttığı küçük aday kümesini süzmek, kıyaslanamayacak kadar ucuzdur. Bu “indeks daraltır, süzgeç inceltir” iş bölümü, sorgu yürütmenin kalbinde yatar.

### 8.4 Birden çok koşulu birleştirmek ve en seçiciyi seçmek

Birden çok indeksli koşul olduğunda, plan daha da inceltir. “VE” ile birleştirilmiş koşullarda, sistem her koşulun ne kadar seçici olduğunu tahmin eder ve **en seçici** olanından başlar — çünkü en az sayıda aday üreten koşul, sonraki süzgeçlerin işini en aza indirir. Bölge bir milyon belgeden bini seçiyor, durum ise yarısını seçiyorsa, plan bölgeden başlar; çünkü binlik bir aday kümesini süzmek, yarım milyonluk bir kümeyi süzmekten çok daha hızlıdır. Bazı sistemler bir adım daha ileri gider ve birden çok indeksin sonuçlarını **kesiştirerek** adayları daha da daraltır.

“VEYA” ile birleştirilmiş koşullarda mantık tersine döner: her koşulun adayları ayrı ayrı bulunur ve **birleştirilir**, çünkü koşullardan herhangi birine uyan her belge sonuca dahildir.

Bu kararların hepsinin altında tek bir soru yatar: hangi yol daha ucuz? Sistem bunu, **maliyet tahmini** yaparak yanıtlar. Her alanın değerlerinin ne kadar dağıldığına dair tuttuğu kabaca istatistiklere bakarak, bir indeksin kaç aday üreteceğini, bir süzgecin ne kadar iş gerektireceğini tahmin eder ve en ucuz görünen planı seçer. Bu tahminler kusursuz değildir; gerçeği yanlış tahmin ederse, sistem kötü bir plan seçip yavaşlayabilir. İyi bir sorgu işleyici, tahminlerini olabildiğince isabetli tutmaya çalışır, ama tahminin doğası gereği hata payı her zaman vardır.

### 8.5 İndeks yoksa: tarama ve süzgeç

Bazen, sorgudaki hiçbir koşul indeksli bir alana denk gelmez. O zaman sistemin elinde tek seçenek kalır: **tarama**. Tüm belgeleri sırayla okur ve her birinde sorgunun koşullarını dener; uyanları sonuca alır, uymayanları atar. Bu, yedinci bölümde kaçınmaya çalıştığımız tam tarama maliyetidir; ama uygun bir indeks yoksa, kaçınılmazdır.

Burada, çoğu zaman gözden kaçan ama önemli bir incelik vardır. Taradığınız her belge için koşulu denemek, belgeyi okuyup içindeki ilgili alanı çözmeyi gerektirir. Oysa belge, beşinci bölümde değindiğimiz gibi, diskte sıkıştırılmış ya da kodlanmış bir biçimde durabilir; onu denetlemek için önce bu biçimden çözmek gerekir. Eğer milyonlarca belgenin **hepsini** baştan sona çözüp sonra çoğunu eler atarsanız, çok büyük bir emek boşa gider. Akıllı bir tasarım,

bir belgenin koşula uyup uymadığını, onu tümüyle çözmeden, kodlanmış biçim üzerinde hızlıca kestirmeye çalışır; yalnızca koşula uyanları tam olarak çözer. Üçüncü kısımda OxiDB’nin tam da böyle, eşleşmeyen belgeleri hiç çözmeden atlayan bir “bayt düzeyinde süzme” yolu kullandığını ve bunun büyük taramalarda hem hızı hem belleği nasıl iyileştirdiğini göreceğiz.

## 8.6 Sıralama, atlama ve sınırlama

Çoğu sorgu, yalnızca “uyanları getir” demez; sonuçların belirli bir düzende, belirli bir miktarda gelmesini de ister: “şu alana göre sıralı, ilk on kayıt.” Bu istekler, sorgu planını önemli ölçüde etkiler.

**Sıralama**, yedinci bölümdeki sıralı indekslerle doğrudan bağlantılıdır. Eğer sorgu, sıralı bir indeksi olan bir alana göre sıralı sonuç istiyorsa, sıralama işini ayrıca yapmaya gerek kalmaz; indeks zaten o sırayı tutar ve sonuçları o sırada gezerek üretebiliriz. Ama sıralanacak alanın indeksi yoksa, sistem önce tüm sonuçları toplar, sonra onları belleğe alıp sıralar — ki bu, büyük sonuç kümelerinde pahalı bir iştir.

**Sınırlama** (limit), yani “yalnızca ilk on” demek, sıralı bir indeksle birleştğinde özellikle güçlüdür. Çünkü indeks sonuçları zaten sıralı verdiği için, sistem onuncu sonucu bulduğu an durabilir; gerisini hiç üretmeye gerek yoktur. Buna **erken sonlanma** denir ve milyonlarca kayıt arasından en büyük ya da en küçük birkaçını bulmayı neredeyse anlık hale getirir. **Atlama** (skip) ise sonuçların baştan belirli bir kısmını geçip kalanını döndürür; çoğu zaman sayfalama için sınırlamayla birlikte kullanılır.

Erken sonlanma yalnızca sıralamayla sınırlı değildir. “Şu koşula uyan **bir** kaydı getir” ya da “şu koşula uyan **ilk** kaydı güncelle” gibi sorgularda da sistem, ilk eşleşmeyi bulduğu an durabilir; kalan belgeleri taramaya gerek yoktur. Üçüncü kısımda OxiDB’nin tekil okuma, tekil güncelleme ve tekil silme işlemlerinde tam olarak bu erken sonlanmayı uyguladığını göreceğiz.

## 8.7 Yalnızca gerekeni döndürmek

Sorgu işleyicinin küçük ama yararlı bir görevi daha vardır. Bir kullanıcı çoğu zaman belgenin tamamını değil, yalnızca birkaç alanını ister: “kullanıcıların yalnızca adlarını ve e-postalarını getir.” Sonucu yalnızca istenen alanlara indirgeme işine **izdüşüm** (projection) denir. İzdüşüm, hem ağ üzerinden taşınan veriyi azaltır hem de — yedinci bölümdeki kapsayan indeksleri hatırlayın — eğer istenen alanların hepsi zaten bir indekste varsa, belgeye hiç dokunmadan yanıt üretmeyi mümkün kılar.

## 8.8 Bildirimsel olmanın asıl kazancı

Bu bölümü, ikinci bölümde attığımız bir tohumun nasıl meyve verdiğini görerek toparlayalım. İlişkisel modelin “ne iste, nasıl’ı sisteme bırak” ilkesinin asıl değeri, tam da burada ortaya çıkar. Kullanıcı yalnızca *ne* istediğini söylediği için, sistem *nasıl* sorusunu özgürce yanıtlayabilir: bir indeks ekleyince, kullanıcı sorgusunu hiç değiştirmeden, sistem o indeksi kullanmaya başlar; veri büyüyüp dağılım değişince, sistem planını ona göre uyarlar; daha iyi bir yürütme yolu mümkün hale gelince, kullanıcı bundan habersiz kazanç sağlar. Eğer sorgular, eski hiyerarşik ve ağ modellerinde olduğu gibi, “şu kayıttan şu bağlantıyı izle” diye adım adım yazılıysaydı, bu esnekliğin hiçbiri mümkün olmazdı. Bildirimsel sorgu ile sorgu işleyicinin akıllı planlaması, aynı madalyonun iki yüzüdür: biri özgürlüğü tanır, diğeri o özgürlüğü performansa çevirir.

## 8.9 Bu bölümün bıraktığı yer

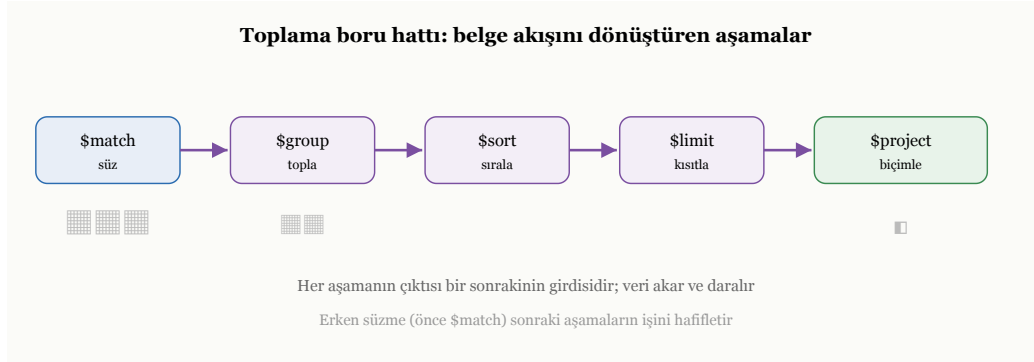
Bu bölümde, bir sorunun bir yanıtı nasıl dönüştüğünü izledik: ham sorgunun ayrıştırılıp yapısal bir nesneye çevrilmesini; planlamanın hangi indeksi kullanacağına, koşulları hangi sırayla uygulayacağına maliyet tahminiyle

karar vermesini; indeksin adayları daraltıp süzgecin onları inceltmesini; indeks yokken taramanın ve eşleşmeyi çözmeden atlamanın inceliğini; sıralama, sınırlama ve erken sonlanmanın gücünü; ve tüm bunların, bildirimsel sorunun tanıdığı özgürlükten doğduğunu gördük.

Şimdiye dek hep tek tek belgeleri bulup süzmekle ilgilendik: “şu koşullara uyan belgeleri getir.” Ama veriye sorabileceğimiz daha zengin bir soru türü daha vardır. Tek tek belgeleri değil, **birçok belgenin toplu görüntüsünü** isteriz: “her bölgedeki ortalama yaş nedir”, “her aydaki toplam satış kaçtır”, “en çok satan on ürün hangileridir.” Bu tür sorular, belgeleri yalnızca süzmekle kalmaz; onları **gruplar, özetler ve dönüştürür**. Bir sonraki bölümde, bu güçlü soru türünü — toplama (aggregation) ve onun ardındaki pipeline modelini — inceleyeceğiz.

## 9 Toplama: Pipeline Modeli, Gruplama ve Pencere Fonksiyonları

Şimdiye dek hep tek tek belgeleri bulup süzmekle ilgilendik: “şu koşullara uyan belgeleri getir.” Ama veriye sorabileceğimiz daha zengin bir soru türü vardır. Tek tek belgeleri değil, **birçok belgenin toplu görüntüsünü** isteriz: “her bölgedeki ortalama yaş nedir”, “her aydaki toplam satış kaçtır”, “en çok satan on ürün hangileridir.” Bu sorular, belgeleri yalnızca süzmekle kalmaz; onları **gruplar, özetler ve dönüştürür**. Bu bölüm, bu güçlü soru türünü — toplama (aggregation) ve onun ardındaki pipeline modelini — inceliyor. Toplama, bir veritabanını bir kayıt deposundan bir analiz aracına dönüştüren yetenektir.



Şekil 9 — Toplama boru hattı: belge akışını dönüştüren ardışık aşamalar.

### 9.1 Süzmenin ötesi: veriyi özetlemek

Önceki bölümdeki sorgular, hep belgeleri **seçmekle** ilgiliydi; sonuç, her zaman var olan belgelerin bir alt kümesiydi. Toplama farklıdır: sonucu, var olan belgelerden **türetilmiş**, çoğu zaman hiçbir tek belgede bulunmayan yeni bir şeydir. “Her bölgedeki ortalama yaş” sorusunun yanıtı, hiçbir kullanıcı belgesinde yazmaz; o, birçok belgenin yaş değerlerini bir araya getirip hesaplanarak üretilir. Toplama, veriden bilgi **damıtmaktır**.

Bu tür soruları yanıtlamak, önceki bölümdeki araçlarla mümkün değildir. Süzgeçler bir belgeyi alır ya da atar; ama “bu bin belgenin ortalamasını al” diyemez. Gruplama, ortalama alma, en büyüğü bulma gibi işlemler, belgeleri **birleştiren** yeni bir mekanizma gerektirir. İşte toplama, bu mekanizmayı sunar.

### 9.2 Pipeline modeli: montaj hattı

Toplamayı düzenlemenin en yaygın ve en zarif yolu, **pipeline** modelidir. Pipeline’ı bir montaj hattı gibi düşünebilirsiniz. Bir uçtan belgeler akarak girer; hat boyunca bir dizi istasyondan geçer; her istasyon belgelere belirli bir işlem uygular ve sonucu bir sonraki istasyona aktarır; diğer uçtan ise nihai sonuç çıkar. Her istasyona pipeline’ın bir **aşaması** (stage) denir. Bir aşamanın çıktısı, doğrudan bir sonraki aşamanın girdisidir.

Bu modelin gücü, **bileşilebilirliğinde** yatar. Karmaşık bir analizi, sıfırdan tek bir dev işlem olarak yazmak yerine, her biri tek bir basit işi yapan küçük aşamaları arka arkaya dizerek kurarsınız. Bir aşama süzer, bir sonraki gruplar, bir sonraki sıralar, bir sonraki ilk onu alır. Tıpkı yapı taşlarıyla bir şey inşa etmek gibi: az sayıda basit, iyi tanımlanmış parçayı birleştirerek, sınırsız çeşitlilikte karmaşık analizler kurabilirsiniz. Bu, pipeline modelini hem anlaşılır hem de esnek kılar.

### 9.3 Pipeline'ın aşamaları

Bir pipeline'da en sık karşılaşılan aşamaları tanıyalım. Bunları, belirli bir ürünün sözdizimiyle değil, yaptıkları işlemleri anlatacağız; çünkü fikirler, adlardan daha kalıcıdır.

**Süzme** aşaması, önceki bölümdeki sorgunun ta kendisidir: koşula uymayan belgeleri akıştan atar. Pipeline'ın genellikle ilk aşaması olarak kullanılır ve nedeni önemlidir: akışı erkenden daraltmak, sonraki tüm aşamaların işini azaltır. Üstelik bu erken süzme, önceki bölümde gördüğümüz indekslerden de yararlanabilir; yani pipeline'ın başındaki bir süzme, tüm koleksiyonu taramak yerine indeksle az sayıda belgeye inebilir.

**Gruplama** aşaması, toplamının kalbidir ve birazdan ona ayrı bir başlık açacağız. Kısaca, belgeleri bir anahtara göre gruplara ayırır ve her grup için özet değerler hesaplar.

**Sıralama, atlama ve sınırlama** aşamaları, önceki bölümde tanıdığımız işlevlerin pipeline içindeki karşılıklarıdır: akışı belirli bir düzene sokar ya da belirli bir kısma indirir. “En çok satan on ürün” sorusu, bir gruplamadan sonra gelen bir sıralama ve bir sınırlamayla yanıtlanır.

**Yeniden şekillendirme** aşamaları, her belgenin biçimini değiştirir: yeni hesaplanmış alanlar ekler, gereksiz alanları atar, alanları yeniden düzenler. Bunlar, akıştaki belgeleri nihai sonucun istediği biçime sokar.

**Açma** aşaması (unwind), ilginç ve güçlü bir dönüşümdür. İçinde bir liste barındıran bir belgeyi alır ve onu, listenin her ögesi için bir tane olmak üzere, **birçok belgeye** açar. Bir siparişin kalemler listesini düşünün; açma aşaması, o tek siparişi, her kalem için ayrı bir belgeye dönüştürür. Böylece liste öğelerini tek tek gruplayıp analiz edebilirsiniz — örneğin “her üründen toplam kaç adet satıldı” sorusunu, kalemleri açıp sonra ürüne göre gruplayarak yanıtlarsınız.

**Birleştirme** aşaması (lookup), başka bir koleksiyondan ilişkili belgeleri getirir. Üçüncü ve dördüncü bölümlerde, belge modelinin ilişkili veriyi ya gömdüğünü ya da ona referansla işaret ettiğini söylemiştik. Referansla bağlanmış veriyi bir araya getirmek gerektiğinde, birleştirme aşaması devreye girer; bu, belge dünyasının, ilişkisel modelin birleştirme gücünden ölçülü bir ödünç almasıdır.

**Çok-yönlü analiz** aşaması (facet), tek bir geçişte birden çok özeti birden üretir. Bir alışveriş sitesinin sonuç sayfasını düşünün: aynı süzölmüş ürün kümesinden, hem kategoriye göre sayıları, hem fiyat aralıklarının dağılımını, hem de en yüksek puanlı birkaç ürünü aynı anda istersiniz. Bu aşama, aynı girdi üzerinde birkaç alt-pipeline'ı paralel çalıştırıp her birinin sonucunu ayrı bir alanda toplar; böylece veriyi tek bir geçişte birden çok açıdan özetlersiniz.

### 9.4 Sıranın önemi

Pipeline'da aşamaların **sırası**, hem sonucu hem de performansı belirler. En temel ilke, süzmeyi olabildiğince erkene almaktır. Eğer önce gruplar, sonra süzerseniz, gereksiz belgeleri de gruplamış olursunuz; oysa önce süzüp sonra gruplarsanız, yalnızca gereken belgeleri gruplarsınız. Akışı erken daraltmak, sonraki her aşamanın taşıması gereken yükü azaltır. Pipeline'ın açık, aşamalı yapısı, sistemin bu sırayı görüp — kullanıcı yazmasa bile — kimi durumlarda kendiliğinden eniyilemesine de olanak tanır; önceki bölümdeki bildirimsel özgürlüğün toplamadaki yankısıdır bu.

### 9.5 Gruplamanın derinliği

Gruplama, toplamının kalbi olduğu için biraz daha yakından bakmayı hak eder. Gruplama iki şey ister: belgeleri hangi anahtara göre gruplayacağınızı (örneğin bölge) ve her grup için ne hesaplayacağınızı.

Her grup için hesaplanan değere bir **biriktirici** (accumulator) aracılığıyla ulaşılır. En yaygın biriktiriciler sezgiseldir: grup içindeki belgeleri **sayma**, bir alanın değerlerini **toplama**, **ortalamasını** alma, en **büyükünü** ya da en **küçükünü** bulma, ya da bir alanın tüm değerlerini bir **listede toplama**. Gruplama kavramsal olarak şöyle işler: sistem belgeleri tek tek gezer, her belgenin grup anahtarına bakar, o anahtara ait “kovaya” belgenin katkısını ekler. Tüm belgeler gezildiğinde, her kovada o grubun özeti hazırır.

Burada önemli bir performans gerçeği vardır: gruplama, doğası gereği, **tüm ilgili belgeleri görmek** zorundadır. Bir grubun ortalamasını, o gruba ait son belgeyi görmeden hesaplayamazsınız. Bu yüzden gruplama, önceki bölümdeki erken sonlanma kestirmesinden yararlanamaz; bir koleksiyonu baştan sona taramayı gerektirebilir. İşte bu yüzden toplama sorguları, tekil aramalara kıyasla çoğunlukla daha pahalıdır.

Yine de zarif bir kestirme vardır. Eğer yalnızca **saymak** istiyorsanız ve gruplama anahtarının bir indeksi varsa — yedinci bölümdeki kapsayan indeksleri hatırlayın — belgelerin hiçbirine dokunmaya gerek kalmaz. İndeks, her anahtarın altında kaç belge olduğunu zaten bilir; saymak için yalnızca bu indeks bilgisini okumak yeterlidir. Üçüncü kısımda OxiDB’nin, indeksli bir alana göre yalnızca sayan gruplamaları, hiç belge okumadan, doğrudan indeksten yanıtladığını göreceğiz.

## 9.6 Pencere fonksiyonları: satırı yok etmeden hesaplamak

Toplamanın bir de bambaşka bir yüzü vardır ve onu gruplamayla karşılaştırarak anlamak en kolaydır. Gruplama, birçok belgeyi tek bir özete **çökertir**: bin satıştan, her bölge için tek bir ortalama kalır. Ama bazı analizlerde, satırları kaybetmek istemeyiz; her belgeyi olduğu gibi tutmak, ama her birine, **komşularına bakarak** hesaplanmış yeni bir değer eklemek isteriz. İşte bunu yapan araca **pencere fonksiyonu** (window function) denir.

Aradaki farkı bir örnekle netleştirelim. Günlük satışlarınız olsun. Gruplama size “bu ayın toplam satışı” gibi tek bir özet verir — günler kaybolur. Pencere fonksiyonu ise her günü olduğu gibi bırakır, ama her güne yanına bir sütun ekler: “yılbaşından bu güne kadarki kümülatif toplam”, ya da “son yedi günün hareketli ortalaması”, ya da “bu günün satış sıralamasındaki yeri”, ya da “bir önceki günün satışı.” Her satır yerinde durur; yalnızca, komşu satırlardan türetilmiş bir bilgiyle zenginleşir.

Pencere fonksiyonu üç şeyi belirleyerek çalışır. Önce belgeleri **bölmelere** ayırır — örneğin her bölgeyi ayrı bir bölüm olarak. Sonra her bölümü bir alana göre **sıralar** — örneğin tarihe göre. En sonra, her belge için, o belgenin çevresindeki bir **pencereye** (örneğin “baştan bu satıra kadar” ya da “son yedi satır”) bakarak istenen değeri hesaplar. Kümülatif toplam, baştan o satıra kadar olan pencereyi toplar; hareketli ortalama, son birkaç satırın penceresini ortalama; sıralama ise satırın bölüm içindeki yerini söyler. Üçüncü kısımda OxiDB’nin pencere fonksiyonlarını tam da böyle — bölümle, sırala, pencere üzerinde hesapla — gerçekleştirdiğini göreceğiz.

Gruplama ile pencere fonksiyonu, analitik soruların iki büyük ailesidir. Gruplama “her grup için tek bir özet ver” derken, pencere fonksiyonu “her satırı koru, ama ona komşularından türeyen bir değer ekle” der. Finansal raporlardan zaman serisi analizlerine, sıralama tablolarından dönemsel değişim hesaplarına kadar pek çok gerçek analiz, bu ikisinin birinden ya da birleşiminden doğar.

## 9.7 Toplamanın performans doğası

Toplama, tekil aramalardan farklı bir performans karakteri taşır ve bunu bilmek önemlidir. Gruplama ve birçok dönüşüm, doğası gereği çok sayıda belgeye, kimi zaman tüm koleksiyona dokunur. Bu maliyeti yönetmek için iyi bir toplama motoru birkaç şeye dikkat eder. Süzmeyi mümkün olduğunca öne çekerek akışı erkenden daraltır. Aşamaları, tüm veriyi belleğe yığmak yerine, belgeleri tek tek akıtarak işleyebildiği ölçüde **akış halinde** çalıştırır; böylece dev bir koleksiyonu, onu bütünüyle belleğe almadan özetleyebilir. Ve mümkün olan her yerde — saymadaki

indeks kestirmesi gibi — indekslerden yararlanır. Yine de toplamının özünde, “çoğu zaman çok şeye dokunmak” vardır; bu yüzden toplama performansını anlamak, tekil sorgu performansını anlamaktan farklı bir bakış gerektirir.

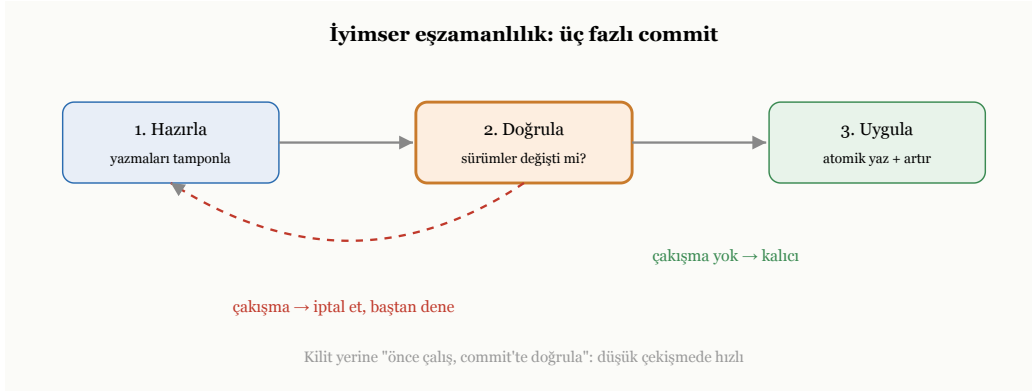
## 9.8 Bu bölümün bıraktığı yer

Bu bölümde, veriye sorabileceğimiz en zengin soru türünü — toplamayı — tanıdık. Toplamanın, belgeleri seçmekle kalmayıp onlardan yeni bilgi damıttığını; pipeline modelinin, karmaşık analizleri basit aşamaları arka arkaya dizerek kurmaya olanak tanıdığını; süzme, gruptama, açma, birleştirme ve çok-yönlü analiz gibi aşamaların ne işe yaradığını; gruptamanın bir anahtara göre biriktiricilerle çalıştığını ve doğası gereği çok şeye dokunduğunu; pencere fonksiyonlarının ise satırları korurken komşulardan değer türettiğini gördük.

Buraya kadar, Kısım II boyunca hep **okumayla** — veriyi saklamak, bulmak, süzmek, özetlemek — ilgilendik. Ama bir veritabanı yalnızca okunmaz; sürekli **yazılır**, hem de çoğu zaman birçok kullanıcı tarafından aynı anda. İşte o an, birinci bölümde değindiğimiz en sinsi sorunlar geri gelir: iki kişi aynı kaydı aynı anda değiştirirse ne olur; bir işlem yarıda kahrırsa tutarlılık nasıl korunur; sistem, bir avuç kullanıcının kaosa sürüklemesine izin vermeden, düzeni nasıl sürdürür? Bir sonraki bölümde, veritabanının belki de en zarif kavramına — işlemlere (transactions) ve onların verdiği “ya hep ya hiç” güvencesine — eğiliyoruz.

## 10 İşlemler: ACID, Yalıtım, Kilitleme, MVCC ve OCC

Kısım II boyunca, dokuzuncu bölümün sonuna kadar, hep okumayla ilgilendik: veriyi saklamak, bulmak, süzmek, özetlemek. Ama bir veritabanı yalnızca okunmaz; sürekli yazılır, hem de çoğu zaman birçok kullanıcı tarafından aynı anda. İşte o an, birinci bölümde değindiğimiz en sinsi sorunlar geri gelir: iki kişi aynı kaydı aynı anda değiştirirse ne olur; bir işlem yarıda kalırsa tutarlılık nasıl korunur; sistem, bir avuç kullanıcının veriyi kaosa sürüklemesine izin vermeden düzeni nasıl sürdürür? Bu bölüm, veritabanının belki de en zarif kavramına — işlemlere — ve onların verdiği güvencelelere eğiliyor.



Şekil 10 — İyimser eşzamanlılıkta üç fazlı commit.

### 10.1 İşlem nedir: bölünmez bir bütün

Birinci bölümdeki para transferi örneğini hatırlayalım: bir hesaptan para düşmek ve diğerine eklemek. Bu iki adım, ayrı ayrı düşünüldüğünde tehlikelidir; çünkü arada bir çökme ya da bir karışıklık, parayı buharlaştırabilir. Asıl istediğimiz, bu iki adımın **tek bir bölünmez bütün** olarak ele alınmasıdır: ya ikisi de olur ya da hiçbiri. İşte bir **işlem** (transaction), tam da budur — birçok işlemi tek bir mantıksal birim olarak gruplayan, “ya hep ya hiç” güvencesi veren bir kavram.

İşlem, kullanıcıya basit bir söz verir: işlemin içindeki adımlar, dışarıdan bakıldığında, tek bir an’da, bölünmeden gerçekleşmiş gibi görünür. Yarıda kalmış, yarısı olmuş bir durum asla görünmez. Bu söz, sandığınızdan çok daha zor tutulur; çünkü hem çökmeye hem de aynı anda çalışan diğer işlemlerle baş etmek gerekir. Bu güvencelerin tümü, geleneksel olarak dört harfle özetlenir: ACID.

### 10.2 Dört güvence: ACID

ACID, bir işlemin verdiği dört sözün İngilizce baş harflerinden oluşur. Bu dördünü tanımak, işlemlerin neyi vaat ettiğini — ve bu bölümün geri kalanında hangi sorunu çözeceğimizi — netleştirir.

**Atomiklik** (atomicity), “ya hep ya hiç” sözüdür: işlemin tüm adımları gerçekleşir ya da hiçbiri gerçekleşmez. Bu güvencenin temeli, altıncı bölümde gördüğümüz yazma-öncesi günlük ve kurtarmadır; işlem yarıda çökse bile, kurtarma ya işlemi tümüyle tamamlar ya da tümüyle geri alır.

**Tutarlılık** (consistency), işlemin veritabanını bir geçerli durumdan başka bir geçerli duruma taşımasıdır; veriye dair kuralların — örneğin “bir hesap bakiyesi eksiye düşemez” — işlem sonunda hâlâ geçerli olmasıdır. Dürüst olmak gerekir ki bu “C”, büyük ölçüde uygulamanın sorumluluğundadır: veritabanı, işlemin atomik ve yalıtılmış olmasını sağlar; ama hangi durumların “geçerli” olduğunu tanımlamak ve gözetmek, çoğunlukla uygulamaya düşer.

**Yalıtım** (isolation), bu bölümün asıl zor konusudur ve birazdan ona uzun uzun eğileceğiz. Kısaca: aynı anda çalışan işlemler birbirinin ayağına basmamalı; sonuç, sanki işlemler sırayla, teker teker çalışmış gibi olmalıdır — gerçekte hepsi iç içe, aynı anda çalışsa bile.

**Dayanıklılık** (durability), altıncı bölümün konusuydu: bir işlem “tamamlandı” dendiğinde, artık hiçbir çökme onu geri alamaz.

Bu dört güvenceden ikisini — atomiklik ve dayanıklılık — daha önce, WAL bölümünde temellendirmiştik. Tutarlılık ise büyük ölçüde uygulamanın işidir. Geriye, gerçekten zor ve bu bölümün kalbinde yatan olan kalır: yalıtım.

### 10.3 Yalıtımın çözdüğü sorun: eşzamanlılık anormallikleri

Yalıtımın neden zor olduğunu anlamak için, onun engellemeye çalıştığı bozulmaları görmek gerekir. Birden çok işlem aynı veriye aynı anda dokunduğunda ortaya çıkabilecek, “anormallik” denen birkaç klasik bozulma vardır.

**Kayıp güncelleme:** İki işlem aynı kaydı okur, ikisi de üzerinde çalışır, sonra sırayla kaydeder. İkincinin kaydı, birincininkini hiç haberi olmadan ezer; birincinin değişikliği kaybolur. Birinci bölümde bu örneği vermiştik; yalıtımın en temel görevi, bunu önlemektir.

**Kirli okuma:** Bir işlem, başka bir işlemin henüz “tamamlandı” dememiş, geçici değişikliğini okur. Eğer o diğer işlem sonradan geri alınırsa, ilk işlem hiç var olmamış bir veriye dayanarak karar vermiş olur.

**Tekrarlanamayan okuma:** Bir işlem aynı kaydı iki kez okur ve arada başka bir işlem o kaydı değiştirdiği için, iki okumada farklı değerler görür. İşlemin gözünde dünya, kendi ortasında değişmiştir.

**Hayalet kayıt:** Bir işlem belirli bir koşula uyan kayıtları sayar; arada başka bir işlem o koşula uyan yeni bir kayıt ekler; ilk işlem aynı sayımı tekrarladığında farklı bir sonuç görür — sanki bir hayalet belirmiştir.

Bu anormalliklerin hepsinin ortak kökü, işlemlerin birbirinin yarım kalmış ya da eşzamanlı çalışmasını “görebilmesidir”. Mükemmel yalıtım, her işleme sanki veritabanında **yalnız kendisi varmış** gibi bir dünya sunmayı amaçlar.

### 10.4 Yalıtım düzeyleri: bir tayf

Mükemmel yalıtım — her işlemin sanki tek başınaymış gibi çalışması — en güçlü güvencedir ve “serileştirilebilirlik” (serializability) diye anılır: sonuç, sanki işlemler bir sıraya dizilip teker teker çalışmış gibi olur. Ama bu güçlü güvence pahalıdır; çünkü işlemlerin birbirini görmesini tümüyle engellemek, eşzamanlılığı ve dolayısıyla performansı kısıtlar.

Bu yüzden veritabanları, yalıtımı bir **tayf** üzerinde sunar. En gevşek uçta, işlemler birbirinin yarım işini görebilir — hızlıdır ama yukarıdaki anormalliklerin çoğuna açıktır. Daha sıkı düzeyler, sırayla kirli okumayı, sonra tekrarlanamayan okumayı, en sonunda hayalet kayıtları da engeller; her sıkı düzey daha güvenlidir ama daha az eşzamanlılığa, yani daha düşük performansa mal olur. Bir veritabanı tasarımcısının ya da kullanıcısının seçimi, çoğu zaman bu tayf üzerinde “ne kadar güvence, ne kadar hız” sorusunu yanıtlamaktır. Mutlak doğru bir nokta yoktur; uygulamanın ne kadar güçlü bir güvenceye gerçekten ihtiyaç duyduğuna bağlıdır.

### 10.5 Yalıtımı sağlamanın üç felsefesi

Peki yalıtım, pratikte nasıl sağlanır? Üç temel felsefe vardır ve her biri, eşzamanlılığa farklı bir tavırla yaklaşır.

### 10.5.1 Kilitleme: kötümser yaklaşım

İlk felsefe **kilitlemedir** ve dünyaya kötümser bakar: “çatışma olacağını varsay, baştan önle.” Bir işlem bir veriye dokunmadan önce, onun üzerinde bir **kilit** alır; o kilit elindeyken, başka hiçbir işlem o veriye dokunamaz, sırada bekler. Tipik olarak iki tür kilit vardır: okuma kilidi (birçok okuyucu aynı anda tutabilir, ama bir yazıcıyı bekletir) ve yazma kilidi (yalnızca bir işleme verilir, herkesi bekletir). Bunu tek şeritli bir köprüye benzetebilirsiniz: köprüde bir anda yalnızca bir araç geçebilir, gerisi bekler.

Kilitleme, güvenliği kesin biçimde sağlar; ama eşzamanlılığı kısıtlar — herkes sırada beklediği için. Üstelik kendine özgü bir tehlike doğurur: **kilitlenme** (deadlock). İki işlem düşünün; birincisi A verisinin kilidini almış, B’yi bekliyor; ikincisi B’nin kilidini almış, A’yı bekliyor. İkisi de ötekinin bırakmasını bekler ve hiçbiri bırakmaz; sonsuza dek donup kalırlar. Veritabanları bu tehlikeyle iki yolla baş eder: ya kilitlenmeyi sezip işlemlerden birini iptal ederek düğümü çözer, ya da kilitlenmenin **hiç oluşmamasını** sağlayacak bir disiplin uygular. En zarif disiplin, kilitleri her zaman aynı, belirli bir sırada almaktır; herkes kilitleri aynı sırayla alırsa, o döngüsel bekleme hiç oluşmaz. Üçüncü kısımda OxiDB’nin tam da böyle, kilitleri sıralı bir düzende alarak kilitlenmeyi baştan imkânsız kıldığını göreceğiz.

### 10.5.2 MVCC: çok sürümlü eşzamanlılık

İkinci felsefe, kilitlemenin “okuyucular ve yazıcılar birbirini bekler” sorununa zarif bir çözüm getirir ve adı **çok sürümlü eşzamanlılık denetimidir** (MVCC). Fikri şudur: bir veriyi değiştirirken eski sürümün üzerine yazma; onun **yeni bir sürümünü** oluştur, eskisini bir süre koru.

Bunun sonucu çarpıcıdır. Bir işlem okumaya başladığında, ona o an’ın bir **anlık görüntüsü** verilir — sanki o an bir fotoğraf çekilmiş gibi. İşlem, çalıştığı süre boyunca hep o tutarlı fotoğrafı görür; başkalarının yaptığı yeni değişiklikler onu etkilemez, çünkü o değişiklikler yeni sürümler yaratır, işlemin gördüğü eski sürüm olduğu gibi durur. Böylece okuyucular yazıcıları, yazıcılar da okuyucuları beklemez: **okuyucu yazıcıyı bloke etmez, yazıcı okuyucuyu bloke etmez**. Bu, kilitlemenin en büyük darboğazını ortadan kaldırır.

MVCC’nin bedeli, eski sürümleri saklamaktır; ve artık hiçbir işlemin görmediği eski sürümleri ara sıra temizlemek gerekir. Burada beşinci bölümle güzel bir bağ kurulur: append-only depolama motorları, veriyi zaten asla üzerine yazmadan, hep yeni sürümler ekleyerek tuttuğu için, MVCC’ye doğal bir zemin sunar — eski sürümler orada zaten vardır.

### 10.5.3 OCC: iyimser yaklaşım

Üçüncü felsefe, kilitlemenin tam tersi bir ruh hâliyle yaklaşır ve adı **iyimser eşzamanlılık denetimidir** (OCC). Varsayımı şudur: çatışmalar aslında nadirdir; çoğu zaman iki işlem aynı veriye aynı anda dokunmaz. Madem öyle, neden baştan kitleyip herkesi bekletelim?

OCC şöyle çalışır. İşlem, hiçbir kilit almadan, serbestçe çalışır; yapacağı değişiklikleri hemen uygulamak yerine bir kenarda **biriktirir**. İş bittiğinde, “tamamla” demeden hemen önce, bir **doğrulama** adımı gelir: işlem, dokunduğu verilerin, kendisi çalışırken başka biri tarafından değiştirilip değiştirilmediğini kontrol eder. Bunu genellikle her veriye iliştilmiş bir **sürüm numarası** ile yapar: işlem veriyi okuduğunda sürüm numarasını hatırlar; tamamlarken o numaranın hâlâ aynı olup olmadığına bakar. Hiçbir şey değişmemişse — ki iyimser varsayıma göre çoğu zaman böyledir — değişiklikler güvenle uygulanır. Ama bir çakışma saptanırsa — yani başka biri o veriyi bu arada değiştirmişse — işlem **iptal edilir** ve baştan denenir.

Bunu, bir mağazada alışverişe benzetebilirsiniz: ürünleri sepete koyarken kimseye sormazsınız (kilit yok); kasaya geldiğinizde, almak istediğiniz şeyin hâlâ uygun olup olmadığı kontrol edilir (doğrulama); bir sorun çıkarsa, o turu baştan yaparsınız (iptal ve yeniden deneme). OCC, çatışmaların nadir olduğu durumlarda muhteşemdir; çünkü hiç

kimse boşuna beklemez. Ama çatışmaların sık olduğu, herkesin aynı veriye saldırdığı durumlarda israftır; çünkü çok sayıda işlem, sona kadar çalışıp sonra iptal edilip yeniden denir. Üçüncü kısımda OxiDB'nin tam olarak bu iyimser yaklaşımı kullandığını — değişiklikleri tamamlanana dek biriktirdiğini, tamamlarken sürüm numaralarını doğruladığını ve çakışma bulursa iptal ettiğini — ayrıntısıyla göreceğiz.

## 10.6 Belge dünyasında işlemler: tek belge kolay, çoğu zor

Dördüncü bölümde, belge modelinde atomikliğin doğal sınırının tek bir belge olduğunu tohumlamıştık; şimdi o tohum meyve veriyor. Belge veritabanlarında, tek bir belgeyi değiştiren bir işlemi atomik kılmak görece kolaydır; çünkü o belge, tek bir bütün olarak yazılır. İşte dördüncü bölümde gömmenin bir avantajı olarak saydığımız şey buydu: ilişkili veriyi tek bir belgede topladığınızda, onları birlikte, tek bir atomik işlemde güncelleyebilirsiniz.

Zorluk, bir işlem **birden çok belgeye** dokunduğunda başlar. O zaman, birkaç ayrı belgenin değişikliğinin hep birlikte ya hep ya hiç gerçekleşmesini güvence altına almak gerekir ki bu, tek belgeye kıyasla daha fazla koordinasyon ister. Veri birden çok makineye dağılmışsa — ki on ikinci bölümün konusu bu olacak — zorluk daha da artar; çünkü artık farklı makinelerdeki değişikliklerin birlikte tamamlanması ya da birlikte geri alınması gerekir. Bunu sağlamak için, işlemin önce tüm taraflara “hazır mısın” diye sorduğu, hepsi “evet” derse “tamamla” dediği iki aşamalı bir mutabakat gibi düzenekler kullanılır; ama bu düzenekler hem yavaştır hem de kendi başına çökme senaryolarıyla doludur.

Buradan, dördüncü bölümdeki tasarım kararının önemi bir kez daha belirir: tutarlı kalması gereken birimi tek bir belgenin içine sığdırabiliyorsanız, işlemler basit ve hızlı kalır; bu birim birçok belgeye yayılıyorsa, daha güçlü ve daha pahalı işlem güvencelerine ihtiyaç duyarsınız.

## 10.7 Bedava güvence yoktur

Bu bölümün her köşesinden aynı ders yükselir: güvence bedavaya gelmez. Daha güçlü yalıtım, daha az eşzamanlılık ve daha düşük performans demektir. Bir işlemin kapsamı genişledikçe — tek belgeden çok belgeye, tek makineden çok makineye — onu tutarlı tutmanın maliyeti artar. Kilitleme güvenlidir ama bekletir; MVCC beklemeyi azaltır ama sürüm saklamayı ve temizlemeyi getirir; OCC çatışma azken uçar ama çatışma çokken israf eder. Doğru seçim, her zaman olduğu gibi, iş yükünüze bağlıdır: çatışmalar ne sıklıkta oluyor, işlemler ne kadar geniş, hangi anormallikleri gerçekten önlemeniz gerekiyor?

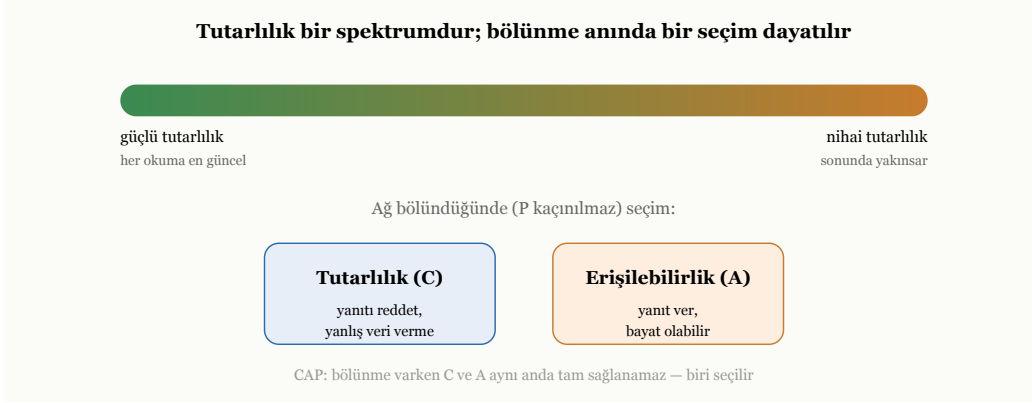
## 10.8 Bu bölümün bıraktığı yer

Bu bölümde, veritabanının eşzamanlı yazmalar ve yarım kalmalar karşısında düzeni nasıl koruduğunu gördük. İşlemin bölünmez bir bütün olduğunu; ACID'in dört güvencesini; yalıtımın engellediği anormallikleri ve yalıtım düzeyleri tayfını; ve yalıtımı sağlamanın üç felsefesini — kötümser kitlemeyi, çok sürümlü MVCC'yi ve iyimser OCC'yi — tanıdık. Tek belgeli işlemlerin kolay, çok belgeli ve dağıtık işlemlerin zor olduğunu ve her güvencenin bir bedeli olduğunu gördük.

Yalıtım, eşzamanlılığın yalnızca bir yüzüdür. Veri tek bir makinede dururken bu kavramlar yeterince zorludur; ama veri birden çok makineye yayıldığında, eşzamanlılık ve tutarlılık tümüyle yeni bir boyut kazanır. Farklı makineler aynı anlık görüntüyü görmeyebilir; bir makinedeki yazma, diğerine henüz ulaşmamış olabilir; “tutarlı” olmanın ne anlama geldiği bile tartışmalı hale gelir. Bir sonraki bölümde, eşzamanlılık ve tutarlılık modellerine — özellikle veri dağıldıkça ortaya çıkan o zorlu seçimlere — daha geniş bir çerçeveden bakacağız.

## 11 Eşzamanlılık ve Tutarlılık

Önceki bölümde yalıtımı — birçok işlemin tek bir makinede birbirinin ayağına basmadan çalışmasını — inceledik. Ama orada hep örtük bir varsayım vardı: verinin tek bir kopyası, tek bir yerde duruyordu. Bu varsayım, gerçek dünyanın büyük sistemlerinde çoğu zaman geçerli değildir. Veri, dayanıklılık, erişilebilirlik ve hız için birden çok makinede, çoğu zaman birden çok kopya halinde tutulur. İşte o an, sandığımızdan çok daha derin bir soru belirir: birden çok kopya varken, “verinin güncel değeri” ne demektir ve “tutarlı olmak” tam olarak neyi ifade eder? Bu bölüm, bu soruyu ele alıyor ve bizi, verinin nasıl dağıtıldığını anlatacak bir sonraki bölüme hazırlıyor.



Şekil 11 — Tutarlılık bir spektrumdur; bölünme anında CAP seçimi dayatılır.

### 11.1 Neden birden çok kopya

Önce, neden tek bir kopyayla yetinmediğimizi kısaca görelim; ayrıntısı bir sonraki bölümün konusu, ama tutarlılık sorununun neden doğduğunu anlamak için gereklidir. Veriyi çoğaltmanın birkaç nedeni vardır. Birincisi **dayanıklılıktır**: tek bir makine, diski bozulursa ya da yanarsa veriyi tümüyle yitirebilir; aynı veriyi birkaç makinede tutarsanız, birinin kaybı felaket olmaktan çıkar. İkincisi **erişilebilirliktir**: bir makine çökse bile, kopyayı tutan diğerleri hizmete devam edebilir. Üçüncüsü **ölçektir**: okuma yükünü birçok kopyaya dağıtarak tek bir makinenin kapasitesinin ötesine geçebilirsiniz. Dördüncüsü **gecikmedir**: kullanıcıya coğrafi olarak yakın bir kopya, uzaktaki bir makineden daha hızlı yanıt verir.

Bu nedenlerin hepsi ikna edicidir ve büyük sistemler bu yüzden kaçınılmaz olarak veriyi çoğaltır. Ama her kopya, bir bedelle gelir ve o bedel, tam da bu bölümün konusudur: kopyalar birbirinden ayrı düşebilir.

### 11.2 Yeni sorun: kopyalar anlaşmazlığa düşebilir

Tek bir kopya varken, “güncel değer” nettir: o tek kopyada ne yazıyorsa odur. Ama birden çok kopya olduğunda, bir an için durup düşünün: bir kopyaya bir yazma yapıldı, ama bu yazma henüz diğer kopyalara ulaşmadı. O anda iki kopyaya bakarsanız, **farklı değerler** görürsünüz. Hangisi “doğru”dur? İkisi de, bir bakıma; biri yeni yazmayı görmüş, diğeri henüz görmemiştir. “Verinin güncel değeri” kavramı, birden çok kopyayla birlikte, bulanıklaşır.

Bunu, aynı bilgiyi tutan birkaç deftere benzetebilirsiniz. Bir deftere yeni bir satır yazdınız; ama o satırı henüz diğer defterlere geçirmediğiniz. Şimdi biri gelip ikinci deftere bakarsa, eski bilgiyi görür. Defterler birbirine eşitlenene kadar, “doğru bilgi” hangi deftere baktığınıza göre değişir. Dağıttık bir veritabanında durum tam olarak budur: kopyalar, eşitlenene kadar, gerçeğin farklı anlık görüntülerini taşır.

İşte tutarlılık, bu anlaşmazlığın ne kadarına izin verileceğiyle ilgilidir. Tutarlılık, bir tayf üzerinde tercih edilen bir özelliktir; bir uçta katı, öteki uçta gevşek davranan sistemler vardır.

### 11.3 Tutarlılık tayfı: katıdan gevşeye

Tayfin bir ucunda **güçlü tutarlılık** (strong consistency) vardır. Güçlü tutarlı bir sistem, sanki tek bir kopya varmış gibi davranır: her okuma, en son yazılmış değeri görür. Bir veriyi yazdıktan sonra, hangi kopyaya bakarsanız bakın, yeni değeri görürsünüz. Bu, üzerine akıl yürütmesi en kolay modeldir — çünkü tıpkı tek makineli dünyadaki gibi davranır. Ama pahalıdır: yeni bir değer tüm kopyalarda geçerli sayılması için, kopyalar arasında **koordinasyon** ve **bekleme** gerekir. Bu koordinasyon, hem yazmaları yavaşlatır hem de — birazdan göreceğimiz gibi — makineler arasındaki iletişim koptuğunda erişilebilirliği tehlikeye atar.

Tayfin öteki ucunda **nihai tutarlılık** (eventual consistency) vardır. Nihai tutarlı bir sistem, kopyaların geçici olarak anlaşmazlığa düşmesine izin verir; yalnızca şunu garanti eder: eğer yazmalar durursa, kopyalar eninde sonunda aynı değere **yakınsar**. Yani bir an için eski veri okuyabilirsiniz, ama sistem arkada sessizce eşitlenmeye devam eder ve sonunda herkes aynı gerçeği görür. Bu model ucuzdur, hızlıdır ve makineler arası iletişim koptuğunda bile hizmete devam edebilir; bedeli, okuyucuların ara sıra eski (bayat) veri görmesi ve uygulamanın buna katlanmak zorunda olmasıdır.

İki uç arasında, pratikte çok işe yarayan ara duraklar vardır. “Kendi yazdığını oku” güvencesi, en azından sizin yaptığınız bir değişikliği sonradan kendinizin göreceğini söyler — başkaları henüz görmese bile. “Tekdüze okuma”, zamanı geriye sarmamayı, yani bir kez yeni bir değer gördükten sonra tekrar eskisine düşmemeyi garanti eder. “Nedensel tutarlılık”, neden-sonuç ilişkisi olan olayların herkese aynı sırada görünmesini sağlar — bir soruya verilen yanıt, soruyu görmeden görünmez. Bu ara modeller, güçlü tutarlılığın tüm maliyetini ödmeden, çıplak nihai tutarlılığın en can sıkıcı tuhaflıklarını giderir.

### 11.4 Bölünme anı ve kaçınılmaz seçim

Dağıtık sistemlerde tutarlılığı anlamanın kalbinde, sade ama derin bir gerçek yatar. Makineler birbiriyle bir ağ üzerinden konuşur ve ağlar bazen kopar. İki grup makine arasındaki iletişim kesildiğinde — buna **bölünme** (partition) denir — sistem ikiye ayrılır; her yarı, diğerinin ne yaptığını bilmez.

İşte tam o an, kaçınılmaz bir seçimle yüzleşirsiniz. Bir yazma isteği geldiğinde, sisteminizin iki seçeneği vardır. Ya isteği kabul edip hizmete devam edersiniz — ama o zaman, diğer yarıdan habersiz olduğunuz için, kopyaların anlaşmazlığa düşmesine, yani tutarsızlığa razı olursunuz. Ya da tutarlılığı korumak için isteği reddedersiniz — “diğer yarıyla konuşamadığım için, doğru olduğundan emin olamadığım bir yanıtı vermem” dersiniz — ama o zaman erişilebilirlikten ödün verirsiniz. Bölünme sırasında, **tutarlılık ile erişilebilirlik arasında** seçim yapmak zorundasınız; ikisine birden sahip olamazsınız. Bu içgörü, dağıtık sistemler kuramının en bilinen sonucudur ve genellikle “CAP” adıyla anılır. Önemli olan, onu doğru anlamaktır: bu bir “her zaman üçten ikisini seç” sloganı değildir; özellikle ağ bölündüğünde, tutarlılık ile erişilebilirlik arasında yapılması gereken bir tercihtir.

Daha az bilinen ama eşit derecede önemli bir incelik vardır: ağ hiç bölünmese bile, tutarlılık bir gecikme bedeliyle gelir. Güçlü tutarlılık daha fazla koordinasyon, daha fazla koordinasyon ise daha fazla bekleme demektir. Yani seçim yalnızca felaket anında değil, sıradan zamanda da karşınızdadır: ne kadar güçlü tutarlılık isterseniz, her işlem o kadar yavaşlar. Tutarlılık, bedava gelmeyen bir lüktür.

## 11.5 Güçlü tutarlılık nasıl sağlanır: otorite ve çoğunluk

Peki birden çok kopyaya rağmen güçlü tutarlılık isteyen bir sistem, bunu nasıl başarır? İki temel fikir vardır ve ikisi de bir sonraki bölümde ayrıntılandırılacak; burada yalnızca özlerini görelim.

Birinci fikir, tek bir **otorite** belirlemektir. Kopyalardan biri “lider” olarak seçilir ve tüm yazmalar önce ondan geçer. Lider, yazmaları belirli bir sıraya koyar ve diğer kopyalara bu sırayla iletir. Böylece, dağınık kopyalar olsa da, yazmaların “doğru sırası” konusunda tek bir karar mercii olur. Bu, anlaşmazlığı önlemenin en sade yoludur.

İkinci ve daha sağlam fikir, **çoğunluk mutabakatıdır**. Bir yazmanın “tamamlandı” sayılması için, tek bir makinenin değil, kopyaların **çoğunluğunun** onu kabul etmesi beklenir. Bir komitenin oy vermesini düşünün: bir kararın geçerli olması için çoğunluğun “evet” demesi gerekir. Çoğunluğun gücü şuradan gelir: herhangi iki çoğunluk, en az bir üyede mutlaka kesişir. Bu yüzden bir yazma çoğunluk tarafından kabul edildiyse ve bir okuma da çoğunluğa danışılıyorsa, okuma o yazmayı mutlaka görür — çünkü kesiştikleri o ortak üye, yazmayı bilmektedir. Çoğunluk mutabakatı, hem güçlü tutarlılığı sağlar hem de azınlığın — örneğin çöken ya da ağdan kopan birkaç makinenin — sistemi yanlış bir karara sürüklemesini engeller. Üçüncü kısımda OxiDB’nin, kümeleme kipinde tam da böyle bir çoğunluk-tabanlı mutabakat protokolü kullandığını göreceğiz.

## 11.6 Ayarlanabilir tutarlılık

Madem tutarlılık bir tayf üzerinde bir tercihtir, neden bu tercihi tek seferde, tüm sistem için sabitleyelim? Birçok modern sistem, tutarlılığı **işlem başına ayarlanabilir** kılar. Aynı veritabanına, bir yazma için “yalnızca bir kopya kabul etsin yeter, hızlı olsun” diyebilir; başka, daha kritik bir yazma için “çoğunluk kabul etmeden tamamlanmış sayma” diyebilirsiniz. Aynı şekilde okumalarda da, “en yakın kopyadan oku, bayat olabilir ama hızlı” ya da “en güncel değeri gör, gerekirse bekle” arasında seçim yapabilirsiniz.

Bu ayarlanabilirlik, üçüncü bölümdeki ilkenin tutarlılıktaki yankısıdır: ihtiyacınız olandan fazlasını ödemeyin. Her yazma çoğunluk onayı beklemek zorunda değildir; her okuma en güncel değeri görmek zorunda değildir. Hangi işlemin ne kadar güçlü bir güvenceye gerçekten ihtiyaç duyduğunu belirler ve gerisinde hızı seçersiniz. Üçüncü kısımda OxiDB’nin dayanıklılık tarafında böyle bir ayar — her yazmada diske zorlayan katı kip ile arada bir zorlayan gevşek kip arasında bir seçim — sunduğunu, ama okuma tarafındaki ince ayar düğmelerinin henüz sınırlı olduğunu dürüstçe ele alacağız.

## 11.7 Tutarlılık ihtiyacı doğruluktan doğar

Bu bölümü bir ilkeyle toparlayalım. Üçüncü bölümde “veri modeli erişim örüntülerini izler” demiştik; tutarlılık için de benzer bir ilke geçerlidir: **tutarlılık ihtiyacı, uygulamanın doğruluk gereksiniminden doğar**. Her uygulama güçlü tutarlılığa muhtaç değildir. Bir sosyal medya beğeni sayısının birkaç saniye geç güncellenmesi kimseyi incitmez; orada nihai tutarlılık fazlasıyla yeterlidir ve getirdiği hız değerlidir. Ama bir banka bakiyesinin ya da bir stok adedinin bayat okunması ciddi sonuçlar doğurabilir; orada güçlü tutarlılık şarttır ve maliyeti haklıdır. Doğru tercih, mutlak değil, uygulamaya özgüdür: yanlış sonucun ne kadar zararlı olduğuna bakar, gereken en zayıf güvenceyi seçer ve gerisinde performansı kazanırsınız.

## 11.8 Bu bölümün bıraktığı yer

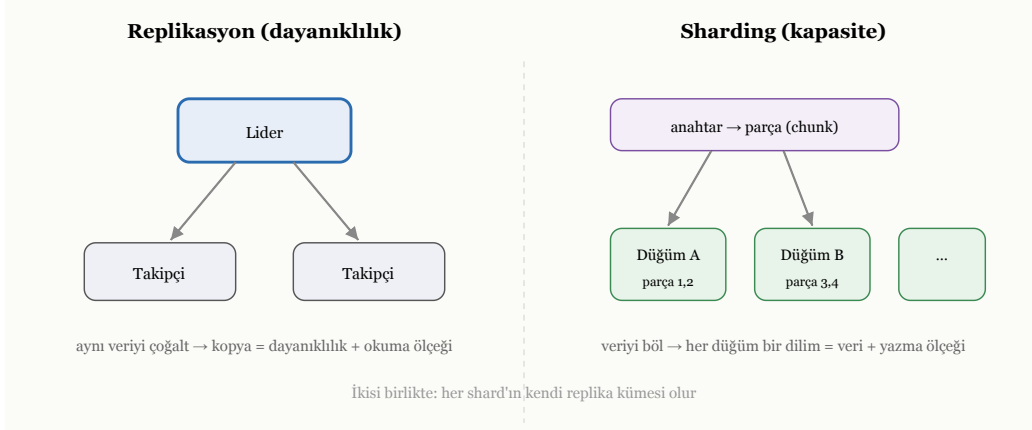
Bu bölümde, eşzamanlılığı tek makinenin ötesine taşıdık ve birden çok kopyayla birlikte gelen tutarlılık sorununu inceledik. Veriyi neden çoğalttığımızı; kopyaların neden anlaşmazlığa düşebildiğini; güçlü tutarlılıktan nihai tutarlılığa uzanan tayfı ve aradaki pratik durakları; ağ bölündüğünde tutarlılık ile erişilebilirlik arasındaki

kaçınılmaz seçimi; güçlü tutarlılığın otorite ve çoğunluk mutabakatıyla nasıl sağlandığını; tutarlılığın işlem başına nasıl ayarlanabildiğini; ve tutarlılık ihtiyacının doğruluk gereksiniminden doğduğunu gördük.

Şimdiye dek hep tutarlılığın **anlamı** üzerinde durduk: kopyalar varken “doğru” ne demektir, hangi güvenceleri seçebiliriz. Henüz konuşmadığımız şey, **mekanizmadır**: veri pratikte birden çok makineye nasıl yayılır, kopyalar nasıl oluşturulup eşitlenir, bir makine çöktüğünde sistem nasıl ayakta kalır ve veri tek bir makineye sığmaz hale geldiğinde nasıl bölünür? Bir sonraki bölümde, ölçeklendirmenin iki büyük tekniğine — replikasyona ve sharding’e — ve bu bölümde tanıdığımız tutarlılık tercihlerini hayata geçiren düzeneklere eğiliyoruz.

## 12 Ölçeklendirme: Replikasyon, Konsensüs ve Sharding

Önceki bölümde, birden çok kopya varken tutarlılığın ne anlama geldiğini — yani işin **anlamını** — inceledik. Ama o kopyaların pratikte nasıl oluşturulduğunu, eşitlendiğini ve bir makine çöktüğünde sistemin nasıl ayakta kaldığını henüz konuşmadık. Bu bölüm, ölçeklendirmenin **mekanizmasını** ele alıyor: veriyi birden çok makineye yaymanın iki büyük tekniğini — replikasyonu ve sharding'i — ve bir önceki bölümde tanıdığımız tutarlılık tercihlerini hayata geçiren düzenekleri. Bu, bir veritabanını tek bir makinenin sınırlarının ötesine taşıyan yetenektir.



Şekil 12 — Replikasyon ve sharding: dayanıklılık ile kapasitenin iki ayrı eksenini.

### 12.1 Neden tek makine yetmez

Bir veritabanı, uzun süre tek bir makinede mutlu yaşayabilir. Sorunlar, üç sınırdan birine dayandığında başlar. Birincisi **kapasitedir**: veri, tek bir makinenin diskine sığmayacak kadar büyüyebilir. İkincisi **iş hacmidir**: gelen istekler, tek bir makinenin işleyebileceğinden fazla olabilir. Üçüncüsü **erişilebilirliktir**: tek makine çökerse, tüm sistem birlikte çöker.

Bu sınırlara iki tür yanıt vardır. Birincisi **dikey ölçeklendirme**: makineyi büyütmek — daha çok bellek, daha hızlı disk, daha çok işlemci. Bu, bir yere kadar işe yarar; ama her zaman bir tavan vardır ve o tavana yaklaştıkça maliyet fırlar. İkincisi **yatay ölçeklendirme**: tek bir dev makine yerine, çok sayıda sıradan makineyi birlikte çalıştırmak. Yatay ölçeklendirmenin tavanı çok daha yüksektir, ama bir bedeli vardır: artık birden çok makineyi koordine etmek zorundasınız ve önceki bölümün tüm tutarlılık zorlukları kapıdadır. Bu bölümdeki iki teknik — replikasyon ve sharding — yatay ölçeklendirmenin iki ayağıdır.

### 12.2 Replikasyon: aynı verinin kopyaları

**Replikasyon**, aynı veriyi birden çok makinede kopya halinde tutmaktır. Önceki bölümde *neden* çoğalttığımızı saymıştık — dayanıklılık, erişilebilirlik, okuma ölçeği, gecikme. Şimdi *nasıl* sorusuna geliyoruz.

En yaygın düzen, **lider-takipçi** modelidir. Kopyalardan biri **lider** olarak belirlenir ve tüm yazmalar ondan geçer. Lider, her değişikliği **takipçi** kopyalara iletir; takipçiler bu değişiklikleri kendi kopyalarına uygular. Okumalar ise hem liderden hem de takipçilerden karşılanabilir; böylece okuma yükü birçok makineye dağılır. Bunu, bir ana şube ile ona bağlı yan şubeleri olan bir kütüphaneye benzetebilirsiniz: yeni kitaplar ana şubeye gelir ve oradan yan şubelere dağıtılır; okuyucular ise en yakın şubeden hizmet alır.

Burada kritik bir tercih, liderin değişikliği takipçilere **ne zaman** kabul edilmiş saydığıdır. **Eşzamanlı** replikasyonda lider, bir yazmayı “tamamlandı” demeden önce takipçilerin onu aldığını bekler; bu güvenlidir — lider çökse bile veri takipçide vardır — ama yavaştır, çünkü her yazma takipçileri beklemek zorundadır. **Eşzamansız** replikasyonda lider beklemez; yazmayı hemen kabul eder ve takipçilere sonradan, kendi hızında iletir; bu hızlıdır, ama bir risk taşır: lider, bir değişikliği takipçilere iletmeden çökerse, o değişiklik kaybolabilir. Bu, doğrudan önceki bölümdeki güçlü-nihai tutarlılık tayfının replikasyondaki yüzüdür.

### 12.3 Lider çökünce: failover ve iki büyük tehlike

Lider-takipçi modelinin asıl sınavı, **lider çöktüğünde** verilir. Sistem ayakta kalmak için, takipçilerden birini yeni lider olarak yükseltmelidir; bu sürece **failover** (devralma) denir. Kulağa basit gelir, ama iki sinsi tehlike içerir.

Birinci tehlike **kayıp yazmadır**: eğer replikasyon eşzamansızsa, yeni lider olacak takipçi, eski liderin son birkaç değişikliğini henüz almamış olabilir. O takipçi lider olunca, o değişiklikler kalıcı olarak yitirilir. İkinci ve daha tehlikeli olanı **ikiye bölünmüş beyindir** (split-brain): eski lider aslında ölmemiş, yalnızca ağdan kopmuşsa ve bu sırada bir takipçi de kendini yeni lider ilan etmişse, ortada **iki lider** belirir. İkisi de yazma kabul eder ve veri iki ayrı, çelişkili gerçeğe ayrılır; bu, onarılması son derece zor bir bozulmadır.

Bu iki tehlike, basit lider-takipçi modelinin yetmediği yeri gösterir. Asıl soru şudur: yeni lidere **kim** karar verir ve aynı anda iki liderin ortaya çıkmasını ne **engeller**? Bu soruların yanıtı, önceki bölümde tohumladığımız bir fikirde yatar: çoğunluk mutabakatı.

### 12.4 Konsensüs: çoğunlukla anlaşmak

**Konsensüs**, bir grup makinenin, bazıları çökse ya da ağdan kopsa bile, ortak bir karar üzerinde güvenle anlaşmasını sağlayan mekanizmadır. Modern dağıtık veritabanlarının çoğu, bu işi çoğunluk oylamasına dayanan bir protokolle yapar; en yaygın olanlardan birinin fikrini burada özetleyeceğiz.

Mantık şöyle işler. Makineler, yapılacak değişikliklerin **sıralı bir günlüğünü** üzerinde anlaşmaya çalışır. Bir değişikliğin günlüğe işlenmiş, yani “tamamlanmış” sayılması için, makinelerin **çoğunluğunun** onu kalıcı olarak kaydetmiş olması gerekir. Aynı şekilde, bir makinenin lider olabilmesi için de çoğunluğun oyunu alması gerekir. Çoğunluğun büyüü, önceki bölümde değindiğimiz şu gerçekten gelir: herhangi iki çoğunluk, en az bir üyede mutlaka kesişir. Bu yüzden iki makine aynı anda lider olamaz — çünkü her ikisinin de çoğunluk oyu alması, en az bir makinenin ikisine birden oy vermesini gerektirir ki bu imkânsızdır. Böylece split-brain, tasarım gereği önlenmiş olur. Aynı kesişme özelliği, kayıp yazmayı da engeller: bir değişiklik çoğunluk tarafından kaydedildiyse, yeni lider seçilirken oy veren çoğunluk o değişikliği bilen en az bir makineyi içerir; dolayısıyla yeni lider o değişikliği asla kaybetmez.

Konsensüsün gücü buradadır: tek bir protokolle hem güçlü tutarlılığı, hem otomatik failover’ı, hem de split-brain güvenliğini birlikte sağlar. Bedeli ise şudur: her yazma, çoğunluğa ulaşp onların onayını beklemek zorundadır — yani bir gidiş-dönüş gecikmesi öder. Ayrıca, çoğunluğun anlamlı olması için yeterli sayıda makine gerekir; tipik olarak tek sayıda makine kullanılır ki “çoğunluk” net tanımlı olsun. Böyle bir küme, makinelerin azınlığının çökmesine dayanır: örneğin beş makineli bir küme, ikisini birden kaybetse bile çalışmaya devam eder, çünkü kalan üç makine hâlâ çoğunluktur. Üçüncü kısımda OxiDB’nin, kümeleme kipinde tam da böyle bir çoğunluk-tabanlı konsensüs protokolü kullandığını ve lider seçimini, replikasyonu, failover’ı ve azınlığın lider seçememesini nasıl sağladığını ayrıntısıyla göreceğiz.

## 12.5 Sharding: veriyi bölmek

Replikasyon, aynı veriyi çoğaltarak erişilebilirlik ve okuma ölçeği sağlar; ama tek başına **kapasite** sorununu çözmez — her kopya yine tüm veriyi taşımak zorundadır. Veri, tek bir makinenin diskine sığmayacak kadar büyüdüğünde ya da yazma hacmi tek bir liderin kaldıramayacağı düzeye çıktığında, bambaşka bir tekniğe ihtiyaç vardır: **sharding** (parçalama). Sharding, veri kümesini makineler arasında **bölmektir**; her makine verinin yalnızca bir dilimini tutar. Böylece hem toplam kapasite hem de toplam yazma hacmi, makine sayısı ile birlikte büyür.

Sharding'in kalbindeki soru şudur: bir belgenin **hangi** makineye gideceğine nasıl karar verilir? Bu kararı veren alana **parça anahtarı** (shard key) denir — belgenin, hangi dilime ait olduğunu belirleyen bir alanı. Anahtarı dilimlere eşlemenin başlıca iki stratejisi vardır.

Birincisi **aralık bölümlemesidir**: anahtarın değer aralıklarına göre bölmek. Bir telefon rehberini A–H, I–P, R–Z diye üçe ayırmak gibi. Bu, aralık sorgularında iyidir — yakın değerler aynı makinededir — ama bir tehlike taşır: eğer yazmalar belirli bir aralıkta yoğunlaşırsa (örneğin son gelen kayıtlar hep en büyük anahtarı alıyorsa), o makine “sıcak nokta” haline gelip aşırı yüklenir, diğerleri boş durur. İkincisi **karma bölümlemesidir**: anahtarı bir karma fonksiyonundan geçirip sonuca göre dilim seçmek. Bu, kayıtları makinelere son derece dengeli dağıtır — sıcak nokta riski azalır — ama aralık yerelliğini kaybeder: yakın değerler farklı makinelere düşer, dolayısıyla aralık sorguları tüm makinelere yayılmak zorunda kalır.

Çoğu sistem, zarif bir dolaylama katmanı ekler. Anahtarları doğrudan makinelere değil, çok sayıda **sanal parçaya** eşler; sonra bu sanal parçaları makinelere dağıtır. Bunun yararı, yeniden dengelemede ortaya çıkar: yeni bir makine eklendiğinde, anahtarları tek tek taşımak yerine, yalnızca birkaç sanal parçayı bir makineden diğerine kaydırmak yeterlidir. Üçüncü kısımda OxiDB'nin sharding katmanının tam da böyle bir sanal-parça eşlemesi kullandığını göreceğiz.

## 12.6 Yönlendirme ve parçalar-arası sorgu

Veri makinelere bölününce, gelen her istek **doğru** makineye gönderilmelidir. Bunu yapan bileşene **yönlendirici** (router) denir — çoğu zaman bir aracı (proxy) olarak çalışır. Yönlendirici, isteğe bakar, parça anahtarını çıkarır, hangi dilime ait olduğunu hesaplar ve isteği o dilimi tutan makineye iletir. Bunu, doğru masaya yönlendiren bir resepsiyona benzetebilirsiniz: anahtarınızı söylersiniz, sizi ait olduğunuz masaya gönderir.

Ama bir sorun vardır. Ya istek, parça anahtarını içermiyorsa? Ya da bir toplama sorgusu, tüm veriye birden bakmayı gerektiriyorsa? O zaman yönlendirici, isteği **tüm** makinelere göndermek, her birinden kısmi yanıtı toplamak ve bunları birleştirmek zorunda kalır. Bu örüntüye **dağıt-topla** (scatter-gather) denir. Bir sayım sorgusunu düşünün: yönlendirici, her dilime “sende kaç tane var” diye sorar, gelen sayıları toplar. Bir gruplama sorgusunda ise her dilim kendi kısmi gruplamasını yapar, yönlendirici bu kısmi sonuçları birleştirip nihai sonucu üretir. Üçüncü kısımda OxiDB'nin parçalar-arası toplama sorgularını tam olarak böyle, her dilimde kısmi hesap yapıp sonra birleştirerek — hem de tek-düğümle birebir aynı sonucu verecek biçimde — yürüttüğünü göreceğiz.

Burada, üçüncü bölümde attığımız bir tohum meyve verir. Kendi içinde bütün olan belgeler sharding'e doğal yatkındır: her belge bağımsız olduğu için, hangi dilime gideceğine kolayca karar verilir ve onu okumak için başka dilimlere bakmak gerekmez. Buna karşılık, yoğun biçimde birbirine bağlı veri — her yönden sorgulanan çok-çok ilişkiler — sharding'de zorlanır; çünkü ilişkili parçalar farklı makinelere düşebilir ve onları bir araya getirmek pahalı, parçalar-arası işlemler ise son derece çetin hale gelir.

## 12.7 İyi bir parça anahtarı ve yeniden dengelemenin yükü

Sharding'in başarısı, büyük ölçüde **parça anahtarının** iyi seçilmesine bağlıdır. İyi bir parça anahtarı üç özelliğe sahiptir. **Yüksek çeşitliliklidir**: çok sayıda farklı değeri vardır, böylece veri ince ince dağılabilir. **Erişimi dengelidir**: hiçbir değer ya da aralık, yükün orantısız bir kısmını üstlenmez; yani sıcak nokta oluşturmaz. Ve **yaygın sorgularla hizalıdır**: en sık yapılan sorgular, bu anahtarı içerir, böylece tüm dilimlere yayılmak yerine tek bir dilimi hedefler. Kötü seçilmiş bir parça anahtarı — örneğin çok az değeri olan ya da yükü tek bir makineye yığan bir alan — sharding'in tüm yararını yok edebilir.

Sharding'in göz ardı edilen bir maliyeti de **yeniden dengelemedir**. Veri büyüdükçe ya da makine eklendikçe çıkarıldıkça, dilimlerin makineler arasında yeniden dağıtılması gerekir; bu, hareket halindeki veriyi tutarlı tutarken yapılması gereken, hassas bir operasyondur. Olgun sistemler, bu dengelemeyi otomatik yöneten bir bileşen barındırır; ama bu otomatik dengeleme, kurması ve doğru işletmesi karmaşık bir yetenektir. Üçüncü kısımda OxiDB'nin sharding katmanını ele alırken, hangi parçaların manuel yapılandırıldığını ve hangi otomasyon düzeylerinin henüz tasarımda olmadığını dürüstçe değerlendireceğiz.

## 12.8 İkisini birleştirmek

Replikasyon ile sharding, birbirinin alternatifi değildir; farklı sorunları çözerler ve büyük sistemlerde **birlikte** kullanılırlar. Tipik bir büyük ölçek topolojisi şöyledir: veri, kapasite ve yazma hacmi için birçok dilime **bölünür** (sharding); ve sonra her dilim, erişilebilirlik ve dayanıklılık için kendi içinde **çoğaltılır** (replikasyon). Böylece sistem hem tek makineye sığmayan veriyi taşır, hem de herhangi bir makinenin çökmesine dayanır. Bu iki tekniğin birleşimi, modern büyük ölçekli veri sistemlerinin standart iskeletidir.

## 12.9 Yine aynı ders: dağıtım bedava değildir

Bu bölüm de, kitap boyunca tekrarlanan o dersi yeniden söyler. Dağıtım, kapasite ve erişilebilirlik satın alır; ama bunu koordinasyon, karmaşıklık ve sorgu kısıtlarıyla öder. Replikasyon, tutarlılık ile gecikme arasında bir tercih dayatır. Konsensüs, güçlü güvenceler verir ama her yazmaya bir çoğunluk gidiş- dönüşü ekler. Sharding, kapasiteyi büyütür ama parçalar-arası sorguları ve işlemleri zorlaştırır ve iyi bir parça anahtarı seçme yükü getirir. Tek bir makinede çalışan basit bir veritabanı, çoğu zaman, dağıtık bir sistemden çok daha az karmaşıktır ve çok daha kolay akıl yürütülür. Bu yüzden bilge bir tasarımcı, dağıtımı bir varsayılan değil, gerçekten ihtiyaç doğduğunda başvurulacak bir araç olarak görür.

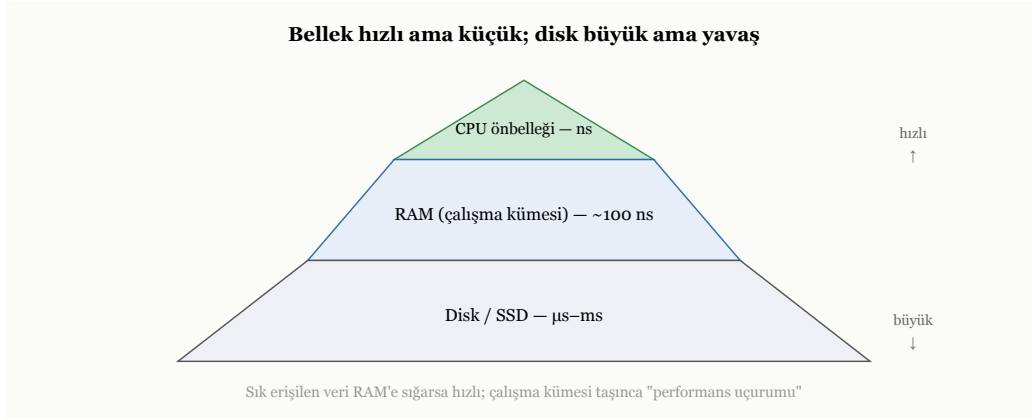
## 12.10 Bu bölümün bıraktığı yer

Bu bölümde, bir veritabanını tek makinenin ötesine taşıyan iki tekniği tanıdık. Replikasyonun aynı veriyi çoğaltarak erişilebilirlik ve okuma ölçeği sağladığını; lider-takipçi modelini, eşzamanlı ve eşzamansız replikasyonu, failover'ın tehlikelerini ve konsensüsün — çoğunluk mutabakatının — bu tehlikeleri nasıl çözdüğünü gördük. Sharding'in veriyi bölerek kapasite ve yazma ölçeği kazandırdığını; parça anahtarını, bölümlene stratejilerini, yönlendirmeyi, dağıt-topla örüntüsünü ve iyi bir anahtar seçmenin önemini öğrendik. Ve ikisinin büyük sistemlerde nasıl birleştirildiğini gördük.

Şimdiye dek hep birden çok makine arasındaki koordinasyondan söz ettik. Ama her bir makinenin, kendi içinde de çözmesi gereken bir kaynak yönetimi sorunu vardır. Beşinci bölümde değinmiştik: bellek hızlı ama küçük ve ucuca, disk yavaş ama büyük ve kalıcıdır. Tek bir düğüm, hangi veriyi bellekte tutacağına, hangisini diske bırakacağına nasıl karar verir; sınırlı belleğini en çok işe yarayacak veriyle nasıl doldurur? Bir sonraki bölümde, bu sessiz ama performansı belirleyen dengeye — bellek, önbellek ve disk arasındaki ödünleşime — eğiliyoruz.

## 13 Bellek, Önbellek ve Disk Ödünleşimi

Önceki iki bölümde, bir veritabanını birçok makineye yaymanın koordinasyon sorunlarıyla uğraştık. Şimdi bakışımızı yeniden tek bir makineye, tek bir düğümün içine çeviriyoruz; çünkü her düğüm, kendi içinde de çözmesi gereken bir kaynak yönetimi sorunu taşır. Beşinci bölümde tohumlamıştık: bellek hızlı ama küçük ve uçucu, disk yavaş ama büyük ve kalıcıdır. Bir düğümün performansının büyük bölümü, tek bir karara dayanır: hangi verinin bellekte tutulacağı, hangisinin diske bırakılacağı. Bu bölüm, o sessiz ama belirleyici dengeyi — bellek, önbellek ve disk arasındaki ödünleşimi — ele alıyor.



Şekil 13 — Bellek hiyerarşisi ve çalışma kümesi: hız ile boyutun ödünleşimi.

### 13.1 Temel asimetri ve oyunun özü

Önce, kitap boyunca birkaç kez değindiğimiz asimetriyi netleştirelim, çünkü bu bölümdeki her şey ondan doğar. Bellek çok hızlıdır ama pahalıdır, sınırlıdır ve elektrik gidince içeriğini kaybeder. Disk çok yavaştır ama ucuzdur, büyüktür ve kalıcıdır. Bu ikisini, bir masa ile bir depoya benzetebilirsiniz: masanız (bellek) küçüktür ama üzerindeki her şeye anında uzanırsınız; depo (disk) kocamandır ama oradan bir şey getirmek zaman alır.

Oyunun özü şudur: çalışmak için ihtiyaç duyduğunuz şeyleri masanın üstünde tutmak, gerisini depoda bırakmak. Bir veritabanının tek bir düğümdeki hızı, asıl olarak bu kararı ne kadar iyi verdiğine bağlıdır. Doğru veri bellekteyse, sistem uçar; yanlış veri bellekteyse ve aradığınız sürekli diskten getirilmek zorundaysa, sistem sürünür. Bütün mesele, sınırlı belleği en çok işe yarayacak veriyle doldurmaktır.

### 13.2 Çalışma kümesi ve uçurum etkisi

Bu kararı anlamanın anahtarı, **çalışma kümesi** (working set) kavramıdır. Herhangi bir anda, verinin yalnızca bir kısmı “sıcaktır” — yani etkin biçimde kullanılmaktadır. Bir e-ticaret sitesinde, o anki kampanyadaki ürünler, son siparişler, aktif kullanıcılar sıcaktır; yıllar önceki kayıtlar ise soğuk, nadiren dokunulan veridir. Çalışma kümesi, işte bu sıcak kısımdır.

Buradan çarpıcı bir gerçek doğar: performans, kademeli bir eğim değil, bir **uçurumdur**. Çalışma kümeniz belleğe sığdığı sürece, sistem hızlıdır — ihtiyaç duyduğunuz her şey zaten masanın üstündedir. Ama çalışma kümesi belleği biraz olsun aşmaya başladığında, sistem aniden çöker; çünkü artık sürekli olarak depoya koşturmak, bir şeyi getirip masaya koymak için başka bir şeyi geri kaldırmak, sonra onu da geri istemek zorunda kalırsınız. Bu sürekli oraya buraya taşıma durumuna **debelenme** (thrashing) denir ve performansı dramatik biçimde düşürür. Bu yüzden bir

veritabanını boyutlandırırken sorulacak en kritik soru, “tüm verim belleğe sığar mı” değil, “**çalışma kümem** belleğe sığar mı” sorusudur.

### 13.3 Önbellek: belleği akılcıca kullanmak

Bir düğüm, sıcak veriyi bellekte tutmak için bir **önbellek** (cache) işletir. Mantığı basittir. Bir veri istendiğinde, sistem önce önbellege bakar. Veri orada varsa — buna “isabet” (hit) denir — hızlıca, diske hiç gitmeden döndürülür. Yoksa — buna “ıska” (miss) denir — diskten getirilir, kullanıcıya verilir ve gelecekte yine istenebileceği umuduyla önbellege yerleştirilir. Önbellek, böylece uygulama ile yavaş disk arasında bir tampon görevi görür; sık istenen veriyi belleğe çekerek disk erişimini en aza indirir.

Önbellek tek bir katmanda olmak zorunda değildir. Sistem, ham disk bloklarını önbellekleyebilir — böylece aynı bloğu tekrar tekrar diskten okumaktan kurtulur. Ya da çözülmüş, kullanıma hazır belgeleri önbellekleyebilir — böylece aynı belgeyi her seferinde yeniden ayrıştırmaktan kurtulur. Ayrıca indeks yapıları da bellekte tutulur, çünkü onlar neredeyse her sorguda kullanılır. Farklı katmanları önbellemek, farklı türden tekrar eden maliyetleri ortadan kaldırır. Üçüncü kısımda OxiDB’nin hem çözülmüş belgeler hem de ham baytlar için ayrı önbellekler tuttuğunu göreceğiz.

### 13.4 Tahliye: yer açmak için ne atılır

Önbellek sonludur; eninde sonunda dolar. Dolduğunda, yeni bir şeyi içeri almak için eskilerden birini **atmak** (tahliye etmek) gerekir. Hangi verinin atılacağına karar veren kurala **tahliye politikası** denir ve bu, önbelletin ne kadar işe yarayacağını doğrudan belirler.

En yaygın politika, “en uzun süredir dokunulmayanı at” ilkesidir — kısaca LRU. Mantığı bir bahse dayanır: yakın zamanda kullandığınız bir şeyi, yine yakında kullanma olasılığınız yüksektir; çok uzun süredir dokunmadığınız bir şeyi ise muhtemelen bir süre daha kullanmayacaksınız. Bu yüzden LRU, en uzun süredir boşta duranı kurban seçer. Bir kütüphanecinin, sık istenen kitapları el arabasında yakınında tutup, aylardır kimsenin sormadıklarını rafa kaldırmasına benzer.

LRU çoğu zaman iyi çalışır, ama bir zayıflığı vardır. Büyük bir tarama düşünün — örneğin tüm koleksiyonu bir kez baştan sona okuyan bir toplama sorgusu. Bu tarama, gerçekte sıcak olmayan bir sürü veriyi önbellege doldurur ve bu sırada asıl sıcak veriyi dışarı atar. Tarama bittiğinde önbellek, bir daha kullanılmayacak soğuk veriyle dolu, asıl ihtiyaç duyulan sıcak veriden ise yoksun kalmıştır. Buna **önbellek kirlenmesi** denir. Bu yüzden olgun sistemler, büyük taramaların önbellege kirlenmesini önleyen, “tarama-dirençli” politikalar kullanır. Bunun çeşitli incelikleri vardır; ama temel ders şudur: iyi bir tahliye politikası, yalnızca “ne zaman kullanıldı” değil, “gerçekten sıcak mı” sorusunu da gözetmeye çalışır.

### 13.5 Önbelletin kendi maliyeti ve sınırlama zorunluluğu

Önbellek bedava bir kazanç değildir; kendi maliyetleri vardır. Önbellege ayrılan bellek, başka hiçbir işe ayrılamayan bellektir. Önbellege yönetmenin — neyin ne zaman kullanıldığını takip etmenin — kendi küçük yükü vardır. Ve en tehlikelisi: eğer bir önbellek sınırsız büyümeye bırakılırsa, tüm belleği yiyip bitirebilir ve sistemi belleksiz bırakıp çökertebilir.

Bu yüzden ciddi sistemler, önbelleklerini bir **bellek bütçesiyle sınırlar**: önbellek, belirli bir boyutu aşamaz; o boyuta ulaştığında, yeni bir şey almak için mutlaka eskisini atar. Böylece bellek kullanımı öngörülebilir kalır ve denetimden çıkmaz. Üçüncü kısımda OxiDB’nin, bellek kullanımının kontrolden çıkmasını önlemek için önbelleklerini tam da böyle sabit bir bütçeyle sınırladığını ve bunun bellek ayak izini nasıl dizginlediğini göreceğiz.

## 13.6 İki felsefe: belleğe öncelikli ve diske öncelikli

Beşinci bölümde, depolama motorlarının iki felsefesini görmüştük; bellek-disk ilişkisinde de benzer bir ikilik vardır ve doğrudan o bölümle bağlanır.

Birinci felsefe **belleğe önceliklidir**: tüm veriyi bellekte tutmak, diski yalnızca kalıcılık için — yani anlık görüntüler ve günlük için — kullanmak. Bu yaklaşımda her okuma, zaten bellekte olan veriye dokunduğu için olağanüstü hızlıdır. Bedeli, belleğin **veriyle birlikte büyümesidir**: veri ikiye katlanınca, gereken bellek de ikiye katlanır. Bu, küçük ve orta ölçekli veri için harikadır, ama veri büyüdükçe bellek hem pahalı hem de bir tavana dayanan bir kısıt haline gelir. Üçüncü kısımda OxiDB'nin varsayılan kipinin tam da bu belleğe öncelikli yaklaşım olduğunu göreceğiz.

İkinci felsefe **diske önceliklidir**: bellekte yalnızca küçük bir şey — örneğin bir kimlik-konum dizini ve sıcak bir alt küme — tutmak, verinin asıl gövdesini diskte bırakmak ve gerektiğinde oradan getirmek. Bu yaklaşımda bellek kullanımı, veriyle birlikte büyümeyiz; öngörülebilir ve düşük kalır. Bedeli, soğuk veriye erişimin diske gitmeyi, yani bir gecikmeyi göze almasıdır. Bu, belleğin değerli ya da verinin çok büyük olduğu durumlar için biçilmiş kaftandır. Üçüncü kısımda OxiDB'nin “disk-öncelikli” kipinin tam olarak bunu yaptığını — taze açılışta tüm veri yerine yalnızca küçük bir dizinin bellekte durduğunu — ve bunun bellek ayak izini nasıl dramatik biçimde küçülttüğünü ayrıntısıyla göreceğiz.

## 13.7 İşletim sistemine güvenmek: belleğe yansıtma

Bellek yönetimini elle yapmanın zarif bir alternatifi vardır ve beşinci bölümde ona değinmiştik: bir dosyayı **belleğe yansıtma** (mmap). Bu teknikte, dosyaya erişmek bellekteki bir adrese erişmek kadar basit hale gelir ve verinin diskten belleğe ne zaman getirileceğine, bellek darda kaldığında neyin geri atılacağına işletim sistemi karar verir.

Bunun güzelliği, önbellek yönetiminin büyük bölümünü işletim sistemine **devretmesidir**. İşletim sistemi zaten dosya sayfalarını bellekte tutan, sıcakları saklayıp soğukları atan olgun bir mekanizmaya sahiptir; mmap, bu mekanizmadan doğrudan yararlanır. Sıcak dosya sayfaları bellekte kalır, soğuklar otomatik olarak atılır ve önemlisi, bunlar disk üzerinde bir kopyaya sahip oldukları için bellek baskı altındayken **geri alınabilir** — yani gerektiğinde serbest bırakılabilir.

Ama bu kolaylığın bir inceliği vardır ve onu görmek önemlidir. Belleğe yansıtılan sayfalar, dokunulduklarında bellekte yer kaplar ve sistemin “bu süreç ne kadar bellek kullanıyor” ölçümüne dahil olur. Bu yüzden, diske öncelikli, mmap'e dayanan bir sistemin bellek kullanımı yanıltıcı görünebilir: taze açılışta çok düşükken, tüm veriye dokunan büyük bir taramadan sonra yükselebilir — çünkü işletim sistemi o sayfaları belleğe çekmiştir. Ama bu yükselen bellek, geri alınabilir bir bellektir; baskı altında işletim sistemi onu sessizce serbest bırakır. “Veritabanım ne kadar bellek kullanıyor” sorusunun, sandığınızdan çok daha incelikli olmasının bir nedeni budur: kalıcı (anonim) bellek ile geri alınabilir (dosya destekli) belleği ayırmak gerekir. Üçüncü kısımda OxiDB'nin disk-öncelikli kipinde tam da bu olguyu — taze açılışta çok düşük, tam iş yükünden sonra yükselen ama geri alınabilir bellek — ölçümlerle göreceğiz.

## 13.8 Sıkıştırma: yer, işlemci ve sıfır-kopya üçgeni

Belleği ve diski yönetmenin bir aracı daha vardır: **sıkıştırma**. Veriyi sıkıştırarak saklamak, hem diskte hem bellekte daha az yer kaplamasını sağlar; böylece aynı belleğe daha çok veri sığar, çalışma kümesinin sığma olasılığı artar. Ama sıkıştırma bedava değildir: sıkıştırılmış veriyi okumak için, her erişimde onu **açmak** gerekir ve bu, işlemci zamanı harcar.

Burada üç yönlü bir ödünleşim belirir: yer, işlemci ve bir üçüncü incelik. Veri sıkıştırılmamışsa ve şifrelenmemişse, onu okumak için hiçbir dönüşüm gerekmez; bellekteki ham baytlara doğrudan, hiç kopyalamadan dokunulabilir —

buna **sıfır-kopya** erişim denir ve son derece hızlıdır. Veri sıkıştırılmışsa, bu sıfır-kopya olanağı kaybolur; her okuma bir açma ve bir kopyalama gerektirir. Dolayısıyla sıkıştırma, yer kazandırırken okuma hızından ödün verebilir. Hangisinin kazandığı, veriye bağlıdır: gerçekten sıkışan, çok okunmayan veri için sıkıştırma kazandırır; az sıkışan ama çok ve hızlı okunan veri için sıkıştırmamak daha iyidir. Üçüncü kısımda OxiDB'nin disk-öncelikli kipinde, sıkıştırılmış ve sıkıştırılmamış depolama arasındaki bu tam ödünleşimi ölçtüğünü ve belirli yüklerde sıkıştırmayı kapatmanın taramaları nasıl hızlandırdığını göreceğiz.

### 13.9 Bu bölümün bıraktığı yer

Bu bölümde, tek bir düğümün performansını belirleyen sessiz dengeyi — bellek, önbellek ve disk arasındaki ödünleşimi — inceledik. Bellek ile diskin temel asimetrisini; çalışma kümesi kavramını ve onun belleğe sığıp sığmamasının yarattığı uçurum etkisini; önbelleğin nasıl çalıştığını ve tahliye politikalarının (özellikle LRU'nun ve önbellek kirlenmesinin) inceliklerini; önbelleğin kendi maliyetini ve sınırlama zorunluluğunu; belleğe öncelikli ve diske öncelikli iki felsefeyi; belleğe yansıtmanın işletim sistemine devrettiği kolaylığı ve bellek ölçümünün inceliğini; ve sıkıştırmamanın yer-işlemci-sıfırkopya üçgenini gördük.

Böylece Kısım II'nin neredeyse tamamını — bir belge veritabanının içeride nasıl çalıştığını — tamamlamış olduk: veriyi sakladık, dayanıklı kıldık, indeksledik, sorguladık, özetledik, tutarlı tuttuk, ölçeklendirdik ve belleğini yönettik. Geriye, OxiDB'ye geçmeden önce ele alınması gereken son bir genel konu kaldı ve o, belki de en çok ihmal edilenidir: tüm bu veriyi **korumak**. Kim okuyabilir, kim yazabilir; veri yetkisiz ellere geçerse ne olur; kimin ne yaptığı nasıl kaydedilir? Bir sonraki bölümde, bir veritabanının güvenlik boyutuna — kimlik doğrulamaya, yetkilendirmeye, şifrelemeye ve denetime — eğilerek Kısım II'yi kapatıyoruz.

## 14 Güvenlik: Kimlik Doğrulama, Yetkilendirme, Şifreleme ve Denetim

Kısım II boyunca, bir belge veritabanının içeride nasıl çalıştığını katman katman açtık: veriyi sakladık, dayanıklı kıldık, indeksledik, sorguladık, özetledik, tutarlı tuttuk, ölçeklendirdik ve belleğini yönettik. Ama tüm bu emek, gözden kaçırılması kolay ama hayati bir şeyi varsayar: veriye yalnızca yetkili kişilerin, yalnızca yetkili biçimde eriştiğini. Veri değerlidir ve çoğu zaman hassastır; bu da onu bir hedef yapar. Bu bölüm, bir veritabanının güvenlik boyutunu — verinin kim tarafından, nasıl erişilebileceğini ve çalınsa bile korunmasını sağlayan mekanizmaları — ele alarak Kısım II'yi kapatıyor.



Şekil 14 — Güvenliğin dört katmanı: kimlik, yetki, şifreleme, denetim.

### 14.1 Tehdit ve katmanlı savunma ilkesi

Güvenliği düşünmenin ilk adımı, neye karşı korunduğumuzu görmektir. Tehdit, hem dışarıdan gelir — sisteme yetkisiz girmeye çalışan saldırganlar — hem de içeriden: yetkisini kötüye kullanan ya da dikkatsizce davranan kullanıcılar. Üstelik veritabanı, bir kuruluşun en değerli verisini tek bir yerde topladığı için, özellikle çekici bir hedeftir.

Bu yüzden güvenlikte tek bir savunma hattına güvenmek tehlikelidir. Asıl ilke, **katmanlı savunmadır** (defense in depth): birbirini tamamlayan birden çok koruma katmanı kurmak, böylece bir katman aşılsa bile diğerlerinin devrede kalması. Bu bölümde göreceğimiz dört mekanizma — kimlik doğrulama, yetkilendirme, şifreleme ve denetim — bu katmanların başlıcalarıdır. Bunları, korumalı bir binaya benzetebiliriz: kapıda kimlik kontrolü (kimlik doğrulama), içeride hangi odalara girebileceğinizi belirleyen kartlı geçiş (yetkilendirme), değerli eşyaların kilitli kasada durması (şifreleme) ve her hareketi kaydeden güvenlik kameraları (denetim). Hiçbiri tek başına yeterli değildir; güçlü güvenlik, hepsinin birlikte çalışmasından doğar.

### 14.2 Kimlik doğrulama: sen kimsin

İlk katman **kimlik doğrulamadır** (authentication) ve tek bir soruyu yanıtlar: “Sen kimsin?” Sisteme erişmek isteyen birinin, iddia ettiği kişi olduğunu kanıtlaması gerekir. Bunu kanıtlamanın en yaygın yolu paroladır; ama parolaların güvenli işlenmesi, sandığınızdan daha incelikli bir iştir.

En temel kural şudur: **parolalar asla düz metin olarak saklanmaz**. Eğer bir veritabanı kullanıcıların parolalarını olduğu gibi tutarsa, o veritabanı bir gün sızdırıldığında — ki sızıntılar olur — tüm parolalar bir çırpıda ele geçer. Bunun yerine, parolanın tek yönlü bir dönüşümü, yani bir **özeti** (hash) saklanır. Bu dönüşüm tek yönlüdür: paroladan özet hesaplanabilir, ama özeten parola geri elde edilemez. Kullanıcı giriş yaptığında, girdiği

parolanın özeti hesaplanır ve saklanan özetle karşılaştırılır; ikisi uyuyorsa parola doğrudur. Böylece veritabanı sızsa bile, ortaya yalnızca geri çevrilemez özetler çıkar.

Bu temel fikir, iki ek önlemle güçlendirilir. Birincisi **tuzlama** (salting): her parolaya, ona özgü rastgele bir değer eklenip öyle özetlenir. Tuzlama, saldırganların önceden hesaplanmış dev özet tablolarıyla parolaları toplu halde çözmesini engeller; çünkü her parolanın tuzu farklı olduğu için, önceden hesaplanmış hiçbir tablo işe yaramaz. İkincisi, özet fonksiyonunu **bilinçli olarak yavaş** seçmektir. Sıradan bir kullanıcı için tek bir parolayı saniyenin küçük bir kesrinde doğrulamak yeterlidir; ama bir saldırgan, milyarlarca olası parolayı deneyerek kaba kuvvetle kırmaya çalışır. Özet hesaplamasını yavaş yapmak, meşru girişi neredeyse hiç etkilemezken, kaba kuvvet saldırısını pratikte imkânsız hale getirir.

Bir incelik daha vardır. Parolanın kendisini ağ üzerinden göndermek tehlikelidir; çünkü hattı dinleyen biri onu yakalayabilir. Bu yüzden olgun sistemler, parolayı hiç göndermeden, kullanıcının onu bildiğini kanıtlamasını sağlayan **meydan-okuma yanıt** (challenge-response) yöntemleri kullanır: sunucu bir soru sorar, kullanıcı parolasını kullanarak ona doğru yanıtı üretir, ama parolanın kendisi hattan hiç geçmez. Üçüncü kısımda OxiDB'nin tam da böyle bir meydan-okuma yanıt protokolü kullandığını ve parolaları yavaş, tuzlanmış bir özetle sakladığını göreceğiz. Parolaların ötesinde, sertifikalar ya da kuruluşun merkezi kimlik sistemine bağlanma gibi daha gelişmiş yöntemler de vardır; üçüncü kısımda OxiDB'nin bunların hangilerini desteklediğini, hangilerinin henüz eksik olduğunu dürüstçe ele alacağız.

### 14.3 Yetkilendirme: ne yapabilirsin

Kimliği doğrulanmış olmak, her şeyi yapabilmek demek değildir. İkinci katman **yetkilendirme** (authorization) ve farklı bir soruyu yanıtlar: “Sen kimsin” değil, “Sen ne yapabilirsin?” Kimliği bilinen bir kullanıcının, hangi veriye erişebileceğine ve hangi işlemleri yapabileceğine karar verir.

Yetkilendirmenin temel ilkesi, **en az ayrıcalıktır** (least privilege): her kullanıcıya, işini yapması için gereken en az yetkiyi vermek, fazlasını vermemek. Bir raporu yalnızca okuması gereken birine yazma yetkisi vermek, gereksiz bir risktir; o hesap ele geçirilirse, verebileceği zarar yetkisiyle sınırlıdır. En az ayrıcalık, olası bir ihlalin etki alanını daraltır.

Yetkileri tek tek her kullanıcıya atamak, çok sayıda kullanıcıda yönetilemez hale gelir. Bu yüzden yaygın yaklaşım, **rol tabanlı erişim denetimidir** (RBAC). Yetkiler, anlamlı rollerde gruplanır — örneğin yalnızca okuyabilen bir rol, okuyup yazabilen bir rol, her şeyi yapabilen bir yönetici rolü — ve kullanıcılara bu roller atanır. Bir kullanıcının ne yapabileceği, sahip olduğu rolden gelir; yetkileri tek tek değil, rol düzeyinde yönetirsiniz. Bu, hem daha basit hem de daha az hata yapılan bir modeldir.

Yetkilendirmenin bir de **ayrıntı düzeyi** boyutu vardır. Erişim, tüm veritabanı düzeyinde verilebilir; ya da daha ince taneli olarak belirli koleksiyonlar düzeyinde; ya da en ince haliyle, **tek tek belgeler** düzeyinde — örneğin “bir kullanıcı yalnızca kendi belgelerini görebilir” gibi bir kural. Ayrıntı düzeyi inceldikçe, koruma güçlenir ama yönetim karmaşıklaşır. Üçüncü kısımda OxiDB'nin hem rol tabanlı bir erişim denetimi sunduğunu hem de belge düzeyinde, “bu belgeye yalnızca sahibi erişebilir” türünden kurallar tanımlamaya olanak verdiğini göreceğiz.

### 14.4 Şifreleme: çalınsa bile okunamaz

Kimlik doğrulama ve yetkilendirme, sisteme **meşru** yoldan erişimi denetler. Ama ya biri bu denetimleri tümüyle atlarsa — diski fiziksel olarak çalarsa ya da ağ trafiğini dinlerse? İşte üçüncü katman, **şifreleme**, tam da bu duruma karşı korur: veriyi öyle bir biçime sokar ki, anahtarı olmayan biri onu ele geçirirse bile okuyamaz.

Şifrelemenin iki ayrı cephesi vardır. Birincisi **aktarım sırasında şifrelemedir**: veri ağ üzerinden giderken şifrelenir, böylece hattı dinleyen biri yalnızca anlamsız baytlar görür. Bu, özellikle veritabanına uzaktan bağlanan sistemlerde vazgeçilmezdir. İkincisi **dururken şifrelemedir** (at rest): veri diske şifrelenmiş olarak yazılır, böylece diski ya da dosyaları çalan biri onları okuyamaz. Beşinci ve altıncı bölümlerde verinin diske nasıl yazıldığını anlatmıştık; dururken şifreleme, o yazma katmanına eklenen bir dönüşümdür: veri diske inmeden şifrelenir, okunurken çözülür.

Şifrelemenin kalbinde, dürüstçe konuşulması gereken bir gerçek yatar: şifreleme, yalnızca **anahtarı** kadar güvenlidir. Veriyi şifrelemek, onu koruyan anahtarı güvende tutmak sorununu doğurur; anahtar ele geçerse, şifreleme hiçbir işe yaramaz. Bu yüzden anahtar yönetimi, şifrelemenin ayrılmaz ve çoğu zaman en zor parçasıdır. Ayrıca bir sınırı da görmek gerekir: dururken şifreleme, çalınan diske karşı korur, ama çalışan sisteme zaten meşru biçimde girmiş bir saldırganı karşı korumaz — çünkü o saldırgan, sistemin verdiği çözülmüş veriyi görür. Daha ileri biçimler — veriyi veritabanının kendisinden bile gizleyen, yalnızca istemcide çözülen şifreleme — bu boşluğu kapatmaya çalışır. Üçüncü kısımda OxiDB'nin hem aktarım sırasında hem de dururken şifrelemeyi desteklediğini, ama bu daha ileri biçimlerin henüz kapsam dışı olduğunu göreceğiz.

## 14.5 Denetim: kim, ne zaman, ne yaptı

İlk üç katman, kötü şeylerin **olmasını engellemeye** çalışır. Dördüncü katman, **denetim** (audit), farklı bir amaca hizmet eder: olan biteni **kaydetmek**. Denetim, “kim, ne zaman, ne yaptı” sorusunu yanıtlayan bir kayıt tutar. Bir şeyi önlemez; ama hesap verebilirliği sağlar, bir ihlal olduğunda neyin nasıl olduğunu sonradan çözmeyi mümkün kılar ve birçok düzenlemenin zorunlu kıldığı izlenebilirliği karşılar. Binadaki güvenlik kameralarına benzer: bir hırsızlığı o an durdurmaz, ama sonradan kimin ne yaptığını gösterir.

Denetimin kendine özgü bir ödünleşimi vardır: ne kadar çok şey kaydederseniz, o kadar eksiksiz bir iz tutarsınız, ama o kadar çok yer harcar ve sistemi o kadar yavaşlatırsınız. Bu yüzden ne kaydedileceğine — her okuma mı, yalnızca yazmalar mı, yalnızca yetki ihlalleri mi — dikkatle karar vermek gerekir. Ayrıca denetim kayıtlarının kendisi de yönetilmelidir: sınırsız büyümeleri için döndürülmeli (eski kayıtlar arşivlenip yenilerine yer açılmalı) ve ideal olarak, bir saldırganın izini silmek için kayıtları sonradan değiştirememesi sağlanmalıdır. Üçüncü kısımda OxiDB'nin, isteğe bağlı olarak açılan ve boyut ya da zamana göre döndürülebilen bir denetim günlüğü tuttuğunu göreceğiz.

## 14.6 Güvenlik bir bütündür ve ödünleşimlerle doludur

Bu dört katmanı ayrı ayrı anlattık, ama asıl ders, onların bir **bütün** oluşturmasıdır ve her biri ötekiler olmadan eksik kalır. Yetkilendirme olmadan kimlik doğrulama anlamsızdır: kim olduğunuzu bilip de ne yapabileceğinizi sınırlamıyorsanız, herkes her şeyi yapabilir. Anahtar yönetimi olmadan şifreleme bir tiyatrodur: anahtar herkesin görebileceği bir yerdeyse, şifrelemenin hiçbir değeri yoktur. Gözden geçirilmeyen denetim yalnızca gürültüdür: hiç kimse bakmıyorsa, kayıt tutmanın bir anlamı kalmaz. Güvenlik, ancak tüm katmanlar birlikte ve tutarlı biçimde çalıştığında gerçek olur.

Ve güvenlik de, kitabın geri kalanı gibi, ödünleşimlerle doludur: daha güçlü güvenlik, çoğu zaman daha az kolaylık ve daha düşük performans demektir. Yavaş parola özetleri, meydan-okuma protokolleri, şifreleme, denetim kaydı — hepsi bir maliyet taşır. Belki de en sık unutulmuş gerçek, güvenlik ihlallerinin çoğunun karmaşık saldırılardan değil, basit **yanlış yapılandırılmalardan** kaynaklandığıdır: açık bırakılmış bir yetki, varsayılan bir parola, kapatılması unutulmuş bir kapı. Bu yüzden iyi bir veritabanının güvenli **varsayılanlarla** gelmesi — güvenliği elde etmek için ekstra çaba değil, onu zayıflatmak için bilinçli çaba gerektirmesi — en az mekanizmaların kendisi kadar önemlidir.

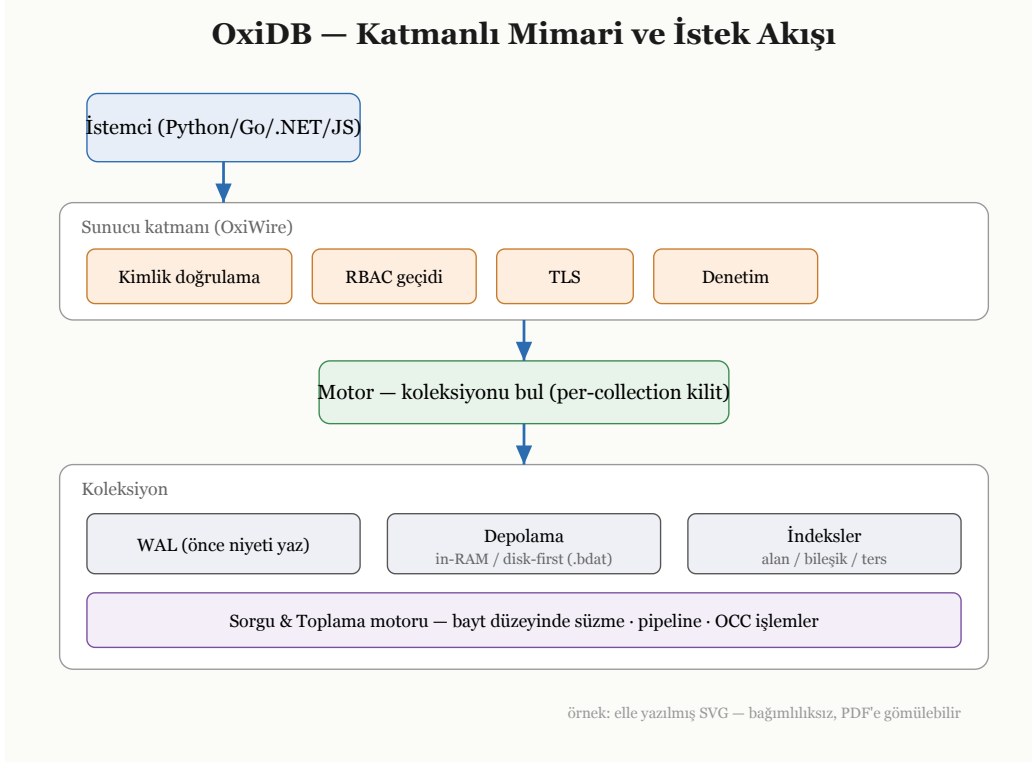
## 14.7 Kısım II'nin sonu ve Kısım III'e geiş

Bu bölümle birlikte Kısım II'yi tamamlamış olduk. Artık bir belge veritabanının içeride nasıl çalıştığına dair eksiksiz bir zihinsel haritamız var. Verinin diske nasıl yazıldığını ve çökmeden nasıl kurtarıldığını biliyoruz. Aradığımızı taramadan nasıl bulduğumuzu, sorularımızın nasıl yanıtlandığını ve veriyi nasıl özetlediğimizi biliyoruz. Eşzamanlı yazmalar karşısında tutarlılığın nasıl korunduğunu, sistemin tek makinenin ötesine nasıl ölçeklendiğini, belleğin nasıl yönetildiğini ve verinin nasıl korunduğunu biliyoruz. Bu harita, herhangi bir belge veritabanına — yalnızca OxiDB'ye değil — uygulanır; çünkü hepsi, bu aynı temel sorunları, bu aynı ilkelerle çözer.

Şimdiye dek hep ilkeler düzeyinde, “bir belge veritabanı bu sorunu *nasil çözer*” sorusuyla kaldık. Üçüncü kısımda ise somuta iniyoruz: tüm bu ilkelerin, gerçek bir motorda — OxiDB'de — nasıl hayata geçtiğini adım adım göreceğiz. Her kavramı, onun OxiDB'deki karşılığına, yapılan mühendislik tercihlerine ve o tercihlerin ardındaki gerekçelere bağlayacağız. Kısım II size bu kitabın sözlüğünü ve dilbilgisini öğretti; Kısım III, o dille yazılmış gerçek bir metni birlikte okuyacağımız yerdir. Bir sonraki bölümde, OxiDB'ye genel bir bakışla ve mimarisinin kuş bakışı görünümüyle başlıyoruz.

## 15 OxiDB’ye Genel Bakış ve Mimari

Kitabın ilk iki kısmı boyunca, hep ilkeler düzeyinde kaldık: bir belge veritabanının çözmesi gereken sorunları ve bu sorunları çözmenin genel yollarını inceledik. Artık somuta iniyoruz. Üçüncü kısım, bu ilkelerin gerçek bir motorda — OxiDB’de — nasıl hayata geçtiğini adım adım gösteriyor. Buradan itibaren her kavramı, onun OxiDB’deki karşılığına, yapılan mühendislik tercihinin ve o tercihin ardındaki gerekçeye bağlayacağız. Bu bölüm, ayrıntılara dalmadan önce bir kuş bakışı sunuyor: OxiDB nedir, hangi parçalardan oluşur ve bu parçalar birbirine nasıl bağlanır?



Şekil 15 — OxiDB’nin katmanlı mimarisi ve bir isteğin akışı.

### 15.1 OxiDB nedir

OxiDB, Rust dilinde yazılmış, hızlı ve gömülebilir bir belge veritabanı motorudur. Verisini, ikinci kısımda tanıdığımız anlamda belgeler — iç içe geçebilen, alanlardan oluşan, kendini tanımlayan nesneler — halinde tutar ve bunları JSON tabanlı bir sorgu diliyle sorgular. OxiDB, bilinçli bir tercihle, yalnızca belge modeline odaklanır; ilişkisel tablolar ve onların sorgu dili, tasarımının bir parçası değildir. Bu, ikinci bölümde gördüğümüz “her model bir ödünleşimdir” ilkesinin somut bir uygulamasıdır: OxiDB, belge modelinin güçlü olduğu yerde ustalaşmayı, her şeye birden yetişmeye çalışmaya yeğler.

OxiDB’yi diğer birçok veritabanından ayıran ilk şey, **tek bir kimliğe sıkışıp kalmamasıdır**. Birinci bölümde, bir veritabanının uygulamayla iki uçtan konuşabileceğini söylemiştik: gömülü bir kütüphane olarak ya da ağ üzerinden bir sunucu olarak. OxiDB, aynı çekirdek motoru üzerine kurulu olarak, her iki uçta da çalışabilir. Bir uygulamanın içine gömülü bir kütüphane olarak yerleşebilir; ayrı bir süreç olarak çalışıp ağ üzerinden birçok istemciye hizmet verebilir; hatta bir web tarayıcısının içinde, küçültülmüş bir biçimde çalışabilir. Bunların hepsinde, altta yatan

mekanizmalar — depolama, indeks, işlem, kurtarma — özünde aynıdır. Bu yüzden ikinci kısımda öğrendiğimiz her şey, OxiDB hangi kılıkta çalışırsa çalışsın geçerlidir.

## 15.2 Katmanların kuş bakışı görünümü

OxiDB'nin mimarisini anlamanın en kolay yolu, onu ikinci kısımda kurduğumuz zihinsel haritayla eşleştirmektir; çünkü OxiDB'nin katmanları, tam da o haritanın parçalarına karşılık gelir. En tepeden en dibe doğru inelim.

En üstte **motor** durur: tüm koleksiyonlara sahip olan, gelen istekleri doğru koleksiyona yönlendiren merkezi bileşen. Motor, her koleksiyonu ayrı bir kilitte yönettiği için, farklı koleksiyonlara erişen istekler birbirini beklemeden, eş zamanlı ilerleyebilir. Koleksiyonlar, ilk yazma geldiğinde kendiliğinden oluşur; önceden tanımlanmaları gerekmez — dördüncü bölümdeki şema esnekliğinin bir yansımasıdır bu.

Her **koleksiyon**, kendi içinde, ikinci kısımda tanıdığımız bileşenleri barındırır: belge baytlarını tutan bir depolama katmanı; dayanıklılığı sağlayan bir yazma-öncesi günlük; sık erişilen veriyi bellekte tutan önbellekler; aramayı hızlandıran alan ve bileşik indeksler; ve eşzamanlılık denetimi için belge başına sürüm sayaçları. Yani her koleksiyon, küçük ölçekte, ikinci kısmın tüm hikâyesini içinde taşır.

Bu koleksiyonun içine indiğimizde, beşinci ve altıncı bölümlerin somut karşılıklarını buluruz. **Depolama katmanı**, ikinci kısımda anlattığımız iki felsefeyi birden sunar: belleğe öncelikli varsayılan kip ile diske öncelikli opsiyonel kip. **Yazma-öncesi günlük**, altıncı bölümdeki dayanıklılık ve kurtarma mekanizmalarını — sağlama toplamları, denetim noktaları, çökmeden geri dönme — hayata geçirir.

Bir üst katmana çıktığımızda, yedinci, sekizinci ve dokuzuncu bölümlerin karşılıklarını görürüz. **İndeksler**, yalnızca temel alan indeksleriyle sınırlı değildir; OxiDB bileşik indeksleri, süreyle dolan (TTL) indeksleri, benzersizlik indekslerini, tam metin için ters indeksleri ve benzerlik araması için vektör indekslerini de destekler. **Sorgu ve toplama motoru**, sekizinci ve dokuzuncu bölümlerde anlattığımız operatörleri ve pipeline aşamalarını — gruplamayı, çok-yönlü analizi, pencere fonksiyonlarını — yürütür. **İşlem yöneticisi**, onuncu bölümdeki iyimser eşzamanlılık denetimini uygular.

En dışta, koleksiyonların ve motorun çevresinde, on ikinci ve on dördüncü bölümlerin karşılıkları yer alır. **Sunucu katmanı**, ağ üzerinden gelen istekleri karşılar; kendi ikili iletişim protokolünü, kimlik doğrulamayı, rol tabanlı erişim denetimini, aktarım şifrelemesini ve denetim günlüğünü barındırır. **Kümeleme katmanı**, on ikinci bölümdeki replikasyonu ve konsensüsü; ayrı bir yönlendirici bileşen ise sharding'i ve parçalar-arası toplamayı sağlar.

## 15.3 Çekirdeğin ötesi: ek yüzeyler

OxiDB, klasik bir belge veritabanının ötesine geçen birkaç ek yüzey de sunar ve bunları kuş bakışı haritaya eklemek, mimarinin genişliğini görmek açısından yararlıdır. Bu yüzeylerin ortak özelliği, hepsinin aynı çekirdek motorun üzerine oturmasıdır.

OxiDB, doğrudan bir HTTP arayüzü ve gerçek zamanlı bir abonelik kanalı sunarak, bir uygulamanın veritabanına web teknolojileriyle, ek bir katman kurmadan bağlanmasına olanak tanır; buna eşlik eden bir kimlik ve belge düzeyinde kural sistemiyle, küçük uygulamaların sunucu yazmadan doğrudan veritabanıyla çalışmasını mümkün kılar. Ayrı bir bellek-içi anahtar-değer katmanı, yaygın bir önbellek protokolüyle uyumlu çalışır; bir mesajlaşma protokolü desteği, veritabanını nesnelerin interneti gibi senaryolara taşır. Büyük ikili nesneler için bir nesne deposu, zamanın belirli bir noktasına geri dönmeyi sağlayan bir kurtarma yeteneği ve dururken şifreleme, bu ek yüzeyleri tamamlar. Bu kitabın odağı çekirdek belge motorudur; bu ek yüzeylere üçüncü kısmın ilerleyen bölümlerinde, çekirdeği anladıktan sonra değineceğiz.

## 15.4 Çalışma alanının yapısı ve istemciler

OxiDB'nin kod tabanı, birkaç ayrı parçaya bölünmüştür ve bu bölünme, mimarisinin mantığını yansıtır. Çekirdekte, belge motorunun kendisi yer alır — depolama, indeks, sorgu, işlem; yani ikinci kısmın tüm mekanizmaları. Bunun üzerine, çekirdeği ağ üzerinden sunan sunucu bileşeni; ve diğer programlama dillerinden çağrılabilmesini sağlayan bir köprü katmanı eklenir. Bu köprü sayesinde, OxiDB yalnızca Rust'tan değil, başka birçok dilden de kullanılabilir.

Bu köprünün üzerine kurulu **istemci kütüphaneleri**, OxiDB'yi çeşitli programlama dillerinden — örneğin Python, Go, .NET ve JavaScript'ten — kullanılabilir kılar. Bu istemciler, ya gömülü kipte çekirdeğe doğrudan bağlanır, ya da sunucu kipinde ağ üzerinden konuşur. İkinci kısımda anlattığımız tüm kavramlar — koleksiyonlar, sorgular, indeksler, işlemler — bu istemcilerin hepsinde aynı biçimde karşımıza çıkar; çünkü hepsi, aynı çekirdek motorun farklı kapılarıdır.

## 15.5 Bir isteğin yaşam döngüsü

Tüm bu parçaların nasıl birlikte çalıştığını görmek için, bir isteğin sistemden nasıl geçtiğini kabaca izleyelim; bu izlek, üçüncü kısmın geri kalanının yol haritası gibidir.

Bir istek iki yoldan gelebilir. Gömülü kipte, doğrudan bir işlev çağrısı olarak; sunucu kipinde ise ağ üzerinden, OxiDB'nin kendi ikili protokolüyle çerçevelenmiş bir mesaj olarak. Sunucu kipinde istek, önce bir güvenlik süzgecinden geçer: kimlik doğrulanır ve rolün bu işleme izni olup olmadığı denetlenir. Bu süzgeçten geçen istek, motora ulaşır; motor, isteğin hedeflediği koleksiyonu bulur. Eğer istek bir okumaysa, sorgu motoru devreye girer: koşulları ayarlar, hangi indeksin işe yarayacağına karar verir, adayları daraltır ve süzer. Eğer istek bir yazmaysa, önce yazma-öncesi günlüğe kaydedilir ve dayanıklı kılınır, sonra depolama katmanına ve indekslere uygulanır. Sonuç, geldiği yoldan geri döner. Kümeleme kipinde, bir yazma ek olarak konsensüs katmanından, sharding'de ise yönlendiriciden geçer.

Bu kısa izlek, aslında bu kitabın bir özetidir: bir istek, ikinci kısımda tek tek anlattığımız tüm katmanlardan sırayla geçer. Üçüncü kısmın geri kalanı, bu izleğin her durağını yakın plana alır.

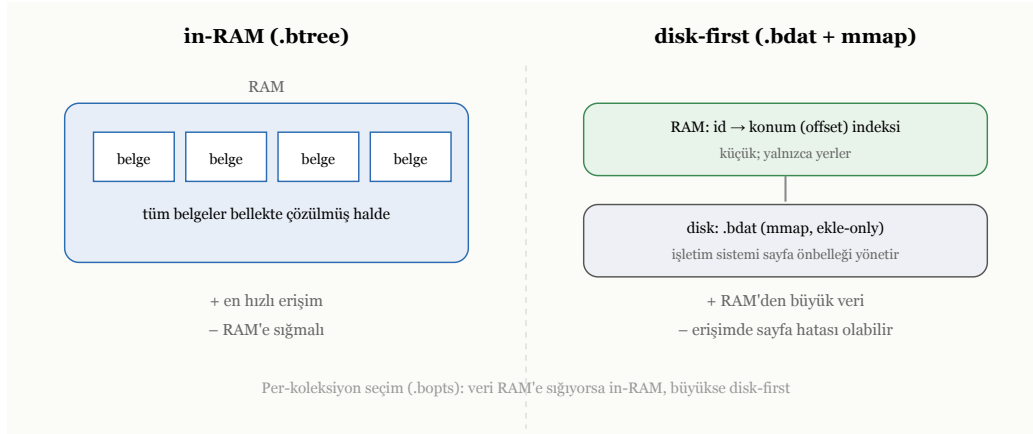
## 15.6 Üçüncü kısmın yol haritası

Üçüncü kısım, isteğin yaşam döngüsünü izleyerek, en dipten en üste doğru ilerleyecek. Önce **depolama katmanına** ineceğiz: OxiDB'nin belleğe öncelikli ve diske öncelikli kiplerini, verinin diskte nasıl yerleştiğini ve belleğe yansıtmanın bellek ayak izini nasıl belirlediğini göreceğiz. Ardından **dayanıklılık** ve kurtarma, sonra **indeksler**, **sorgu motoru** ve **toplama pipeline'ı** gelecek — her biri, ikinci kısımda kurduğumuz ilkeyi OxiDB'nin somut tercihine bağlayarak. **İşlemler**, **sıkıştırma**, ve ardından çekirdeğin ötesindeki ek yüzeyler — tam metin arama, nesne deposu, şifreleme, kurtarma — ele alınacak. Sonra **sunucu** katmanına ve protokolüne, oradan **ölçeklendirmeye** — Raft kümesi ve sharding'e — ve **istemcilere** geçeceğiz. Kısım, OxiDB'nin bellek optimizasyonunu ve gerçek bir karşılaştırmalı değerlendirmesini ele alarak kapanacak.

Bu yolculuğun her durağında, tanıdık bir kavramı yeniden göreceksiniz — ama bu kez soyut değil, somut. İkinci kısım size dili öğretti; üçüncü kısım, o dille yazılmış gerçek bir metni birlikte okuyacağımız yerdir. Bir sonraki bölümde, en dipten, depolama katmanından başlıyoruz: OxiDB belgeleri tam olarak nasıl saklar ve neden iki ayrı depolama kipi sunar?

## 16 OxiDB'nin Depolama Katmanı: In-RAM ve Disk-First, mmap, .bdat / .btree

Önceki bölümde OxiDB'nin mimarisine kuş bakışı baktık ve isteğin yaşam döngüsünü izleyerek en dipten, depolama katmanından başlayacağımızı söylemiştik. İşte buradayız. Beşinci ve on üçüncü bölümlerde, bir depolama motorunun belge baytlarını diske nasıl yerleştirdiğini ve bellek ile disk arasındaki ödünleşimi genel ilkeler düzeyinde tanımiştık. Bu bölüm, o ilkelerin OxiDB'de nasıl somutlaştığını gösteriyor: OxiDB belgeleri tam olarak nasıl saklar, neden iki ayrı depolama kipi sunar ve bu seçimin bellek ile hız üzerindeki sonuçları nelerdir?



Şekil 16 — in-RAM ile disk-first depolama kiplerinin karşılaştırması.

### 16.1 Çekirdek: kimlikten baytlara

Beşinci bölümde, bir depolama motorunun belgenin içeriğiyle ilgilenmediğini, onun için bir belgenin “bir kimliğe bağlı bir bayt yığını” olduğunu söylemiştik. OxiDB'nin depolama çekirdeği tam olarak böyle çalışır: her belgeye bir kimlik verir ve o kimlikten belgenin baytlarına giden bir eşleme tutar. Üstüne kurulu tüm katmanlar — sorgu, indeks, işlem — bu çekirdeğe iki temel istekle gelir: “şu kimlikli belgeyi sakla” ve “şu kimlikli belgeyi geri ver.”

Bu eşlemenin altında, eş zamanlı erişime izin veren bir yapı yatar. Birçok istek aynı anda farklı belgelere dokunabildiği için, depolama çekirdeği, parça parça kilitlenebilen — yani bir bölümüne dokunulurken başka bir bölümüne paralel erişilebilen — bir yapı kullanır. Bu, on beşinci bölümde değindiğimiz “koleksiyon içinde eş zamanlı erişim” yeteneğinin temelidir.

Belgenin kendisi, bu çekirdekte ham JSON metni olarak değil, dördüncü bölümde öngördüğümüz gibi **daha zengin, daha sıkı bir ikili biçimde** tutulur. Dışarıya, kullanıcıya JSON olarak görünen belge, içeride bu ikili biçime kodlanır; bu biçim hem yer açısından daha tutumludur hem de bir alana, tüm belgeyi baştan ayrıştırmadan doğrudan erişmeye olanak tanır. Bu inceliğin sorgu hızına katkısını, ileride sorgu motorunu ele alırken yeniden göreceğiz.

### 16.2 İki kip, iki felsefe

OxiDB'nin depolama katmanının en belirleyici özelliği, on üçüncü bölümde tanıdığımız iki felsefeyi — belleğe öncelikli ve diske öncelikli — birden sunmasıdır. Bu iki kip, aynı çekirdek arayüzün arkasında, verinin nerede yaşadığına dair taban tabana farklı iki karar verir.

### 16.2.1 Belleğe öncelikli kip: varsayılan

Varsayılan kipte, her belgenin baytları **bellekte yerleşik** olarak durur. Yani tüm koleksiyon, kelimenin tam anlamıyla, bellekteki o eş zamanlı eşlemenin içinde yaşar. Diske yazılan dosya — uzantısıyla anılırsa, **.btree** dosyası — yalnızca bu bellek içeriğinin periyodik bir **anlık görüntüsüdür**; veritabanı açıldığında bu dosya bütünüyle belleğe geri yüklenir. Bu yüzden OxiDB, varsayılan kipinde, aslında disk üzerinde kalıcılığa sahip bir **bellek-içi veritabanı** gibi davranır.

Bunun sonucu, on üçüncü bölümün belleğe öncelikli felsefesinin tam karşılığıdır. Okumalar olağanüstü hızlıdır, çünkü her belge zaten bellektedir; hiçbir okuma diske gitmez. Bedeli ise belleğin **veriyle birlikte büyümesidir**: her belgenin baytları bellekte yer kapladığı için, milyon belgelik bir koleksiyon, yüzlerce megabayt yerleşik bellek tüketir. Bu, veri belleğe rahatça sığıldığı sürece mükemmel bir tercihtir; ama veri büyüdükçe, bellek hem pahalı hem de sınırlayıcı bir kısıt haline gelir.

### 16.2.2 Disk-öncelikli kip: opsiyonel

İkinci kip, on üçüncü bölümün diske öncelikli felsefesini hayata geçirir ve isteğe bağlı olarak açılır. Bu kipte, bellekte tüm belgeler değil, yalnızca küçük bir **kimlik-konum dizini** durur: her belgenin kimliğinden, o belgenin disk üzerindeki yerine giden, belge başına yalnızca birkaç düzine bayt tutan kompakt bir eşleme. Belgelerin asıl baytları ise, beşinci bölümdeki append-only felsefeyle yazılan bir veri dosyasında — **.bdat** dosyasında — durur ve gerektiğinde oradan okunur.

Bu kararın bellek üzerindeki etkisi çarpıcıdır. Yerleşik bellek artık veriyle birlikte büyümeyiz; yalnızca o küçük dizin kadar yer kaplar. Milyon belgelik bir koleksiyonun bellekteki yükü, yüzlerce megabayttan birkaç düzine megabayta iner. Önemlisi, OxiDB bu kipte yeni bir depolama motoru icat etmez; beşinci ve altıncı bölümlerde anlattığımız, append-only, mmap ile okunan, soft-delete ve sağlama desteği olan sağlamlaşmış depolama bileşenini yeniden kullanır. Yani disk-öncelik, köklü bir yeniden yazım değil, var olan dayanıklı altyapının zarif bir biçimde yeniden kullanılmasıdır.

## 16.3 Dosyaların dili: .btree, .bdat ve .bopts

OxiDB'nin hangi koleksiyonun hangi kipte olduğunu bilmesi gerekir; bunu, dosya adlarındaki bilinçli bir ayrımla yapar. Belleğe öncelikli bir koleksiyonun anlık görüntüsü **.btree** uzantısıyla; disk-öncelikli bir koleksiyonun append-only veri dosyası ise **.bdat** uzantısıyla yazılır. Bu iki uzantı kasıtlı olarak farklıdır: veritabanı açıldığında, bir koleksiyonun yanında hangi dosyayı gördüğüne bakarak onun hangi kipte olduğunu anlar.

Buna eşlik eden üçüncü bir dosya, **.bopts**, on ikinci bölümde tanıdığımız per-koleksiyon seçeneklerini kalıcı kılar. OxiDB, depolama kipini ve sıkıştırma gibi tercihleri her koleksiyon için ayrı ayrı belirlemeye izin verir; ve bu tercihleri **.bopts** dosyasına yazar. Böylece bir koleksiyon, hangi tercihle oluşturulduysa, yeniden açıldığında o tercihle açılır — sistemin o anki ortam ayarlarından bağımsız olarak. Bu, on ikinci bölümdeki “per-koleksiyon ayar” fikrinin somut bir uygulamasıdır: aynı veritabanında, bir koleksiyon belleğe öncelikli, başka biri disk-öncelikli olabilir ve her biri kendi kimliğini korur.

## 16.4 Belleğe yansıtmanın somut etkisi

Disk-öncelikli kipte, **.bdat** veri dosyası belleğe yansıtılır — on üçüncü bölümde değindiğimiz mmap tekniği. Bir belge okumak, kimlik-konum dizininden konumu bulmak ve ardından belleğe yansıtılmış dosyanın o konumundan baytları okumaktır. Veri zaten bellekteyse bu çok hızlıdır; değilse, işletim sistemi onu diskten getirir ve bellek baskı altındayken yine sessizce geri atabilir.

Bunun somut sonucu, on üçüncü bölümde tartıştığımız bellek ölçümü inceliğinin canlı bir örneğidir. Disk-öncelikli kipte, beş yüz bin belgelik bir koleksiyonu taze açtığınızda, sistemin yerleşik belleği yalnızca birkaç on megabayt mertebesinde — çünkü henüz yalnızca o küçük dizin bellektedir, belgelerin gövdesi diskte beklemektedir. Buna karşılık aynı veriyi belleğe öncelikli kipte açtığınızda, tüm belgeler belleğe yüklendiği için yerleşik bellek kat kat yüksektir. İki kip arasındaki bu taze-açılış farkı, disk-öncelik tercihinin asıl kazancıdır.

Ama on üçüncü bölümün dürüst uyarısını burada da tekrarlamak gerekir: tüm veriye dokunan büyük bir taramadan sonra, disk-öncelikli kipin belleği de yükselir — çünkü işletim sistemi dokunulan sayfaları belleğe çeker. Bu yükselen bellek geri alınabilir bir bellektir; baskı altında işletim sistemi onu serbest bırakır. Bu yüzden disk-öncelik, çalışma kümesi tüm veriden küçük olan yükler için en çok kazandırır; tüm veriyi sürekli baştan sona tarayan yükler için kazancı azalır.

## 16.5 Append-only olmanın sonuçları

Disk-öncelikli kip, beşinci bölümdeki append-only felsefeyi izlediği için, o felsefenin iki sonucunu da devralır. Birincisi, var olan veriyi yerinde değiştirmemesidir: bir belge güncellendiğinde, eski baytlar `.bdat` dosyasında olduğu gibi kalır, yeni baytlar dosyanın sonuna eklenir ve kimlik-konum dizini yeni konuma güncellenir; eski konum ölü alana dönüşür. İkincisi, bu ölü alanın zamanla biriktiğidir ve onu geri kazanmak için sıkıştırmaya — yirmi ikinci bölümün konusuna — ihtiyaç vardır.

Bu append-only doğanın bir başka sonucu, yazma hızındadır. Beşinci bölümde gördüğümüz gibi, dosyanın sonuna ardışık ekleme, diskin en sevdiği yazma biçimidir; bu yüzden disk-öncelikli kipte yazmalar, toplu olarak ve ardışık biçimde yapıldığında oldukça verimlidir.

## 16.6 Sıkıştırma: yer, işlemci ve sıfır-kopya

On üçüncü bölümde sıkıştırmanın yer-işlemci-sıfırkopya üçgenini tanıtmıştık; OxiDB'nin disk-öncelikli kipi, bu üçgenin tam bir uygulamasını sunar. `.bdat` dosyasındaki kayıtlar, sıkıştırılmış olarak ya da — isteğe bağlı bir tercihle — sıkıştırılmadan saklanabilir. Sıkıştırılmış saklama, diskte daha az yer kaplar, ama her okumada baytların açılmasını gerektirir. Sıkıştırılmamış saklama daha çok yer kaplar, ama on üçüncü bölümde tanıttığımız sıfır-kopya erişimi mümkün kılar: baytlar zaten ham haldeyse, belleğe yansıtılmış dosyadan doğrudan, hiç kopyalamadan ve açmadan kullanılabilir.

Hangisinin kazandığı, on üçüncü bölümdeki ilkeye uyar: veriye bağlıdır. Gerçekten iyi sıkışan, seyrek taranan veri için sıkıştırma kazandırır; az sıkışan ama sık ve büyük taranan veri için sıkıştırmamak daha iyidir. OxiDB üzerinde yapılan ölçümler bu dengeyi somut biçimde gösterir: küçük, yapılandırılmış belgelerden oluşan tipik bir veri kümesinde, sıkıştırma diskte kayda değer bir yer kazandırmazken, her taramada belgeleri açma zorunluluğu büyük bir maliyet getiriyordu; sıkıştırmayı kapatmak, bu tür yükleri belirgin biçimde hızlandırıyordu. Bu, on üçüncü bölümün soyut ödünleşiminin gerçek bir sistemdeki yankısıdır.

## 16.7 Açılış ve kurtarmaya kısa bir bakış

Bir koleksiyon açıldığında, depolama katmanının kendini kullanılabilir bir duruma getirmesi gerekir ve bu, kipe göre değişir. Belleğe öncelikli kipte, sistem `.btree` anlık görüntüsünü bütünüyle belleğe geri yükler. Disk-öncelikli kipte ise, `.bdat` dosyasındaki yaşayan kayıtları tarayarak kimlik-konum dizinini yeniden kurar — her belge kendi kimliğini taşıdığı için bu mümkündür. Her iki kipte de, bu adımın ardından yazma-öncesi günlük devreye girip son değişiklikleri uzlaştırır; ama bu, bir sonraki bölümün konusudur.

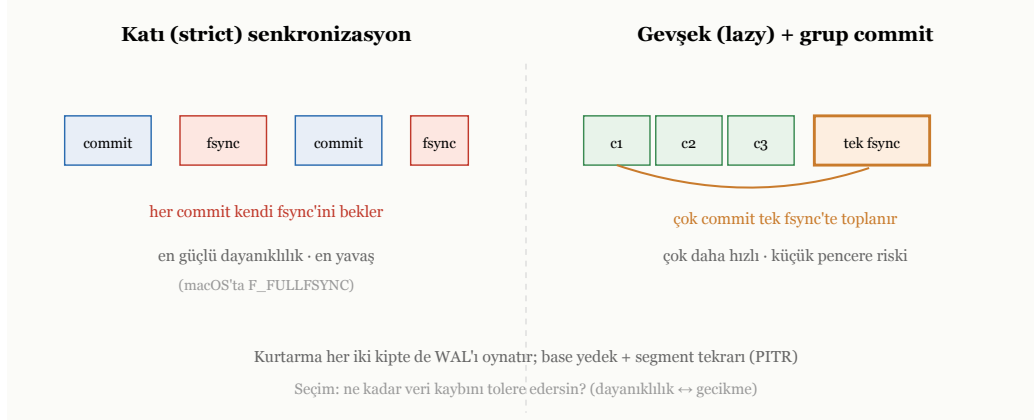
## 16.8 Bu bölümün bıraktığı yer

Bu bölümde, OxiDB'nin depolama katmanını yakın plana aldık. Çekirdeğin, belgeleri bir kimlikten baytlara eşleyen, eş zamanlı erişime açık bir yapı olduğunu; belgelerin içeride zengin bir ikili biçimde tutulduğunu gördük. İki kipi tanıdık: tüm veriyi bellekte tutan, hızlı ama bellek-yoğun belleğe öncelikli varsayılanı; ve yalnızca kompakt bir dizini bellekte tutup veriyi mmap ile diskte bırakan, bellek-tutumlu disk-öncelikli kipi. Dosya uzantılarının (`.btree`, `.bdat`, `.bopts`) bu kipleri ve per-koleksiyon tercihleri nasıl kodladığını; belleğe yansıtmanın bellek ayak izini nasıl belirlediğini; append-only olmanın sonuçlarını; ve sıkıştırmanın somut ödünleşimini gördük.

Buraya kadar hep verinin nasıl **saklandığından** söz ettik. Ama altıncı bölümde öğrendiğimiz gibi, bir veriyi diske yazmak, onun çökmeye karşı güvende olduğu anlamına gelmez. OxiDB, bir yazmanın gerçekten dayanıklı olduğundan nasıl emin olur; çökmeden nasıl kurtulur; ve dayanıklılık ile hız arasındaki o gerilimi — her commit'i diske zorlamak ile arada bir zorlamak arasındaki tercihi — nasıl yönetir? Bir sonraki bölümde, OxiDB'nin yazma-öncesi günlüğüne ve dayanıklılık mekanizmalarına eğiliyoruz.

## 17 OxiDB’de WAL, Dayanıklılık ve Kurtarma; Katı ve Gevşek Senkronizasyon

Önceki bölümde, OxiDB’nin belgeleri nasıl sakladığını gördük. Ama altıncı bölümde öğrendiğimiz gibi, bir veriyi diske yazmak, onun çökmeye karşı güvende olduğu anlamına gelmez. Bu bölüm, OxiDB’nin bir yazmanın gerçekten dayanıklı olduğundan nasıl emin olduğunu, çökmeden nasıl kurtulduğunu ve altıncı bölümde tanıdığımız o dayanıklılık tayfında — her commit’i diske zorlamak ile arada bir zorlamak arasında — nereye oturduğunu ele alıyor. Bu, OxiDB’nin verdiği en temel sözlerden birinin — “tamamlandı dediysem, kaybolmaz” — ardındaki düzenektir.



Şekil 17 — Katı ve gevşek senkronizasyon; grup commit.

### 17.1 OxiDB’nin yazma-öncesi günlüğü

Altıncı bölümdeki ilkeyi hatırlayalım: asıl veriyi değiştirmeden önce, ne yapacağını ayrı bir günlüğe yaz ve o günlüğü dayanıklı kıl; dayanıklılık günlükten gelir, asıl güncelleme tembelce yapılır. OxiDB, bu ilkeyi doğrudan uygular. Her koleksiyonun, yalnızca sona eklenerek büyüyen bir yazma-öncesi günlüğü vardır.

Bu günlüğe yazılan her kayıt, altıncı bölümde anlattığımız iki korumayı taşır. Birincisi, kaydın içeriğinden hesaplanan bir **sağlama toplamıdır** (CRC); bu, kurtarma sırasında yarım kalmış ya da bozulmuş bir kaydı yakalamayı sağlar. İkincisi, kaydın hangi işleme ait olduğunu belirten bir **işlem kimliğidir**; bu, ileride işlemleri ele alırken önemli olacak. Bir yazma geldiğinde, OxiDB önce bu kaydı günlüğe ekler ve onu dayanıklı kılar; ancak ondan sonra, değişikliği bir önceki bölümdeki depolama katmanına — belleğe öncelikli kipte bellekteki eşlemeye, disk-öncelikli kipte **.bdat** dosyasına — ve indekslere uygular. Yani günlük her zaman asıl veriden **önce** dayanıklı olur; çökme tam o sırada olsa bile, niyet günlükte kayıtlıdır.

Burada altıncı bölümün bir incelğini somut görürüz. Bir kaydı “dayanıklı kılmak”, yalnızca diske göndermek değil, oraya gerçekten oturduğundan emin olmaktır; altıncı bölümde, bazı sistemlerde sıradan boşaltmanın veriyi yalnızca disk denetleyicisinin ön belleğine bıraktığını söylemiştik. OxiDB bu konuda tavizsizdir: dayanıklılık gerektiğinde, veriyi fiziksel ortama gerçekten işleyen, tam bir boşaltma kullanır. Bu güçlü garanti, az sonra göreceğimiz gibi, bir hız bedeli taşır — ama verdiği söz de o kadar güçlüdür.

### 17.2 Katı dayanıklılık: varsayılan tercih

Altıncı bölümde dayanıklılığın bir tayf üzerinde tercih edildiğini söylemiştik. OxiDB, bu tayfın katı ucunu **varsayılan** olarak seçer: her commit, “tamamlandı” denmeden önce günlüğe yazılır ve diske gerçekten boşaltılır.

Bu, en güçlü güvencedir — bir işlem tamamlandı dindikten sonra hiçbir çökme onu geri alamaz — ve OxiDB'nin güvenliğe verdiği önceliği yansıtır.

Ama bu güvencenin dürüstçe konuşulması gereken bir maliyeti vardır ve bu maliyet, OxiDB üzerinde yapılan ölçümlerde açıkça görülür. Tek bir belgeyi güncelleyen tek bir işlem düşünün. Bu işlem, tek başına bir günlük yazması ve tek bir tam boşaltma gerektirir; ve tam boşaltma, fiziksel ortama gerçek bir yazma olduğu için, bu makinede milisaniyeler mertebesinde — bilgisayar ölçeğinde upuzun bir süre — alır. Ölçümlerde, katı kipte tek bir belge güncellemesi yaklaşık dört milisaniye sürüyordu. Bu, OxiDB'nin, varsayılan ayarlarıyla MongoDB'nin arkasında kaldığı az sayıdaki işlemden biridir; çünkü MongoDB varsayılan ayarında her yazmayı diske zorlamaz, bellekten onaylayıp günlüğü sonradan, toplu olarak boşaltır. Yani bu karşılaştırma, elma ile armuttur: OxiDB her tekil yazmayı gerçekten dayanıklı kılarken bir bedel öder; MongoDB o bedeli erteler, daha az dayanıklılık karşılığında daha hızlı görünür.

### 17.3 Grup commit: bedeli toplu yazmada eritmek

Peki OxiDB, her yazma için bir tam boşaltma ödüyorsa, çok sayıda belge eklerken nasıl hızlı kalır? Yanıt, altıncı bölümde tanıttığımız grup commit fikrindedir. Bir tam boşaltma, ister bir kaydı ister binlerce kaydı diske işlesin, kabaca aynı süreyi alır. Bu yüzden OxiDB, toplu bir ekleme yaparken, her belgeyi ayrı ayrı boşaltmaz; tüm partiyi günlüğe yazar ve **tek bir boşaltmayla** birden dayanıklı kılar. Böylece tam boşaltmanın maliyeti, partideki belge sayısına bölünür ve belge başına neredeyse hiç iner.

Bunun somut sonucu çarpıcıdır. Ölçümlerde, beş binlik partiler halinde yapılan toplu eklemede OxiDB, her partiyi tek boşaltmayla dayanıklı kıldığı için, MongoDB ile başa baş ekleme hızına ulaşıyordu — hem de MongoDB'nin yapmadığı bir şeyi, her partiyi gerçekten diske boşaltmayı, yaparak. Yani katı dayanıklılık, toplu yazmalarda bir dezavantaj olmaktan çıkıyordu; çünkü grup commit, güvenceden hiç ödün vermeden maliyeti eritiyordu. Tek tek yazmada görünen dört milisaniyelik fark, toplu yazmada kayboluyordu; çünkü orada amorti edilecek bir parti vardı, tek tek güncellemede ise yoktu.

### 17.4 Gevşek senkronizasyon: hızı seçmek

OxiDB, katı dayanıklılığı varsayılan yapar, ama bunu zorunlu kılmaz. Altıncı bölümdeki tayfın gevşek ucu da, isteğe bağlı bir ayarla — bir ortam değişkeniyle — açılabilir. Gevşek kipte, bir yazma günlüğe yazılır ama hemen diske boşaltılmaz; boşaltma, her commit'te değil, arka planda çalışan bir iş parçacığı tarafından belirli aralıklarla, toplu olarak yapılır. Yazma, bellekten onaylanır ve hemen geri döner.

Bu kipin hızı, ölçümlerde net biçimde görülür: gevşek kipte, tek bir belge güncellemesi, katı kipteki yaklaşık dört milisaniyeden, yaklaşık seksen yedi mikrosaniyeye iniyordu — yani kırk katından fazla hızlanıyor ve MongoDB'nin varsayılan hızını bile geçiyordu. Çünkü artık her güncelleme bir tam boşaltma beklemiyordu. Bedeli, altıncı bölümde söylediğimiz risktir: bir çökme olursa, henüz boşaltılmamış son birkaç yazma kaybolabilir. Yani gevşek kip, hızı küçük bir veri kaybı riski karşılığında satın alır ve bu, MongoDB'nin varsayılan dayanıklılık modeline çok benzer bir tercihtir. OxiDB'nin yaptığı şey, bu tercihi kullanıcının eline vermektir: güvenlik mi, hız mı — duruma göre seçersiniz.

### 17.5 Kurtarma: çökmeden geri dönmek

OxiDB'nin dayanıklılık vaadi, asıl sınavını çökmeden sonra verir. Sistem yeniden açıldığında, kendini tutarlı bir duruma getirmek için altıncı bölümdeki kurtarma sürecini uygular; bu süreç, bir önceki bölümdeki iki depolama kipine göre küçük farklarla işler.

Önce bir **taban** kurulur. Belleğe öncelikli kipte bu, **.btree** anlık görüntüsünün belleğe yüklenmesidir. Disk-öncelikli kipte ise, **.bdat** dosyasındaki yaşayan kayıtların taranıp kimlik-konum dizininin yeniden kurulmasıdır. Bu taban hazır olduktan sonra, OxiDB **yazma-öncesi günlüğü taban üzerine oynatır**: günlükteki, henüz tabana yansımamış değişiklikleri yeniden uygular. Böylece, çökmeden hemen önce dayanıklı kılınmış her değişiklik geri gelir.

Bu yeniden oynatmanın güvenli olması, altıncı bölümdeki “etkisiz tekrarlanabilirlik” ilkesine dayanır ve OxiDB bunu somut biçimde sağlar: değişiklikler belge kimliğiyle ilişkilendirildiği için, aynı eklemeyi iki kez oynatmak zararsızdır — ikinci kez, aynı kimliğe yazılır ve sonucu değiştirmez. Bu yüzden kurtarma, bir değişikliğin tabana zaten yansıyor yansımadığından emin olmasa bile, onu güvenle yeniden uygulayabilir.

Kurtarmanın yarım kalmış son kaydı nasıl ele aldığı, altıncı bölümdeki sağlama toplamı mekanizmasının doğrudan uygulamasıdır. Çökme, en son günlük kaydının yazılmasının ortasında olmuş olabilir; o kayıt yarım kalmış, bozulmuş olabilir. OxiDB, her kaydı oynatırken sağlama toplamını yeniden hesaplar; uyuşmuyorsa, o kaydın yarım kaldığını anlar, onu güvenle atar ve ondan önceki son sağlam noktadan devam eder. Böylece yarım yazma, sessizce bozuk veriye dönüşmek yerine, açıkça fark edilip temizlenir.

## 17.6 Denetim noktası ve günlüğün dizginlenmesi

Altıncı bölümde, günlüğün sonsuza dek büyüemeyeceğini ve periyodik olarak denetim noktalarıyla dizginlenmesi gerektiğini söylemiştik. OxiDB de, günlükteki değişiklikler asıl depoya güvenle yansdıktan sonra, o noktaya kadarki günlük kayıtlarını geri kazanır; böylece günlük baştan büyüme devam edebilir.

Burada OxiDB’nin bir tasarım inceliği vardır ve onu görmek öğreticidir. Performans için, asıl depoya yazma — yani denetim noktası — tembelce, arada bir yapılır; bu arada, tamamlanmış bir değişikliğin tek dayanağı bir süre yalnızca günlük olabilir. Bu, hız kazandıran bilinçli bir tercihtir; ama kurtarma mantığının buna uygun tasarlanmasını gerektirir, çünkü kurtarma, asıl depoda henüz görünmeyen ama günlükte dayanıklı olan değişiklikleri de geri getirmek zorundadır. OxiDB’nin geliştirilmesinde, bu inceliğin gözden kaçtığı bir durumun nasıl bir kurtarma hatasına yol açtığı ve nasıl düzeltildiği, dayanıklılık mantığının ne kadar dikkat gerektirdiğinin iyi bir örneğidir.

## 17.7 WAL her iki kibin de önünde durur

Altıncı bölümde, yazma-öncesi günlüğün depolama felsefesinin bir alternatifi değil, onun önünde duran ayrı bir katman olduğunu vurgulamıştık. OxiDB’de bu, açıkça görülür: aynı günlük ve aynı dayanıklılık makinesi, hem belleğe öncelikli hem de disk-öncelikli kipin önünde durur. Hangi kipte olursanız olun, bir yazma önce aynı günlüğe gider; yalnızca o yazmanın “asıl depoya” uygulanma biçimi — bellekteki eşlemeye mi yoksa **.bdat** dosyasına mı — kipe göre değişir. Böylece dayanıklılık, depolama kipinden bağımsız, ortak bir güvence olarak kalır.

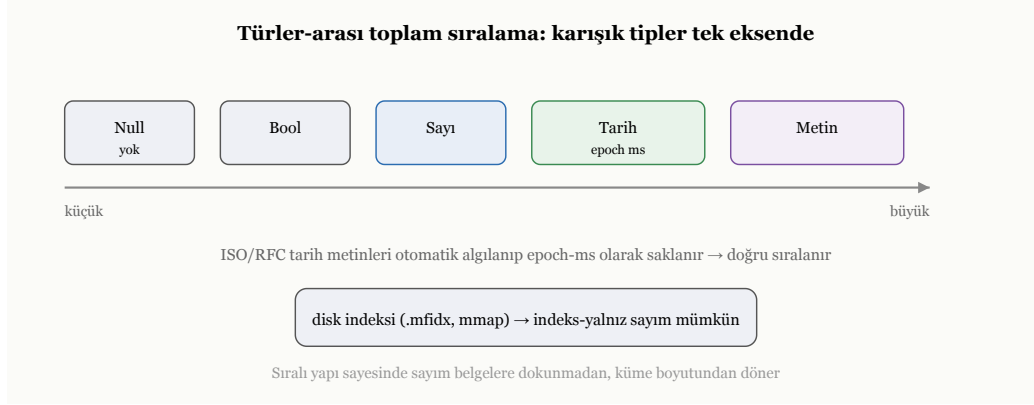
## 17.8 Bu bölümün bıraktığı yer

Bu bölümde, OxiDB’nin dayanıklılık mekanizmasını yakın plana aldık. Her koleksiyonun, sağlama toplamı ve işlem kimlikli kayıtlardan oluşan bir yazma-öncesi günlük tuttuğunu; her yazmanın önce bu günlüğe gidip dayanıklı kılındığını, sonra depoya uygulandığını gördük. Katı dayanıklılığın varsayılan olduğunu ve bunun tek tek güncellemelerde bir hız bedeli taşıdığını, ama grup commit sayesinde toplu yazmalarda bu bedelin eridiğini; gevşek kipin ise hızı bir kayıp riski karşılığında nasıl artırdığını ölçümlerle gördük. Kurtarmanın taban kurma, günlük oynatma, etkisiz tekrarlanabilirlik ve yarım-kayıt temizleme adımlarını ve denetim noktasının günlüğü nasıl dizginlediğini inceledik.

Artık verimiz hem kalıcı hem de çökmeye dayanıklı biçimde duruyor. Ama yedinci bölümde gördüğümüz gibi, veriyi güvenle saklamak yetmez; onu taramadan hızla bulabilmek de gerekir. Bir sonraki bölümde, OxiDB'nin aramayı hızlandıran yapılarına — alan indekslerine, bileşik indekslere ve disk-öncelikli kipte belleğe yansıtılan indekslere — ve bunların yedinci bölümdeki ilkelerle nasıl örtüştüğüne eğiliyoruz.

# 18 OxiDB’de İndeksler: Alan, Bileşik ve mmap Tabanlı Disk İndeksleri

Önceki iki bölümde, OxiDB’nin veriyi nasıl sakladığını ve onu çökmeye karşı nasıl güvence altına aldığını gördük. Artık verimiz hem kalıcı hem dayanıklı; ama yedinci bölümde öğrendiğimiz gibi, onu taramadan hızla bulabilmek de gerekir. Bu bölüm, OxiDB’nin aramayı hızlandıran yapılarını — alan indekslerini, bileşik indeksleri ve disk-öncelikli kipte belleğe yansıtılan indeksleri — ele alıyor ve bunların yedinci bölümdeki ilkelerle nasıl örtüştüğünü gösteriyor.



Şekil 18 — Türler-arası toplam sıralama ve mmap tabanlı disk indeksi.

## 18.1 Alan indeksleri: sıralı bir eşleme

Yedinci bölümde bir indeks, “bir alanın değerlerinden o değere sahip belgelerin konumuna giden bir eşleme” olarak tanımlamıştık. OxiDB’nin alan indeksleri tam olarak budur: indekslenen her alan için, o alanın her değerinin altında, o değere sahip belgelerin kimliklerini tutan bir yapı. Bu yapı **sıralı** tutulur ve bu, yedinci bölümdeki üç armağanı birden getirir: belirli bir değere sahip kayıtları bulmak (eşitlik), bir değer aralığındaki kayıtları bulmak (aralık) ve sonuçları o alana göre sıralı vermek (sıralı getirme).

OxiDB, bu sıralı yapıyı, bellekte verimli çalışacak biçimde tasarlar. Değerleri, işaretçilerle birbirine bağlı dağınık bir ağaç yerine, bitişik ve sıralı bir düzende tutar; böylece bir değeri ararken, belleğin oraya buraya sıçramadan, birbirine yakın bölgelerde ikili arama yapması yeterli olur. Bu, büyük indekslerde, modern işlemcilerin bellek erişim biçiminden en iyi şekilde yararlanmayı sağlar. Yeni yazmalar ise, her seferinde bu sıralı düzeni baştan bozmamak için, önce küçük bir tampona alınır ve periyodik olarak sıralı yapıya katılır; böylece eklemeler amorti edilmiş biçimde ucuz kalır.

## 18.2 Türler arası sıralama: IndexValue inceliği

Yedinci bölümde “sıralı indeks” derken, üstü örtük bir varsayım vardı: değerlerin sıralanabilir olması. İlişkisel bir sütunda bu kolaydır, çünkü her sütunun tek bir türü vardır. Ama belge dünyasında, dördüncü bölümde gördüğümüz gibi, aynı alan farklı belgelerde farklı türde değerler taşıyabilir — birinde sayı, birinde metin, birinde tarih. Sıralı bir indeks kurabilmek için, OxiDB’nin bu **farklı türleri bile tek bir bütünsel sıraya** koyabilmesi gerekir.

OxiDB bunu, türler arasında sabit bir sıralama tanımlayarak çözer: boş değerler en önce, sonra mantıksal değerler, sayılar, tarihler ve metinler belirli bir düzende gelir. Böylece tür karışık olsa bile, indeks tutarlı bir sıra koruyabilir. Özellikle zarif bir ayrıntı, tarihlerle ilgilidir. Tarihler belgelerde çoğu zaman metin olarak — yaygın tarih biçimlerinde — yazılır; ama metin olarak sıralandığında, tarihler doğru kronolojik sıraya girmeyebilir. OxiDB,

yaygın biçimlerdeki tarih metinlerini **otomatik olarak tanır** ve onları sayısal bir zaman damgasına çevirip öyle indeksler; böylece tarih aralığı sorguları doğru biçimde sıralanır. Bu, yedinci bölümün soyut “sıralı indeks” fikrini, belge verisinin tür çeşitliliğine uydurmak için yapılmış somut bir mühendislik tercihidir.

### 18.3 Bileşik indeksler ve önek kuralı

Yedinci bölümde, birden çok alanı birlikte süzen sorgular için bileşik indeksleri ve onların önek kuralını tanıdık. OxiDB, birden çok alanın birleşimi üzerine kurulu bileşik indeksleri destekler ve bunlar, tıpkı yedinci bölümdeki telefon rehberi gibi davranır: alanların belirli bir sırayla dizildiği bu indeks, sıralamanın baştan başlayan bir önekini kullanan sorgulara yarar. Böylece “şu bölgedeki, şu yaş aralığındaki” gibi çok koşullu sorgular, tüm koleksiyonu taramak yerine, doğrudan bileşik indeksten hızlıca yanıtlanır.

### 18.4 İndeksten yanıt: belgeye dokunmadan saymak

Yedinci bölümdeki kapsayan indeks fikrinin — sorgunun ihtiyaç duyduğu her şeyi indeksin tek başına sağlaması — OxiDB’deki en çarpıcı uygulaması, saymadır. “Şu değere sahip kaç belge var” ya da indeksli bir alana göre “her grupta kaç belge var” sorularını yanıtlamak için, OxiDB belgelerin hiçbirine dokunmaz; indeksteki her değer altında kaç kimlik olduğunu zaten bildiği için, yalnızca bu sayıları okur. Dokuzuncu bölümde, indeksli bir alana göre yalnızca sayan gruplamaların belgelere hiç dokunmadan yanıtlanabileceğini söylemiştik; işte bunun somut karşılığı budur. Ölçümlerde bu, OxiDB’nin en belirgin üstünlüklerinden biriydi: indeksli sayım, belgeleri okuyan bir yaklaşıma kıyasla onlarca kat hızlıydı.

Bu hızlı yolun OxiDB’nin gelişimindeki öyküsü, mühendislik tercihlerinin nasıl incelikler içerdiğini güzel gösterir. Bu sayım kestirmesi, başlangıçta yalnızca belleğe öncelikli indekslerde çalışıyordu; disk-öncelikli, belleğe yansıtılmış indeksler için ise devreye girmiyor, sistem belgeleri taramaya düşüyordu. Bu, disk-öncelikli kipte sayım gruplamalarını gereksiz yere yavaşlatıyordu. Bu hızlı yolun disk-öncelikli indeksleri de kapsayacak biçimde yeniden etkinleştirilmesi, o yükleri yeniden onlarca kat hızlandırdı.

### 18.5 İndeks destekli sıralama ve erken sonlanma

Sekizinci bölümde, sıralı bir indeksin “şu alana göre sıralı ilk on kayıt” sorusunu nasıl hızlandırdığını ve erken sonlanmanın gücünü görmüştük. OxiDB, bunu doğrudan uygular: böyle bir sorgu, ilgili alanın sıralı indeksini baştan gezerek, onuncu kaydı bulunduğu an durur; geri kalanı hiç üretmez. Milyonlarca kayıt arasından en büyük ya da en küçük birkaçını bulmak, bu sayede neredeyse anlık olur.

Burada da, OxiDB’nin gelişiminden öğretici bir öykü vardır. Disk-öncelikli kipte, indeks destekli sıralamanın dayandığı iç gezinme, belleğe yansıtılmış indekslerde boş sonuç veriyordu; yani disk-öncelikli bir koleksiyonda sıralı sorgular, sessizce hiçbir şey döndürmüyordu — bir hata, üstelik fark edilmesi zor bir hata, çünkü sistem çökmüyor, yalnızca yanlışlıkla boş sonuç veriyordu. Bu durumun fark edilip düzeltilmesi, hem doğru sonuçları geri getirdi hem de bize, sessiz yanlışlıkların çöken sistemlerden daha tehlikeli olabileceğini bir kez daha hatırlattı. Bu yüzden bu davranış, sonradan, her iki kipte de çalıştığını güvence altına alan bir sınama ile korumaya alındı.

### 18.6 Disk-öncelikli indeksler: indeksi de bellekten çıkarmak

On üçüncü ve on altıncı bölümlerde, disk-öncelikli kipin belge gövdelerini bellekten diske taşıyarak bellek ayak izini küçülttüğünü gördük. Ama bir incelik kalmıştı: indeksler de bellekte yer kaplar ve büyük bir koleksiyonda birçok indeks, kayda değer bir bellek tüketebilir. OxiDB, disk-öncelikli kipte bu sorunu da çözer: indeksleri de diske taşır.

Bu kipte, indeksler kendi belleğe yansıtılmış dosyalarında — uzantısıyla anılırsa `.mfidx` dosyalarında — durur ve gerektiğinde sayfa sayfa belleğe getirilir; yalnızca son yazmaları tutan küçük bir bellek-içi tampon kalır. Daha da önemlisi, veritabanı yeniden açıldığında, bu indeksler baştan kurulmaz; belleğe yansıtılarak neredeyse anında yüklenir. Bunun sonucu, on üçüncü bölümün disk-öncelikli felsefesinin indekslere kadar uzanmasıdır: beş yüz bin belgelik ve birkaç indeksli bir koleksiyonu taze açan bir süreç, yalnızca birkaç megabayt yerleşik bellekle açılır — hem belge gövdeleri hem de indeksler, yerleşik bellekten çıkmış, gerektiğinde diskten getirilen, geri alınabilir veriye dönüşmüştür.

Dürüst olmak gerekir ki bu yaklaşımın sınırları da vardır. Bir indeksi ilk kez **kurmak** — sıfırdan oluşturmak — yine de tüm veriyi gezmeyi ve bir miktar geçici bellek kullanmayı gerektirir; bu, sürekli değil, kuruluş anına özgü bir yüküdür. Ayrıca bazı indeks türleri henüz tümüyle belleğe yansıtılmış değildir. Bunlar, disk-öncelikli kipin zamanla olgunlaşan, ama temel kazancını — yerleşik belleği veriyle birlikte büyümekten kurtarmayı — bugün de sunan yönleridir.

## 18.7 Diğer indeks türleri

OxiDB’nin indeks ailesi, temel alan ve bileşik indekslerin ötesine geçer ve yedinci bölümde değindiğimiz birkaç türü daha kapsar. **Benzersizlik indeksleri**, bir alanda aynı değerin iki kez yazılmasını engeller; yani yedinci bölümde saydığımız o yan faydayı — aramayı hızlandırırken bir bütünlük kuralı dayatmayı — sunar. **Süreyle dolan indeksler**, belgelere bir ömür biçer: belirli bir süre sonra dolan kayıtlar, arka planda çalışan bir süreç tarafından otomatik olarak temizlenir; oturum verisi ya da geçici kayıtlar için biçilmiş kaftandır. **Tam metin indeksleri**, yedinci bölümdeki ters indeks fikrini hayata geçirir ve metin içinde sözcük aramayı, alaka puanlamasıyla birlikte mümkün kılar; bunlara yirmi üçüncü bölümde döneceğiz. **Vektör indeksleri** ise, tam eşleşme yerine “benzerlik” aramasını — birbirine yakın anlamlı ya da yakın özellikli kayıtları bulmayı — destekler.

## 18.8 İndekslerin bedeli ve dayanıklılığı

Yedinci bölümün iki dersini OxiDB bağlamında tekrar etmekte yarar var. Birincisi, indekslerin bedavaya gelmediğidir: her yazma, yalnızca belgeyi değil, o belgeyi ilgilendiren her indeksi de güncellemek zorundadır. Bir koleksiyonda ne kadar çok indeks varsa, her yazma o kadar çok ek iş doğurur; bu yüzden OxiDB’de de, “her ihtimale karşı her şeyi indeksleme” değil, erişim örüntülerine göre seçici indeksleme doğru yaklaşımdır.

İkincisi, indekslerin de dayanıklı olması gerektiğidir. OxiDB, indeksleri diske kalıcı kılar; disk-öncelikli kipte yeniden açılışa belleğe yansıtılarak anında yükler, belleğe öncelikli kipte ise gerektiğinde yeniden kurar ya da kayıtlı biçiminden okur. Bir önceki bölümdeki kurtarma süreciyle el ele, indekslerin asıl veriyle tutarlı kalması — bir çökme sonrası bile — sessiz ama kritik bir görevdir.

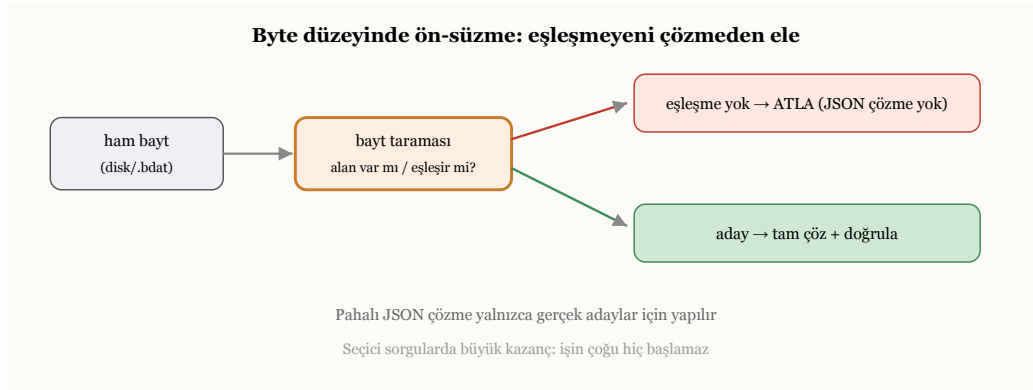
## 18.9 Bu bölümün bıraktığı yer

Bu bölümde, OxiDB’nin aramayı hızlandıran yapılarını yakın plana aldık. Alan indekslerinin, sıralı ve bellek-dostu bir yapıyla eşitlik, aralık ve sıralı getirmeyi birden desteklediğini; türler arası sıralamanın ve otomatik tarih tanınmanın bu sıralamayı belge verisine nasıl uyarladığını; bileşik indekslerin önek kuralını; indeksten doğrudan saymanın gücünü; indeks destekli sıralamayı ve erken sonlanmayı; disk-öncelikli kipte indekslerin de belleğe yansıtılarak bellekten çıkarıldığını; ve diğer indeks türlerini gördük. Bu arada, OxiDB’nin gelişiminden iki öğretici öyküyü — disk indekslerinde sayım kestirmesinin yeniden etkinleştirilmesini ve disk sıralamasındaki sessiz boş-sonuç hatasının düzeltilmesini — bu mekanizmaların ne kadar dikkat gerektirdiğinin örnekleri olarak izledik.

Ama indeksler, yedinci bölümde söylediğimiz gibi, tek başlarına yalnızca araçtır. Bir kullanıcının sorusunu alıp hangi indeksin işe yarayacağına karar veren, veriyi en az iş yaparak süzecek bir plana dönüştüren akıl, sorgu işleyicidir. Bir sonraki bölümde, OxiDB'nin sorgu motorunu — operatörlerini, indeks destekli yollarını ve eşleşmeyen belgeleri hiç çözmeden atlayan o ince bayt düzeyinde süzme tekniğini — sekizinci bölümdeki ilkelerle bağlayarak ele alacağız.

## 19 OxiDB'nin Sorgu Motoru: Operatörler, İndeks Destekli Yollar ve Bayt Düzeyinde Süzme

Önceki bölümde, OxiDB'nin aramayı hızlandıran indekslerini tanıdık ve yedinci bölümde olduğu gibi, indekslerin tek başına yalnızca araç olduğunu söyledik. Bir kullanıcının sorusunu alıp hangi indeksin işe yarayacağına karar veren, veriyi en az iş yaparak süzecek bir plana dönüştüren akıl, sorgu işleyicidir. Bu bölüm, OxiDB'nin sorgu motorunu — desteklediği operatörleri, indeks destekli yürütme yollarını ve eşleşmeyen belgeleri hiç çözmeden atlayan o ince bayt düzeyinde süzme tekniğini — sekizinci bölümdeki ilkelerle bağlayarak ele alıyor.



Şekil 19 — Byte düzeyinde ön-süzme: eşleşmeyi çözmeden eleme.

### 19.1 Sorgunun ayrıştırılması ve operatör dağarcığı

Sekizinci bölümde, sorgu işlemenin ilk adımının, ham sorguyu koşulların ve mantıksal bağların bir ağacına çevirmek olduğunu görmüştük. OxiDB de tam olarak bunu yapar: gelen JSON tabanlı sorguyu, üzerinde akıl yürütebileceği yapısal bir koşul ağacına ayrıştırır. Bu ağaçtaki koşullar, sekizinci bölümde saydığımız zengin çeşitliliği yansıtır.

En temelde **karşılaştırma** koşulları vardır: bir alanın bir değere eşit ya da eşit olmaması; bir değerden büyük, küçük ya da bir aralıkta olması. **Üyelik** koşulları, bir alanın belirli bir değerler kümesine girip girmediğini sorar. **Varlık** koşulu, bir alanın belgede bulunup bulunmadığını denetler. Belgeler iç içe ve listeli olabildiği için, belgeye özgü koşullar da vardır: bir listenin belirli bir öğeyi içermesi, belirli bir uzunlukta olması, belirli koşullara uyan bir öğe barındırması; bir alanın belirli bir türde olması; bir metnin bir kalıba uyması. Tüm bu koşullar, **mantıksal bağlarla** — ve, veya, ne...ne, değil — birleştirilebilir. Son olarak, bir alanı başka bir alanla karşılaştıran, yani tek bir belge içinde alanlar arası bir ilişki kuran koşullar da vardır. Bu dağarcık, sekizinci bölümün soyut “koşul ağacı”nı, belge verisinin tüm zenginliğiyle somutlaştırır.

### 19.2 İndeksli koşullar ile süzgeç koşulları

OxiDB'nin sorgu motorunu anlamanın anahtarı, bu koşulları **iki sınıfa** ayırmaktır. Bazı koşullar bir indeksten yararlanabilir; bazıları yararlanamaz. Eşitlik, aralık ve üyelik gibi koşullar, eğer ilgili alanın bir indeksi varsa, indeks üzerinden hızlıca az sayıda aday belgeye inebilir. Buna karşılık, bir listenin belirli koşullara uyan bir öğe barındırıp barındırmadığı, bir alanın türü, bir sayının belirli bir bölme kalanına sahip olup olmadığı, alanlar arası bir karşılaştırma ya da bir olumsuzlama gibi koşullar, doğaları gereği bir indeksle yanıtlanamaz; bunlar yalnızca belgeleri tek tek inceleyen bir **süzgeç** olarak uygulanabilir.

İşte sekizinci bölümdeki “indeks daraltır, süzgeç inceltir” iş bölümü, OxiDB’de tam olarak bu ayrım üzerine kuruludur. OxiDB, bir sorguyu aldığı anda onu, indeksle çözülebilen bir parça ile yalnızca süzgeçle çözülebilen bir parçaya ayırır. İndeksli parçayı kullanarak az sayıda aday belge belirler; sonra, kalan süzgeç koşullarını yalnızca bu adaylar üzerinde dener. Bir milyon belgeyi taramak yerine, indeksin daralttığı küçük aday kümesini süzmek, sekizinci bölümde gördüğümüz gibi kıyaslanamayacak kadar ucuzdur.

### 19.3 Yürütme: indeks, süzgeç ve tarama

Bu ayırmadan, sekizinci bölümdeki yürütme mantığının somut karşılığı doğar. OxiDB, bir sorguyu yürütürken önce indeksli parçayı çalıştırıp aday belgelerin kimliklerini elde eder. Eğer indeks, sorgunun **tüm** koşullarını zaten karşılıyorsa — yani süzgeçle inceltilecek bir şey kalmamışsa — adaylar doğrudan sonuçtur ve hiçbir ek denetim gerekmez. Aksi halde, kalan süzgeç koşulları adaylar üzerinde uygulanır ve yalnızca uyanlar sonuca alınır.

Sorgunun hiçbir koşulu indeksli bir alana denk gelmezse, sekizinci bölümde gördüğümüz son çare devreye girer: tarama. OxiDB tüm belgeleri gezer ve her birinde süzgeç koşullarını dener. Bu, kaçınılması istenen tam tarama maliyetidir; ama uygun bir indeks yoksa başka yol yoktur. İşte tam burada, OxiDB’nin en ayırt edici tekniği devreye girer.

### 19.4 Bayt düzeyinde süzme: çözmeden elemek

Sekizinci bölümde, tarama ya da süzme sırasında gözden kaçan ama kritik bir inceliğe değinmiştik. Bir belgenin koşula uyup uymadığını denemek, normalde onu diskteki ya da bellekteki kodlanmış biçiminden, kullanıma hazır bir nesneye **çözmeyi** gerektirir. Eğer milyonlarca belgenin hepsini baştan sona çözüp sonra çoğunu eler atarsanız, çok büyük bir emek boşa gider. Sekizinci bölümde, akıllı bir tasarımın bir belgenin koşula uyup uymadığını onu tümüyle çözmeden kestirmeye çalıştığını söylemiştik; OxiDB’nin **bayt düzeyinde süzme** tekniği, tam olarak budur ve bu bölümün vitrin konusudur.

Fikir şudur. OxiDB, bir belgeyi süzgeçten geçirirken, onu önce nesneye çevirmez; bunun yerine, belgenin kodlanmış baytları üzerinde, doğrudan koşulu kontrol eder. Belgenin yapılandırılmış ikili biçimi — dördüncü ve on altıncı bölümlerde değindiğimiz, JSON’un daha zengin akrabası — bir alanın değerine, tüm belgeyi çözmeden atlamaya olanak tanır. Böylece koşula uymayan belgeler, hiç çözülmeden, yalnızca baytlarına bakılarak elenir. Yalnızca koşula **uyan** belgeler, sonuç için gereken biçime dönüştürülür. Yani sistem, sonunda atacağı belgeler için çözme emeğini hiç harcamaz.

Bunun somut etkisi, OxiDB üzerinde yapılan ölçümlerde çarpıcı biçimde görülür. İndeksin yardım edemediği, çok sayıda belgeyi süzen bir sorgu düşünün — örneğin bir koşula uyan yüz binlerce belge. Eski yaklaşım, eşleşen tüm bu belgeleri — hatta eşleşmeyenleri de denerken — nesnelere çevirip bellekte yüzlerce megabayt tutuyordu. Bayt düzeyinde süzme ise, eşleşmeyenleri hiç çözmeden atladığı için, hem belleği çok daha az kullanıyor hem de daha hızlı çalışıyordu — çünkü çözme, işin en pahalı kısmıdır ve bu yaklaşım onu çoğu belge için tümüyle ortadan kaldırır. Bu, sekizinci bölümün “atacağın şeyi çözme” içgörüsünün gerçek bir sistemdeki, ölçülmüş karşılığıdır.

Bu tekniğin OxiDB’deki gelişim öyküsü de öğreticidir, çünkü doğru çözümün ilk denemede bulunmadığını gösterir. İlk bir deneme, belgeleri yine çözüyor ama bunu farklı bir biçimde yapıyordu; ölçümler bu denemenin aslında **yavaşlattığını** ortaya koydu ve geri alındı. Asıl kazanç, ancak eşleşmeyen belgelerin hiç çözülmediği — yalnızca baytlarına bakılıp atıldığı — doğru tasarımla geldi. Burada, kitap boyunca tekrarlanan bir ders bir kez daha belirir: bir eniyilemenin gerçekten işe yarayıp yaramadığı, ancak ölçülerek bilinir; sezgi, çoğu zaman yanıltıcıdır.

Dürüst bir sınır da belirtmek gerekir. Bu bayt düzeyinde yol, süzülebilir koşullar için geçerlidir. Sonuçların sıralanması, atlanması ya da sınırlandırılması gerektiğinde, sistem yine de nesne tabanlı yola döner; çünkü sıralama,

belgeleri karşılaştırılabilir bir biçimde elde etmeyi gerektirir. Yani bayt düzeyinde süzme, büyük süzme işlerini hızlandırır, ama sıralama gibi işlerin sahibi hâlâ nesne yoludur.

## 19.5 Tekil işlemlerde erken sonlanma

Sekizinci bölümde, “şu koşula uyan bir kaydı getir” ya da “ilk eşleşeni güncelle” gibi tekil işlemlerin, ilk eşleşmeyi bulduğu an durabileceğini görmüştük. OxiDB bunu doğrudan uygular: tekil okuma, tekil güncelleme ve tekil silme işlemleri, aradıkları ilk belgeyi bulduğu an dururlar; kalan belgeleri taramaya gerek yoktur. Bu, çok sayıda belge arasından tek bir kaydı bulup işlem yapmayı, tüm koleksiyonu gezmeden, hızlı hale getirir.

## 19.6 Sunucu yoluyla ilişki

OxiDB sunucu kipinde çalıştığında, sorgu yürütme ile ağ üzerinden yanıt gönderme birbirine bağlanır. Bir okuma isteğinin yanıtı, mümkün olan her yerde bayt düzeyinde yoldan üretilir: eşleşen belgelerin baytları, çözülüp yeniden kodlanmadan, doğrudan ağ biçimine aktarılır ve istemciye gönderilir. Bayt düzeyinde yolun uygulanamadığı durumlarda — örneğin sıralama gerektiğinde — sistem nesne tabanlı yola geri döner. Sorgu motorunun bu ağ yoluyla nasıl bütünleştiği, sunucu protokolünü ele aldığımız ileriki bölümde daha net olacak; şimdilik akılda tutulacak nokta, bayt düzeyinde süzmenin yalnızca bir iç eniyileme değil, ağ üzerinden gönderilen yanıtın da daha hafif ve hızlı olmasını sağlayan, uçtan uca bir kazanç olduğudur.

## 19.7 Bildirimsel özgürlüğün yansıması

Sekizinci bölümü, bildirimsel sorgunun tanıdığı özgürlükle kapatmıştık: kullanıcı yalnızca *ne* istediğini söylediği için, sistem *nasıl* sorusunu özgürce yanıtlayabilir. OxiDB’de bu özgürlük somut biçimde görülür. Aynı sorgu, ilgili alanın bir indeksi varsa indeks destekli yoldan, yoksa bayt düzeyinde süzmeli taramadan yürütülür; ve siz sorgunuzu hiç değiştirmeden bir indeks eklediğinizde, OxiDB o indeksi kullanmaya başlar. Kullanıcı “ne”, motor “nasıl” sorusuyla ilgilenir; bu ayrım, sekizinci bölümde söylediğimiz gibi, bildirimsel sorgu ile akıllı yürütmenin aynı madalyonun iki yüzü olduğunu bir kez daha gösterir.

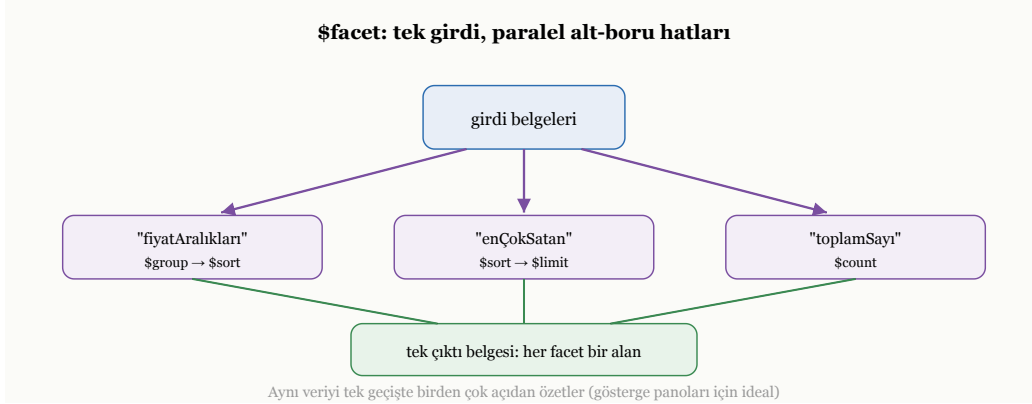
## 19.8 Bu bölümün bıraktığı yer

Bu bölümde, OxiDB’nin sorgu motorunu yakın plana aldık. Sorgunun zengin operatör dağarcığını ve nasıl bir koşul ağacına ayrıştırıldığını; koşulların indeksli ve süzgeç sınıflarına ayrılmasını ve “indeks daraltır, süzgeç inceltir” iş bölümünün bu ayrım üzerine kurulduğunu; yürütmenin indeks, süzgeç ve tarama yollarını; ve OxiDB’nin vitrin tekniği olan bayt düzeyinde süzmeyi — eşleşmeyen belgeleri hiç çözmeden eleyerek hem belleği hem hızı iyileştiren yaklaşımı — gördük. Tekil işlemlerdeki erken sonlanmayı, sunucu yoluyla bütünleşmeyi ve bildirimsel özgürlüğün bu motordaki yansımasını izledik.

Buraya kadar hep tek tek belgeleri bulup süzmekle ilgilendik. Ama dokuzuncu bölümde gördüğümüz gibi, veriye sorabileceğimiz daha zengin bir soru türü daha vardır: belgeleri gruplayan, özetleyen ve dönüştüren toplama. Bir sonraki bölümde, OxiDB’nin toplama pipeline’ını — gruplamayı, çok-yönlü analizi ve pencere fonksiyonlarını, ki bunların bazılarını bu kitap yazılırken motora ekledik — dokuzuncu bölümdeki ilkelerle bağlayarak ele alacağız.

## 20 OxiDB'nin Toplama Pipeline'ı: Gruplama, \$facet ve Pencere Fonksiyonları

Önceki bölümlerde, OxiDB'nin tek tek belgeleri nasıl bulup süzdüğünü gördük. Ama dokuzuncu bölümde öğrendiğimiz gibi, veriye sorabileceğimiz daha zengin bir soru türü vardır: belgeleri gruplayan, özetleyen ve dönüştüren toplama. Bu bölüm, OxiDB'nin toplama pipeline'ını dokuzuncu bölümdeki ilkelerle bağlayarak ele alıyor. Bu bölümün ayrı bir özelliği var: anlatacağımız yeteneklerin bir kısmı — çok-yönlü analiz ve pencere fonksiyonları — tam da bu kitap yazılırken OxiDB'ye eklendi; bu yüzden onları yalnızca bir kullanıcı olarak değil, gelişim öyküleriyle birlikte anlatabiliyoruz.



Şekil 20 — \$facet: tek girdi üzerinde paralel alt-boru hatları.

### 20.1 Pipeline ve aşamaları

Dokuzuncu bölümde toplamayı bir montaj hattına benzetmiştik: belgeler bir uçtan akar, bir dizi aşamadan geçer, her aşama akışı dönüştürüp bir sonrakine verir, diğer uçtan nihai sonuç çıkar. OxiDB'nin toplama motoru tam olarak böyle çalışır ve dokuzuncu bölümde tanıdığımız aşamaların hepsini sunar: süzme, gruplama, sıralama, atlama, sınırlama, yeniden şekillendirme, sayma, listeyi belgelere açma, başka koleksiyondan ilişkili veri getirme, çok-yönlü analiz ve pencere fonksiyonları. Bu aşamalar arka arkaya dizilerek, basit yapı taşlarından karmaşık analizler kurulur — dokuzuncu bölümdeki bileşilebilirlik ilkesinin somut karşılığıdır bu.

### 20.2 Baştaki süzmeyi öne çekmek

Dokuzuncu bölümde, en temel toplama eniyilemesinin süzmeyi olabildiğince erkene almak olduğunu söylemiştik; çünkü akışı erkenden daraltmak, sonraki her aşamanın yükünü azaltır. OxiDB bunu somut biçimde yapar: bir pipeline'ın başında bir süzme aşaması varsa, OxiDB onu pipeline'ın geri kalanından **ayırıp**, belgeleri depodan okurken doğrudan uygular. Üstelik bu baştaki süzme, on dokuzuncu bölümde gördüğümüz indeks destekli yollardan yararlanabilir; yani pipeline'ın girdisi, tüm koleksiyon taranmadan, indeksle daraltılmış bir aday kümesi olabilir. Böylece toplama, daha işin başında, üzerinde çalışacağı veri miktarını en aza indirir.

### 20.3 Akış halinde gruplama

Dokuzuncu bölümde gruplamanın toplamamanın kalbi olduğunu ve doğası gereği çok sayıda belgeye dokunduğunu söylemiştik. OxiDB, bu maliyeti yönetmek için gruplamayı **akış halinde** yapar: tüm belgeleri belleğe yığıp sonra gruplamak yerine, belgeleri depodan tek tek akıtarak, her birinin grup anahtarına ve biriktiricilere katkısını anında işler. Böylece dev bir koleksiyon, bütünüyle belleğe alınmadan gruplanabilir.

Burada, on dokuzuncu bölümdeki bayt düzeyinde fikrin gruplamaya uygulanmış bir biçimini görürüz. Bir grup hesabı, çoğu zaman belgenin yalnızca birkaç alanına ihtiyaç duyar: grup anahtarına ve toplanan ya da ortalaması alınan alana. OxiDB, gruplama sırasında belgeyi bütünüyle çözmek yerine, kodlanmış baytlarından **yalnızca gereken alanları** çıkarır; gerisini hiç dokunmadan geçer. Bu, on dokuzuncu bölümdeki “atacağın şeyi çözme” ilkesinin gruplamadaki yankısıdır: gruplama için gereksiz olan alanlar, hiç çözülmez. Biriktiriciler — sayma, toplama, ortalama, en büyük, en küçük — bu akış üzerinde, her grup için bir özet biriktirerek çalışır.

Dokuzuncu bölümün bir kestirmesi de OxiDB’de doğrudan karşımıza çıkar. Eğer gruplama yalnızca **saymaysa** ve grup anahtarının bir indeksi varsa, OxiDB belgelere hiç dokunmaz; on sekizinci bölümde gördüğümüz gibi, indeksten doğrudan her grubun belge sayısını okur. Bu, dokuzuncu bölümde değindiğimiz “indeksli sayma gruplaması, belgelere dokunmadan yanıtlanabilir” fikrinin somut karşılığıdır ve ölçümlerde belirgin bir hız kazancı sağlar.

## 20.4 Çok-yönlü analiz: \$facet

Dokuzuncu bölümde çok-yönlü analizi, aynı süzölmüş veri kümesinden birden çok özeti tek bir geçişte üretmek olarak tanımlamıştık — bir alışveriş sitesinin sonuç sayfasındaki, kategoriye göre sayılar, fiyat dağılımı ve en iyi birkaç ürünün hep birlikte istenmesi gibi. OxiDB’nin bu yeteneği, tam da bu kitap yazılırken eklendi ve fikri dokuzuncu bölümdekiyle birebir aynıdır: aynı girdi üzerinde birkaç alt-pipeline’ı çalıştırıp her birinin sonucunu ayrı bir alanda toplamak.

Uygulaması zarif biçimde sadedir. OxiDB, bu aşamaya gelen belgeleri bir kez hazırlar ve her alt-pipeline’ı, bu aynı girdi üzerinde, var olan pipeline motorunu yeniden kullanarak çalıştırır. Sonuç, tek bir belgedir; o belgenin her alanı, bir alt-pipeline’ın sonuç listesidir. Bu sayede, üç ayrı sorgu yapıp veriyi üç kez taramak yerine, tek bir geçişte üç özet birden üretilir. Sadeliğin bir gereği olarak, bir alt-pipeline’ın içine yine bir çok-yönlü analiz koymak ya da oraya bir yan etki — örneğin başka bir koleksiyona yazma — sığdırmak engellenir; çünkü bunlar, çok-yönlü analizin “aynı veriyi birden çok açıdan özetle” amacına aykırıdır.

## 20.5 Pencere fonksiyonları

Dokuzuncu bölümde, gruplamanın yanında ikinci büyük analitik biçimi — satırları çökertmeden, her satıra komşularından türeyen bir değer ekleyen pencere fonksiyonlarını — tanımlamıştık. Kümülatif toplam, hareketli ortalama, sıralama, bir önceki satırın değeri gibi hesaplar bu aileye girer. OxiDB’nin bu yeteneği de bu kitap yazılırken eklendi ve dokuzuncu bölümdeki üç adımlı modeli doğrudan izler.

OxiDB, pencere fonksiyonunu uygularken önce belgeleri bir alana göre **bölümlere** ayırır — örneğin her bölgeyi ayrı bir bölüm yapar. Sonra her bölümü, belirtilen sıralama alanına göre düzenler — örneğin tarihe göre. En sonra, her belge için, o belgenin çevresindeki bir pencereye bakarak istenen değeri hesaplar ve onu belgeye yeni bir alan olarak ekler — ama belgeyi silmeden, her satırı koruyarak. Pencere üzerindeki biriktiriciler — toplam, ortalama ve benzerleri — kümülatif toplamı ya da hareketli ortalamayı üretir; sıralama işlevleri, satırın bölüm içindeki yerini verir; kaydırma işlevi ise bir önceki ya da sonraki satırın değerini getirir. Dokuzuncu bölümde söylediğimiz gibi, gruplama “her grup için tek özet ver”, pencere fonksiyonu ise “her satırı koru, ona komşularından bir değer ekle” der; OxiDB bu ikisini ayrı ama bütünleyici araçlar olarak sunar.

## 20.6 Toplamanın performans gerçeği: disk-öncelikli durum

Dokuzuncu bölümde toplamanın, tekil aramalardan farklı bir performans karakteri taşıdığını ve doğası gereği çok sayıda, kimi zaman tüm belgelere dokunduğunu söylemiştik. OxiDB’de bu gerçek, on altıncı bölümdeki depolama

kipleriyle ilginç bir biçimde etkileşir ve burada dürüstçe ele alınmayı hak eder.

Belleğe öncelikli kipte, gruplamanın akış halinde okuduğu her belge zaten bellektedir; bu yüzden toplama hızlıdır. Disk-öncelikli kipte ise her belge, on altıncı bölümde gördüğümüz gibi, belleğe yansıtılmış `.bdat` dosyasından okunur. Toplama tüm belgelere dokunduğu için, bu okuma maliyeti her belge için tekrarlanır. Eğer `.bdat` sıkıştırılmışsa, her belgenin okunması bir de açma — yani işlemci işi — gerektirir; ve toplama tüm koleksiyonu gezdiği için, bu açma maliyeti birikir. OxiDB üzerinde yapılan ölçümler bunu net biçimde gösterdi: disk-öncelikli, sıkıştırılmış kipte tüm koleksiyonu tarayan bir gruplama, belleğe öncelikli kipe kıyasla belirgin biçimde — milyon belgelik bir veride kat kat — yavaşlıyordu.

Bu yavaşlığın iki ayrı kökü vardı ve ikisi de bu kitabın yazımı sırasında giderildi. Birincisi, sayma gruplamalarında indeksten yararlanan o hızlı yolun disk-öncelikli indekslerde devre dışı kalmasıydı; bunu on sekizinci bölümde anlattık ve yeniden etkinleştirilmesi, sayma gruplamalarını taramadan kurtardı. İkincisi, sıkıştırmanın getirdiği açma maliyetiydi; on altıncı bölümde değindiğimiz sıkıştırmasız saklama seçeneği, her belgeyi açma zorunluluğunu ortadan kaldırarak — ve sıfır-kopya erişimi mümkün kılarak — tüm koleksiyonu tarayan gruplamaları büyük ölçüde hızlandırdı. Böylece disk-öncelikli kip, sıkıştırmasız ve indeks destekli haliyle, toplama performansında belleğe öncelikli kibe çok yaklaştı. Bu öykü, dokuzuncu, on üçüncü ve on altıncı bölümlerdeki ilkelerin — toplamanın çok şeye dokunması, sıkıştırmanın işlemci bedeli ve indeksten yararlanma — gerçek bir sistemde nasıl iç içe geçtiğinin güzel bir örneğidir.

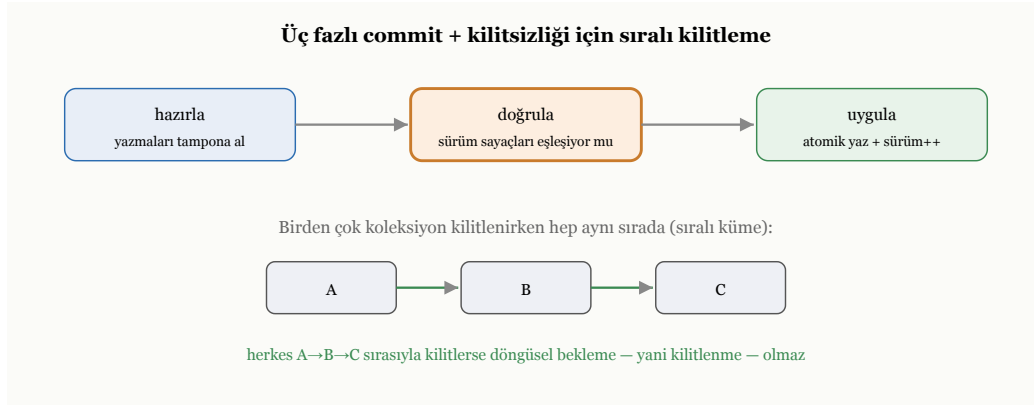
## 20.7 Bu bölümün bıraktığı yer

Bu bölümde, OxiDB'nin toplama pipeline'ını yakın plana aldık. Aşamalarının dokuzuncu bölümdeki montaj hattını nasıl somutlaştırdığını; baştaki süzmenin öne çekilip indeksle daraltıldığını; gruplamanın akış halinde, yalnızca gereken alanları çözerek çalıştığını; sayma gruplamalarının indeksten doğrudan yanıtlandığını; çok-yönlü analizin aynı girdiyi birkaç açıdan tek geçişte özetlediğini; ve pencere fonksiyonlarının satırları koruyarak komşulardan değer türettiğini gördük. Ayrıca disk-öncelikli kipte toplamanın performans gerçeğini ve onu iyileştiren iki düzeltmeyi — bu kitap yazılırken yapılan iki işi — dürüstçe izledik.

Buraya kadar, Kısım III boyunca hep okuma tarafıyla — saklama, bulma, süzme, özetleme — ilgilendik. Ama onuncu bölümde gördüğümüz gibi, bir veritabanının asıl zorlu sınavı, eşzamanlı yazmalar ve “ya hep ya hiç” güvencesidir. Bir sonraki bölümde, OxiDB'nin işlem mekanizmasını — onuncu bölümde tanıdığımız iyimser eşzamanlılık denetimini ve onun üç fazlı tamamlama düzenini — somut olarak ele alacağız.

## 21 OxiDB’de İşlemler: İyimser Eşzamanlılık ve Üç Fazlı Commit

Kısım III boyunca, buraya kadar hep okuma tarafıyla ilgilendik: OxiDB’nin veriyi nasıl sakladığını, dayanıklı kıldığını, indekslediğini, sorguladığını ve özetlediğini gördük. Ama onuncu bölümde öğrendiğimiz gibi, bir veritabanının asıl zorlu sınavı, eşzamanlı yazmalar ve “ya hep ya hiç” güvencesidir. Bu bölüm, OxiDB’nin işlem mekanizmasını — onuncu bölümde tanıdığımız iyimser eşzamanlılık denetimini ve onun üç fazlı tamamlama düzenini — somut olarak ele alıyor.



Şekil 21 — Üç fazlı commit ve kilitlenmeyi önleyen sıralı kitleme.

### 21.1 OxiDB neden iyimser yolu seçti

Onuncu bölümde, yalıtımı sağlamanın üç felsefesini görmüştük: kötümser kitleme, çok sürümlü MVCC ve iyimser OCC. OxiDB, bunlardan **iyimser eşzamanlılık denetimini** seçer. Bu seçimin altında, onuncu bölümde tanımladığımız iyimser varsayım yatar: çoğu zaman çatışmalar nadirdir; iki işlem aynı belgeye aynı anda dokunmaz. Madem öyle, baştan kilitleyip herkesi bekletmek yerine, işlemleri serbestçe çalıştırıp çatışmayı yalnızca tamamlama anında denetlemek daha verimlidir.

Onuncu bölümdeki mağaza-kasası benzetmesini hatırlayalım: ürünleri sepete koyarken kimseye sormazsınız, kasada almak istediğinizin hâlâ uygun olup olmadığı kontrol edilir, bir sorun çıkarsa o turu baştan yaparsınız. OxiDB’nin işlemleri tam olarak böyle davranır. Bu yaklaşım, çatışmaların nadir olduğu tipik belge iş yüklerinde — ki belgeler çoğu zaman bağımsız bütünler olduğu için çatışmalar gerçekten nadirdir — kimseyi boşuna bekletmediği için hızlıdır.

### 21.2 İyimser akışın üç fazı

OxiDB’nin işlemleri, onuncu bölümdeki iyimser akışı **üç fazlı bir tamamlama** düzeniyle hayata geçirir. Bu üç fazı tek tek görelim, çünkü OCC’nin somut işleyişi tam olarak buradadır.

İşlem çalışırken, yaptığı değişiklikleri hemen asıl depoya uygulamaz; onları bir kenarda **biriktirir**. Yani işlem boyunca, dışarıdan bakan hiç kimse bu yarım değişiklikleri görmez; depo, işlem tamamlanana dek dokunulmamış gibi durur. Bu, onuncu bölümde OCC’nin “değişiklikleri biriktir” adımının karşılığıdır. İşlem, aynı zamanda dokunduğu her belgenin **sürüm numarasını** hatırlar; çünkü OxiDB, her belgeye, her değiştiğinde artan bir sürüm sayacı iliş­tirir.

İşlem “tamamla” dediğinde, üç faz devreye girer. Birinci faz, **hazırlıktır**: biriktirilen tüm değişiklikler bir araya getirilir. İkinci faz, **doğrulamadır**: işlemin dokunduğu her belgenin sürüm numarasının, işlem onu okuduğundan bu yana değişip değişmediği kontrol edilir. Eğer tüm sürümler hâlâ aynıysa — ki iyimser varsayıma göre çoğu zaman böyledir — hiçbir çatışma yok demektir ve üçüncü faza geçilir. Üçüncü faz, **tamamlamadır**: biriktirilen değişiklikler asıl depoya uygulanır, ilgili belgelerin sürüm numaraları artırılır ve değişiklikler, bir önceki bölümlerde gördüğümüz yazma-öncesi günlüğe yazılarak dayanıklı kılınır.

Ya doğrulama fazı başarısız olursa? Yani işlem çalışırken, dokunduğu bir belgeyi başka biri değiştirmiş ve onun sürüm numarasını artırmışsa? O zaman bir **çatışma** saptanmış demektir; işlem iptal edilir ve değişiklikleri uygulanmaz. Çağırın taraf, işlemi baştan deneyebilir. Bu, onuncu bölümde anlattığımız “çatışmada iptal et ve yeniden dene” davranışının tam karşılığıdır; sürüm numaraları, çatışmayı saptamanın aracıdır.

## 21.3 Dayanıklılıkla bağ

İşlemler, on yedinci bölümdeki dayanıklılık mekanizmasıyla doğrudan bütünleşir. Bir işlem tamamlandığında, biriktirilmiş değişiklikleri tek seferde yazma-öncesi günlüğe yazılır; on yedinci bölümde, her günlük kaydının bir işlem kimliği taşıdığını söylemiştik — işte o kimlik, bir kaydın hangi işleme ait olduğunu belirtir ve kurtarmada işlemleri bir bütün olarak ele almayı sağlar. İşlemin atomikliği — ya hep ya hiç — iki şeyden gelir: değişikliklerin yalnızca tamamlama anında, hep birlikte uygulanması ve bunların günlükle dayanıklı kılınması. Bir çökme olursa, kurtarma, tamamlanmış işlemlerin değişikliklerini bir bütün olarak geri getirir; yarım kalmış, hiç tamamlanmamış bir işlemin biriktirilmiş ama uygulanmamış değişiklikleri ise zaten depoya hiç inmediği için kaybolur — ki bu da istenen davranıştır.

## 21.4 Kilitlenmeye karşı tasarımla bağımsızlık

OxiDB iyimser bir sistem olduğu için, çoğu zaman hiç kilit almaz; bu, onuncu bölümdeki kilitlenme tehlikesini büyük ölçüde ortadan kaldırır. Ama bir işlemin, birden çok koleksiyona birden dokunması gereken durumlar vardır ve bu gibi yerlerde, koleksiyonların kilitlerinin alınması gerekebilir. İşte burada OxiDB, onuncu bölümde tanıttığımız en zarif disipline başvurur: kilitleri her zaman **aynı, belirli bir sırada** almak.

Onuncu bölümde, kilitlenmenin döngüsel bir bekleme olduğunu — birinci işlemin A’yı tutup B’yi, ikincinin B’yi tutup A’yı beklemesini — anlatmıştık. Eğer her işlem, kilitleri her zaman aynı sırayla alırsa, bu döngü hiç oluşmaz; çünkü iki işlem de önce aynı kilidi almaya çalışır ve biri diğerini beklerken ters bir bağımlılık kurulmaz. OxiDB, koleksiyon kilitlerini sıralı bir düzende aldığı için, kilitlenme **tasarım gereği imkânsızdır** — onu sezip çözmeye çalışan bir mekanizmaya bile gerek kalmaz. Bu, onuncu bölümdeki soyut “sıralı kilit disiplini” fikrinin, gerçek bir sistemde bir kilitlenme sınıfını tümüyle ortadan kaldıran somut bir uygulamasıdır.

## 21.5 Tek belge ile çok belge

Dördüncü ve onuncu bölümlerde, belge dünyasında atomikliğin doğal sınırının tek bir belge olduğunu vurgulamıştık. OxiDB’de bu doğrudan görülür: tek bir belgeyi değiştiren bir işlem, zaten atomiktir, çünkü o belge tek bir bütün olarak yazılır; burada karmaşık bir işlem makinesine gerek yoktur. Buraya kadar anlattığımız üç fazlı, sürüm-doğrulamalı işlem düzeneği, asıl olarak **birden çok belgeye ya da birden çok işleme** birden dokunan, hepsinin birlikte tutarlı kalması gereken durumlar içindir. Bu, dördüncü bölümdeki “tutarlı kalması gereken birim ne kadar büyük” sorusunun OxiDB’deki yankısıdır: birimi tek bir belgeye sığdırabiliyorsanız işler basit kalır; birim birçok belgeye yayıldığında, bu işlem düzeneği devreye girer.

## 21.6 Küme durumunda işlemler

Onuncu ve on ikinci bölümlerde, işlemlerin tek makinede zor, birçok makinede daha da zor olduğunu görmüştük. OxiDB tek bir düğümde, az önce anlattığımız iyimser düzeneği kullanır. Bir kümede çalıştıyındaysa, tamamlanmış bir işlemin biriktirilmiş değışiklikleri, on ikinci bölümdeki konsensüs katmanına **tek bir bütün olarak** verilir; böylece işlemin tüm değışiklikleri ya birlikte replikasyona girer ya da hiçbiri girmez. Bunun ayrıntılarına, ölçeklendirmeyi ele aldığımız ileriki bölümde döneceğiz; şimdilik akılda tutulacak nokta, işlemin “ya hep ya hiç” niteliğinin, tek düğümden kümeye taşındığında da korunduğudur.

## 21.7 İyimserliğin bedeli

Onuncu bölümün dürüst dersini OxiDB bağlamında tekrar etmek gerekir: iyimser yaklaşım her zaman en iyisi değildir. Çatışmaların nadir olduğu durumlarda muhteşemdir; kimse boşuna beklemez ve işlemler hızla tamamlanır. Ama çatışmaların sık olduğu, birçok işlemin aynı belgeye saldırdığı durumlarda israfli olabilir; çünkü o işlemler sona kadar çalışıp, doğrulama fazında çatışma bulup iptal edilir ve yeniden denenir — yapılan iş boşa gider. OxiDB’nin iyimser tercihi, belge veritabanlarının tipik iş yüküne — çoğunlukla bağımsız belgelere dokunan, düşük çatışmalı yüklere — uygundur; ama herkesin aynı birkaç belgeye yarıştığı bir senaryoda, bu tercihin bedeli artar. Onuncu bölümde söylediğimiz gibi, doğru seçim her zaman iş yüküne bağlıdır.

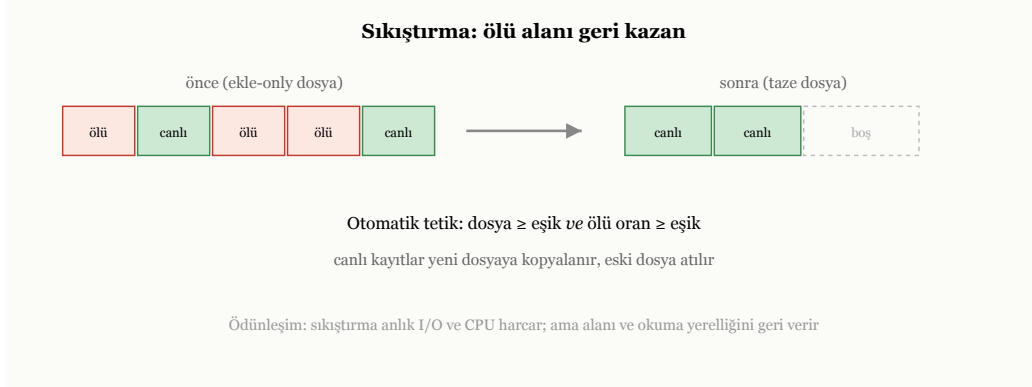
## 21.8 Bu bölümün bıraktığı yer

Bu bölümde, OxiDB’nin işlem mekanizmasını yakın plana aldık. OxiDB’nin iyimser eşzamanlılık denetimini neden seçtiğini; değışiklikleri biriktirip, tamamlama anında üç fazlı bir düzenle — hazırlık, sürüm doğrulama ve tamamlama — işleyişini; çatışmanın sürüm numaralarıyla nasıl saptanıp iptale yol açtığını; işlemlerin dayanıklılıkla nasıl bütünleştğini; kilitlenmenin sıralı kilit disipliniyle nasıl tasarımdan dışlandığını; tek belge ile çok belge ayrımını; küme durumundaki davranışı; ve iyimserliğin bedelini gördük.

İşlemleri ele alırken, OxiDB’nin disk-öncelikli kipinin append-only doğasına birkaç kez değindik. Beşinci ve on altıncı bölümlerde söylediğimiz gibi, append-only depolama veriyi asla üzerine yazmaz; her güncelleme yeni bir kayıt ekler ve eskisi ölü alana dönüşür. Bu ölü alan zamanla birikir ve onu geri kazanmak gerekir. Bir sonraki bölümde, OxiDB’nin bu temizlik işini — sıkıştırmayı (compaction), onu ne zaman ve nasıl yaptığını, hatta bu kitap yazılırken eklenen otomatik tetikleyicisini — ele alacağız.

## 22 OxiDB’de Sıkıştırma: Ölü Alan ve Otomatik Tetikleme

İşlemleri ele alırken, OxiDB’nin disk-öncelikli kipinin append-only doğasına birkaç kez değindik. Beşinci ve on altıncı bölümlerde söylediğimiz gibi, append-only depolama veriyi asla üzerine yazmaz; her güncelleme yeni bir kayıt ekler ve eskisi ölü alana dönüşür. Bu ölü alan zamanla birikir ve onu geri kazanmak gerekir. Bu bölüm, OxiDB’nin bu temizlik işini — sıkıştırmayı, onu güvenli nasıl yaptığını ve bu kitap yazılırken eklenen otomatik tetikleyicisini — ele alıyor. Sıkıştırma, append-only bir motorun ayrılmaz, sessiz bakım işidir.



Şekil 22 — Sıkıştırma: ölü alanın taze dosyaya kopyalanarak geri kazanılması.

### 22.1 Ölü alan nereden gelir

Beşinci bölümdeki muhasebe defteri benzetmesini hatırlayalım: append-only bir depo, hiçbir eski satırı silmez; bir kaydı güncellemek için onun yeni hâlini defterin sonuna yazar ve en son yazılanı geçerli sayar. Bunun kaçınılmaz sonucu, eski hâllerin defterde öylece durmaya devam etmesidir. Bir belgeyi yüz kez güncellerseniz, disk-öncelikli kipin veri dosyasında o belgenin yüz kopyası birikir; yalnızca sonuncusu geçerlidir, gerisi **ölü alandır**. Silmeler de benzer biçimde, kaydı fiziksel olarak çıkarmak yerine ölü olarak işaretler.

Bu olgunun kritik bir sonucu vardır: veri dosyasının boyutu, **yaşayan verinin** boyutuyla değil, **yapılan yazma sayısı**yla büyür. Yoğun güncelleme alan bir koleksiyonun veri dosyası, içindeki gerçek, geçerli veri küçük kalsa bile, zamanla çoğu ölü kayıttan oluşan bir dev haline gelebilir. Bu hem yer israfıdır hem de on altıncı bölümde gördüğümüz okuma ve tarama maliyetlerini artırır; çünkü daha büyük bir dosyada gezinmek gerekir. İşte bu şişkinliği gidermek, sıkıştırmamanın görevidir.

### 22.2 Sıkıştırma ne yapar

Sıkıştırma, beşinci bölümde tanımladığımız temizlik işidir: veri dosyasını baştan yazıp, yalnızca **yaşayan** kayıtları taze bir dosyaya kopyalamak; ölü kayıtları geride bırakmak. Muhasebe defteri dolup taşıduğında oturup yalnızca hâlâ geçerli satırları temiz bir deftere geçirmeye benzer. İşlem bittiğinde, taze dosya yalnızca geçerli veriyi içerir ve eski, şişmiş dosyanın yerini alır; biriken tüm ölü alan geri kazanılmıştır. OxiDB üzerinde yapılan ölçümlerde, yoğun güncelleme sonrası şişmiş bir veri dosyasının, sıkıştırmayla birkaç kat küçüldüğü — ve bu sırada tüm yaşayan verinin ve indeksli sorguların hem sıkıştırma sırasında hem de yeniden açılışa eksiksiz kaldığı — doğrulandı.

Belleğe öncelikli kipte ise sıkıştırma farklı bir anlam taşır. O kipte append-only bir veri dosyası yoktur; belgeler bellekteki eşlemede yerinde güncellenir, dolayısıyla biriken bir ölü alan da yoktur. Orada “sıkıştırma”, yalnızca güncel bellek içeriğinin taze bir anlık görüntüsünü diske yazmaktan ibarettir. Yani sıkıştırma, asıl olarak disk-öncelikli kipin bir ihtiyacıdır.

## 22.3 Zor kısım: yaşayan sistemde güvenle yapmak

Sıkıştırmanın asıl zorluğu, onu veritabanı çalışmaya devam ederken, okuma ve yazmalar sürerken güvenle yapmaktır. Burada, on birinci bölümdeki eşzamanlılık kaygılarının somut bir örneği belirir ve OxiDB'nin çözümü öğreticidir.

Sorunu görelim. Disk-öncelikli kipte bir belgeyi okumak iki adımdır: kimlik-konum dizininden belgenin dosyadaki konumunu bulmak ve sonra o konumdan baytları okumak. Şimdi düşünün: bir okuyucu konumu öğrenmiş, tam o baytları okuyacakken, sıkıştırma araya girip dosyayı baştan yazsa ve belgeler yeni dosyada başka konumlara taşınsa ne olur? Okuyucunun elindeki konum, artık eski dosyaya aittir; yeni dosyada o konumda bambaşka bir şey olabilir. Bu, sessiz bir veri bozulmasıdır.

OxiDB bunu, veri dosyası tutamağı üzerinde bir okuma-yazma engeliyle çözer. Normal işlemler — okumalar ve yazmalar — bir **paylaşımlı okuma kilidi** tutar ve bu kilit, hem konumu bulma hem de baytları okuma adımlarının ikisini birden kapsar; yani bir konum, her zaman onu okuduğu dosyaya karşı kullanılır, asla başka bir dosyaya karşı değil. Sıkıştırma ise **dışlayıcı bir yazma kilidi** alır: bu kilidi tutarken hiçbir okuma ya da yazma araya giremez; sıkıştırma, taze dosyayı kurar, kimlik-konum dizinini yeni konumlara göre yeniden eşler ve dosyayı bütünüyle değiştirir; ancak bundan sonra kilidi bırakır. Böylece hiçbir konum, hiçbir zaman değiştirilmiş bir dosyaya karşı kullanılmaz. Bu, on birinci bölümdeki eşzamanlılık ilkelerinin — paylaşımlı ve dışlayıcı erişimin dikkatli düzenlenmesinin — gerçek bir doğruluk sorununu nasıl çözdüğünün somut bir örneğidir.

## 22.4 İndeksler neden sağ kalır

Sıkıştırmanın zarif bir yanı, indeksleri bozmadan çalışmasıdır ve bunun nedeni öğreticidir. On sekizinci bölümde gördüğümüz gibi, OxiDB'nin alan indeksleri belgeleri **kimliğe** göre tutar, fiziksel konuma göre değil. Sıkıştırma ise yalnızca belgelerin dosyadaki **konumlarını** değiştirir — kimliklerini değil. Bu yüzden veri dosyasını baştan yazmak, indeksleri geçersiz kılmaz; yalnızca kimlikten konuma giden o küçük eşlemenin yeniden kurulması yeterlidir. Kimlik ile fiziksel konum arasındaki bu ayrışma — indekslerin kimliğe, dizinin konuma bakması — sıkıştırmayı, indekslere hiç dokunmadan yapılabilir kılar. İyi bir soyutlama ayrımının, karmaşık bir işi nasıl basitleştirdiğinin güzel bir örneğidir bu.

## 22.5 Açık çağrıdan otomatik tetiklemeye

Sıkıştırmanın OxiDB'deki gelişimi, bu kitap yazılırken yaşanan iki aşamalı bir öyküdür ve onu anlatmak, mühendislik kararlarının nasıl olgunlaştığını gösterir.

İlk aşamada, sıkıştırma **açıkça çağrılan** bir işlemdi: ne zaman gerek duyduğunuza siz karar verir ve sıkıştırmayı elle başlattırdınız. Bu, az önce anlattığımız tüm güvenlik özelliklerine — yaşayan sistemde güvenli yeniden yazma, indekslerin sağ kalması — sahipti ve ölçümlerle doğrulanmıştı; ama ne zaman çalıştırılacağına karar vermek kullanıcıya kalıyordu.

İkinci aşamada, bir **otomatik tetikleyici** eklendi. Artık OxiDB, periyodik bakım sırasında ucuz bir ölü-alan ölçütüne bakar ve gerektiğinde sıkıştırmayı kendisi başlatır. Ölçüt şöyledir: veri dosyası hem belirli bir **asgari boyutu** aşmışsa hem de içeriğinin belirli bir **oranından fazlası ölüyse**, sıkıştırma tetiklenir. Ölü oran, basitçe, dosyanın ne kadarının yaşayan veri **olmadığıyla** ölçülür — yaşayan baytların dosya boyutuna oranı ne kadar düşükse, ölü oran o kadar yüksektir.

İki koşulun birlikte aranması bilinçlidir. Asgari boyut koşulu, küçük dosyalar için boşuna sıkıştırma yapılmasını engeller; çünkü küçük bir dosyada kazanılacak yer azdır, sıkıştırmanın maliyetine değmez. Ölü oran koşulu ise,

ancak gerçekten kayda değer bir ölü alan biriktiğinde sıkıştırma yapılmasını sağlar; az miktarda ölü alan için dosyayı baştan yazmak israf olurdu. İki eşik birden sağlandığında, yani dosya hem yeterince büyük hem de yeterince ölüyse, sıkıştırma devreye girer.

Bu otomatik tetikleyicinin zarif bir özelliği, **kendini sınırlamasıdır**. Bir sıkıştırma yapıldığında, ölü alan sıfırlanır; dolayısıyla ölçüt bir süre daha sağlanmaz ve sıkıştırma boşuna tekrar tekrar tetiklenmez. Sistem, yalnızca ölü alan yeniden birikip eşiği aştığında tekrar sıkıştırır. Bu eşikler ayarlanabilir ve on ikinci bölümde gördüğümüz gibi per-koleksiyon belirlenebilir; istenirse otomatik tetikleme tümüyle kapatılıp sıkıştırma elle yönetilebilir.

## 22.6 Sıkıştırmanın bedeli ve dengesi

Sıkıştırma bedava değildir: tüm yaşayan veriyi baştan yazmayı gerektirir, yani bir disk yükü taşır; ve dışlayıcı kilidi tuttuğu kısa süre boyunca diğer işlemleri bekletir. Bu yüzden onu sürekli değil, ölü alan gerçekten büyüdüğünde yapmak gerekir — az önceki eşiklerin amacı tam olarak budur. Sıkıştırma, beşinci bölümde söylediğimiz gibi, arka planda yürüyen, amorti edilmiş bir bakım işidir: dosyayı sürekli temiz tutmaya çalışmak yerine, kirlilik belli bir düzeye ulaştığında toplu bir temizlik yapmak, hem maliyeti hem de kesintiye en aza indirir.

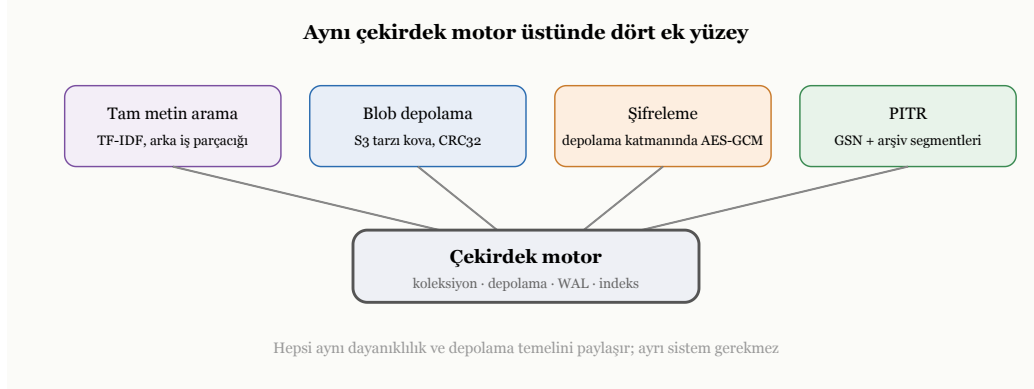
## 22.7 Bu bölümün bıraktığı yer

Bu bölümde, append-only bir motorun ayrılmaz bakım işini — sıkıştırmayı — OxiDB bağlamında ele aldık. Ölü alanın nereden geldiğini ve veri dosyasını yazma sayısı ile nasıl şişirdiğini; sıkıştırmanın yalnızca yaşayan kayıtları taze bir dosyaya geçirip ölü alanı geri kazandığını; bunu yaşayan bir sistemde güvenle yapmak için kullanılan okuma-yazma engelini; indekslerin kimliğe dayandığı için neden sağ kaldığını; açık çağrıdan otomatik tetiklemeye uzanan gelişim öyküsünü ve otomatik tetikleyicinin ölü-oran ölçütünü, kendini sınırlamasını ve dengesini gördük.

Buraya kadar Kısım III'te, OxiDB'nin çekirdek belge motorunu — depolama, dayanıklılık, indeks, sorgu, toplama, işlem ve sıkıştırma — eksiksiz dolaştık. Ama on beşinci bölümde söylediğimiz gibi, OxiDB klasik bir belge veritabanının ötesine geçen birkaç ek yetenek de sunar. Bir sonraki bölümde, bu ek yüzeylerden başlıcalarını — tam metin aramayı, büyük ikili nesneler için nesne deposunu, dururken şifrelemeyi ve zamanın bir noktasına geri dönmeyi sağlayan kurtarmayı — ele alacağız.

## 23 OxiDB'nin Ek Yüzeyleri: Tam Metin Arama, Blob Depolama, Şifreleme ve PITR

On beşinci bölümde, OxiDB'nin klasik bir belge veritabanının ötesine geçen birkaç ek yetenek sunduğunu söylemiştik. Kısım III boyunca buraya kadar çekirdek belge motorunu — depolama, dayanıklılık, indeks, sorgu, toplama, işlem ve sıkıştırma — dolaştık. Bu bölüm, çekirdeğin etrafındaki dört önemli ek yüzeyi ele alıyor: tam metin aramayı, büyük ikili nesneler için blob depolamayı, dururken şifrelemeyi ve zamanın bir noktasına geri dönmeyi sağlayan kurtarmayı. Bu dört yüzeyin ortak özelliği, hepsinin aynı çekirdek motorun üzerine oturması ve isteğe bağlı olmasıdır; kullandığınız kadarının bedelini ödersiniz.



Şekil 23 — Tek çekirdek motor üstündeki dört ek yüzey.

### 23.1 Tam metin arama: ters indeksin somutu

Yedinci bölümde, metnin içinde sözcük aramayı mümkün kılan **ters indeksi** — “sözcük, o sözcüğü içeren belgeler” eşleşmesini — tanımiştık ve metin aramanın alaka düzeyiyle sıralama gerektirdiğini söylemiştik. OxiDB'nin tam metin arama yeteneği, bu fikrin doğrudan uygulamasıdır.

Bu yetenek birkaç bakımdan dikkate değerdir. Birincisi, yalnızca düz metni değil, **zengin biçimleri** de indeksleyebilmesidir: web sayfalarından, ofis belgelerinden, taşınabilir belge dosyalarından, hatta görüntülerden metin çıkarıp indeksleyebilir. Yani bir belgeye eklenmiş bir dosyanın içeriği bile aranabilir hale gelir. İkincisi, indeklemeyi **arka planda** yapmasıdır. Metni sözcüklere ayırıp ters indeksi güncellemek pahalı bir iştir; bunu her yazmanın ortasında yapmak, yazmaları yavaşlatırdı. Bunun yerine OxiDB, indeksleme işlerini bir kuyruğa koyar ve onları ayrı bir çalışan iş parçacığında, yazma yolunun dışında işler. Bunun bir sonucu vardır ve onu dürüstçe söylemek gerekir: arama indeksi, yazmalarla **nihai olarak tutarlıdır** — yani yeni yazılan bir belge, çok kısa bir gecikmeyle aranabilir hale gelir. Bu, on birinci bölümdeki nihai tutarlılık fikrinin, bir alt sistem düzeyindeki küçük bir yankısıdır: hız uğruna, anlık tutarlılıktan biraz ödün verilir.

Üçüncüsü, **alaka puanlamasıdır**. Yedinci bölümde, bir sözcüğü içeren yüzlerce belgenin hepsinin aynı ölçüde ilgili olmadığını söylemiştik. OxiDB, sonuçları, bir sözcüğün belgede ne sıklıkta geçtiğine ve genel olarak ne kadar yaygın bir sözcük olduğuna bakan, klasik terim-sıklığı yaklaşımının olgunlaşmış bir biçimini kullanarak puanlar. Böylece, sözcüğün yoğun geçtiği kısa bir belge, onu bir kez içeren uzun bir belgenin önüne çıkar. Bu, yedinci bölümün “alaka düzeyi” kavramının somut karşılığıdır.

## 23.2 Blob depolama: büyük ikili nesnelere yer açmak

Dördüncü bölümde, sınırsız büyüyen ya da çok büyük verileri bir belgenin içine gömmenin tuzağına değinmiştik: belge şişer, her okumada o yük taşınır. Büyük ikili nesneler — görüntüler, dosyalar, medya — bu tuzağın en belirgin örneğidir; bir megabaytlık bir görüntüyü bir belgenin içine gömmek, o belgeyi her okuduğunuzda o megabaytı da taşımak demektir.

OxiDB, bu ihtiyaca ayrı bir **blob depolama** yüzeyiyle yanıt verir. Bu yüzey, büyük ikili nesneleri belgelerin dışında, kovalar halinde düzenlenmiş bir nesne deposunda tutar; her nesnenin yanında, ona dair üst veriler ve içeriğinin bozulmadığını doğrulayan bir bütünlük damgası bulunur. Bu, yaygın bir bulut nesne deposu arayüzüne benzer ve onunla aynı mantığı izler: yapılandırılmış belge verisini bir yerde, büyük opak baytları başka bir yerde tutmak. Dördüncü bölümün ilkesiyle tam örtüşür: belgeleri yalın ve hızlı okunur tutmak için, büyük ve gömülmesi sakıncalı veriyi belgeden ayırıp ona referansla işaret etmek. Böylece belgeleriniz küçük kalır, büyük nesneler ise onlara uygun, ayrı bir depoda verimli biçimde yönetilir.

## 23.3 Dururken şifreleme: depolama sınırında koruma

On dördüncü bölümde, şifrelemenin iki cephesinden söz etmiştik: aktarım sırasında ve dururken. OxiDB, dururken şifrelemeyi, depolama katmanına **saydam** biçimde yerleştirir. “Saydam” olması şu anlama gelir: üstteki katmanlar — sorgu, indeks, işlem — şifrelemenin varlığından habersizdir; şifreleme, yalnızca veri diske inmeden hemen önce ve diskten okunduktan hemen sonra, depolama sınırında devreye girer. On altıncı bölümde, baytların diske yazılmadan önce bir hazırlık adımından — sıkıştırma ve ardından şifreleme — geçtiğine değinmiştik; işte dururken şifreleme tam o adımda yapılır.

Bu yetenek isteğe bağlıdır: motora bir şifreleme anahtarı verirsiniz açılır, vermezseniz hiçbir bedeli olmaz. Açıldığında, diske yazılan her şey şifrelenir; böylece diski ya da dosyaları çalan biri, on dördüncü bölümde söylediğimiz gibi, yalnızca anlamsız baytlar görür. Ama o bölümün dürüst uyarısı burada da geçerlidir: şifreleme yalnızca anahtarı kadar güvenlidir, ve dururken şifreleme çalışan diske karşı korur, çalışan sisteme zaten meşru biçimde girmiş bir saldırgana karşı değil. Anahtarın güvenli tutulması, bu korumanın ayrılmaz parçasıdır.

## 23.4 Zaman-noktasına kurtarma: çökmenin ötesinde

Altıncı bölümde, kurtarmanın sizi en son tamamlanmış duruma — çökmeden hemen öncesine — geri getirdiğini görmüştük. Ama bazen istenen, çökmeden geri dönmek değil, **zamanda geriye gitmektir**. Hatalı bir güncelleme tüm bir koleksiyonu bozmuş olabilir; yanlış bir komut önemli veriyi silmiş olabilir; kötü bir dağıtım, saatler boyunca hatalı veri yazmış olabilir. Bu durumlarda, sistemin son tutarlı haline değil, **belirli bir geçmiş ana** dönmek istersiniz. OxiDB’nin zaman-noktasına kurtarma yeteneği — kısaca PITR — tam da bunu sağlar.

Bu yeteneğin nasıl çalıştığını kavramsal olarak görelim. OxiDB, bu yetenek açıkken, dayanıklı kılınan her yazmaya **küresel, sürekli artan ve duvar saatiyle damgalanmış** bir sıra numarası verir. Yazma-öncesi günlük dolup döndükçe, eski bölümleri silinmek yerine **arşivlenir** — yani mühürlenmiş parçalar halinde saklanır. Bir yedek alındığında, o yedek bir taban anlık görüntüsü ile birlikte, “bu yedek şu sıra numarasına kadarki durumu içerir” diyen bir su damgası taşır. Belirli bir noktaya geri dönmek istediğinizde, OxiDB bu tabandan başlar ve arşivlenmiş günlüğü, seçtiğiniz ana — bir sıra numarasına, bir zaman damgasına ya da “en sona” — kadar ileri **oynatır**. Üstelik bu oynatma, işlem sınırlarına saygı gösteren, tutarlı kesim noktalarında durur; yani yarım kalmış bir işlemin ortasında değil, temiz bir noktada geri dönersiniz.

Bu model — küresel sıra numarası, arşivlenen günlük, taban yedek ve ileri oynatma — altıncı bölümdeki dayanıklılık fikrinin zengin bir uzantısıdır. Çökmeden kurtarma, günlüğü taban üzerine son ana kadar oynatır; PITR ise aynı

günlüğü, **seçtiğiniz** bir ana kadar oynatır. İkisi de aynı temel fikre — niyeti günlüğe yaz, sonra oynat — dayanır; PİTR, o fikri zaman boyunca esnetir. Bu da isteğe bağlıdır; kapalıyken hiçbir maliyeti yoktur.

## 23.5 Ortak iplik: tek motor, isteğe bağlı yetenekler

Bu dört yüzeyin hepsinin ortak bir özelliği vardır ve onu görmek, OxiDB'nin tasarım felsefesini özetler. Hepsi, aynı çekirdek motorun üzerine oturur; hiçbirini ayrı, kopuk bir sistem değildir. Ve hepsi isteğe bağlıdır: tam metin araması istemiyorsanız indeksleme yapılmaz; büyük nesneleriniz yoksa blob deposu boş kalır; şifreleme istemiyorsanız anahtar vermezsiniz; zaman-noktasına kurtarmaya ihtiyacınız yoksa arşivleme hiç çalışmaz. Kullanmadığınız bir yetenek, bir bedel getirmez. Bu, on beşinci bölümde değindiğimiz “tek çekirdek, çok yüz” felsefesinin bir başka yüzüdür: çekirdek sade ve odaklı kalır, ek yetenekler ise onun üzerine, gerektiğinde devreye giren katmanlar olarak eklenir.

Bir başka ortak nokta, her birinin bu kitapta daha önce kurduğumuz bir kavramın somut bir örneği olmasıdır. Tam metin arama, yedinci bölümün ters indeksidir; blob depolama, dördüncü bölümün “büyüyü gömme, ayır” ilkesidir; dururken şifreleme, on dördüncü bölümün şifreleme cephesidir; zaman-noktasına kurtarma, altıncı bölümün dayanıklılık fikrinin zamana yayılmış halidir. Yani bu ek yüzeyler, OxiDB'ye özgü tuhaflıklar değil, kitabın ilk iki kısmında öğrendiğimiz ilkelerin gerçek dünyadaki uygulamalarıdır.

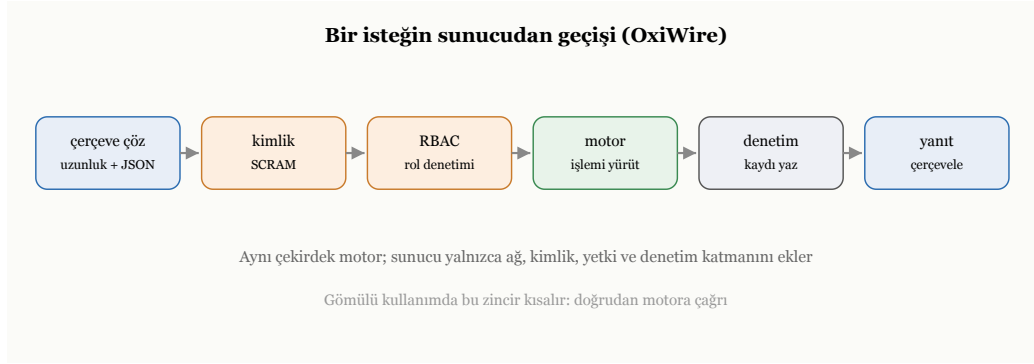
## 23.6 Bu bölümün bıraktığı yer

Bu bölümde, OxiDB'nin çekirdeğin ötesindeki dört ek yüzeyini ele aldık. Tam metin aramanın, arka planda çalışan, zengin biçimleri bile indeksleyen ve alaka puanlamasıyla sıralayan bir ters indeks olduğunu; blob depolamanın büyük ikili nesneleri belgelerden ayrı, kova tabanlı bir nesne deposunda tuttuğunu; dururken şifrelemenin depolama sınırında saydam biçimde çalışan, isteğe bağlı bir koruma olduğunu; ve zaman-noktasına kurtarmanın, dayanıklılık fikrini zamanda geriye gidebilecek biçimde genişlettiğini gördük. Hepsinin aynı motora oturduğunu, isteğe bağlı olduğunu ve daha önce öğrendiğimiz ilkelerin somut örnekleri olduğunu da gördük.

Buraya kadar OxiDB'nin yeteneklerinden söz ettik, ama bu yeteneklere uzaktan, bir ağ üzerinden nasıl erişildiğine hiç değinmedik. Gömülü kipte her şey doğrudan işlev çağrılarıyla olur; ama OxiDB bir sunucu olarak çalıştığında, istemcilerle bir ağ protokolü üzerinden konuşmalı, kimlik doğrulamalı, yetkilendirmeli ve güvenliği sağlamalıdır. Bir sonraki bölümde, OxiDB'nin sunucu katmanını — kendi ikili iletişim protokolünü, kimlik doğrulamasını, rol tabanlı erişim denetimini ve denetim günlüğünü — on dördüncü bölümdeki güvenlik ilkeleriyle bağlayarak ele alacağız.

## 24 OxiDB'nin Sunucu Katmanı: OxiWire Protokolü, Kimlik Doğrulama, RBAC ve Denetim

Buraya kadar OxiDB'nin yeteneklerinden söz ettik, ama bu yeteneklere uzaktan, bir ağ üzerinden nasıl erişildiğine değinmedik. Birinci ve on beşinci bölümlerde gördüğümüz gibi, OxiDB gömülü kipte çalışırken her şey doğrudan işlev çağrılıyla olur; orada ağ, kimlik doğrulama, protokol diye bir sorun yoktur. Ama OxiDB bir **sunucu** olarak çalıştığında, istemcilerle bir ağ protokolü üzerinden konuşmalı, kimlik doğrulamalı, yetkilendirmeli ve güvenliği sağlamalıdır. Bu bölüm, OxiDB'nin sunucu katmanını — kendi iletişim protokolünü, kimlik doğrulamasını, rol tabanlı erişim denetimini ve denetim günlüğünü — on dördüncü bölümdeki güvenlik ilkeleriyle bağlayarak ele alıyor.



Şekil 24 — Bir isteğin sunucu katmanlarından geçişi.

### 24.1 Çerçeveleme sorunu ve OxiWire protokolü

Bir ağ bağlantısı, özünde, kesintisiz bir bayt akışıdır; mesajlar arasında doğal bir sınır yoktur. Bu yüzden bir protokolün çözmesi gereken ilk sorun, **çerçevelemedir**: alıcının, bir mesajın nerede bitip diğerinin nerede başladığını bilmesi. OxiDB bunu, her mesajın önüne onun uzunluğunu yazarak çözer: alıcı önce uzunluğu okur, sonra tam o kadar bayt okuyarak mesajı bütün olarak alır, ardından bir sonraki mesajın uzunluğunu bekler. Bu basit ama sağlam çerçeveleme, akış üzerinde mesajları net biçimde ayırır.

Mesajın içeriği iki biçimde kodlanabilir. Birincisi, insan tarafından okunabilir, JSON tabanlı bir biçimdir; hata ayıklamak ve basit istemciler yazmak kolaydır. İkincisi, **OxiWire** adı verilen, daha hızlı bir ikili biçimdir. İkili biçimin avantajı, on dokuzuncu bölümdeki bayt düzeyinde fikrle bağlanır: bir sorgu yanıtı, belgeleri nesnelere çevirip yeniden metne dökmek zorunda kalmadan, doğrudan ikili biçimde gönderilebilir. Böylece sunucu yolu, yalnızca sorgu işlemede değil, ağ üzerinden yanıt göndermede de gereksiz dönüşümlerden kaçınır.

Bir istek, özünde, bir **komut** ve onun argümanlarından oluşur: ne yapılacağı (ekle, bul, güncelle, topla...) ve hangi koleksiyon üzerinde, hangi sorguyla. Sunucu bu isteği alır, çözer ve birazdan göreceğimiz aşamalardan geçirerek işler. OxiDB'nin neden genel amaçlı bir web protokolü yerine kendi protokolünü kullandığı, verimlilikle: veritabanı yüküne uygun, hafif bir çerçeveleme ve ikili bir seçenek sunar. Yine de, on beşinci ve yirmi üçüncü bölümlerde değindiğimiz gibi, web istemcileri için ayrı bir HTTP arayüzü de mevcuttur; her istemci, kendine en uygun kapıdan girer.

## 24.2 Bağlantı modeli ve el sıkışma

Bir sunucu, aynı anda birçok istemci bağlantısını karşılamak zorundadır. OxiDB, gelen istekleri işlemek için bir çalışan havuzu kullanır: belirli sayıda iş parçacığı, gelen isteklere paralel olarak hizmet verir. Uzun süre boşta kalan bağlantılar, kaynakları boşa tutmamak için bir süre sonra kapatılır. Bu ayarlar — kaç çalışan, ne kadar boşta kalma süresi, hangi adres ve veri dizini — ortam değişkenleriyle yapılandırılır; sunucuyu işletmenin pratik düğmeleridir bunlar.

Bir istemci ilk bağlandığında, kimlik doğrulamadan önce küçük bir **el sıkışma** gerçekleşir: sunucu kendini tanıtır, yeteneklerini bildirir ve iki taraf nasıl konuşacaklarını kararlaştırır. Bu, kimlik doğrulamanın öncesinde, herhangi bir gizli bilgi gerektirmeyen bir adımdır; asıl güvenlik denetimleri ondan sonra başlar.

## 24.3 Kimlik doğrulama: SCRAM ile parolayı hattan geçirmeden

On dördüncü bölümde, kimlik doğrulamanın iki temel kuralını görmüştük: parolalar asla düz metin saklanmaz, yavaş ve tuzlanmış bir özetle tutulur; ve parolanın kendisi ağ üzerinden gönderilmez, bir meydan-okuma yanıt yöntemiyle bilindiği kanıtlanır. OxiDB, sunucu kimlik doğrulamasında tam olarak bu iki kuralı uygular.

OxiDB, parolaları doğrularken bir meydan-okuma yanıt protokolü kullanır: sunucu bir soru gönderir, istemci parolasını kullanarak ona doğru yanıtı üretir, ama parolanın kendisi hattan hiç geçmez. Böylece bağlantıyı dinleyen biri, parolayı yakalayamaz. Parolaların kendisi ise, on dördüncü bölümün ilk kuralına uygun olarak, yavaş ve tuzlanmış bir özetle saklanır; böylece veritabanı sızsa bile parolalar geri elde edilemez ve kaba kuvvet saldırıları pratikte imkânsız hale gelir. Bu, on dördüncü bölümdeki soyut güvenlik ilkelerinin gerçek bir sistemde nasıl somutlaştığının net bir örneğidir.

## 24.4 Yetkilendirme: rol tabanlı geçit

Kimlik doğrulandıktan sonra, on dördüncü bölümdeki ikinci soru gelir: bu kullanıcı ne yapabilir? OxiDB, bunu rol tabanlı bir erişim denetimiyle yanıtlar ve üç temel rol tanımlar. **Yönetici** rolü her şeyi yapabilir. **Okuma-yazma** rolü, belge ekleme, güncelleme, silme, indeks oluşturma, toplama, işlem gibi veriyle çalışmanın olağan işlemlerini yapabilir, ama kullanıcı yönetimi gibi yönetimsel işlemleri yapamaz. **Okuma** rolü ise yalnızca okuyabilir — bulma, sayma, toplama gibi veriyi değiştirmeyen işlemler.

Her gelen komut, işlenmeden önce bu rol denetiminden geçer: kullanıcının rolünün, o komutu çalıştırmaya izni var mı? İzni yoksa, komut daha motora ulaşmadan reddedilir. Bu, on dördüncü bölümdeki en az ayrıcalık ilkesinin somut bir geçididir: her kullanıcı, yalnızca rolünün izin verdiği işlemleri yapabilir. OxiDB ayrıca, bir kullanıcının rolünü veritabanı düzeyinde geçersiz kılmaya da izin verir; böylece aynı kullanıcı, farklı veritabanlarında farklı yetkilere sahip olabilir.

Bu geçidin gerçekten işlediğine dair, bu kitap yazılırken yaşanan küçük ama öğretici bir örnek vardır. OxiDB'ye yeni bir komut — koleksiyonları belirli depolama seçenekleriyle oluşturan bir komut — eklendiğinde, onu yalnızca işleyicide tanımlamak yetmedi; komutun, rol denetim tablosuna da, hangi rollerin onu çalıştırabileceğini belirtecek biçimde eklenmesi gerekti. Aksi halde, komut var olsa bile, hiçbir rolün ona izni olmadığı için reddedilirdi. Bu, rol geçidinin dekoratif değil, gerçek bir kapı olduğunu — her komutun ondan açıkça geçmesi gerektiğini — gösterir.

## 24.5 Aktarım şifrelemesi ve denetim

On dördüncü bölümün diğer iki katmanı da sunucuda karşımıza çıkar. **Aktarım şifrelemesi**, sunucu bağlantısının şifreli bir kanala sarılmasıyla sağlanır; böylece hattı dinleyen biri yalnızca anlamsız baytlar görür ve istek ile yanıtın

içeriği ağ üzerinde korunur.

**Denetim** ise, on dördüncü bölümde anlattığımız “kim, ne zaman, ne yaptı” kaydını tutar. OxiDB’de bu yetenek isteğe bağlıdır; açıldığında, işlemler bir denetim günlüğüne kaydedilir. On dördüncü bölümde, denetim kayıtlarının kendisinin de yönetilmesi gerektiğini — sınırsız büyümeleri için döndürülmeleri gerektiğini — söylemiştik. OxiDB bunu olgun biçimde yapar: denetim günlüğü, boyuta göre, geçen zamana göre ya da takvim sınırlarına göre döndürülebilir ve döndürülen eski kayıtlar isteğe bağlı olarak sıkıştırılabilir. Böylece denetim, sistemin uzun süre çalışmasını engelleyen, kontrolsüz büyüyen bir yük olmaktan çıkar; on dördüncü bölümdeki “denetim kayıtları yönetilmeli” ilkesinin somut karşılığıdır bu.

## 24.6 İsteğin sunucudaki yolu

Bu parçaları bir araya getirip, bir isteğin sunucudaki yolunu izleyelim; bu, on beşinci bölümdeki yaşam döngüsünün sunucuya özgü ayrıntısıdır. İstek, ağ üzerinden çerçevelenmiş bir mesaj olarak gelir ve çözülür. Önce, henüz kimlik doğrulanmamışsa, el sıkışma ve kimlik doğrulama adımları tamamlanır. Sonra, komut **rol geçidinden** geçer: rolün bu komuta izni yoksa, istek burada reddedilir. Geçidi aşan istek, artık tanıdık yola girer: motora ulaşır, hedef koleksiyon bulunur ve istek — okuma ya da yazma — önceki bölümlerde anlattığımız mekanizmalarla işlenir. Sonuç, çerçevelenip istemciye geri gönderilir; eğer denetim açıksa, bu işlem günlüğe de kaydedilir.

Burada, on beşinci bölümde değindiğimiz birleştirici noktayı yeniden görürüz: sunucu, çekirdek motorun üzerine giydirilmiş bir ağ kabuğudur. Kimlik doğrulama, yetkilendirme, protokol, denetim — bunların hepsi, isteği asıl işleyen çekirdeğe ulaşmadan önceki ve sonraki katmanlardır. Çekirdek, gömülü kipte de sunucu kipinde de aynıdır; sunucu yalnızca ona ağ üzerinden, güvenli bir biçimde erişim sağlar. İlerideki bölümde göreceğimiz gibi, OxiDB’nin küme kipi bile, istekleri aynı işleyici yolundan geçirir; yalnızca araya replikasyon ve yönlendirme ekler.

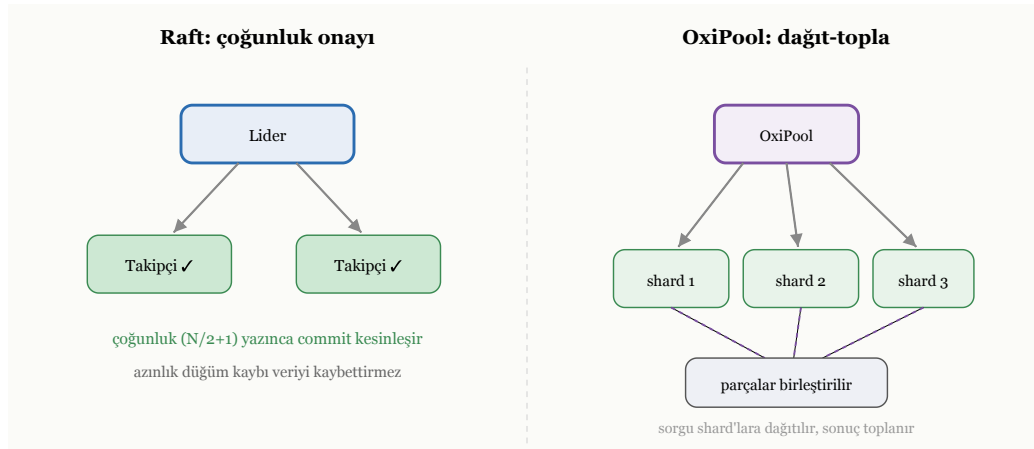
## 24.7 Bu bölümün bıraktığı yer

Bu bölümde, OxiDB’nin sunucu katmanını yakın plana aldık. Çerçeveleme sorununu ve OxiWire protokolünün uzunluk-önekli, JSON ya da ikili biçimli mesajlarını; bağlantı havuzunu ve el sıkışmayı; SCRAM tabanlı kimlik doğrulamayı ve parolanın hattan hiç geçmemesini; üç rollü yetkilendirme geçidini ve onun gerçek bir kapı olduğunu; aktarım şifrelemesini ve döndürülebilen denetim günlüğünü; ve bir isteğin sunucudaki uçtan uca yolunu gördük. Tüm bunların, çekirdek motorun üzerine giydirilmiş bir güvenlik ve iletişim kabuğu olduğunu da gördük.

Şimdiye dek hep tek bir sunucu düğümünden söz ettik. Ama on ikinci bölümde öğrendiğimiz gibi, bir veritabanı tek makinenin sınırlarına dayandığında, birçok makineye yayılmak gerekir. Bir sonraki bölümde, OxiDB’nin ölçeklendirme katmanını — on ikinci bölümdeki konsensüsü hayata geçiren Raft tabanlı kümeyi ve sharding’i sağlayan yönlendiriciyi — somut olarak, hatta bu kitap yazılırken doğruladığımız davranışlarla birlikte ele alacağız.

## 25 OxiDB’de Ölçeklendirme: Raft Kümesi ve OxiPool ile Sharding

Şimdiye dek hep tek bir sunucu düğümünden söz ettik. Ama on ikinci bölümde öğrendiğimiz gibi, bir veritabanı tek makinenin sınırlarına dayandığında, birçok makineye yayılmak gerekir. On ikinci bölümde ölçeklendirmenin iki büyük tekniğini — aynı veriyi çoğaltan replikasyonu ve veriyi bölen sharding’i — genel ilkeler düzeyinde tanıdık. Bu bölüm, bu iki tekniğin OxiDB’de nasıl hayata geçtiğini ele alıyor: replikasyon için Raft tabanlı kümeyi ve sharding için OxiPool adlı yönlendiriciyi. Bu bölümün ayrı bir değeri, anlatacağımız davranışların bir kısmının bu kitap yazılırken çok-düğümlü testlerle **doğrulanmış** olmasıdır; böylece on ikinci bölümün vaatlerini yalnızca tarif etmekle kalmayıp, gösterilmiş davranışlarla bağlayabiliyoruz.



Şekil 25 — Raft çoğunluk onayı ve OxiPool dağıt-topla.

### 25.1 Raft kümesi: replikasyon ve konsensüs

On ikinci bölümde, güçlü tutarlı bir replikasyonun iki fikirden — tek bir otorite ve çoğunluk mutabakatı — beslendiğini görmüştük. OxiDB’nin küme kipi, tam da bunu yapan bir çoğunluk-konsensüs protokolüne dayanır. Bir grup düğüm vardır; biri **lider** olur; tüm yazmalar liderden geçer ve bir yazma, düğümlerin **çoğunluğu** onu kalıcı kıldığında “tamamlanmış” sayılır.

Bir yazmanın küme içindeki yolu şöyledir. İsteği alan düğüm, onun bir yazma olduğunu tanır ve replikasyona uygun bir günlük girdisi hazırlayıp konsensüs grubuna önerir. Çoğunluk bu girdiyi kabul ettiğinde, girdi tamamlanmış sayılır ve her düğümde uygulanır. Burada zarif bir nokta vardır: girdiyi her düğümde işleyen mekanizma, yirmi dördüncü bölümdeki tek-düğüm işleyicinin **aynısıdır**. Yani motorun tüm davranışı — depolama, indeks, sorgu — kümede de birebir aynıdır; yalnızca yazmalar, uygulanmadan önce konsensüsten geçer. Küme, çekirdeğin üzerine eklenen bir replikasyon ve sıralama katmanıdır; çekirdeği değiştirmez.

Liderin çökmesi, on ikinci bölümde gördüğümüz failover sürecini tetikler: bir takipçi, çoğunluğun oyunu alarak yeni lider olur. Çoğunluğun büyüüsü — herhangi iki çoğunluğun mutlaka kesişmesi — burada iki güvenceyi birden sağlar: aynı anda iki lider olamaz (split-brain önlenir) ve tamamlanmış hiçbir yazma kaybolmaz; çünkü yeni lideri seçen çoğunluk, o yazmayı bilen en az bir düğümü içerir.

## 25.2 Kümenin doğrulanması

On ikinci bölümün vaatleri, bu kitap yazılırken gerçek, çok-düğümlü bir test paketiyle sınılandı; bu, kitabın “anlattığını gösterme” yaklaşımının iyi bir örneğidir. Dört düğümlü bir küme kurulup şu senaryolar tek tek doğrulandı: tüm düğümlerin aynı lider üzerinde anlaşması; lidere yazılan belgelerin tüm düğümlere yayılması; liderin öldürülüp bir takipçinin devralması; failover sonrası verinin tutarlı kalması; düğümlerin azınlığının kaybedilmesine rağmen çalışmaya devam edilmesi; ve en kritik güvenlik özelliği — çoğunluğunu yitiren bir azınlığın yeni bir lider **seçmemesi**, yani split-brain’in oluşmaması. Bu senaryoların hepsi beklendiği gibi çalıştı. Yani on ikinci bölümde soyut olarak anlattığımız konsensüs vaatleri — lider seçimi, replikasyon, otomatik failover ve çoğunluk güvenliği — OxiDB’de yalnızca tasarımda değil, gösterilmiş davranışta da geçerli.

Yirmi birinci bölümdeki işlemlerle de bir bağ vardır. Bir küme kipinde, tamamlanmış bir işlemin biriktirilmiş değişiklikleri, konsensüs katmanına **tek bir bütün** olarak verilir; böylece işlemin tüm değişiklikleri ya birlikte replikasyona girer ya da hiçbiri girmez. İşlemin “ya hep ya hiç” niteliği, tek düğümden kümeye taşındığında da korunur.

Bu gücün bedeli, on ikinci bölümde söylediğimizdir: her yazma, çoğunluğa ulaşp onların onayını beklediği için bir gidiş-dönüş gecikmesi öder. Güçlü tutarlılık, liderden geçen ve çoğunlukla mühürlenene bu yolla sağlanır.

## 25.3 OxiPool: sharding ile veriyi bölmek

Raft kümesi, aynı veriyi çoğaltarak erişilebilirlik ve dayanıklılık sağlar; ama on ikinci bölümde gördüğümüz gibi, tek başına kapasite sorununu çözmez — her kopya yine tüm veriyi taşır. Veri tek bir makineye sığmaz hale geldiğinde, sharding gerekir. OxiDB bunu, bağımsız OxiDB parçalarının önünde duran **OxiPool** adlı bir yönlendiriciyle sağlar. Her parça, normal bir OxiDB örneğidir — ki istenirse o parça kendi içinde bir Raft kümesi de olabilir; bu, on ikinci bölümün “shard’la, sonra her shard’ı replikasyonla çoğalt” topolojisidir.

OxiPool, on ikinci bölümdeki sharding mekaniğini doğrudan uygular. Bir **parça anahtarı** belirlenir ve anahtarlar, doğrudan parçalara değil, çok sayıda **sanal parçaya** eşlenir; sanal parçalar da parçalara dağıtılır. Bu dolaylama, on ikinci bölümde gördüğümüz yeniden dengeleme kolaylığını sağlar: bir parça eklendiğinde, anahtarları tek tek taşımak yerine, yalnızca birkaç sanal parçayı kaydırmak yeterlidir.

Yönlendirme şöyle işler. Parça anahtarını taşıyan bir istek — örneğin o anahtara göre bir sorgu ya da o anahtarı içeren bir belge ekleme — doğrudan o anahtarın ait olduğu tek parçaya gönderilir. Anahtarı içermeyen bir istek ya da bir toplama sorgusu ise, on ikinci bölümdeki **dağıt-topla** örüntüsünü tetikler: istek tüm parçalara gönderilir, her birinden kısmi yanıt alınır ve bunlar birleştirilir. Bir sayım, parçaların sayılarını toplar; bir bulma, parçaların sonuçlarını birleştirir; bir gruplama ise, on ikinci bölümde anlattığımız gibi, parça-yerel bir hesaba ve bir birleştirme hesabına bölünerek, tek-düğümle birebir aynı sonucu üretecek biçimde yürütülür.

Bu cross-shard toplama birleştirmesi de bu kitap yazılırken uçtan uca bir testle doğrulandı. Gerçek bir OxiPool yönlendiricisi, üç bağımsız parçanın önüne kuruldu; belgeler parça anahtarına göre parçalara dağıtıldı; ve bir sayımın parçalar boyunca toplandığı, bir bulmanın birleştirildiği, hem küresel hem de anahtar başına bir gruplamanın bölünüp birleştirilerek tek-düğüm beklentisiyle birebir eşleştiği, ve parça anahtarı taşıyan bir sorgunun tek bir parçaya yönlendirilip doğru sonucu verdiği doğrulandı. Yani on ikinci bölümün scatter-gather ve parçalar-arası toplama vaatleri de, gösterilmiş davranışla destekleniyor.

Burada, üçüncü ve on ikinci bölümlerde attığımız bir tohum meyve verir: kendi içinde bütün olan belgeler sharding’e doğal yatkındır. OxiPool’un belge başına yönlendirmesinin temiz çalışmasının nedeni budur — her belge bağımsız olduğu için, hangi parçaya gideceğine kolayca karar verilir ve onu okumak için başka parçalara bakmak gerekmez.

## 25.4 Dürüst sınırlar

On ikinci bölümde, dağıtımın bedava olmadığını ve sharding'in olgun bir otomasyon gerektirdiğini söylemiştik. OxiDB bağlamında bu sınırları dürüstçe belirtmek gerekir. OxiDB'nin sharding'i, parçaları **otomatik dengeleyen** bir bileşene henüz sahip değildir; parçalar yapılandırılır, kendiliğinden yeniden dağıtılmaz. Cross-shard *toplama* vardır, ama parçalar arasında veriyi otomatik taşıyan bir denge mekanizması yoktur. Benzer biçimde, on birinci bölümde değindiğimiz ayarlanabilir tutarlılık düğmeleri — örneğin “en yakın takipçiden oku” gibi okuma tercihleri ya da işlem başına ince ayarlanan yazma güvenceleri — OxiDB'de henüz sınırlıdır; baskın yol, liderden geçen ve çoğunlukla mühürlenmiş güçlü tutarlı yoldur. Bunlar, sistemin olgunlaştıkça doldurabileceği boşluklardır; ama bir kitabın görevi, sistemin yapabildikleri kadar yapamadıklarını da dürüstçe göstermektir.

## 25.5 İkisini birleştirmek

On ikinci bölümde, replikasyon ile sharding'in birbirinin alternatifi değil, tamamlayıcısı olduğunu ve büyük sistemlerde birlikte kullanıldığını söylemiştik. OxiDB bu birleşimi destekler: veri, kapasite için OxiPool ile parçalara bölünür; ve her parça, erişilebilirlik için kendi içinde bir Raft kümesi olarak çoğaltılabilir. Böylece sistem hem tek makineye sığmayan veriyi taşır, hem de herhangi bir makinenin çökmesine dayanır. Bu, on ikinci bölümde anlattığımız standart büyük ölçek topolojisinin OxiDB'deki karşılığıdır.

Ama on ikinci bölümün kapanış dersini de tekrarlamak gerekir: dağıtım, kapasite ve erişilebilirlik kazandırır, ama koordinasyon, gecikme ve sorgu kısıtları getirir. Tek bir düğümde çalışan OxiDB, çoğu zaman bir kümeden ya da sharding'li bir kurulumdan çok daha basittir ve çok daha kolay akıl yürütülür. OxiDB'nin ölçeklendirme yeteneklerini, bir varsayılan değil, gerçekten ihtiyaç doğduğunda başvurulacak araçlar olarak görmek doğru olur.

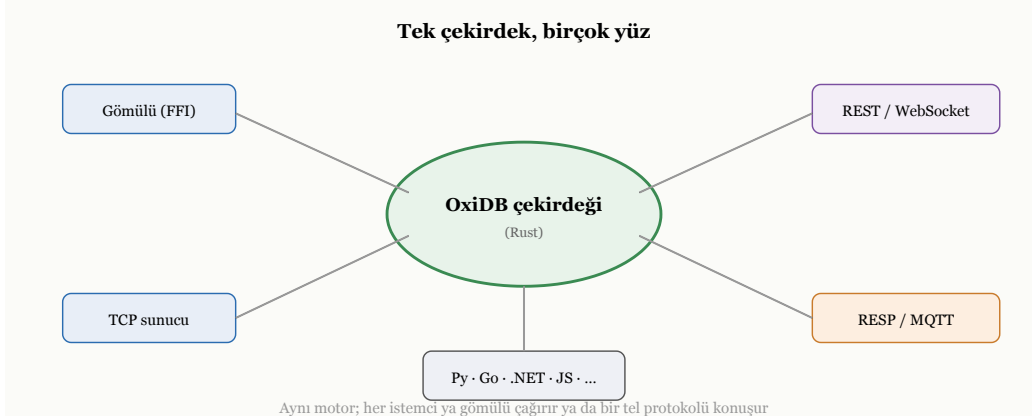
## 25.6 Bu bölümün bıraktığı yer

Bu bölümde, OxiDB'nin ölçeklendirme katmanını ele aldık. Raft tabanlı kümenin, on ikinci bölümdeki konsensüsü — lider seçimi, çoğunluk mutabakatı, otomatik failover ve split-brain güvenliğini — nasıl hayata geçirdiğini ve bunların çok-düğümlü testlerle nasıl doğrulandığını gördük. OxiPool'un, parça anahtarı, sanal parçalar ve dağıt-topla ile sharding'i nasıl sağladığını ve cross-shard toplamanın uçtan uca nasıl doğrulandığını izledik. Dürüst sınırları — otomatik dengeleyicinin ve ince tutarlılık ayarlarının henüz eksik olduğunu — ve iki tekniğin birleşik topolojisini gördük.

Buraya kadar OxiDB'yi hep motor ve sunucu tarafından, yani veriyi sağlayan taraftan tanıdık. Ama bir veritabanı, ona erişen uygulamalar kadar değerlidir. Bir sonraki bölümde, OxiDB'ye farklı programlama dillerinden nasıl erişildiğini — istemci kütüphanelerini, gömülü ve sunucu kiplerini ve aynı çekirdeğin nasıl birçok dilin önüne çıktığını — ele alacağız.

## 26 Uyumluluk Katmanları ve İstemciler

Önceki bölümün sonunda, bir veritabanının ona erişen uygulamalar kadar değerli olduğunu söylemiştik. Buraya kadar OxiDB’yi hep veriyi sağlayan taraftan — motor ve sunucu olarak — tanıdık. Bu bölüm, madalyonun öteki yüzüne, OxiDB’ye **nasıl erişildiğine** bakıyor: farklı programlama dillerinden kullanmayı sağlayan istemci kütüphanelerine ve OxiDB’yi başka ekosistemlerin protokolleriyle konuşturan uyumluluk katmanlarına. Bu bölüm, on beşinci bölümde tanıttığımız “tek çekirdek, çok yüz” felsefesinin en somut görünümüdür.



Şekil 26 — Tek çekirdek, birçok istemci ve protokol yüzü.

### 26.1 Çekirdeğe iki kapı

Birinci bölümde, bir veritabanına iki uçtan erişilebileceğini söylemiştik: gömülü olarak ya da bir sunucu üzerinden. OxiDB’ye erişim de bu iki kapıdan birinden geçer. **Gömülü** kapıda, uygulamanız OxiDB ile aynı süreçte çalışır ve veriyi doğrudan, bir ağ aradan geçmeden, işlev çağrılarıyla erişir; bu çok hızlıdır ve hiçbir sunucu kurulumu gerektirmez. **Sunucu** kapısında ise uygulamanız, yirmi dördüncü bölümdeki OxiWire protokolüyle ağ üzerinden bağlanır; bu, aynı veriyi birçok uygulamanın aynı anda erişmesini sağlar.

İstemci kütüphanelerinin işi, bu iki kapı arasındaki farkı uygulamadan **gizlemektir**. Bir istemci kütüphanesi kullandığınızda, ister gömülü ister sunucu kipinde çalışsın, karşınıza aynı kavramlar — koleksiyonlar, sorgular, indeksler, toplama, işlemler — aynı biçimde çıkar; çünkü hepsi, on beşinci bölümde söylediğimiz gibi, aynı çekirdek motorun farklı kapılarıdır.

### 26.2 Diğer dillere köprü: FFI

OxiDB’nin çekirdeği Rust dilinde yazılmıştır; peki başka dillerden nasıl kullanılır? Yanıt, **C uyumlu bir köprü** katmanındadır. Çekirdek, C dilinin çağırma kurallarına uyan bir arayüz sunar; ve neredeyse her programlama dili, C ile konuşabildiği için, bu köprü sayesinde OxiDB’yi gömülü olarak çağırabilir. Bu, yaygın bir tekniktir: düşük seviyeli, performans-kritik bir çekirdeği bir kez yazmak ve onu, bu C köprüsü üzerinden, birçok yüksek seviyeli dile açmak. Böylece OxiDB’nin gömülü hızı, yalnızca Rust’a değil, başka dillere de sunulur.

### 26.3 İstemci kütüphaneleri

Bu köprünün ve sunucu protokolünün üzerine kurulu istemci kütüphaneleri, OxiDB’yi çeşitli programlama dillerinden — örneğin Python, Go, .NET ve JavaScript’ten — kullanılabilir kılar. Her istemci, ya gömülü köprüye

doğrudan bağlanır, ya da sunucuya ağ üzerinden konuşur; bazı istemciler, tek bir arayüzün arkasında her iki kipi birden sunar, böylece uygulamanız gömülünden sunucuya geçerken kodunu değiştirmez.

İstemcilerin ortak özelliği, hepsinin aynı komut dağıtıcısını yansıtmasıdır; çünkü hepsi aynı çekirdeğe konuşur. Bu birörnekliğin somut bir örneği, bu kitap yazılırken yaşandı. OxiDB'ye yeni bir yetenek — koleksiyonları belirli depolama seçenekleriyle oluşturma — eklendiğinde, bu yetenek önce çekirdekte ve sunucu protokolünde tanımlandı; sonra istemcilerin her birine, bu komutu çağıran küçük bir sarmalayıcı eklendi. Python'da bir, Go'da bir, .NET'te bir, JavaScript'te bir. Hepsi aynı altta yatan komutu çağırıyordu; yalnızca her dilin kendi deyimine — Python'un adlandırılmış argümanlarına, Go'nun işlev-seçenek desenine — uygun bir biçim alıyordu. Bu, istemcilerin neden hepsinin aynı kavramları sunduğunu güzel gösterir: onlar bağımsız veritabanları değil, tek bir motorun farklı dillerdeki kapılarıdır.

## 26.4 Uyumluluk katmanları: başka protokolleri konuşmak

İstemci kütüphaneleri, uygulamaların OxiDB'nin kendi protokolüyle konuşmasını kolaylaştırır. Ama bazı uygulamalar, zaten **başka bir protokolü** konuşacak biçimde yazılmıştır ve OxiWire öğrenmek istemez. İşte burada OxiDB'nin uyumluluk katmanları devreye girer: OxiDB'yi, var olan araçların ve istemcilerin tanıdığı yüzlerle sunarak, onların az değişiklikle ya da hiç değişmeden çalışmasını sağlar.

Birinci uyumluluk yüzü, bir **bellek-içi anahtar-değer katmanıdır**. Bu katman, yaygın bir önbellek protokolüyle uyumlu çalışır; böylece o protokolü konuşan mevcut önbellek istemcileri ve araçları, OxiDB'ye doğrudan bağlanabilir. İkinci bölümde tanıdığımız anahtar-değer modelini hatırlayın; OxiDB, kendi belge motorunun üzerine, bu sade ve yaygın anahtar-değer yüzünü de giydirir ve sıralı kümeler, yayınla-abone ol gibi tanıdık yetenekleri sunar.

İkinci uyumluluk yüzü, bir **mesajlaşma protokolü** desteğidir. Bu, nesnelerin interneti gibi, çok sayıda aygıtın yayınla-abone ol biçiminde haberleştiği senaryolar içindir; ve anahtar-değer katmanının yayınla-abone ol kanallarıyla çapraz çalışabilir, yani bir protokolden yayınlanan bir mesaj diğerinden dinlenebilir.

Üçüncü ve belki en geniş uyumluluk yüzü, **web yüzeyidir**: doğrudan bir HTTP arayüzü, gerçek zamanlı bir abonelik kanalı, bir kimlik sistemi ve belge düzeyinde kurallar. Yirmi üçüncü ve on beşinci bölümlerde değindiğimiz bu Firebase benzeri yüz, web ve mobil uygulamaların, araya bir sunucu katmanı koymadan, doğrudan OxiDB ile — gerçek zamanlı güncellemeler, kimlik doğrulama ve belge başına erişim kurallarıyla — çalışmasını mümkün kılar. JavaScript istemcisi, tam da bu web yüzeyi üzerinden konuşur.

## 26.5 Tek çekirdek, çok yüz

Tüm bu erişim biçimlerinin — gömülü çağrı, OxiWire sunucusu, dil istemcileri, uyumluluk katmanları — ortak bir noktası vardır ve o, on beşinci bölümdeki felsefenin özüdür: hepsi aynı çekirdek motora oturur. Python istemcisiyle yazılan bir belge, web yüzeyinden okunabilir; anahtar-değer katmanı ile konan bir değer, aynı motorun verisidir. Hangi kapıdan girerseniz girin, ardındaki depolama, indeks, işlem ve dayanıklılık mekanizmaları aynıdır. OxiDB'nin bu genişliği — tek bir motoru bu kadar çok yüzle sunması — onu klasik bir belge veritabanından ayıran belirgin özelliklerinden biridir.

Bu genişliğin bir bedeli olduğunu da dürüstçe söylemek gerekir. Her yüz, bakımı gereken bir yüzeydir; ve uyumluluk katmanları, çoğu zaman yabancı protokolün **tamamını** değil, “yeterli” bir altkümesini konuşur — yani o protokolün her inceliğini değil, en yaygın kullanımını destekler. Bu, makul bir ödünleşimdir: amaç, başka bir protokolün birebir kopyası olmak değil, o ekosistemin araçlarının çoğunun OxiDB ile çalışmasını sağlamaktır. Yine de, bir uyumluluk

katmanını kullanırken, onun bir köprü olduğunu — hedef sistemin yerine geçen tam bir taklit değil — akılda tutmak gerekir.

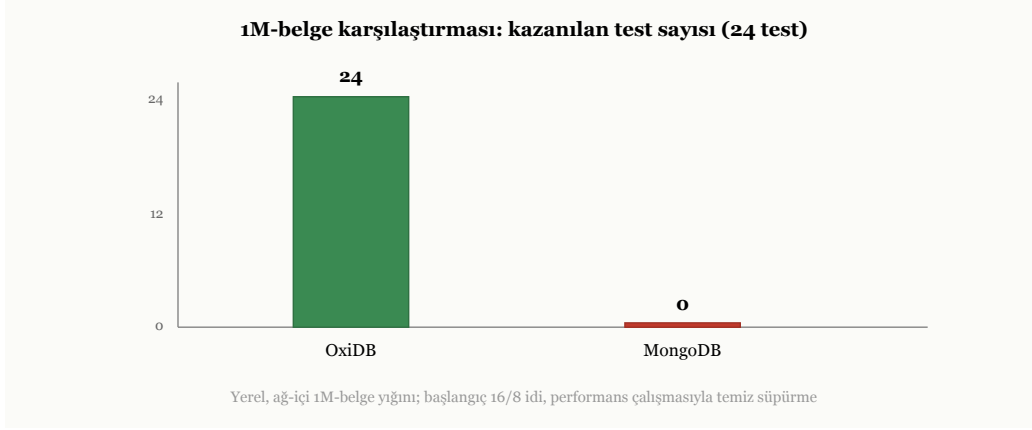
## 26.6 Bu bölümün bıraktığı yer

Bu bölümde, OxiDB’ye erişimin iki kapısını — gömülü ve sunucu — ve istemci kütüphanelerinin bu farkı nasıl gizlediğini gördük. Çekirdeği başka dillere açan C köprüsünü; istemcilerin neden hepsinin aynı kavramları sunduğunu ve bunun bu kitap yazılırken eklenen bir yetenekle nasıl somutlaştığını izledik. Anahtar-değer, mesajlaşma ve web olmak üzere üç uyumluluk yüzünü ve OxiDB’nin başka ekosistemlerin araçlarını nasıl ağırladığını gördük. Hepsinin aynı çekirdeğe oturduğunu — “tek çekirdek, çok yüz” — ve bu genişliğin dürüst bedelini de gördük.

Böylece, OxiDB’nin tüm katmanlarını — depolamadan dayanıklılığa, indeksten sorguya, işlem den ölçeklendirmeye, sunucudan istemcilere — dolaşmış olduk. Geriye, kitabı kapatmadan önce bir adım geri çekilip bütüne bakmak kaldı. Son bölümde, OxiDB’nin bu kitap boyunca birçok kez değindiğimiz bellek optimizasyonunu bir araya getirecek, onu MongoDB ile karşılaştıran ölçümleri dürüstçe değerlendirecek ve baştan beri izlediğimiz ilkelerin gerçek bir sistemde nasıl bir bütün oluşturduğu üzerine düşüneceğiz.

## 27 Bellek Optimizasyonu ve Karşılaştırmalı Değerlendirme

Kısım III boyunca, OxiDB'nin her katmanını tek tek dolaştık. Bu son bölüm, bir adım geri çekilip bütüne bakıyor. İki şeyi bir araya getiriyor: kitap boyunca birçok kez değindiğimiz bellek optimizasyonu hikâyesini ve OxiDB'yi olgun bir emsalle — MongoDB ile — karşılaştıran ölçümlerin dürüst bir değerlendirmesini. Amacımız bir galip ilan etmek değil; bu kitabın baştan beri savunduğu gibi, her sayının altındaki ödünleşimi görmek ve OxiDB'nin tercihlerini, onların gerçek sonuçlarıyla birlikte anlamaktır.



Şekil 27 — 1M-belge karşılaştırmasında kazanılan test sayısı.

### 27.1 Belleği yerleşik yığından çıkarma yolculuğu

On üçüncü ve on altıncı bölümlerde, belleğe öncelikli bir veritabanının temel kısıtını görmüştük: yerleşik bellek, veriyle birlikte büyür. OxiDB'nin varsayılan kipi tam da böyledir ve küçük-orta ölçekte mükemmeldir; ama veri büyüdükçe bellek, hem pahalı hem de sınırlayıcı bir kısıt haline gelir. OxiDB'nin bellek optimizasyonu, bu kısıt adım adım gevşetmenin hikâyesidir ve bu kitap yazılırken izlediğimiz bir yoldur.

Yol, etki sırasıyla şu durakları içerir. Önce **önbellekler bir bütçeyle sınırlandı**: on üçüncü bölümde anlattığımız gibi, kontrolsüz büyüyen önbellekler, sabit bir bellek bütçesine bağlandı; bu, bellek kullanımını öngörülebilir kıldı. Sonra **belge gövdeleri bellekten çıkarıldı**: on altıncı bölümdeki disk-öncelikli kip, yerleşik bellekte yalnızca kompakt bir kimlik-konum dizini bırakarak, belge başına yüzlerce baytlık yükü birkaç düzine bayta indirdi. Ardından **indeksler de çıkarıldı**: on sekizinci bölümde gördüğümüz belleğe yansıtılmış indeksler, indekslerin de yerleşik bellekten çıkmasını sağladı. Sonra **sorgu sırasındaki geçici bellek dizginlendi**: on dokuzuncu bölümdeki bayt düzeyinde süzme, büyük bir sonucun yüzlerce megabayt nesneye dönüşmesini önledi. Sıkıştırma, ölü alanı geri kazandı; ve sıkıştırmasız kip, tarama yükünü hafifletti.

Bu durakların birleşik sonucu çarpıcıdır. Disk-öncelikli kipte, beş yüz bin belgelik ve birkaç indeksli bir koleksiyonu taze açan bir süreç, yalnızca birkaç megabayt yerleşik bellekle açılır — oysa belleğe öncelikli yaklaşımda aynı veri yüzlerce megabayt tükettirdi. Hem belge gövdeleri hem de indeksler, yerleşik bellekten çıkmış, gerektiğinde diskten getirilen, geri alınabilir veriye dönüşmüştür. Bu yolculuk, on üçüncü bölümdeki “belleğe öncelikli kısıt” sorununun, bir dizi bilinçli mühendislik tercihiyle nasıl aşıldığının somut bir örneğidir.

### 27.2 Bellek ölçümünün dürüst yüzü

On üçüncü bölümde, “veritabanım ne kadar bellek kullanıyor” sorusunun aldatıcı derecede zor olduğunu söylemiştik; OxiDB'nin ölçümleri bunu canlı biçimde gösterir. Disk-öncelikli kipte taze açılışta bellek çok düşüktür

— çünkü henüz yalnızca o küçük dizin bellektedir. Ama tüm veriye dokunan büyük bir taramadan sonra, bellek yükselir; çünkü işletim sistemi dokunulan sayfaları belleğe çeker. Bu yükselen bellek, on üçüncü bölümde anlattığımız gibi, geri alınabilir bir bellektir; baskı altında işletim sistemi onu serbest bırakır.

Bu yüzden iki sistemi yalnızca bir andaki bellek sayısıyla karşılaştırmak yanıltıcıdır. OxiDB, taze açılışta emsalinden kat kat az bellek kullanır; ama tüm veriyi baştan sona tarayan bir iş yükünden sonra, iki sistemin yerleşik belleği birbirine yakınsar — çünkü her ikisi de dokunulan veriyi belleğe çeker. Dürüst sonuç şudur: disk-öncelik kipinin bellek kazancı, çalışma kümesi tüm veriden küçük olan yükler için en büyüktür; tüm veriyi sürekli tarayan yükler için ise bu kazanç azalır. Bu, bir zayıflık değil, on üçüncü bölümde tarif ettiğimiz çalışma kümesi gerçeğinin doğal bir sonucudur.

### 27.3 Karşılaştırmanın felsefesi

Bir karşılaştırmalı değerlendirme, koşulları açıkça belirtilmedikçe, bir yalandan ibarettir. Bu yüzden ölçümlere geçmeden önce, onları nasıl okumak gerektiğini söyleyelim. Bu kitaptaki ölçümler, iki sistemi aynı makinede, aynı veriyle ve emsalin belleği sınırlanmış bir biçimiyle karşılaştırdı. Ama daha önemlisi, bir karşılaştırmanın amacı bir galip ilan etmek değildir; her sonucun altındaki **ödünleşimi** görmektir. Tek bir sayıyı seçip “şu sistem daha hızlı” demek kolaydır; zor ve dürüst olan, o sayının hangi tercihten doğduğunu söylemektir. Aşağıda, OxiDB’nin nerede kazandığını, nerede berabere kaldığını ve nerede kaybettiğini — ve her birinin **neden** öyle olduğunu — bu kitabın anlattığı mekanizmalara bağlayarak göreceğiz.

### 27.4 OxiDB nerede kazanır

OxiDB’nin belirgin biçimde öne geçtiği yerler, bu kitapta anlattığımız mekanizmaların doğrudan meyveleridir. İndeksli **tam eşleşme** sorgularında — belirli bir e-posta adresine sahip belgeyi bulmak gibi — OxiDB, emsalini kat kat geçer; bu, yedinci ve on sekizinci bölümlerdeki sıralı indekslerin gücüdür. **Saymada** üstünlük daha da belirgindir; çünkü on sekizinci ve dokuzuncu bölümlerde gördüğümüz gibi, OxiDB indeksli bir alana göre saymayı belgelere hiç dokunmadan, doğrudan indeksten yapar. **Sıralı ilk-N** sorgularında, sekizinci ve on dokuzuncu bölümlerdeki erken sonlanma sayesinde OxiDB öne çıkar. **İndeks kurmada**, toplu silmede ve belleğe öncelikli kipte eşzamanlı okumalarda da OxiDB güçlüdür. Bu kazançların hiçbirisi sihir değildir; her biri, kitabın bir bölümünde anlattığımız somut bir tasarımın sonucudur.

### 27.5 OxiDB nerede berabere kalır

Bazı işlemlerde, iki sistem başa baş gider: eşitlik ve aralık sorguları gibi. Ama en öğretici beraberlik, **eklemededir**. OxiDB, toplu eklemede emsaliyle başa baş hız tutturur; ve bunu, altıncı ve on yedinci bölümlerde anlattığımız gibi, **her partiyi gerçekten diske boşaltarak** yapar. Emsali ise, varsayılan ayarında bu boşaltmayı yapmaz; yazmayı bellekten onaylar. Yani OxiDB, başa baş hıza, daha **güçlü** bir dayanıklılık güvencesiyle ulaşır. Bu, bir sonraki başlıkta döneceğimiz can alıcı dürüstlük noktasıdır.

### 27.6 OxiDB nerede kaybeder ve neden

Dürüst bir değerlendirme, kaybedilen yerleri ve nedenlerini de açıkça söylemek zorundadır. OxiDB’nin emsalinin gerisinde kaldığı her yer, bu kitabın anlattığı bir ödünleşime kadar izlenebilir.

Birincisi, **tek bir belgenin güncellenmesidir**. OxiDB, varsayılan katı dayanıklılık kipinde her commit’i gerçekten diske boşalttığı için, tek bir güncelleme bir tam boşaltma — milisaniyeler mertebesinde bir maliyet — öder. Emsali, varsayılan ayarında bu boşaltmayı yapmadığı için çok daha hızlı görünür. Ama on yedinci bölümde

gördüğümüz gibi, bu elma ile armuttur: OxiDB her tekil yazmayı gerçekten dayanıklı kılarken bir bedel öder, emsali o bedeli erteler. Gevşek kipe geçildiğinde — yani aynı dayanıklılık modeli seçildiğinde — OxiDB’nin tek belge güncellemesi, emsalini bile geçer. Yani buradaki “kayıp”, bir performans yetersizliği değil, bir güvenlik tercihinin görünür maliyetidir.

İkincisi, **disk-öncelikli kipte, büyük ölçekte eşzamanlı tekil okumalardır**. On üçüncü bölümde gördüğümüz gibi, çalışma kümesi belleği aştığında, rastgele tekil okumalar diske fault eder ve yavaşlar. Milyon belgelik bir veride, çok sayıda eşzamanlı rastgele okuma, disk-öncelikli kipte belirgin biçimde yavaştı; çünkü belge gövdeleri diskte, belleğe yansıtılmış halde duruyordu. Bu, disk-öncelik tercihinin beklenen bedelidir — bellek kazancının karşılığında, soğuk veriye erişimin gecikmesi.

Üçüncüsü, **disk-öncelikli, sıkıştırılmış kipte tüm koleksiyonu tarayan toplamalardır**. Yirminci ve on altıncı bölümlerde anlattığımız gibi, toplama tüm belgelere dokunur; sıkıştırılmış kipte her belgenin açılması gerekir ve bu maliyet birikir. Ama bu, kitap yazılırken giderildi: indeksli sayma kestirmesinin disk indekslerinde yeniden etkinleştirilmesi ve sıkıştırmasız kip, bu yükleri büyük ölçüde hızlandırdı. Yani bu kayıp, bir kez teşhis edilip mekanizmaya bağlandıktan sonra, doğru tercihlerle büyük ölçüde kapandı.

Bu üç kaybın ortak özelliği, hepsinin bu kitabın anlattığı bir ödünleşime dayanmasıdır: dayanıklılık-hız, bellek-gecikme, yer-işlemci. Hiçbiri açıklanamaz bir zayıflık değildir; her biri, bilinçli bir tercihin ölçülmüş sonucudur.

## 27.7 Karşılaştırmanın asıl dersi: dayanıklılık merceği

Bu değerlendirmenin en önemli dürüstlük noktası şudur: hız sayılarını, dayanıklılık güvencelerini karşılaştırmadan okumak yanıltıcıdır. İki veritabanını yalnızca “kaç işlem saniyede” diye karşılaştırmak, eğer biri her yazmayı diske boşaltıyor, diğeri bellekten onaylıyorsa, elma ile armuttur. OxiDB’nin varsayılanı, emsalinin varsayılanından daha güçlü bir dayanıklılık sunar; ekleme ve güncelleme sayıları, ancak bu mercekten okunduğunda anlam kazanır. Bir karşılaştırmayı dürüst kılan, sayıları vermek değil, o sayıların hangi güvenceler altında elde edildiğini söylemektir.

## 27.8 İlkelerin bir bütün oluşturması

Bu son bölüm, aslında tüm kitabın bir özetidir. OxiDB’nin her kazancı, her beraberliği ve her kaybı, bu kitabın ilk iki kısmında öğrendiğimiz bir ilkeye ve bir ödünleşime kadar izlenebilir. Sıralı indeksler tam eşleşmeyi ve saymayı hızlandırır; katı dayanıklılık tekil yazmaları yavaşlatır ama güvenceyi güçlendirir; disk-öncelik belleği kurtarır ama soğuk okumayı yavaşlatır; sıkıştırma yer kazandırır ama tarama işlemcisini artırır. Karşılaştırmalı ölçümler, bu kitabın soyut ilkelerinin **ölçülebilir** hale gelmiş halidir.

İkinci bölümde “her model bir ödünleşimdir, gümüş kurşun yoktur” demiştik; bu son bölüm, aynı dersi sistem düzeyinde tekrarlar. OxiDB, evrensel ilkelerin somut bir cisimleşmesidir; ayırt edici tercihleri — tek çekirdek-çok yüz, belleğe öncelikli ve disk-öncelikli arasındaki bellek ayarı, belge odağı, bayt düzeyinde teknikler — sihir değil, bilinçli ödünleşimlerdir. Onun nerede parladığını ve nerede zorlandığını anlamak, bu ödünleşimleri görmekten geçer; ve bu kitap, baştan beri, o ödünleşimleri görmeyi öğretmeye çalıştı.

## 27.9 Bu bölümün ve ana metnin sonu

Bu bölümle birlikte, kitabın ana yolculuğunu tamamladık. Belge veritabanlarının ne olduğunu temelden kurduk; içeride nasıl çalıştıklarını katman katman açtık; ve tüm bu ilkelerin OxiDB adlı gerçek bir motorda nasıl hayata geçtiğini adım adım izledik. İkinci bölümde söylediğimiz gibi, ilk iki kısım size bu alanın sözlüğünü ve dilbilgisini öğretti; üçüncü kısım, o dille yazılmış gerçek bir metni birlikte okudu. Artık bir belge veritabanına baktığınızda —

ister OxiDB olsun ister başkası — onun verdiđi her sözün altındaki mekanizmayı ve her hızın ardındaki ödünleşimi görebilecek bir bakışa sahipsiniz.

Kitabın geri kalanında, başvurmak isteyebileceğiniz iki ek bulacaksınız: kitap boyunca kullandığımız terimlerin bir sözlüğü ve daha derine inmek isteyenler için bir kaynaklar listesi. Ama asıl yolculuk burada tamamlandı. Bir veritabanının mütevazı vaadiyle — bir şeyi hatırlamak — başladık ve o vaadin altındaki zarif, çetin ve birbirine bağı mekanizmaların tümünü dolaştık. Umarız, bir daha bir veritabanına “veriyi koy, sonra geri al” diye baktığınızda, o basit cümlenin altında ne kadar derin bir mühendisliğin yattığını hatırlarsınız.

# Ek A — Sözlük

Bu sözlük, kitap boyunca kullanılan başlıca terimleri kısaca tanımlar. Tanımlar, terimin ilk geçtiği bölümdeki ayrıntılı anlatımın yerini tutmaz; hızlı bir hatırlatma olarak düşünülmüştür.

## ACID

Bir işlemin verdiği dört güvencenin kısaltması: atomiklik, tutarlılık, yalıtım ve dayanıklılık.

## Açma (unwind)

Bir toplama aşaması; içinde liste barındıran bir belgeyi, listenin her ögesi için bir tane olmak üzere birçok belgeye genişletir.

## Anahtar-değer modeli

Veriyi bir anahtardan opak bir değere giden eşleme olarak tutan, en yalın veri modeli.

## Append-only (yalnızca-ekleme)

Var olan veriyi yerinde değiştirmeyen, her yazmayı dosyanın sonuna ekleyen depolama yaklaşımı; ardışık yazma hızlıdır, ölü alan biriktirir.

## Atomiklik

Bir işlemin tüm adımlarının ya hep birlikte gerçekleşmesi ya da hiç gerçekleşmemesi güvencesi.

## Ayrıştırma (parsing)

Ham bir sorguyu, sistemin üzerinde işlem yapabileceği yapısal bir koşul ağacına çevirme adımı.

## B-ağacı

Veriyi sıralı ve dengeli tutan, hem tekil hem aralık aramalarında verimli bir ağaç yapısı; sayfa tabanlı depolama motorlarının ve sıralı indekslerin temelidir.

## Bayt düzeyinde süzme

OxiDB'nin, bir belgeyi koşula uyup uymadığını anlamak için onu nesneye çevirmeden, kodlanmış baytları üzerinde denetleyen ve eşleşmeyenleri hiç çözmeden eleyen tekniği.

## Belge (doküman)

Alanlardan oluşan, değerleri skaler, liste ya da iç içe belge olabilen, kendini tanımlayan veri birimi.

## Bileşik indeks

Birden çok alanın birleşimi üzerine kurulu indeks; alan sırasının baştan başlayan bir önekini kullanan sorgulara yarar (önek kuralı).

## Biriktirici (accumulator)

Gruplama sırasında her grup için bir özet değer hesaplayan işlem; sayma, toplama, ortalama, en büyük, en küçük gibi.

## CAP

Ağ bölündüğünde, bir dağıtık sistemin tutarlılık ile erişilebilirlik arasında seçim yapmak zorunda olduğunu ifade eden içgörü.

## Çalışma kümesi (working set)

Verinin, herhangi bir anda etkin biçimde kullanılan sıcak kısmı; belleğe sığıp sığmaması performansı belirler.

## Çoğunluk (quorum)

Bir kararın geçerli sayılması için gereken düğüm çoğunluğu; herhangi iki çoğunluğun kesişmesi, konsensüsün güvenliğini sağlar.

**Dağıt-topla (scatter-gather)**

Sharding’de, parça anahtarı içermeyen bir isteğin tüm parçalara gönderilip kısmi yanıtların birleştirilmesi örüntüsü.

**Dayanıklılık (durability)**

Bir işlem tamamlandı dindikten sonra, hiçbir çökmenin onu geri alamaması güvencesi.

**Denetim (audit)**

“Kim, ne zaman, ne yaptı” sorusunu yanıtlayan, hesap verebilirlik için tutulan kayıt.

**Denetim noktası (checkpoint)**

Yazma-öncesi günlüğün, değişiklikler asıl depoya güvenle yansıdıktan sonra kısaltıldığı senkronizasyon anı.

**Disk-öncelikli (disk-first)**

OxiDB’nin, bellekte yalnızca kompakt bir kimlik-konum dizini tutup belge gövdelerini diske bırakan, bellek-tutumlu depolama kipi.

**Erken sonlanma**

Bir sorgunun, gereken sonuca ulaştığı an durup geri kalanı üretmemesi; sıralı indeksli “ilk N” sorgularında ve tekil işlemlerde kullanılır.

**Eşzamansız replikasyon**

Liderin, bir yazmayı takipçilere iletmeyi beklemeden onayladığı, hızlı ama kayıp riski taşıyan replikasyon biçimi.

**Failover (devralma)**

Lider çöktüğünde, bir takipçinin yeni lider olarak yükseltilmesi süreci.

**FFI**

Bir dilde yazılmış çekirdeği, C uyumlu bir arayüz üzerinden başka dillerden çağrılabilir kılan köprü.

**fsync**

Yazılmış verinin gerçekten kalıcı ortama işlendiğinden emin olmak için verilen, güçlü ama yavaş boşaltma emri.

**Gömme (embedding)**

İlişkili veriyi, ona ait olduğu belgenin içine yerleştirme; yerellik kazandırır, çoğaltma ve sınırsız büyüme riski getirir.

**Gruplama**

Belgeleri bir anahtara göre gruplara ayırıp her grup için biriktiricilerle özet hesaplayan, satırları çökerten toplama işlemi.

**İndeks**

Bir alanın değerlerinden belgelerin konumuna giden, aramayı hızlandıran, veriden türetilmiş yardımcı yapı.

**İşlem (transaction)**

Birçok işlemi tek bir bölünmez birim olarak ele alan, “ya hep ya hiç” güvencesi veren kavram.

**İyimser eşzamanlılık denetimi (OCC)**

Çatışmaların nadir olduğunu varsayan, kilit almadan çalışıp tamamlama anında sürüm doğrulayan ve çatışmada iptal eden yaklaşım; OxiDB’nin kullandığı model.

**JSON**

JavaScript nesne gösteriminden türeyen, dilden bağımsız, metinsel veri değiş-tokuş biçimi; belge yapısının yazıya dökülmüş hâli.

**Kapsayan indeks (covering)**

Bir sorgunun ihtiyaç duyduğu her şeyi tek başına sağlayan ve böylece belgeye hiç dokunmadan yanıt üretilmesini mümkün kılan indeks durumu.

### **Kilitlenme (deadlock)**

İki işlemin her birinin, diğerinin tuttuğu kaynağı beklemesiyle oluşan, çözülmediğinde sonsuza dek süren döngüsel bekleme.

### **Konsensüs**

Bir grup makinenin, bazıları çökse bile ortak bir karar üzerinde güvenle anlaşmasını sağlayan, çoğunluk oylamasına dayanan mekanizma.

### **Koleksiyon**

Belgelerin gruplandığı birim; ilişkisel tablonun karşılığıdır ama belgelerin aynı biçimde olmasını zorunlu kılmaz.

### **Kurtarma (recovery)**

Çökme sonrası, yazma-öncesi günlüğü oynatarak veritabanını tutarlı bir duruma getiren süreç.

### **LSM ağacı**

Yazmaları bellekte biriktirip sıralı parçalar halinde diske yazan ve bunları arka planda birleştiren, yazma-yoğun log-yapılı depolama tasarımı.

### **mmap (belleğe yansıtma)**

Bir dosyayı belleğin bir parçasıymış gibi erişilebilir kılan ve önbellek yönetimini büyük ölçüde işletim sistemine devreden teknik.

### **MVCC**

Verinin birden çok sürümünü tutarak okuyucuların tutarlı bir anlık görüntü görmesini, okuyucu ile yazıcının birbirini beklememesini sağlayan yaklaşım.

### **Nihai tutarlılık**

Kopyaların geçici olarak anlaşmazlığa düşmesine izin veren ama yazmalar durursa aynı değere yakınsamayı garanti eden tutarlılık biçimi.

### **Normalleştirme**

Her bilgiyi tek bir yerde tutarak çoğaltmayı önleme; ilişkisel modelin güncellemeyi ucuzlatan ilkesi.

### **Ölü alan**

Append-only depolamada, güncellenen ya da silinen kayıtların geride bıraktığı, artık geçerli olmayan veri; sıkıştırma ile geri kazanılır.

### **Önek kuralı**

Bir bileşik indeksin, ancak alan sırasının baştan başlayan bir önekini kullanan sorgulara yaradığı kural.

### **Pencere fonksiyonu**

Her belgeyi koruyarak, ona komşu belgelerden oluşan bir pencereye dayalı bir değer ekleyen analitik işlem; kümülatif toplam, hareketli ortalama, sıralama gibi.

### **Pipeline**

Toplamanın, her biri akışı dönüştürüp bir sonrakine veren aşamalardan oluşan, bileşilebilir modeli.

### **PITR (zaman-noktasına kurtarma)**

Veritabanını yalnızca son tutarlı duruma değil, geçmişteki belirli bir ana geri döndürebilme yeteneği.

### **RBAC (rol tabanlı erişim denetimi)**

Yetkileri rollerde gruplayıp kullanıcılara rol atayarak yetkilendirmeyi yöneten model.

**Replikasyon**

Aynı veriyi birden çok makinede kopya halinde tutma; dayanıklılık, erişilebilirlik, okuma ölçęi ve gecikme için yapılır.

**Saęlama toplamı (checksum)**

Bir kaydın içerięinden hesaplanan, yarım kalmıř ya da bozulmuř kayıtları yakalamaya yarayan doęrulama deęeri.

**Sayfa**

Diskten tek seferde okunup yazılan, sabit boyutlu blok; sayfa tabanlı depolama motorlarının temel birimi.

**Seęicilik**

Bir indeks alanının deęerlerinin ne kadar az belgeyle eřleřtięi; yüksek seęicilik, indeksi daha yararlı kılar.

**Sharding (paręalama)**

Veri kümesini makineler arasında bölerek kapasiteyi ve yazma hacmini ölçeklendirme teknięi.

**Sıfır-kopya**

Sıkıştırılmamıř ve şifrelenmemiř veriye, belleęe yansıtılmıř dosyadan hiç kopyalamadan ve çözmeden doęrudan erişebilme.

**Sıkıştırma (compaction)**

Append-only bir veri dosyasını bařtan yazıp yalnızca yařayan kayıtları tutarak ölü alanı geri kazanan bakım iřlemi.

**Split-brain (ikiye bölünmüř beyin)**

Bir aę bölünmesinde, aynı anda iki liderin ortaya çıkıp veriyi çeliřkili iki gerçeęe ayırması tehlikesi; çoęunluk mutabakatı bunu önler.

**Sürüm numarası**

Her belgeye iliřtirilen, her deęiřiklikte artan sayaç; iyimser eřzamanlılık denetiminde çatıřmayı saptamaya yarar.

**řema-okumada / řema-yazmada**

řemanın okuma anında (uygulamada, örtük) mı yoksa yazma anında (veritabanında, açık) mı dayatıldığını ayıran kavramlar.

**Tahliye (eviction)**

Dolu bir önbellekte, yeni bir řeye yer açmak için bir öęenin atılması; yaygın bir politika “en uzun süredir kullanılmayanı at” (LRU) ilkesidir.

**Tam tarama (full scan)**

Aranan kaydı bulmak için tüm belgeleri tek tek okuma; uygun bir indeks yoksa bařvurulan, maliyetli yol.

**Ters indeks (inverted index)**

“Sözcük, o sözcüęü içeren belgeler” eřlemesi; metin içinde sözcük aramayı ve alaka puanlamasını mümkün kılar.

**Tutarlılık (tek makine / daęıtık)**

Tek makinede, verinin geęerli kurallara uyması; daęıtık sistemde, kopyaların ne ölçüde aynı deęeri gösterdięi (güçlüden nihaiye uzanan bir tayf).

**Veri modeli**

Verinin nasıl yapılandırıldığını, iliřkilerin nasıl ifade edildiğini ve hangi iřlemlerle erişildiğini belirleyen temel karar.

**WAL (yazma-öncesi günlük)**

Asıl veriyi deęiřtirmeden önce niyeti kaydedip dayanıklı kılan, dayanıklılık ve çökme güvenlięinin temelindeki günlük.

**Yalıtım (isolation)**

Aynı anda çalışan işlemlerin birbirini etkilememesi; sonucun, işlemler sırayla çalışmış gibi olması güvencesi.

**Yerellik**

İlişkili veriyi bir arada tutarak tek bir okumayla almayı sağlama; belge modelinin gömme yoluyla sunduęu kazanç.

**Yetkilendirme (authorization)**

Kimlięi bilinen bir kullanıcının hangi işlemleri yapabileceęine karar verme.

## Ek B — Kaynaklar

Bu kitap, belge veritabanlarını ve OxiDB’yi temelden, kod kullanmadan anlatmayı amaçladı. Daha derine inmek isteyenler için, burada konuları temalara göre gruplanmış bir okuma ve inceleme rehberi sunuyoruz. Belirli kaynakları, kesin yayın künyeleri yerine ne içerdikleriyle tanıttık; çünkü amaç, bir bibliyografya değil, hangi yöne bakacağınıza dair bir pusula vermektir.

### 27.10 Genel: veri-yoğun sistemlerin temelleri

Bu kitabın ikinci kısmında ele aldığımız ilkelerin — depolama, dayanıklılık, indeksleme, replikasyon, tutarlılık — derinlemesine ve bütünsel bir incelemesi için, Martin Kleppmann’ın *Designing Data-Intensive Applications* adlı eseri, alanın en yaygın başvurulanan modern kaynağıdır. Veritabanı motorlarının iç mekanizmalarına — depolama düzenleri, B-ağaçları, LSM ağaçları, işlem ve dağıtım — daha doğrudan inen bir kaynak için Alex Petrov’un *Database Internals* kitabı önerilir. Bu iki eser, bu kitabın ikinci kısmının doğal devamı niteliğindedir.

### 27.11 Veri modelleri ve ilişkisel temel

Üçüncü bölümde değindiğimiz ilişkisel modelin temelini atan, E. F. Codd’un 1970 tarihli “A Relational Model of Data for Large Shared Data Banks” makalesi, alanın kurucu metnidir ve mantık-fizik ayrışması fikrinin kaynağıdır. Belge modelinin pratiğini ve gömme-referans kararını daha somut görmek için, yaygın belge veritabanlarının (örneğin MongoDB’nin) veri modelleme kılavuzları, kavramları gerçek senaryolarla ilişkilendirir.

### 27.12 JSON ve belge biçimleri

Dördüncü bölümde tarihçesini anlattığımız JSON’un sade tanımı ve dilbilgisi için [json.org](https://json.org) adresi, biçimin kaynağıdır. Biçimin resmî, kesin tanımları, yazılım endüstrisi ve internet standart kuruluşlarının yayımladığı şartnamelerde bulunur. Veritabanlarının içeride kullandığı, JSON’un daha zengin ikili akrabaları (belge veritabanlarının ikili kodlamaları gibi) hakkında, ilgili sistemlerin biçim belgeleri ayrıntı sağlar.

### 27.13 Depolama motorları ve indeksleme

Beşinci ve yedinci bölümlerin konuları — B-ağaçları, log-yapılı ve LSM depolama, indeks yapıları — klasik veritabanı ders kitaplarında ve yukarıda anılan *Database Internals* eserinde kapsamlı biçimde işlenir. Tam metin aramanın temelindeki alaka puanlama yöntemleri (terim sıklığı temelli yaklaşımlar ve onların olgun biçimleri), bilgi erişimi (information retrieval) alanının standart kaynaklarında ele alınır.

### 27.14 İşlemler, eşzamanlılık ve tutarlılık

Onuncu ve on birinci bölümlerin konuları — ACID, yalıtım düzeyleri, kilitleme, çok sürümlü ve iyimser eşzamanlılık denetimi — veritabanı kuramının klasik metinlerinde derinlemesine incelenir. Dağıtık tutarlılık tarafında, on birinci bölümde değindiğimiz CAP içgörüsü Eric Brewer’a; ilgili pratik tutarlılık modelleri ise dağıtık sistemler yazınına dayanır.

### 27.15 Ölçeklendirme ve konsensüs

On ikinci ve yirmi beşinci bölümlerde anlattığımız konsensüs, OxiDB’nin de dayandığı Raft protokolünün temelinde yatar; Diego Ongaro ve John Ousterhout’un “In Search of an Understandable Consensus Algorithm” başlıklı

çalışması, Raft'ı anlaşılır biçimde tanıtan kurucu metindir ve konsensüsün neden çoğunlukla çalıştığını kavramak için en iyi başlangıçtır.

## 27.16 OxiDB'nin kendi kaynakları

Üçüncü kısımda anlattığımız OxiDB'nin somut tasarımına daha yakından bakmak isteyenler için en doğrudan kaynak, projenin açık kaynak deposudur ([parisxmas/OxiDB](https://github.com/parisxmas/OxiDB)). Deponun içinde, bu kitapta değindiğimiz birçok mühendislik kararının ardındaki gerekçeyi bulabileceğiniz birkaç materyal vardır. Mimari karar kayıtları ([docs/decisions/](#) altındaki belgeler), disk-öncelikli depolama ve sıkıştırma gibi konularda yapılan tercihleri ve ödünleşimleri belgeler. Karşılaştırmalı değerlendirme materyali ([tests/benchmark-1m/](#) altındaki rapor), yirmi yedinci bölümde özetlediğimiz ölçümlerin ayrıntısını içerir. Sürüm değişiklik günlüğü ([CHANGELOG.md](#)), bu kitap yazılırken eklenen yeteneklerin — çok-yönlü analiz, pencere fonksiyonları, sıkıştırmasız kip, per-koleksiyon ayarlar — gerekçeleriyle birlikte kaydını tutar. Ve doğrulama testleri (örneğin küme ve parçalar-arası toplama testleri), yirmi beşinci bölümde anlattığımız davranışların nasıl sınındığını gösterir.

## 27.17 Son bir söz

Bu kaynaklar, bir bitiş değil, bir başlangıçtır. Bu kitabın amacı, size belirli bir sistemi ezberletmek değil, herhangi bir belge veritabanına baktığınızda onun altındaki ilkeleri ve ödünleşimleri görebilecek bir bakış kazandırmaktı. O bakışla donanmış olarak, yukarıdaki kaynaklara — ya da karşınıza çıkacak herhangi bir veri sistemine — artık çok daha hazırlıklı yaklaşabilirsiniz. İyi okumalar.